CSCI 566: Deep Learning and its Applications

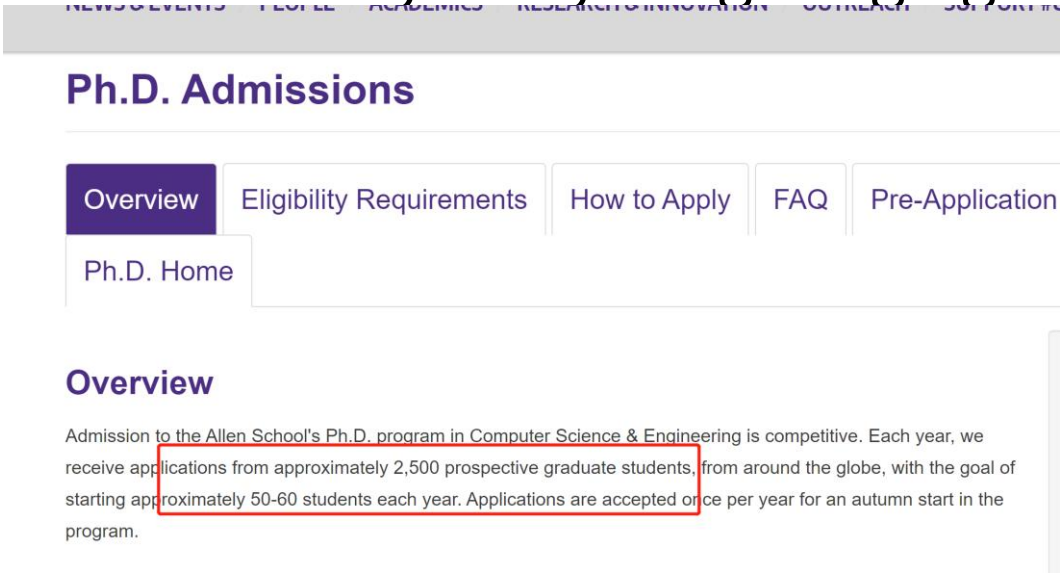
Yue Zhao
Thomas Lord Department of Computer Science
University of Southern California

Credits to previous versions of USC CSCI566, CMU 10601/701, Stanford CS 229, 231n

USC Viterbi School of Engineering

Current Ph.D. Application Situation

For top universities, e.g., top 50 in the U.S., the number of applications increases like crazy – anything ranging from 2%~10% (like us)



USC Viterbi School of Engineering

Current Ph.D. Application Situation

However, many decent universities face the issue of not getting enough applicants – which is good opportunity if you are looking into industry jobs after the Ph.D. time.

- Top 100-200
- Not in the huge cities but in less popular locations

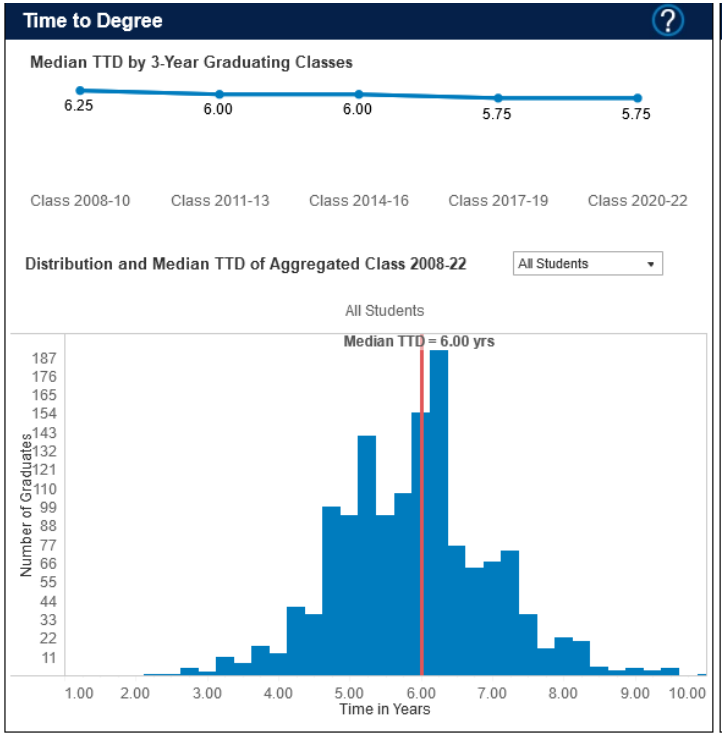
Should I do it?

Pros:

- An alternative when industry goes bad
- Better leverage/title in a few years

Cons:

- Waste of time
- May not generate any impact
- Even may not get it (~49% in the U.S.)
- Have to find something between June to next September…



USC Viterbi School of Engineering

How to secure a position?

For top universities, we value:

- Research potentials – via letters from top researchers in the field
- Publications – the track record to show you will excel in the future
    - Top venues only
- Nothing else (many of time, connection is all u need ☹)

For decent universities, we value:

- Good undergraduate school (top ones from each country) with good GPAs
- Maybe some publications to show interest (optional)

How to prepare for it?

What is more urgent?

- Consider whether you want to do it
- Understand and find the topic to work on
- Identify potential research mentors at USC to accumulate research experience – PhD students are most appropriate to yield publications
    - Note: course recommendation letters mean NOTHING
    - Unfortunately, I have too much on my plate
- Reach out (cold email) to potential Ph.D. advisors by research interest and shared background

Convolutional Neural Networks (CNN)

Recap on NN

Different NN models

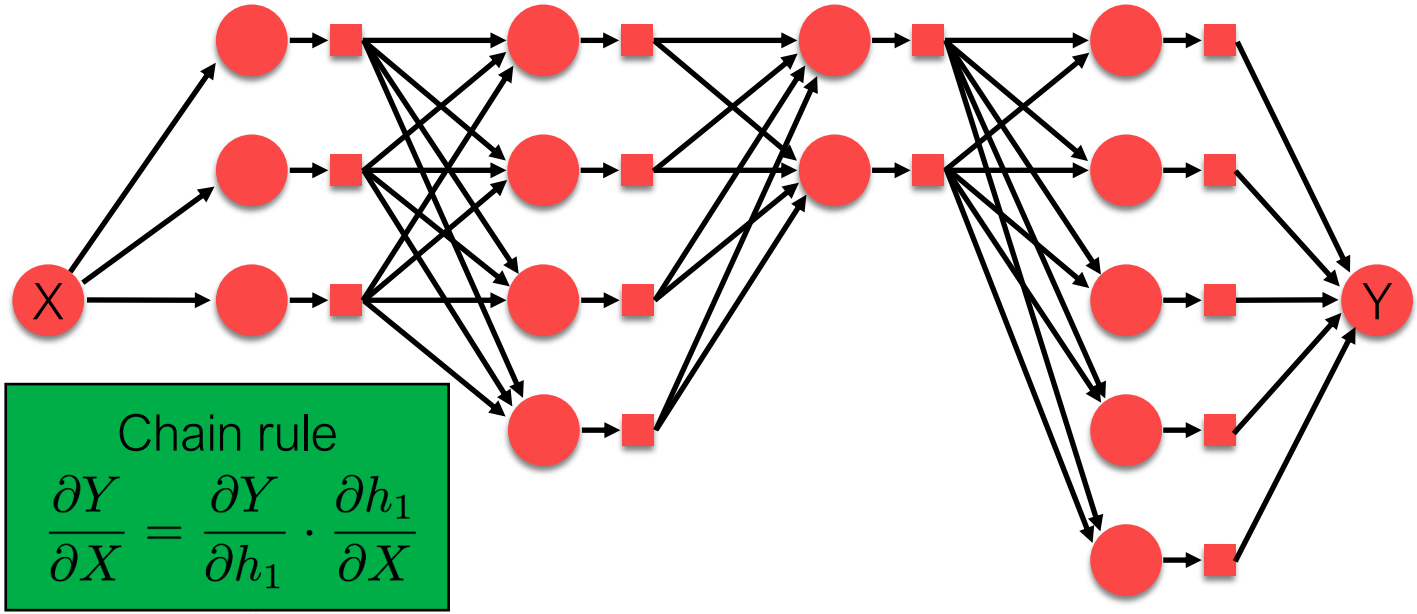
Feedforward networks (CNNs, …)

Recurrent networks (RNNs, …)

Memory networks

…

Recap: Backward Propagation



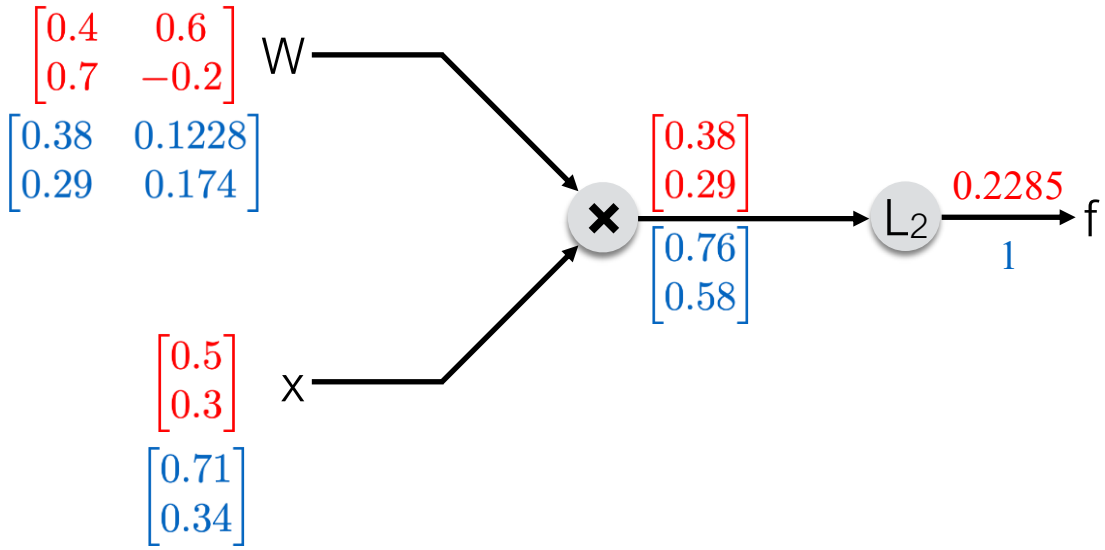
Chain rule

$$\frac{\partial Y}{\partial X} = \frac{\partial Y}{\partial h_1} \cdot \frac{\partial h_1}{\partial X}$$

Backward propagation

Recap: Computational Graph

A vectorized example: $f(x, W) = ||W \cdot x||^2 = \Sigma_{i=1}^{n}(W \cdot x)_i^2$



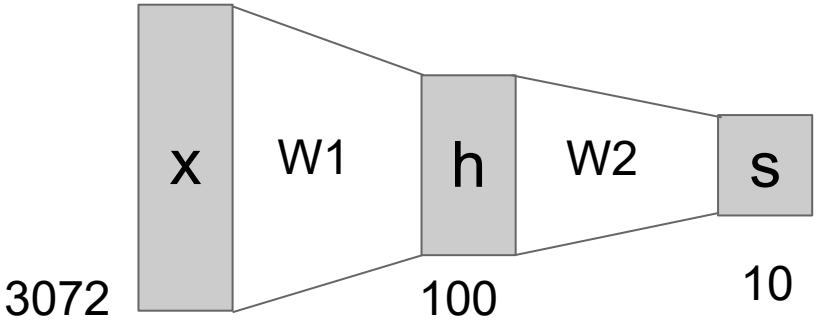
Recap : Neural Networks

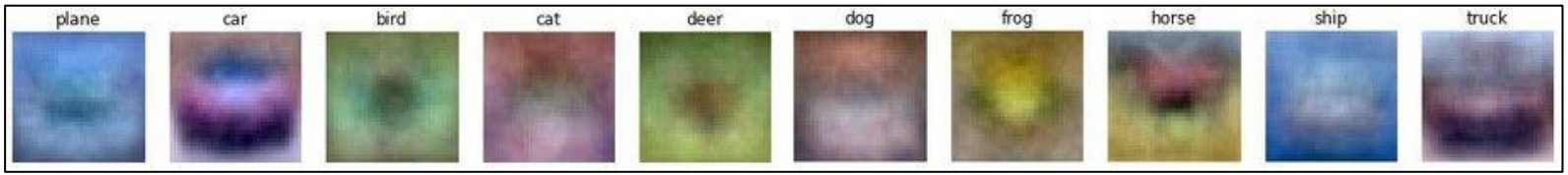
Linear score function:

$$f = Wx$$

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$





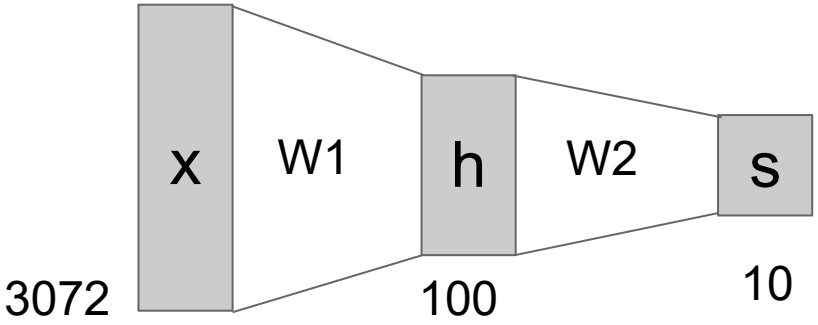
Recap : Neural Networks

Linear score function:

$$f = Wx$$

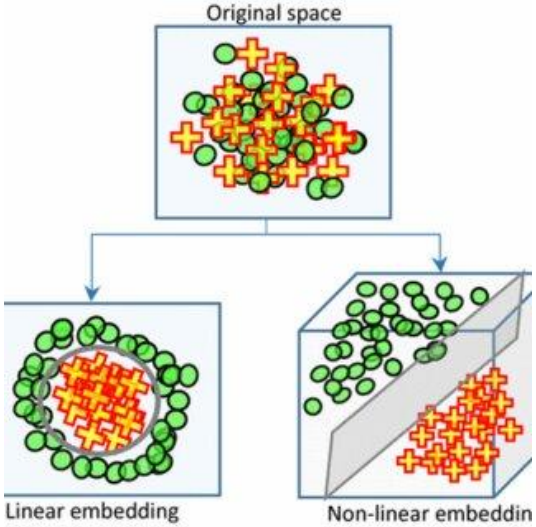
2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$



That is why NN is often called “feature extractors” – learn a good space

Recap : Neural Networks



That is why NN is often called “feature extractors” – learn a good space

Next: Convolutional Neural Networks

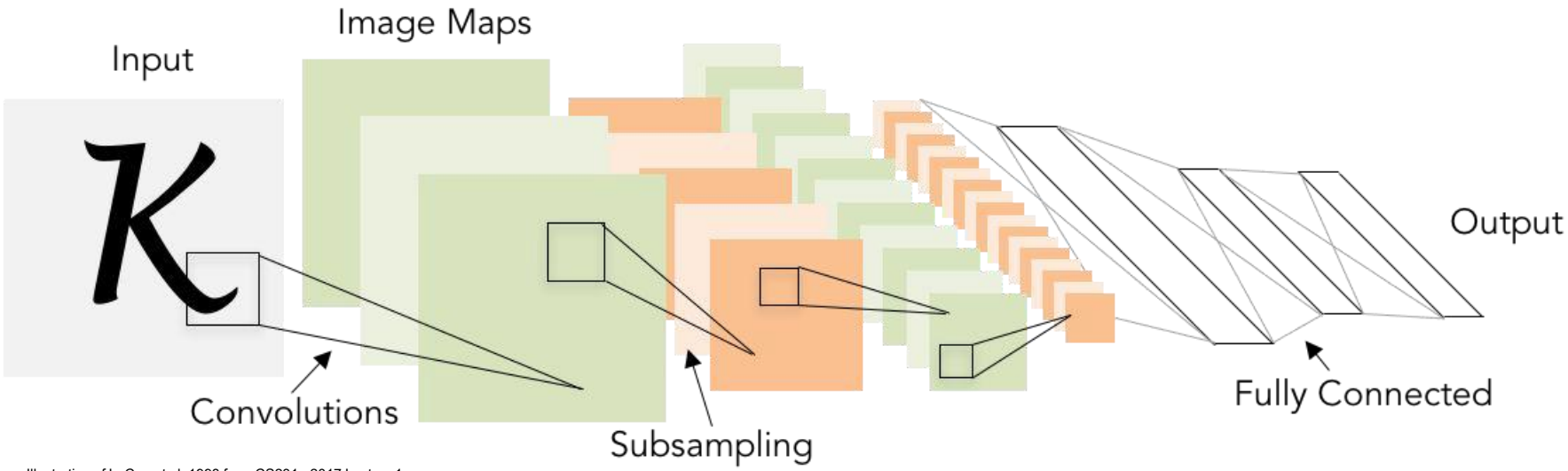


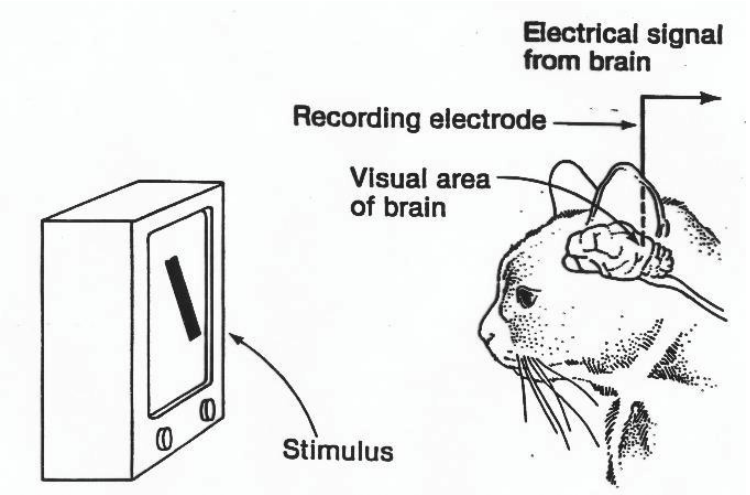
Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

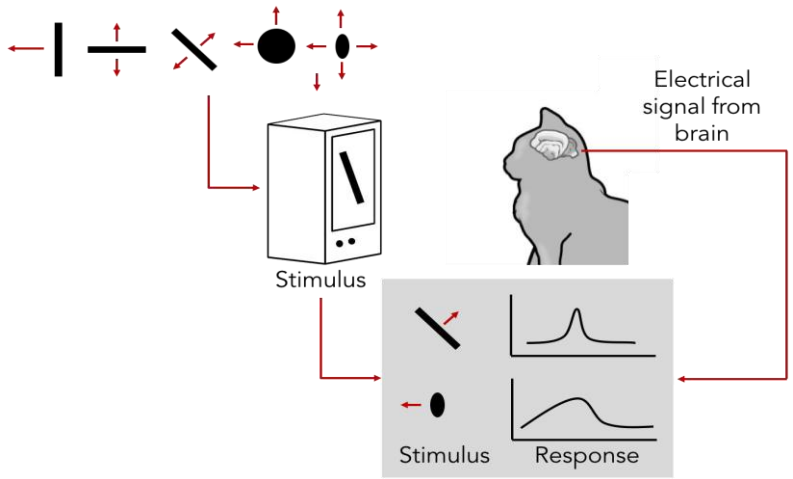
Convolutional Neural Networks (CNN)

History

History

Hubel & Wiesel 1959



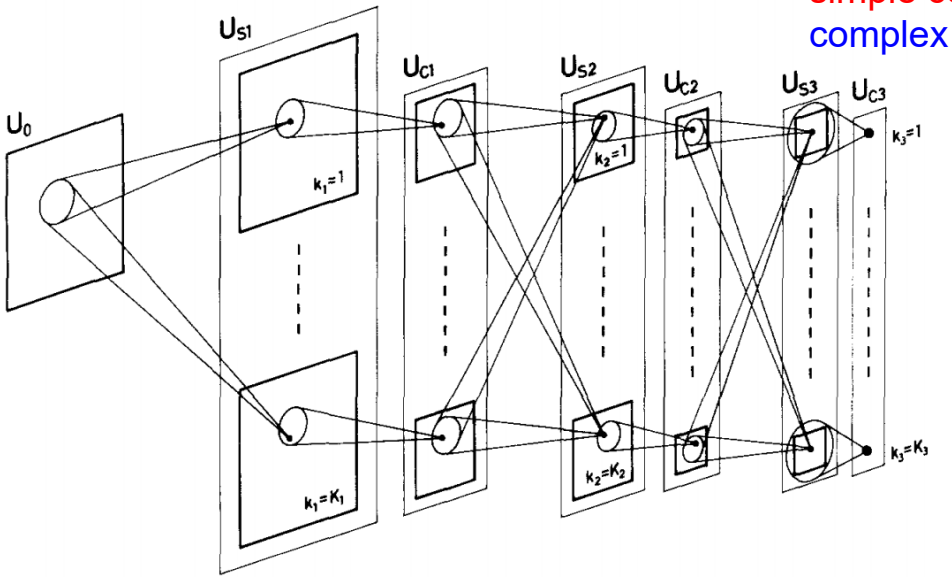


Understanding how the system constructs complex representation from simple stimulus features.

History

Neocognitron 1980

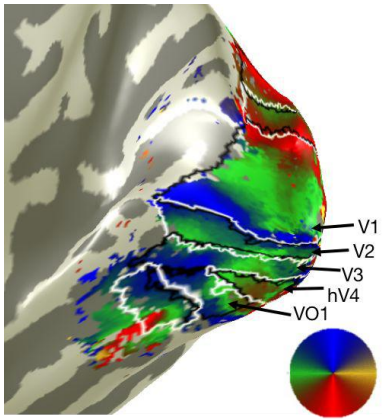
"sandwich" architecture (SCSCSC…)
simple cells: modifiable parameters
complex cells: perform pooling



Fukushima, Kunihiko. Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition. Competition and cooperation in neural nets 1982.

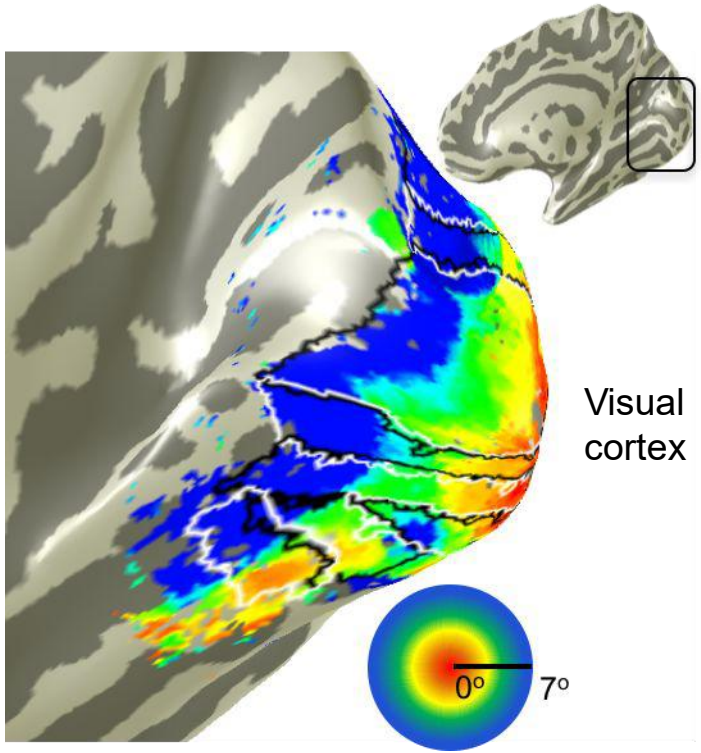
History

Topographical mapping in the cortex:
nearby cells in cortex represent
nearby regions in the visual field



There is a direct relationship between the spatial layout of the retina and the spatial layout of the visual cortex.

Human brain



Visual cortex

Retinotopy images courtesy of Jesse Gomez in the Stanford Vision & Perception Neuroscience Lab.

Hierarchical organization

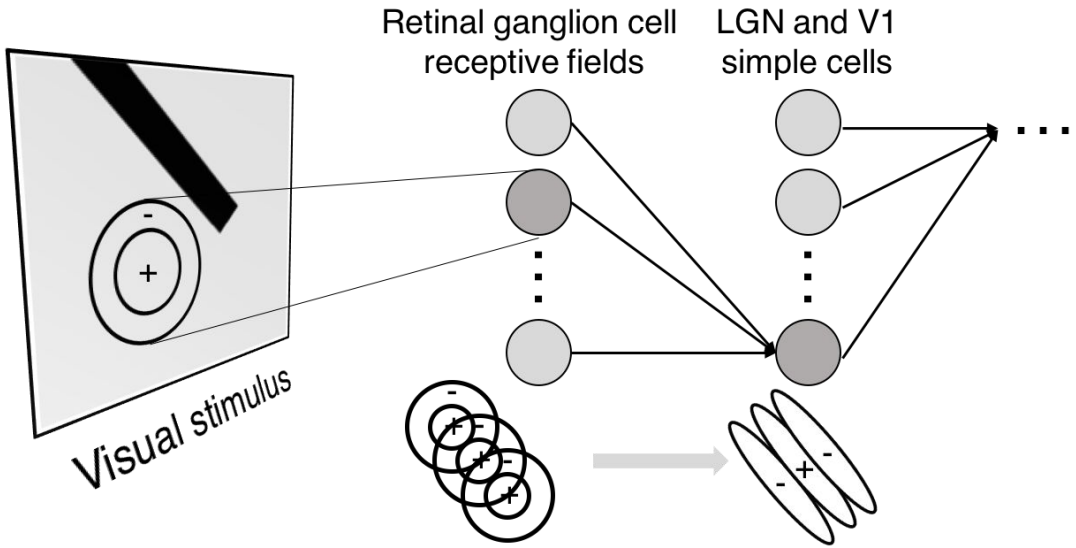
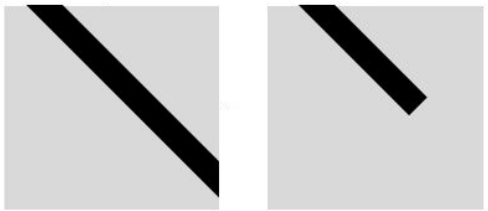


Illustration of hierarchical organization in early visual pathways by Lane McIntosh, copyright CS231n2017

Simple cells:
Response to light orientation

Complex cells:
Response to light orientation and movement

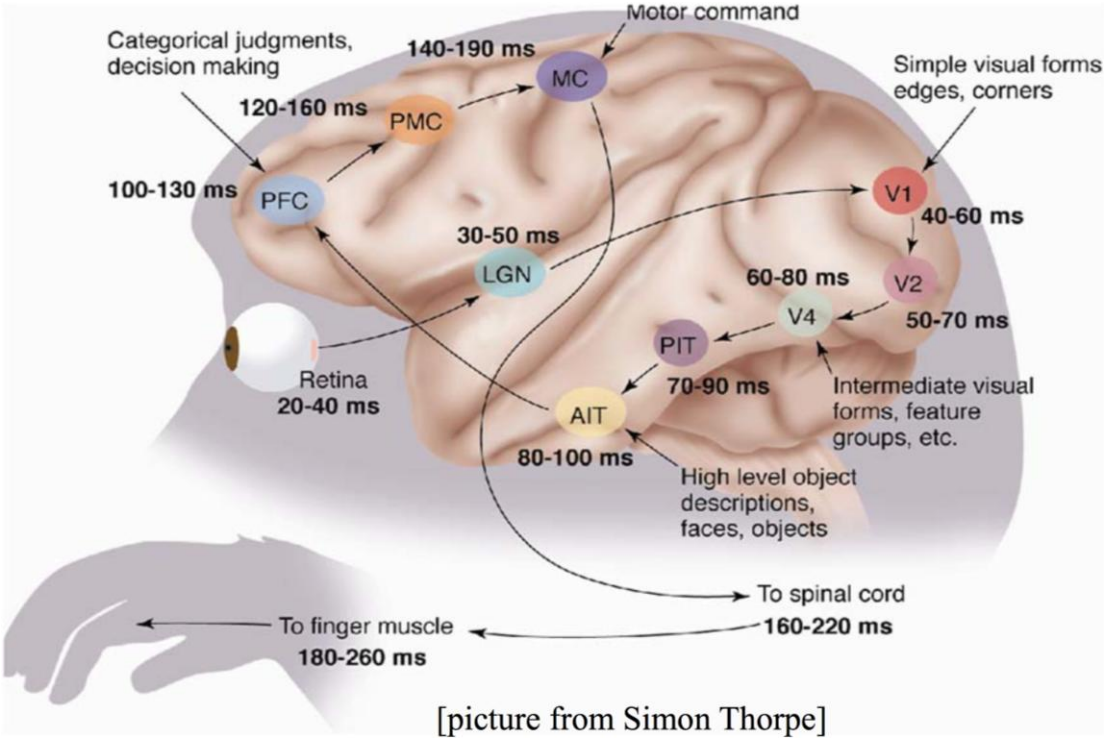
Hypercomplex cells:
response to movement with an end point



No response

Response (end point)

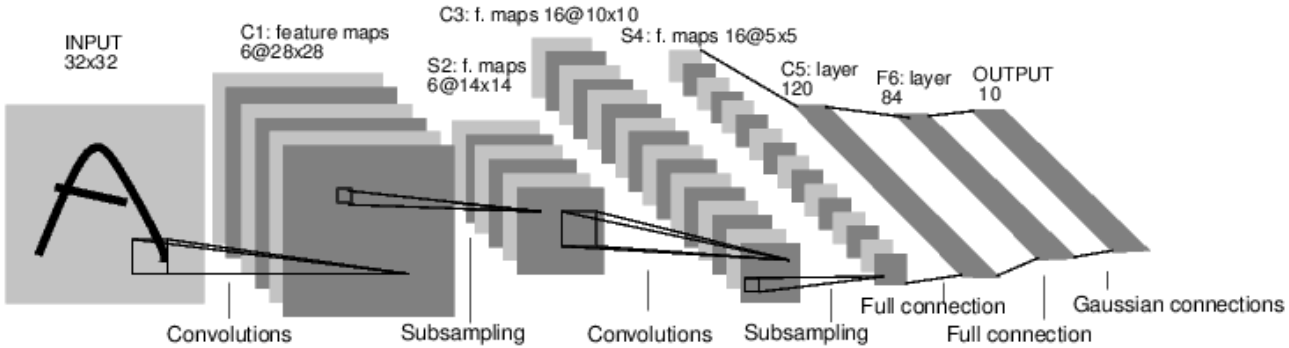
Deep Learning may be Motivated by Human Brain



[picture from Simon Thorpe]

History

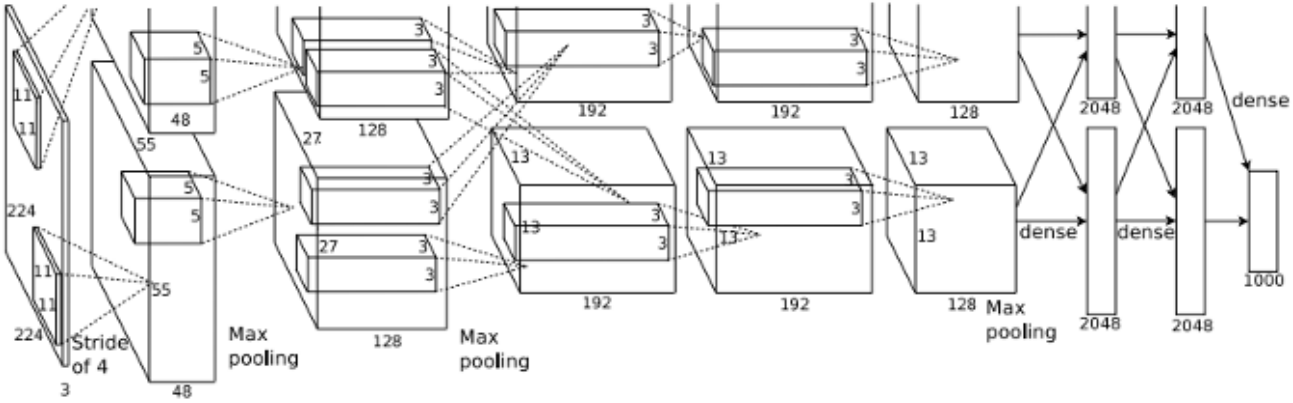
LeNet



Y. LeCun, et. al. Handwritten digit recognition with a back-propagation network. NIPS 1989.

History

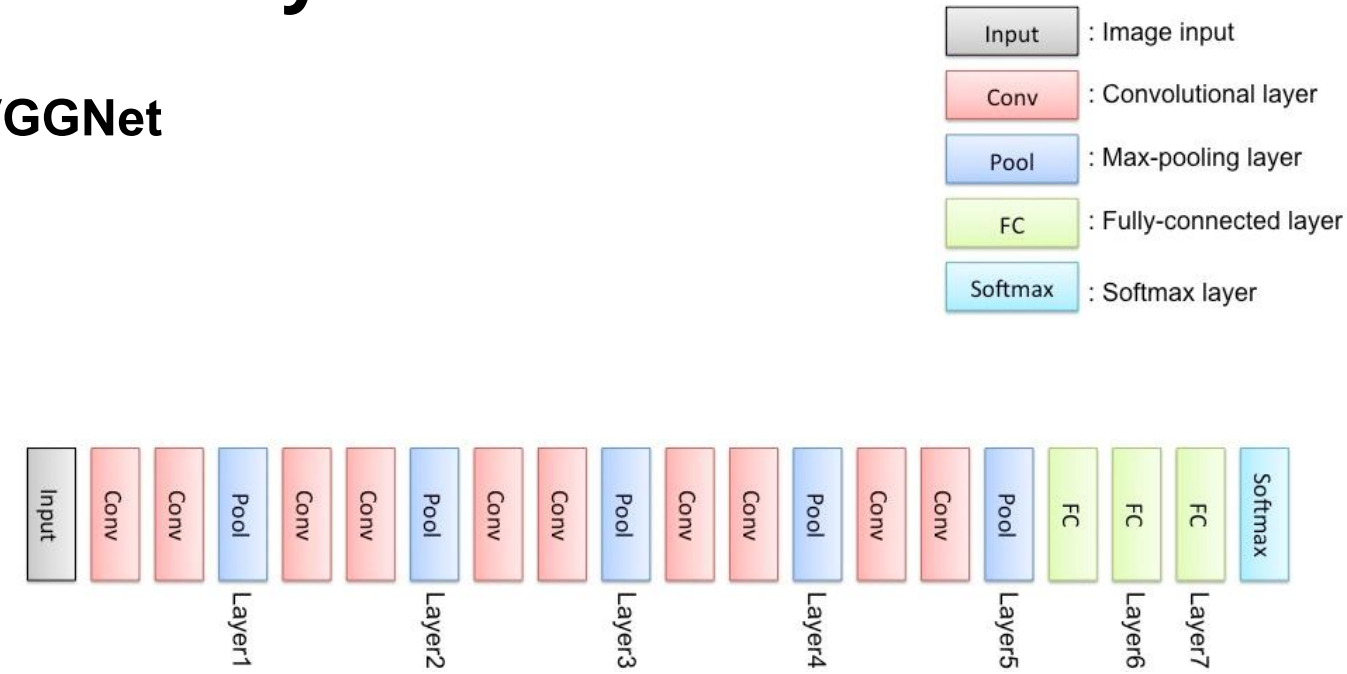
AlexNet



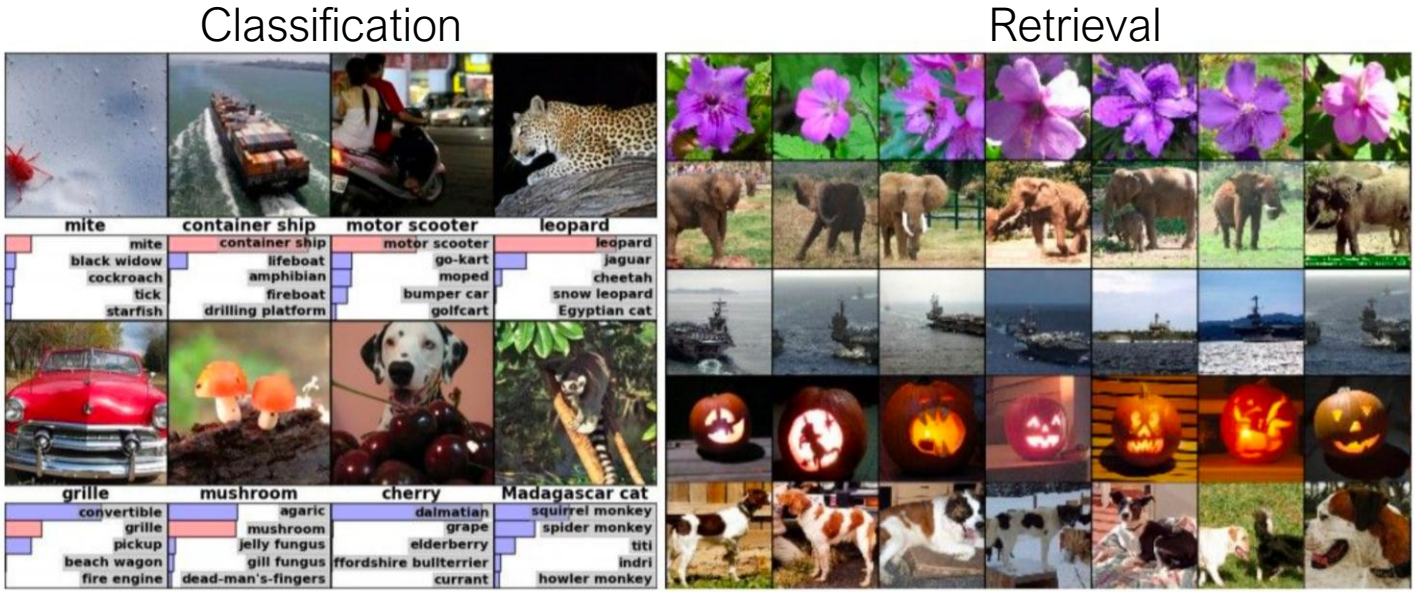
AlexNet is the name of a convolutional neural network (CNN) architecture, designed by Alex Krizhevsky in collaboration with **Ilya Sutskever (OpenAI thing in 2023 ☺)** and Geoffrey Hinton.

History

VGGNet



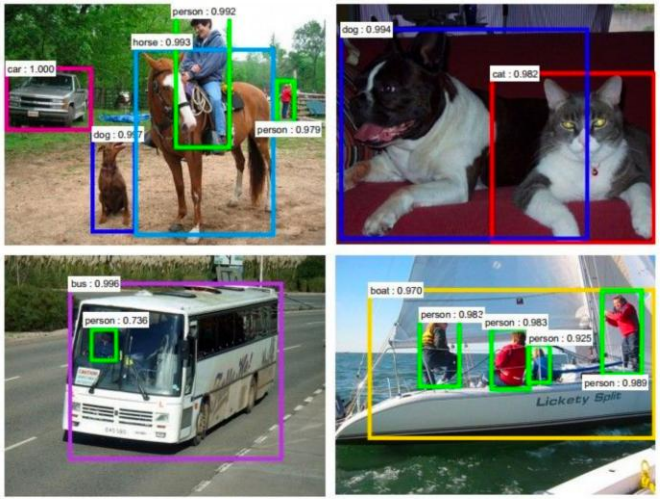
Convolutional NNs



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Convolutional NNs

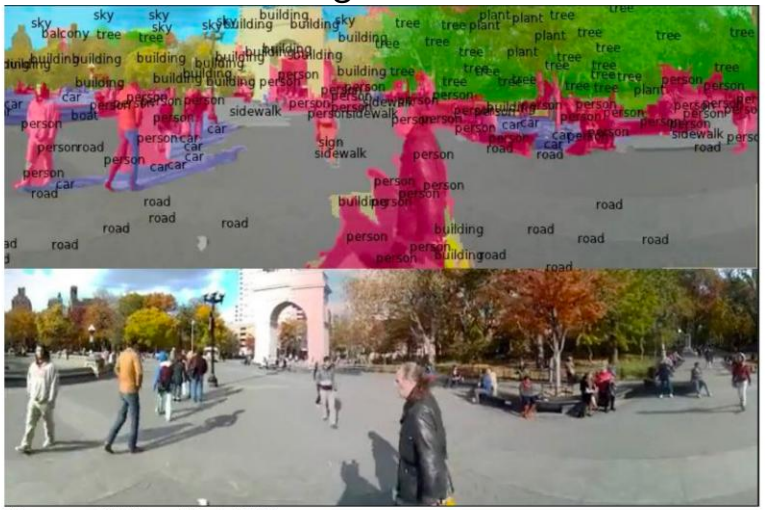
Detection



Figures copyright Shaoqing Ren, Kaiming He, Ross Girschick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

Segmentation

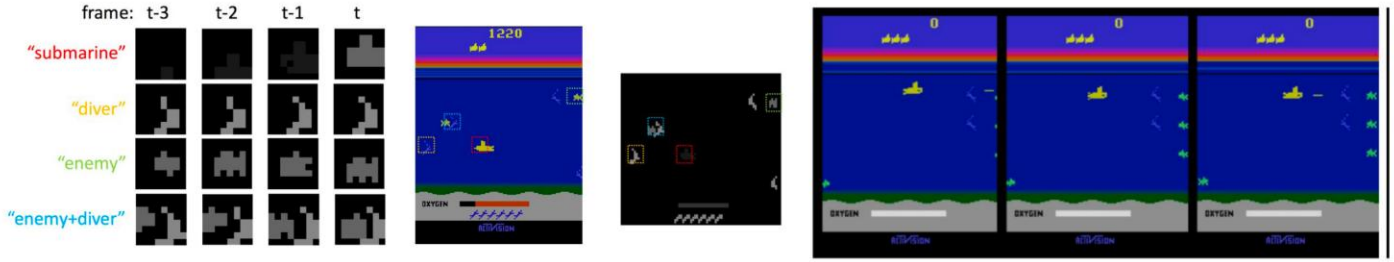


Figures copyright Clement Farabet, 2012. Reproduced with permission.

[Farabet et al., 2012]

Convolutional NNs

Game AI



[Guo et al. 2014]

Figures copyright Xiaoxiao Guo, Satinder Singh, Honglak Lee, Richard Lewis, and Xiaoshi Wang, 2014. Reproduced with permission.

Arts



Original image is CC0 public domain
Starry Night and Tree Roots by Van Gogh are in the public domain
Bokeh image is in the public domain
Stylized images copyright Justin Johnson, 2017;

Gatys et al, "Image Style Transfer using Convolutional Neural Networks", CVPR 2016
Gatys et al, "Controlling Perceptual Factors in Neural Style Transfer", CVPR 2017

Slide credit: CS 231N

Convolutional NNs

Image Captioning

No errors



A white teddy bear sitting in the grass

Minor errors



A man in a baseball uniform throwing a ball

Somewhat related



A woman is holding a cat in her hand

[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]



A man riding a wave on top of a surfboard



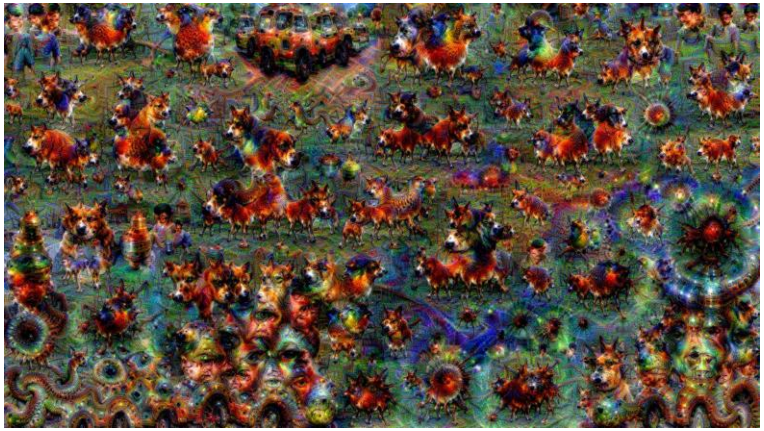
A cat sitting on a suitcase on the floor

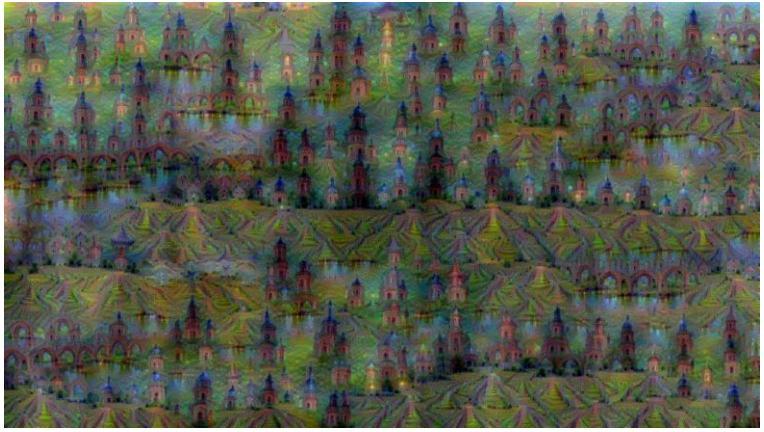


A woman standing on a beach holding a surfboard

All images are CC0 Public domain:
https://pixabay.com/en/luggage-antique-cat-1643010/
https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/
https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/
https://pixabay.com/en/woman-female-model-portrait-adult-983967/
https://pixabay.com/en/handstand-lake-meditation-496008/
https://pixabay.com/en/baseball-player-shortstop-infield-1045263/

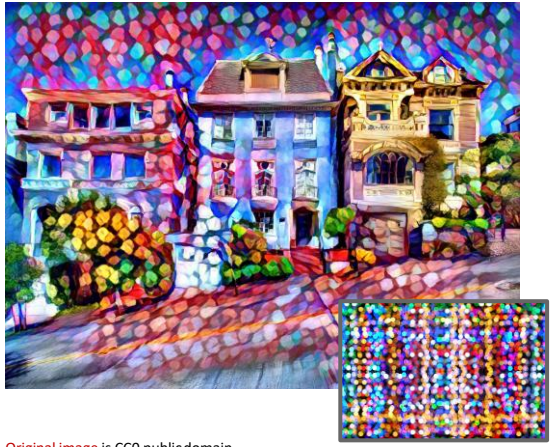
Slide credit: CS 231N





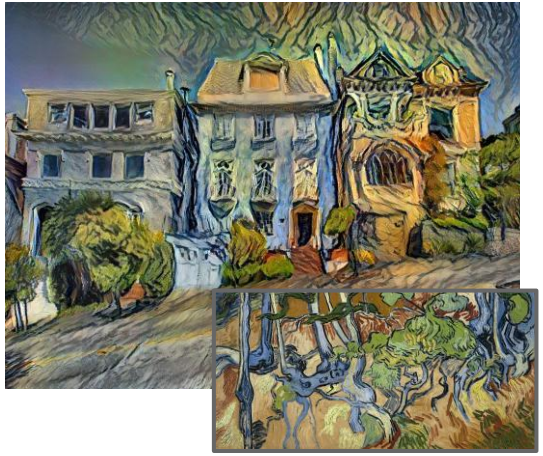
Figures copyright Justin Johnson, 2015. Reproduced with permission. Generated using the Inceptionism approach from a blog post by Google Research.





Original image is CC0 public domain
Starry Night and Tree Roots by Van Gogh are in the public domain
Bokeh image is in the public domain
Stylized images copyright Justin Johnson, 2017; reproduced with permission

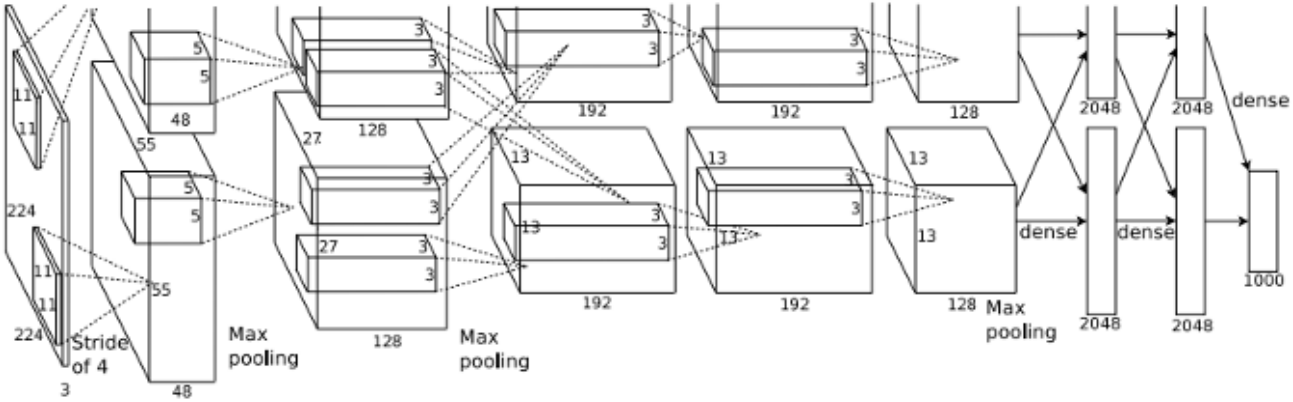




Gatys et al, "Image Style Transfer using Convolutional Neural Networks", CVPR 2016
Gatys et al, "Controlling Perceptual Factors in Neural Style Ti

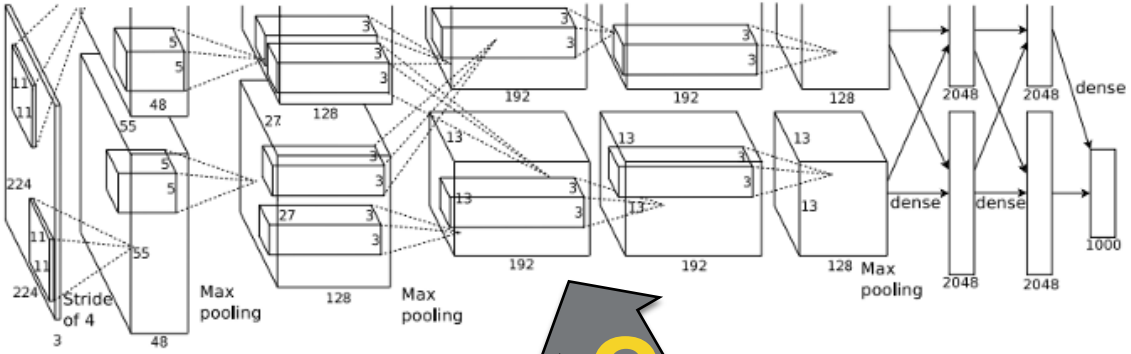
Slide credit: CS 231N

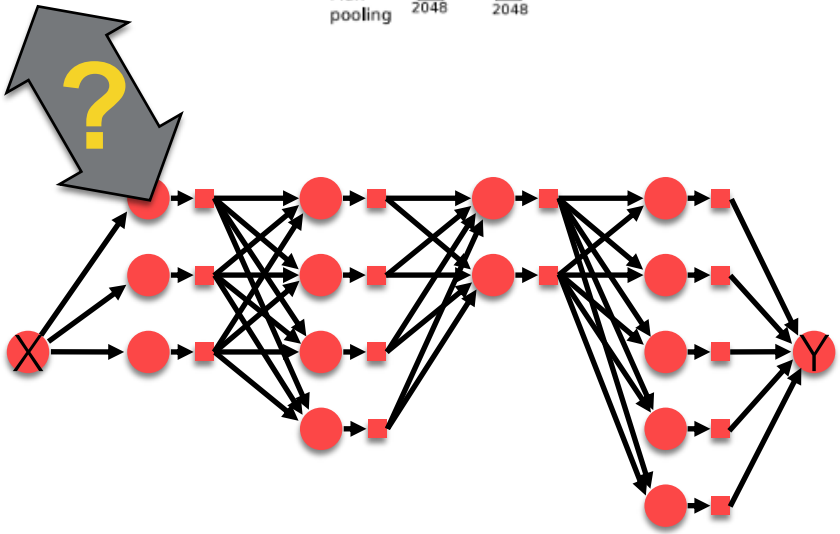
AlexNet in details



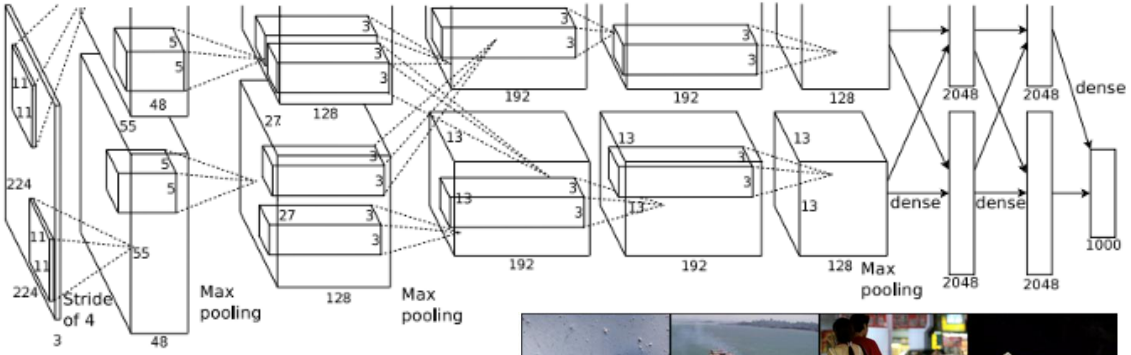
A Krizhevsky, et. al. ImageNet Classification with Deep Convolutional Neural Networks. NIPS 2012.

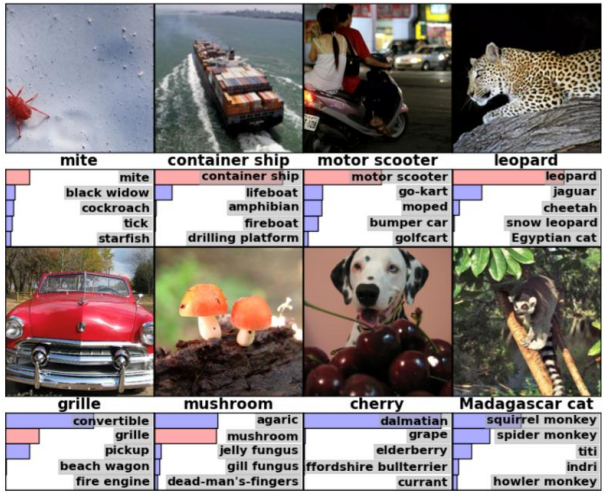
AlexNet in details





AlexNet in details

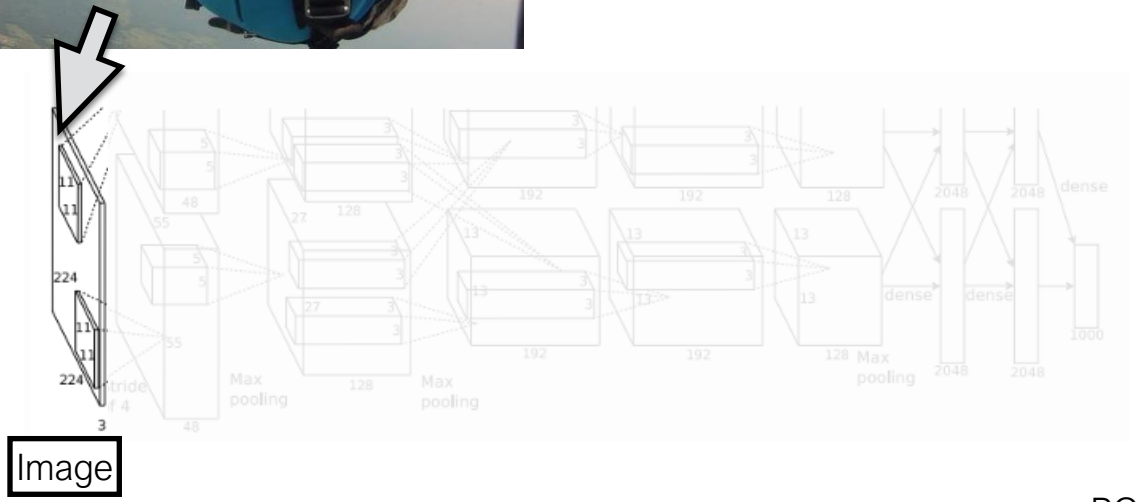




USC Viterbi School of Engineering

AlexNet in details



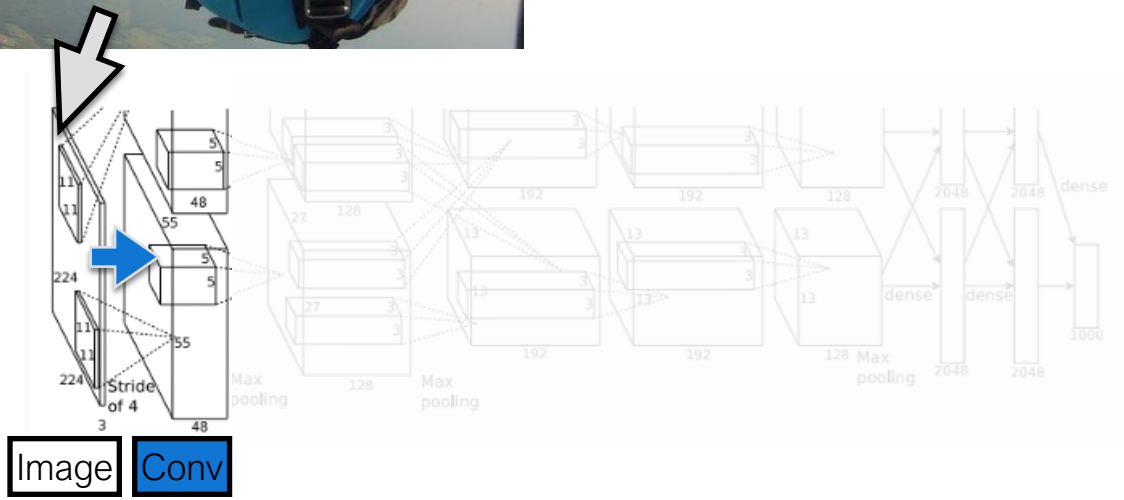


Image

PC: Daniel Yang

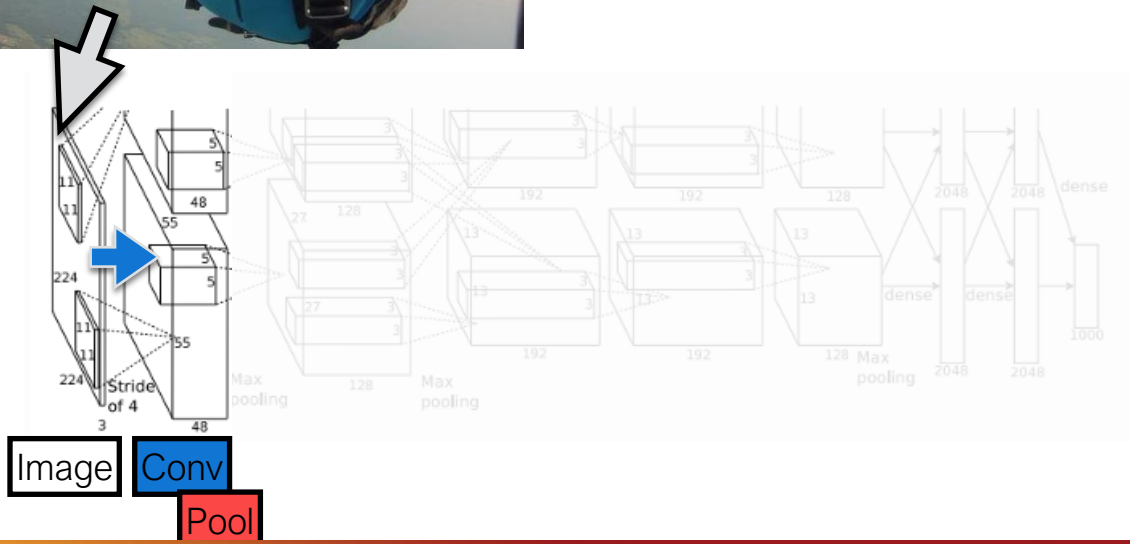
AlexNet in details





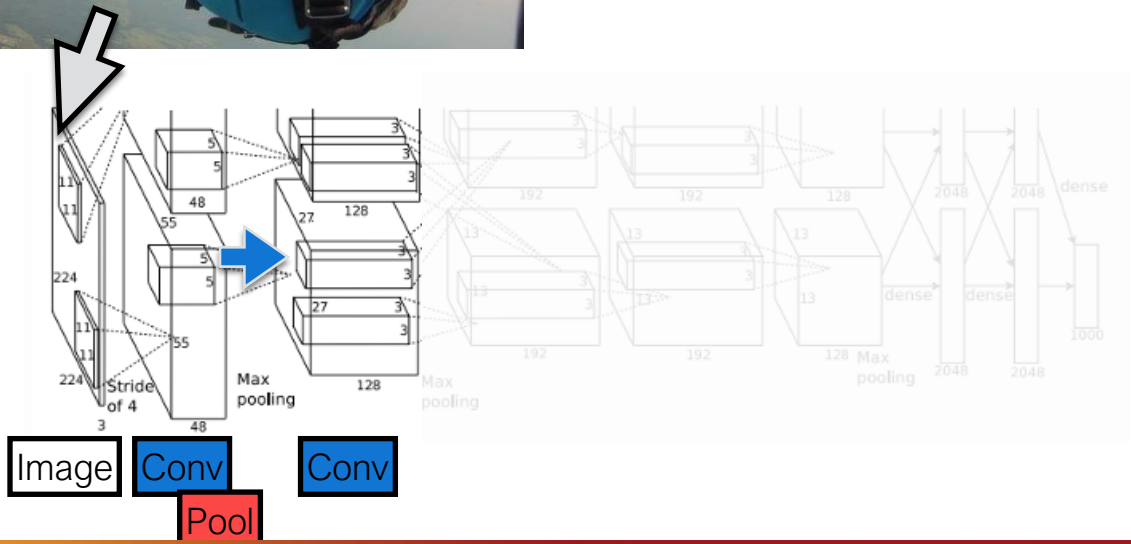
AlexNet in details





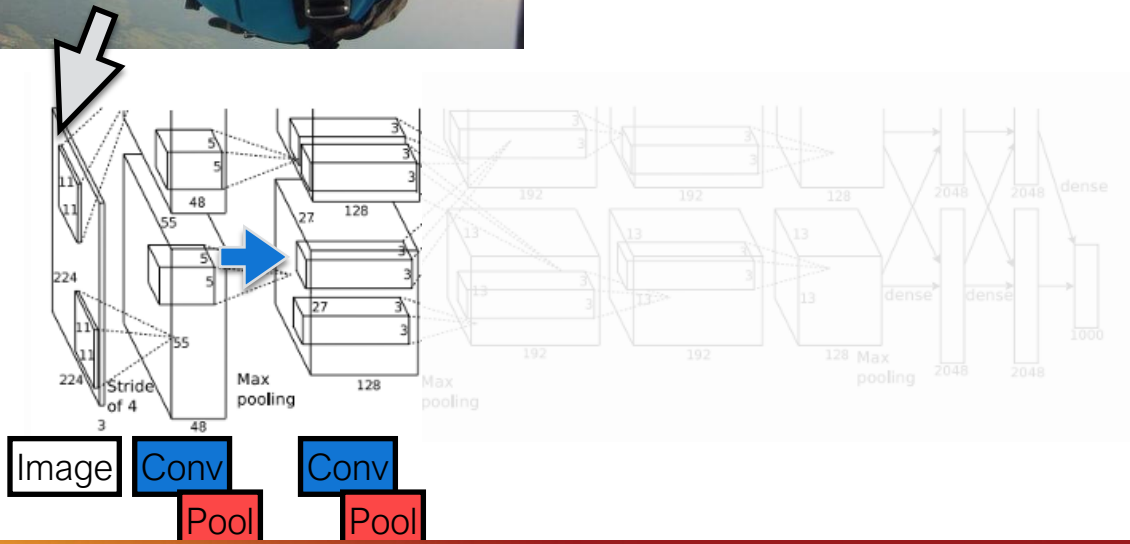
AlexNet in details





AlexNet in details





AlexNet in details



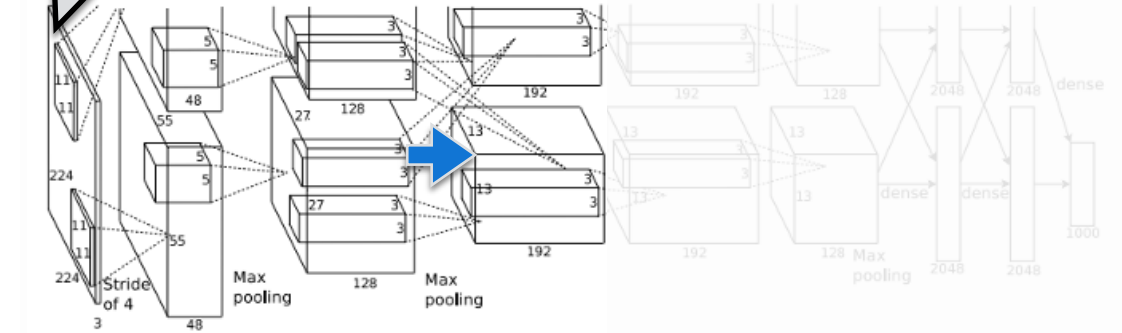
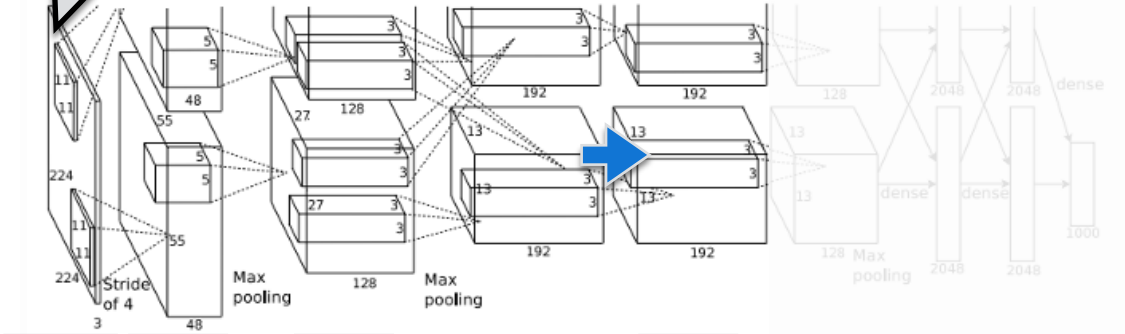


Image | Conv | Pool | Conv | Pool | Conv

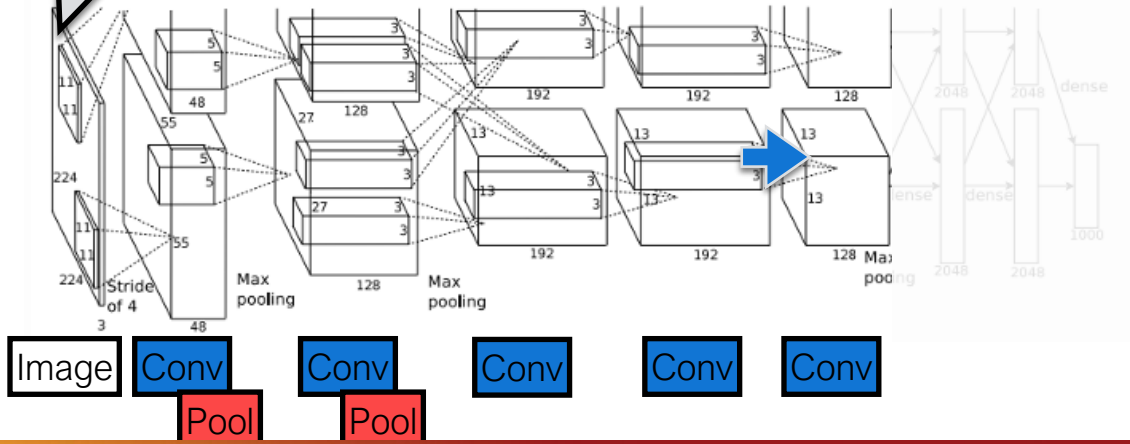
AlexNet in details





AlexNet in details





AlexNet in details



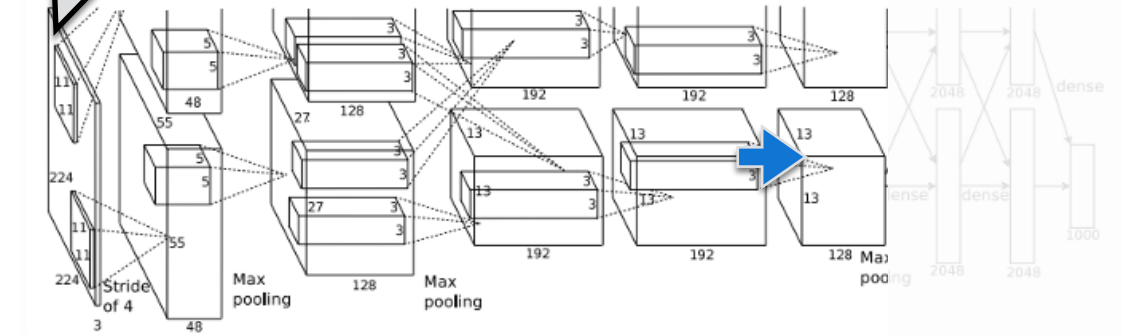
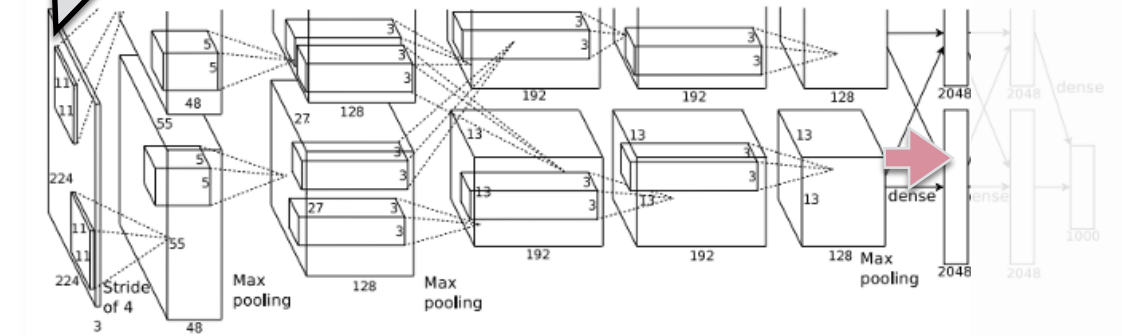


Image | Conv + Pool | Conv + Pool | Conv | Conv | Conv + Pool

USC Viterbi School of Engineering

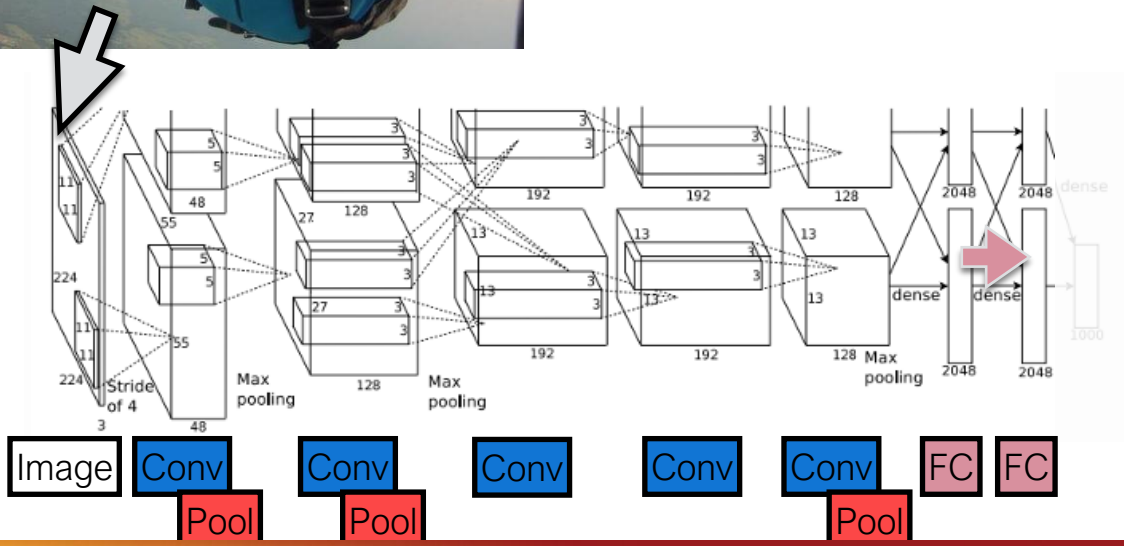
AlexNet in details





AlexNet in details

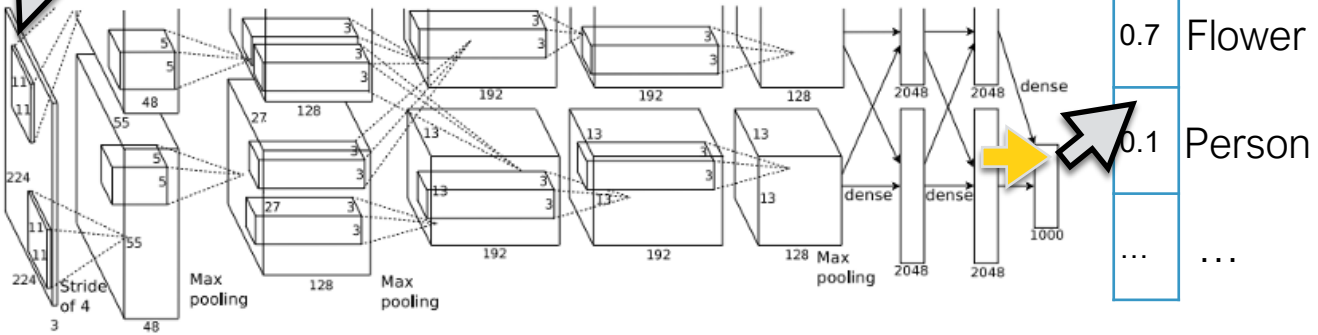




USC Viterbi School of Engineering

AlexNet in details

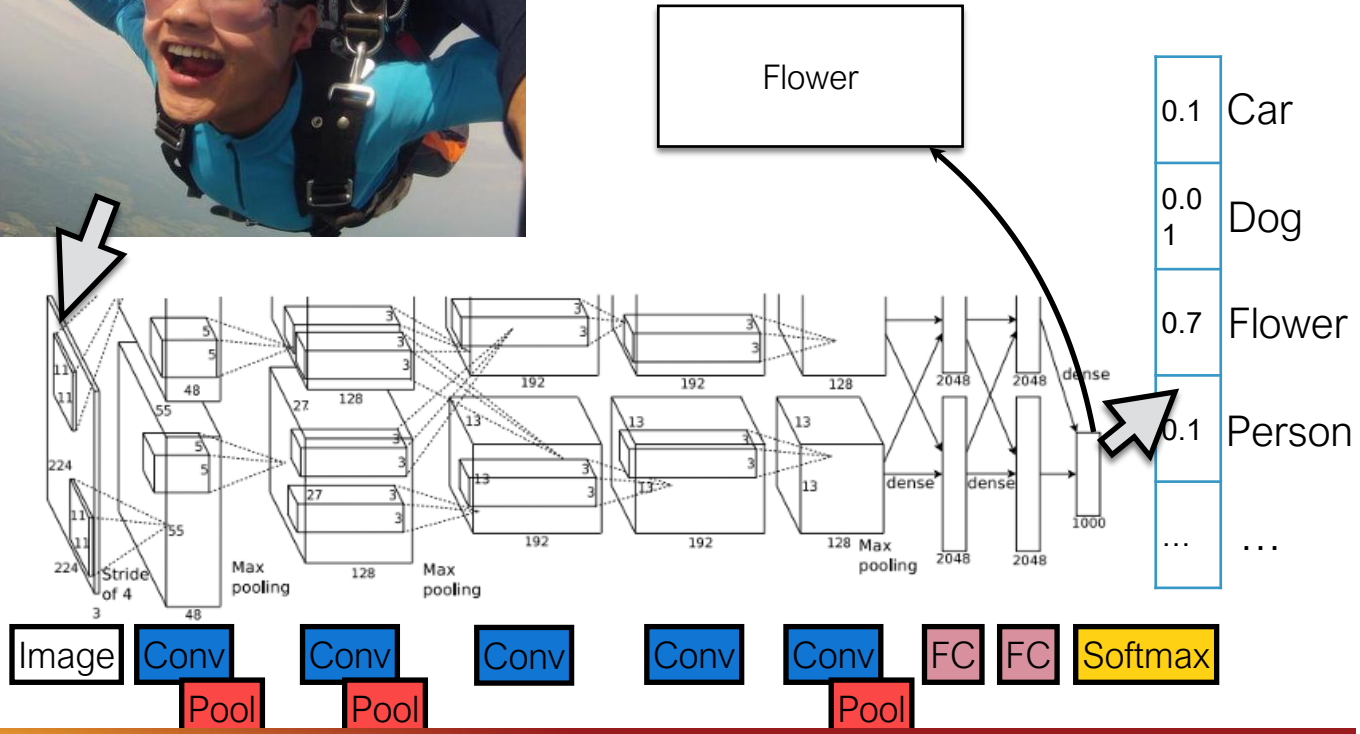




USC Viterbi School of Engineering

AlexNet in details





USC Viterbi School of Engineering

Convolutional Neural Networks (CNN)

Layers

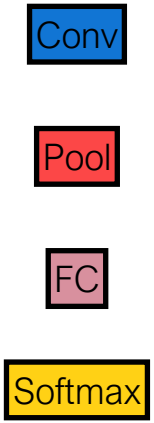
What are these layers?

Convolutional layer

Pool layer

Fully connected layer

Softmax layer



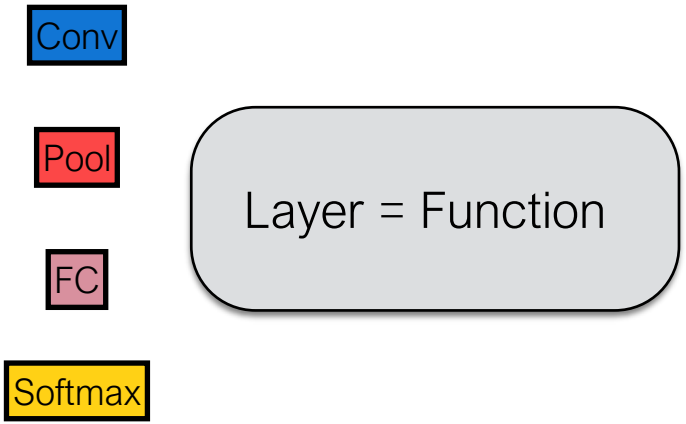
What are these layers?

Convolutional layer

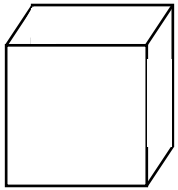
Pool layer

Fully connected layer

Softmax layer

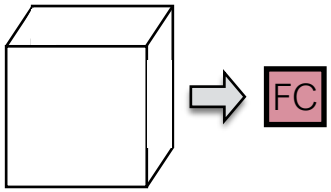


Fully connected layer (FC)



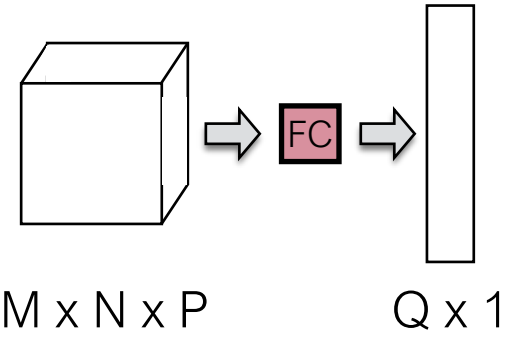
M x N x P

Fully connected layer (FC)

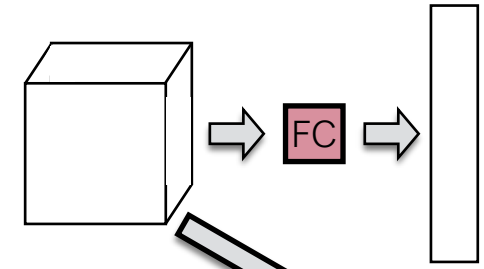


M x N x P

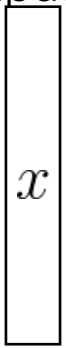
Fully connected layer (FC)



Fully connected layer (FC)

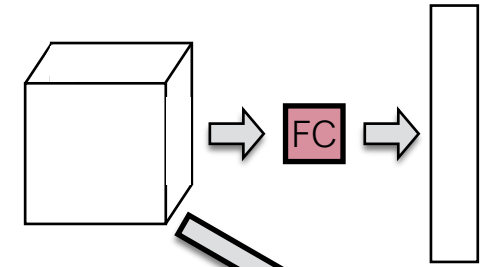


FC operation takes an input, stretches it.

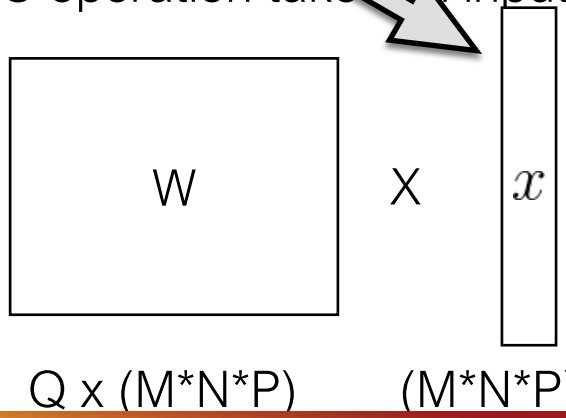


(M*N*P) x 1

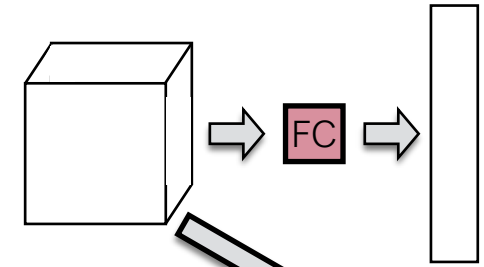
Fully connected layer (FC)



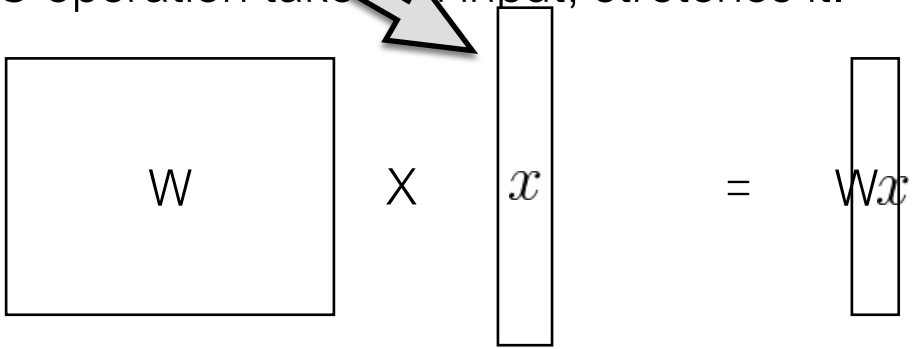
FC operation takes an input, stretches it.



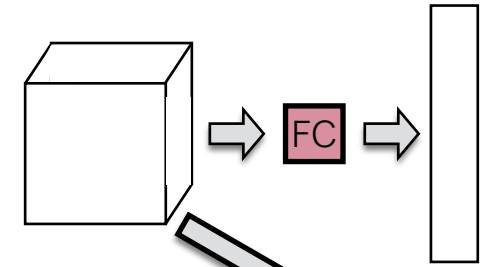
Fully connected layer (FC)



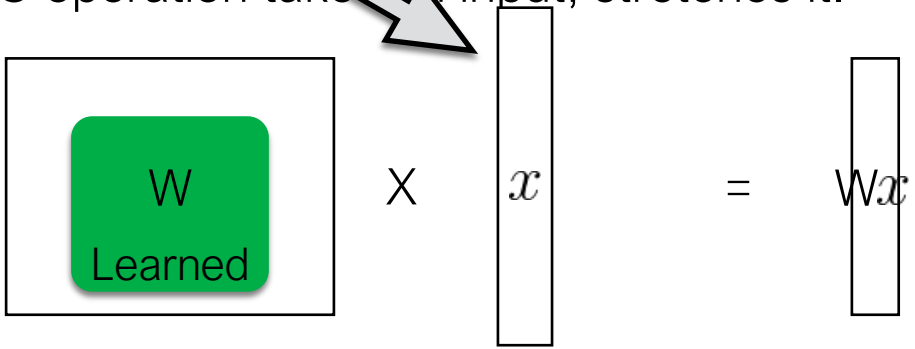
FC operation takes an input, stretches it.



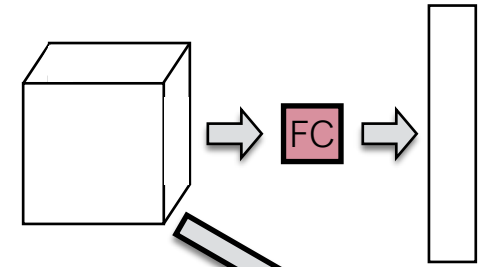
Fully connected layer (FC)



FC operation takes an input, stretches it.

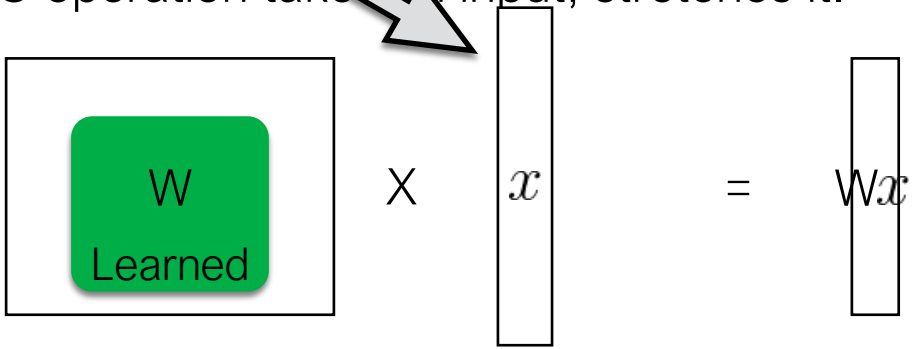


Fully connected layer (FC)

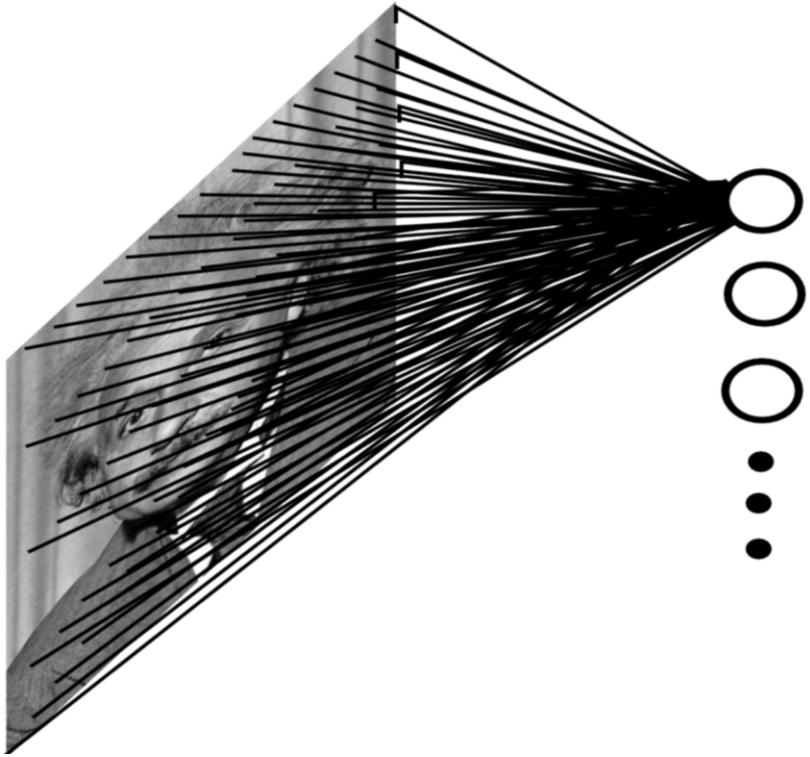


Why is this called FC?

FC operation takes an input, stretches it.



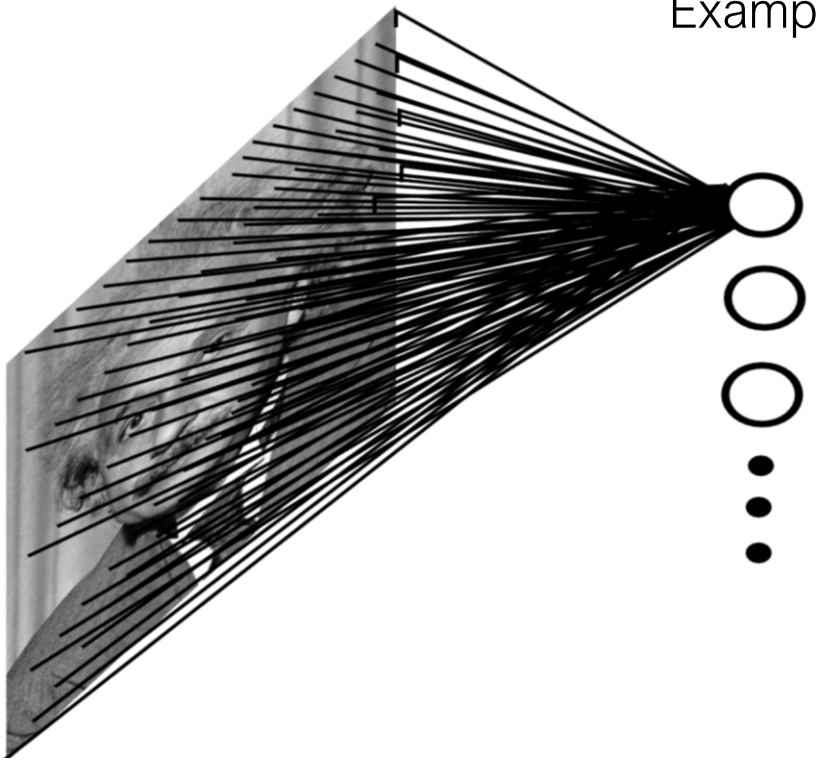
Fully connected layer (FC)



Slide credit: Marc'Aurelio Ranzato

Fully connected layer (FC)

Example: 200x200 image



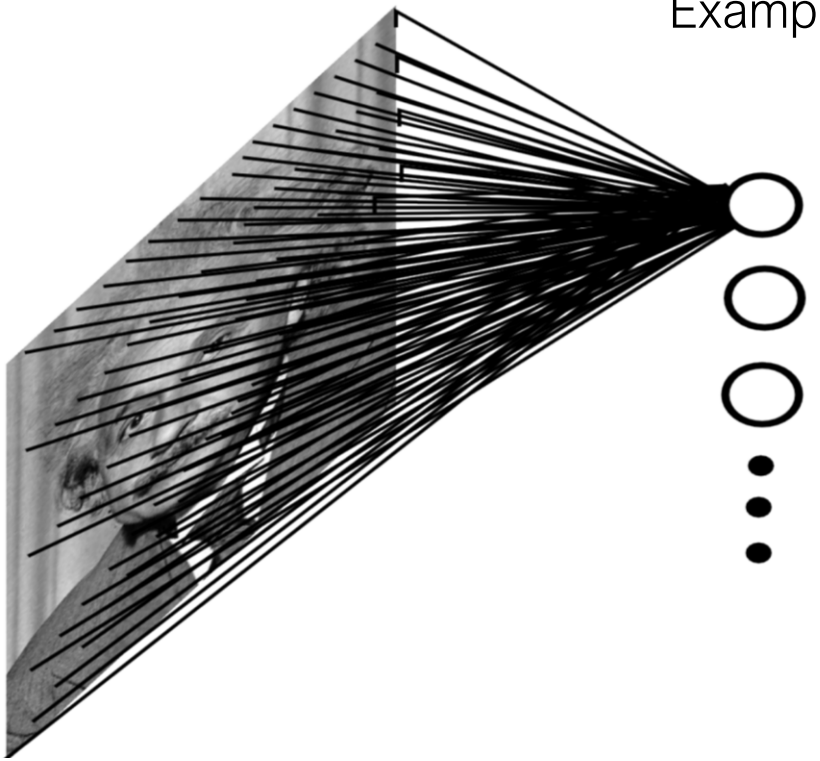
Slide credit: Marc'Aurelio Ranzato

USC Viterbi School of Engineering

Fully connected layer (FC)

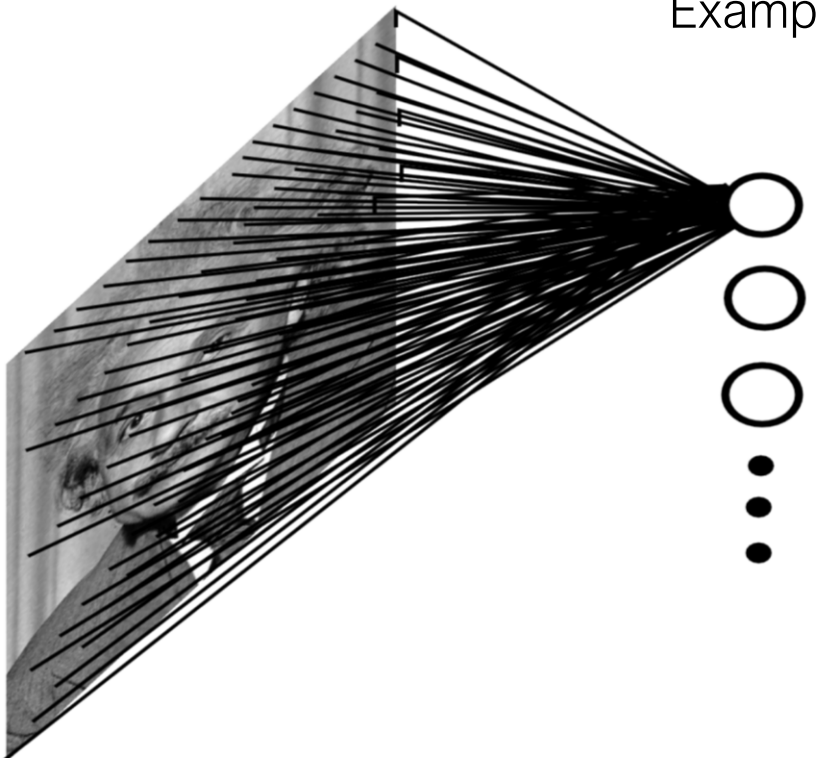
Example: 200x200 image

40k hidden units



Slide credit: Marc'Aurelio Ranzato

Fully connected layer (FC)



Example: 200x200 image

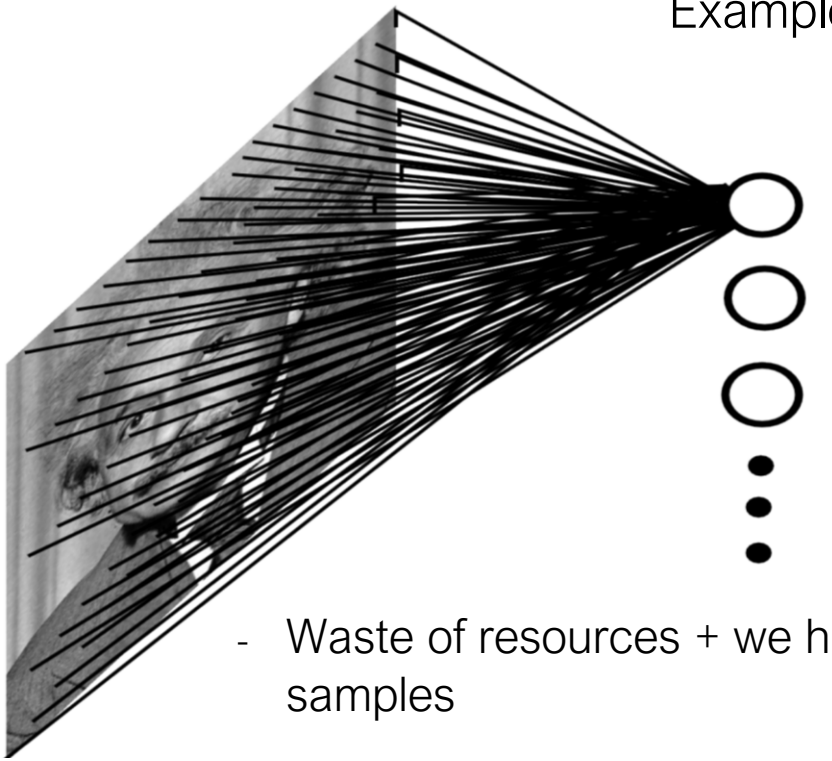
40k hidden units

~2B parameters!!!

Why?

Slide credit: Marc'Aurelio Ranzato

Fully connected layer (FC)



Example: 200x200 image

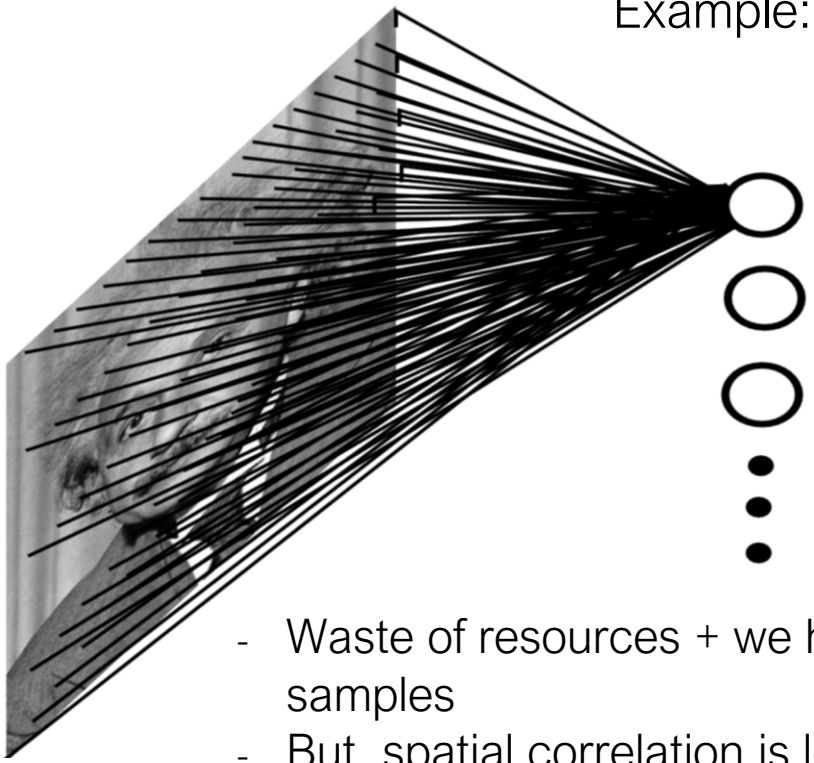
40k hidden units

~2B parameters!!!

- Waste of resources + we have not enough training samples

Slide credit: Marc'Aurelio Ranzato

Fully connected layer (FC)



Example: 200x200 pixel image

40k hidden units

~2B parameters!!!

- Waste of resources + we have not enough training samples
- But, spatial correlation is local!

Slide credit: Marc'Aurelio Ranzato

Recap: Neural Networks

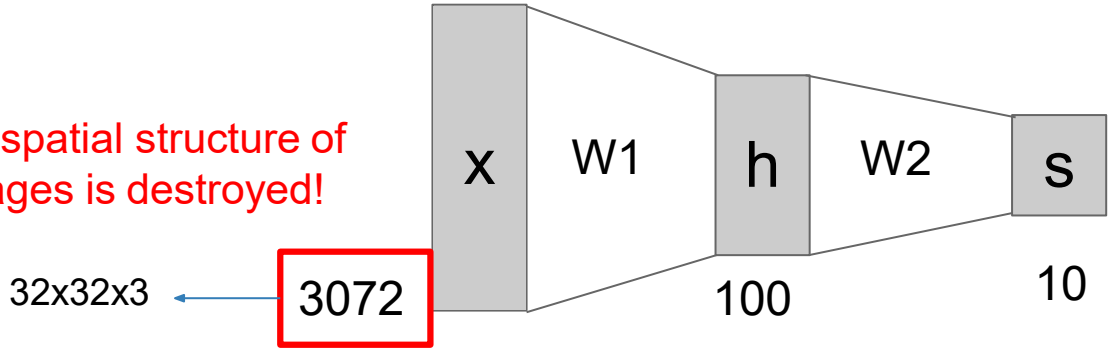
Linear score function:

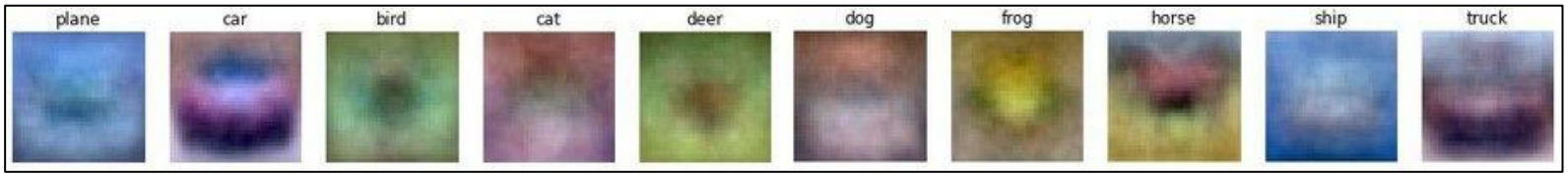
$$f = Wx$$

2-layer Neural Network

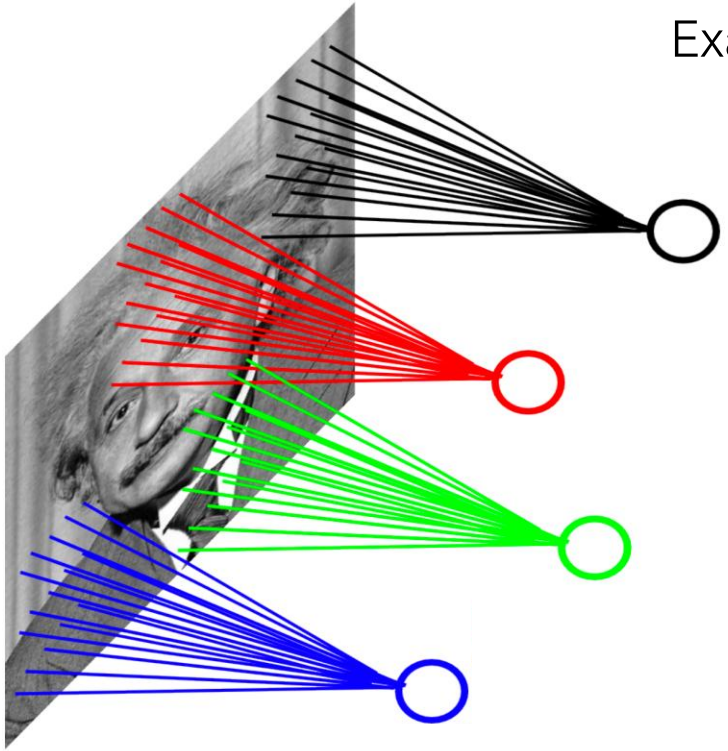
$$f = W_2 \max(0, W_1 x)$$

The spatial structure of images is destroyed!





Locally connected layer



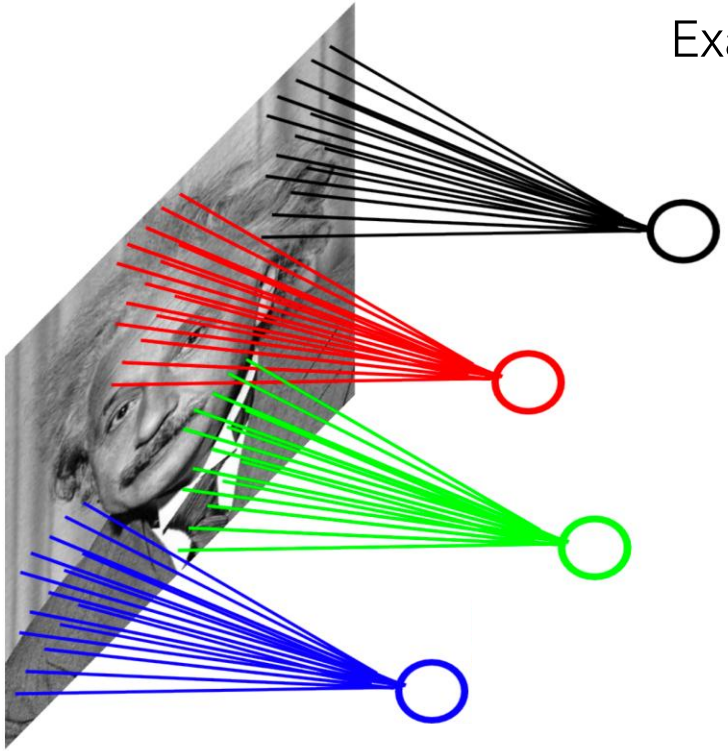
Example: 200x200 image

40k hidden units

Filter size: 10x10

Slide credit: Marc'Aurelio Ranzato

Locally connected layer



Example: 200x200 image

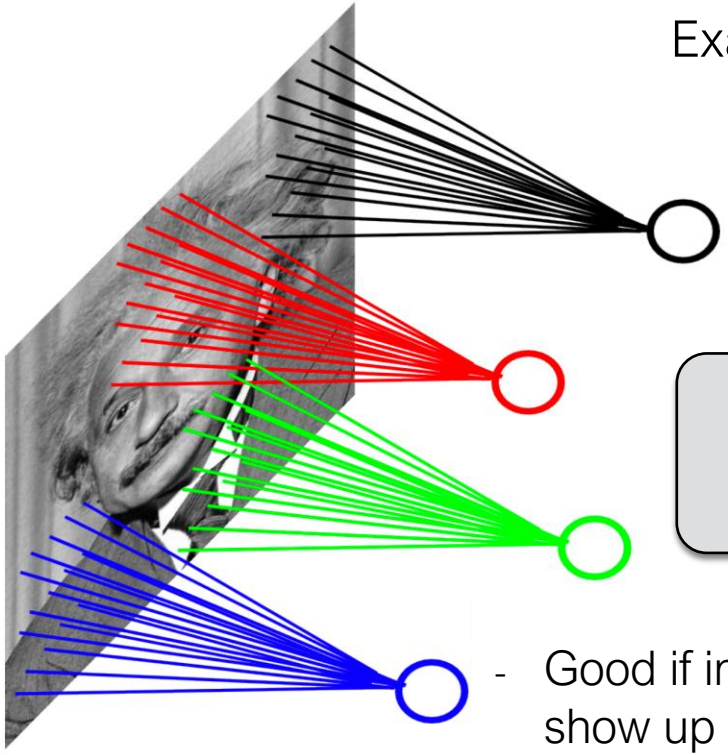
40k hidden units

Filter size: 10x10

4M parameters

Slide credit: Marc'Aurelio Ranzato

Locally connected layer



Example: 200x200 image

40k hidden units

Filter size: 10x10 pixels

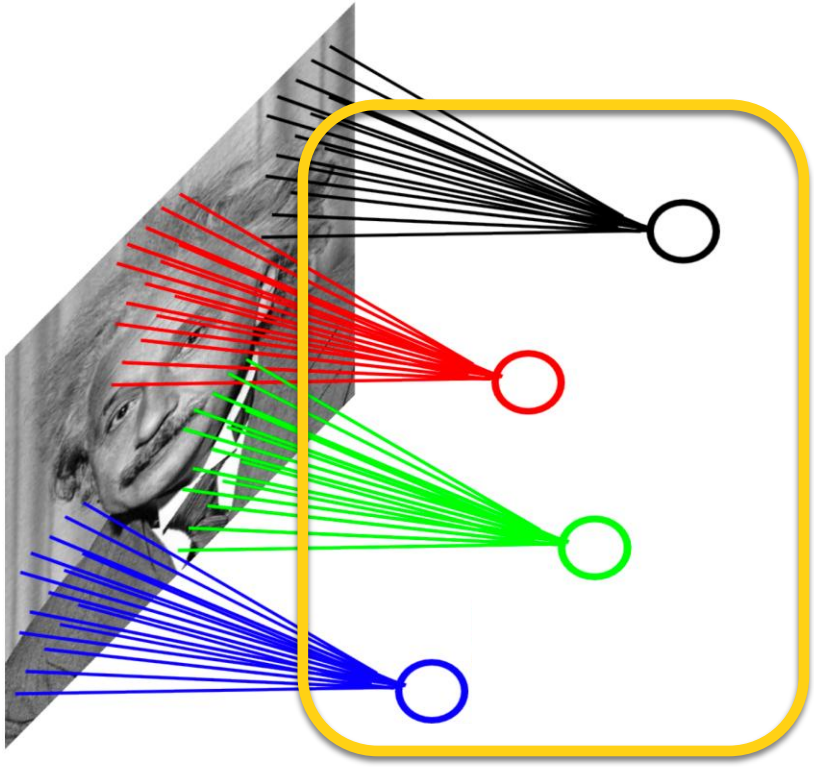
4M parameters (40k x 100)

What can we do?

- Good if images are registered (i.e. parts show up at the same locations)

Slide credit: Marc'Aurelio Ranzato

Locally connected layer

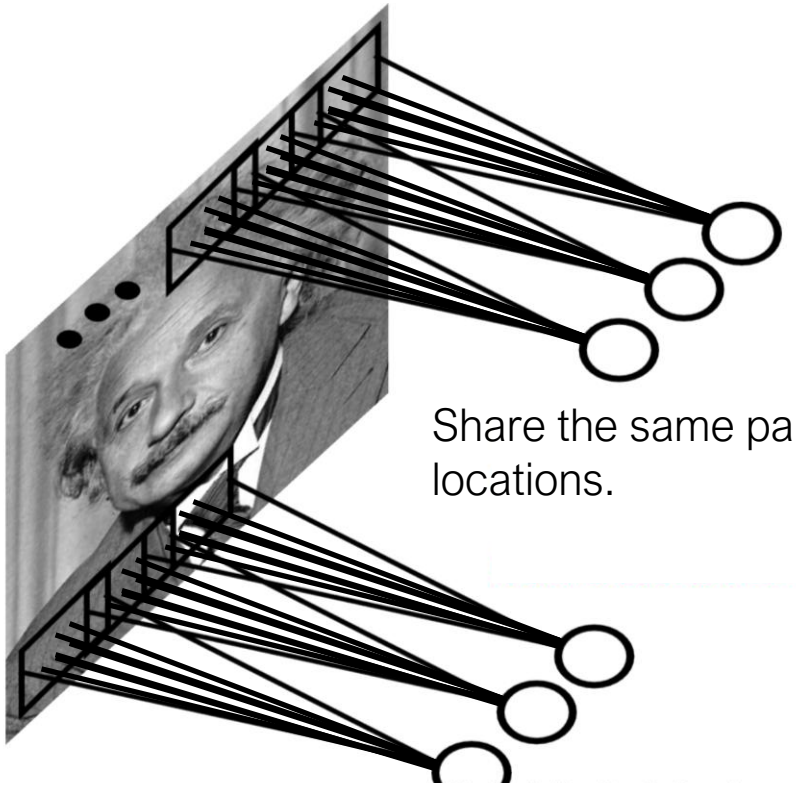


Share all local weights

Slide credit: Marc'Aurelio Ranzato

USC Viterbi School of Engineering

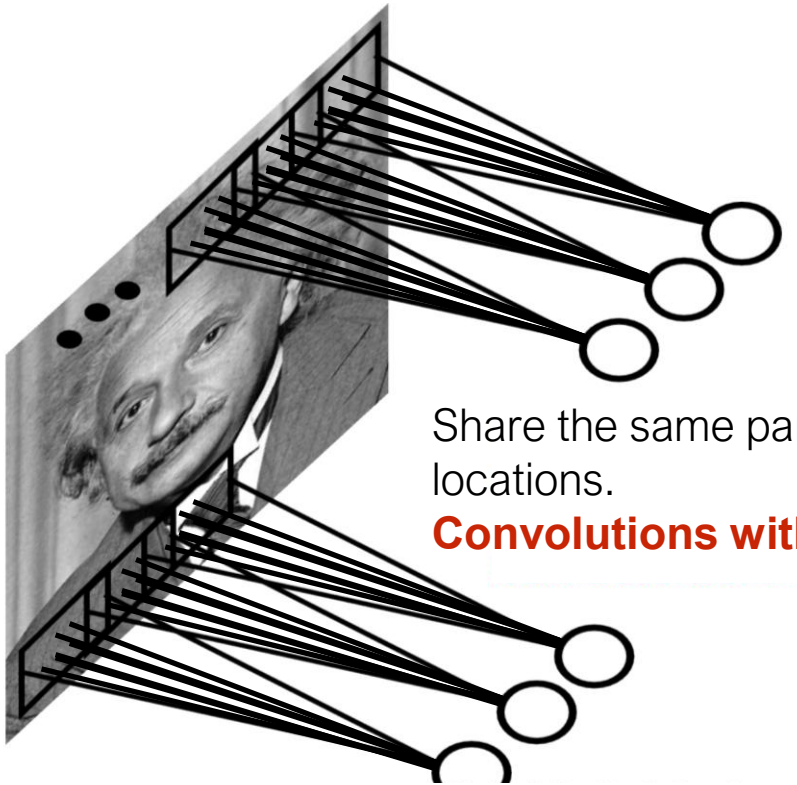
Convolutional layer



Share the same parameters across different locations.

Slide credit: Marc'Aurelio Ranzato

Convolutional layer



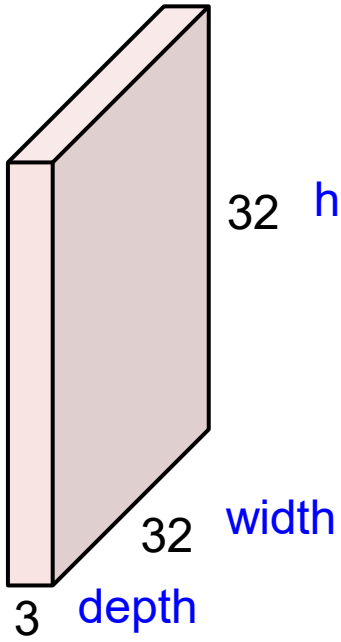
Share the same parameters across different locations.

Convolutions with learned kernels (weights)

Slide credit: Marc'Aurelio Ranzato

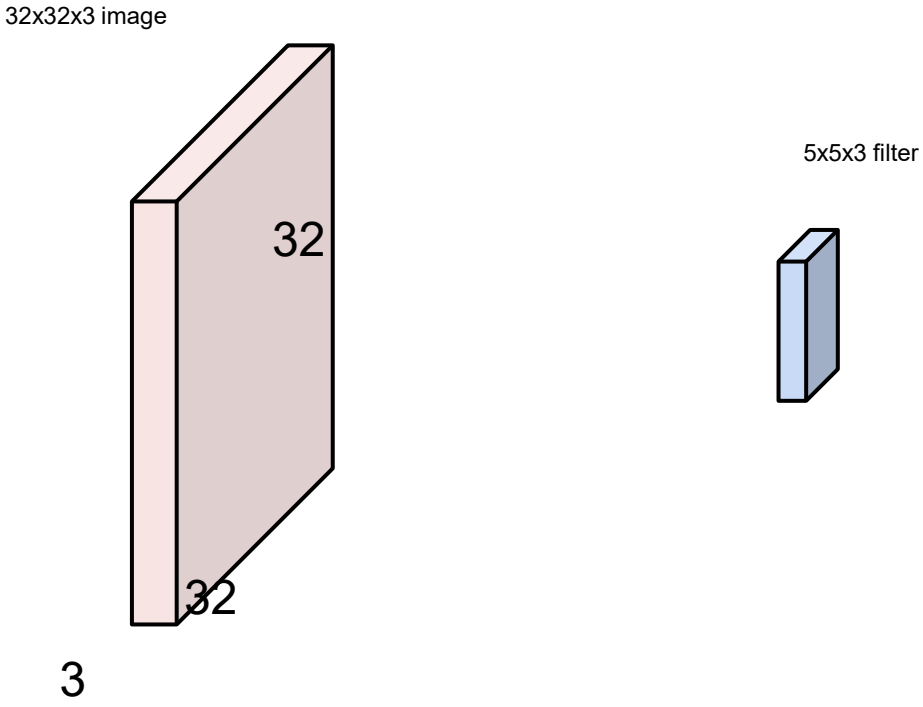
Convolutional layer

32x32x3 image -> preserve spatial structure



Slide credit: CS 231N

Convolutional layer



Convolve the filter with the image

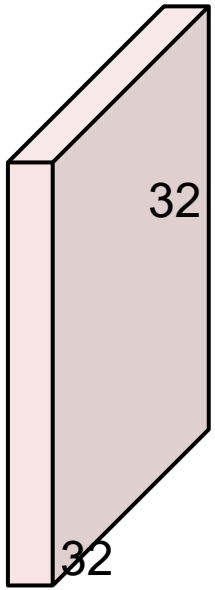
i.e. “slide over the image spatially, computing dot products”

Slide credit: CS 231N

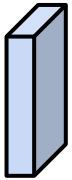
Convolutional layer

Standard filters extend the full depth of the input volume

32x32x3 image



5x5x3 filter

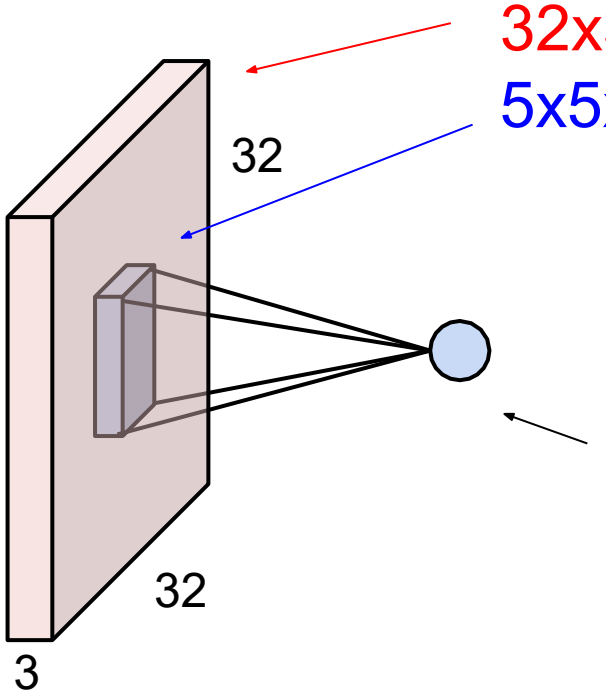


Convolve the filter with the image

i.e. “slide over the image spatially, computing dot products”

Slide credit: CS 231N

Convolutional layer

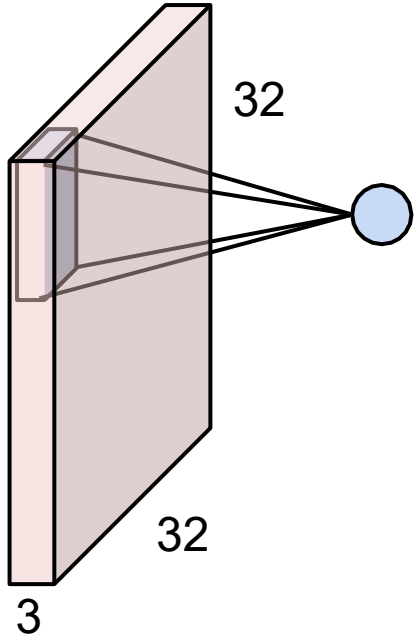


1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

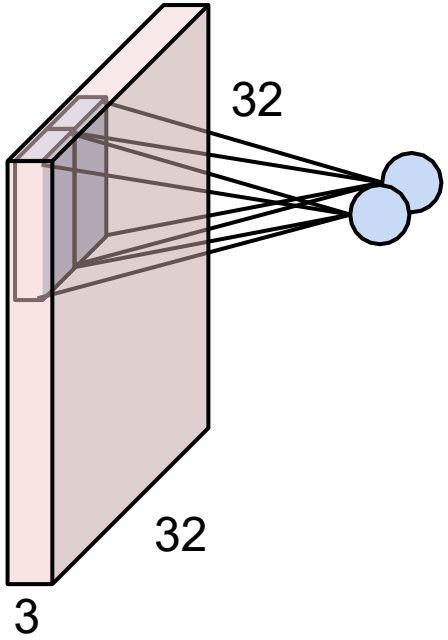
Slide credit: CS 231N

Convolutional layer



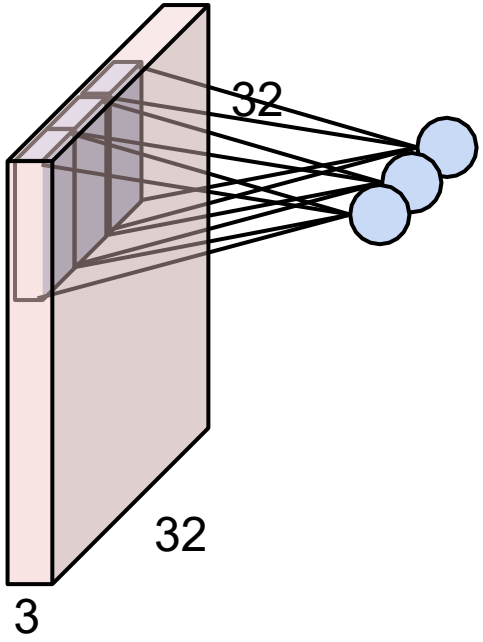
Slide credit: CS 231N

Convolutional layer



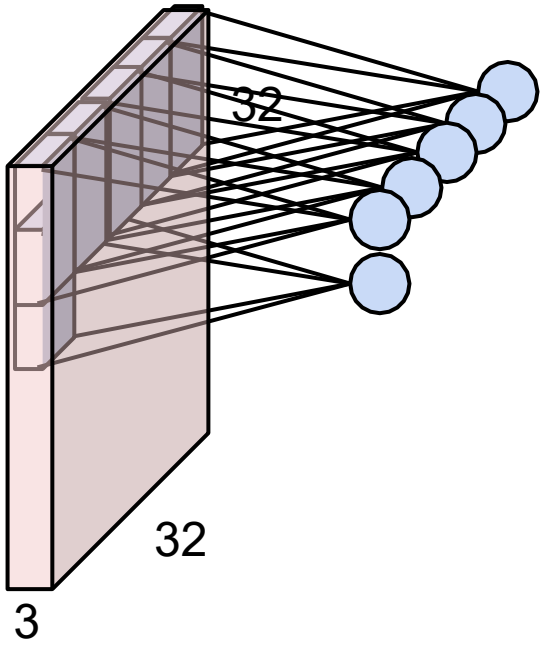
Slide credit: CS 231N

Convolutional layer



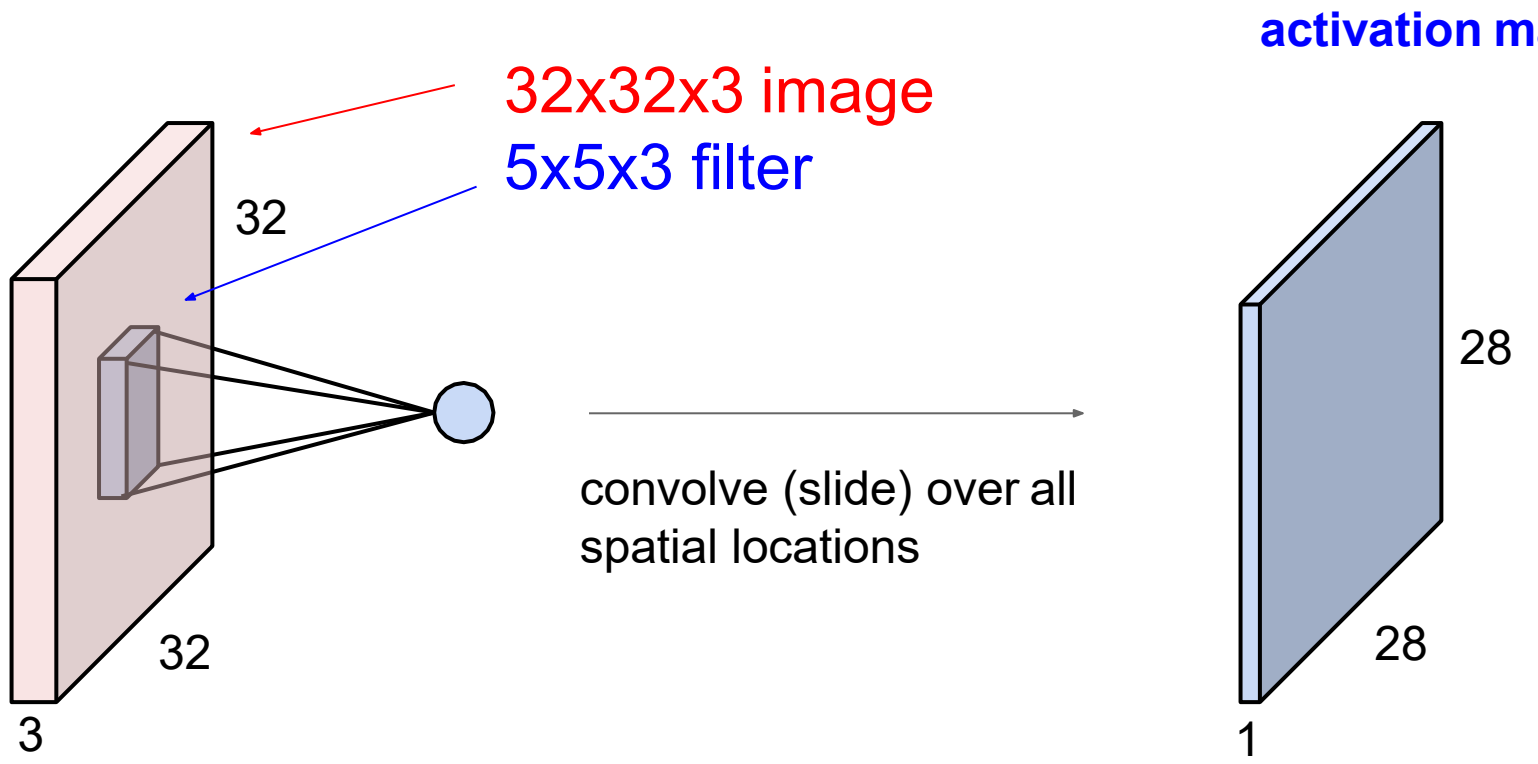
Slide credit: CS 231N

Convolutional layer



Slide credit: CS 231N

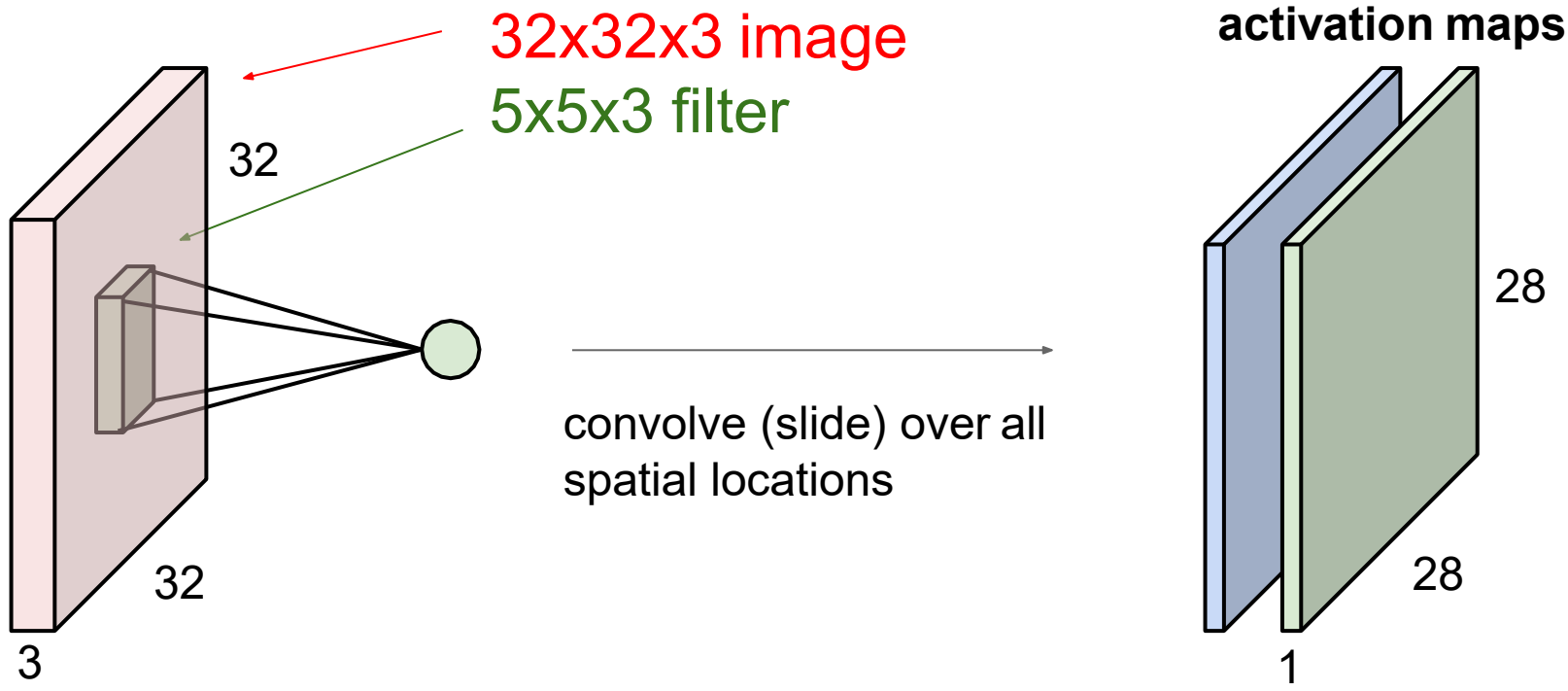
Convolutional layer



Slide credit: CS 231N

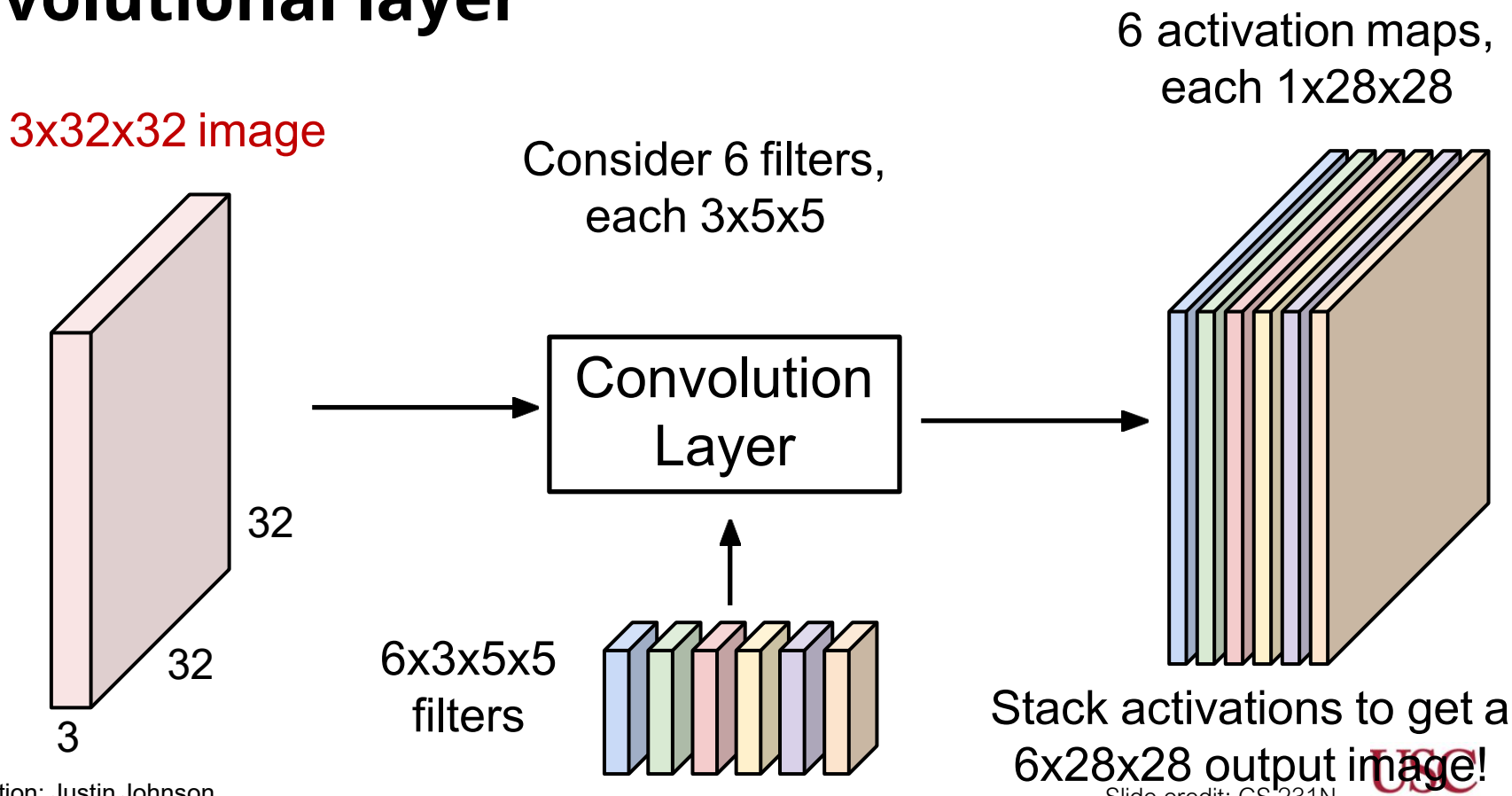
Convolutional layer

consider a second, green filter



Slide credit: CS 231N

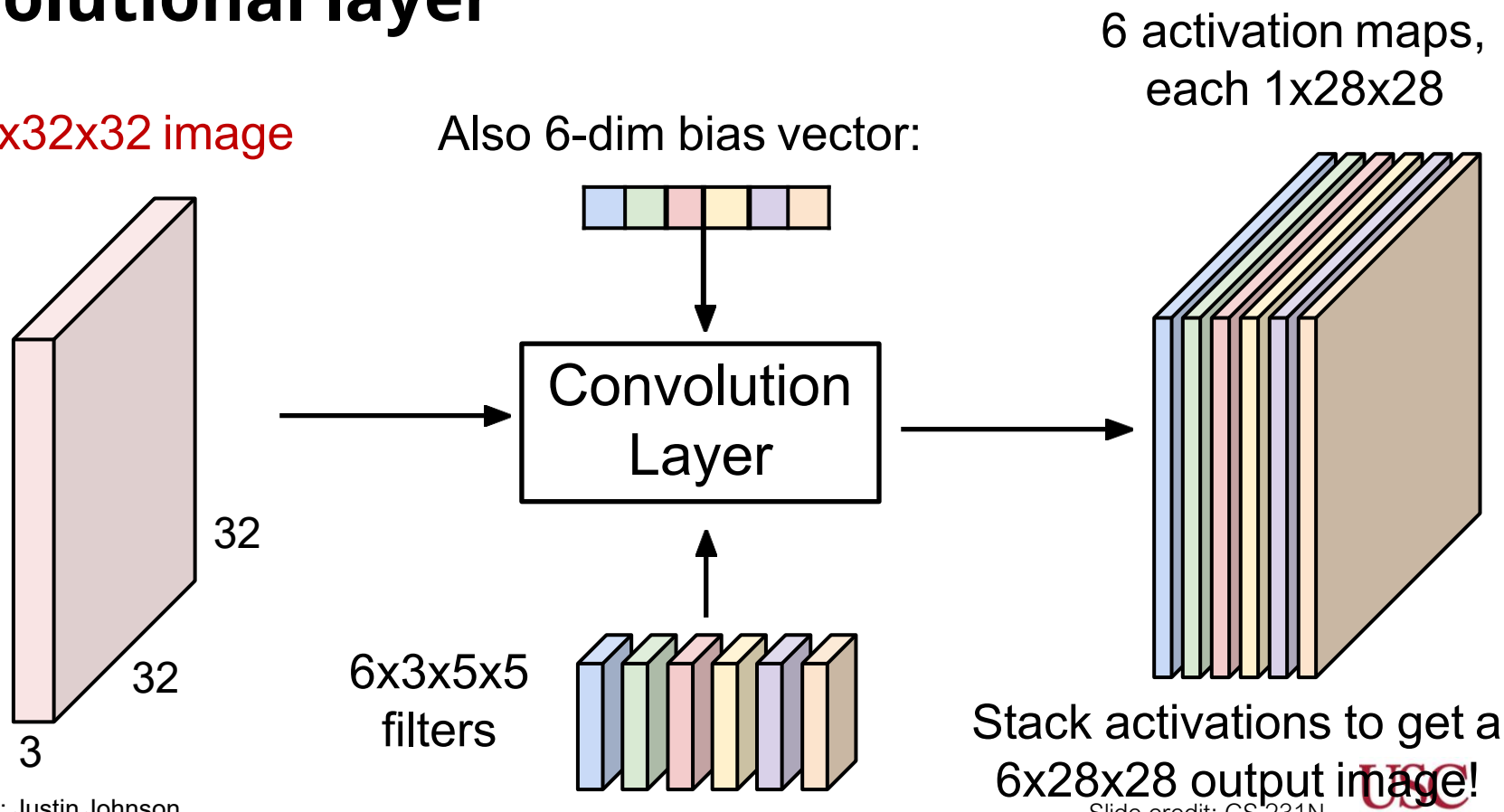
Convolutional layer



Slide inspiration: Justin Johnson

Slide credit: CS 231N

Convolutional layer



Slide inspiration: Justin Johnson

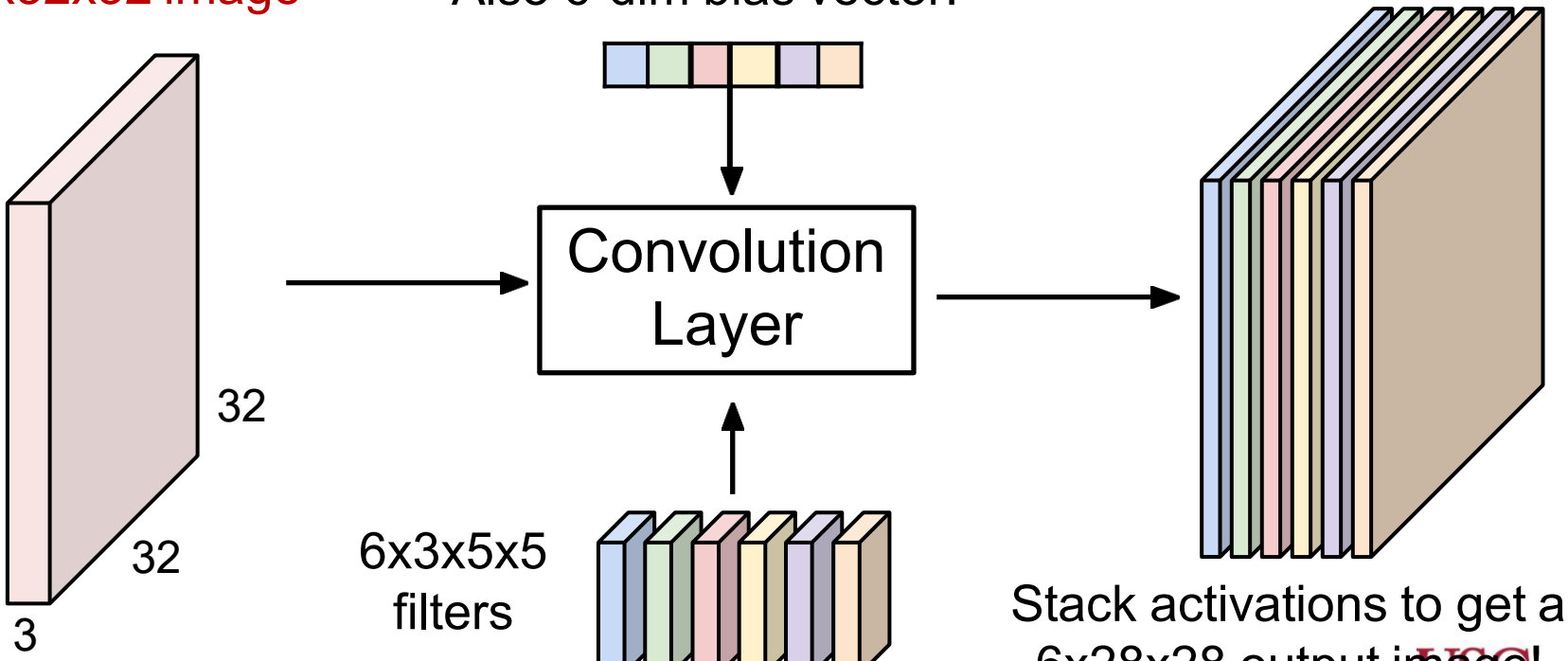
Slide credit: CS 231N

Convolutional layer

28x28 grid, at each point a 6-dim vector

3x32x32 image

Also 6-dim bias vector:

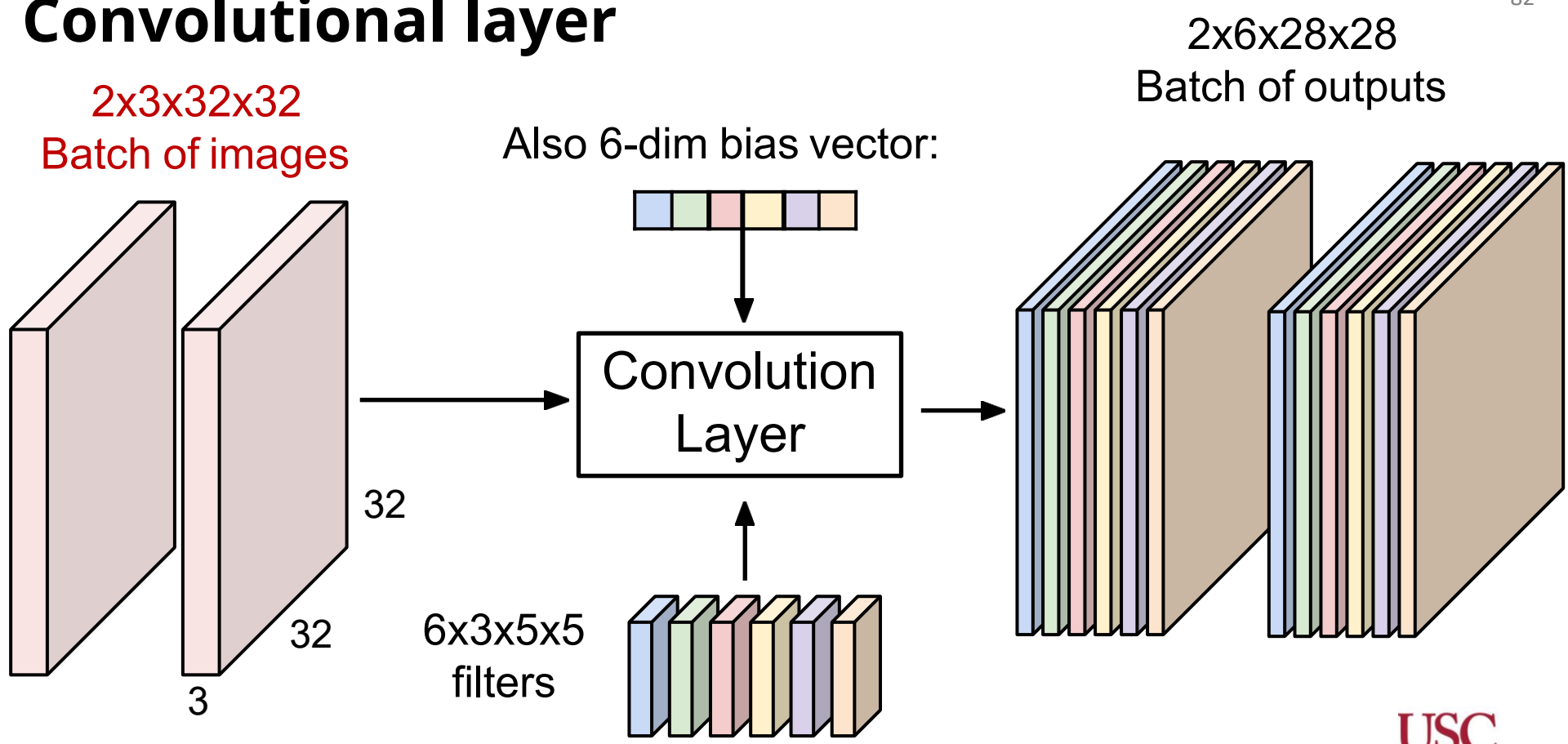


Stack activations to get a 6x28x28 output image!

Slide inspiration: Justin Johnson

Slide credit: CS 231N

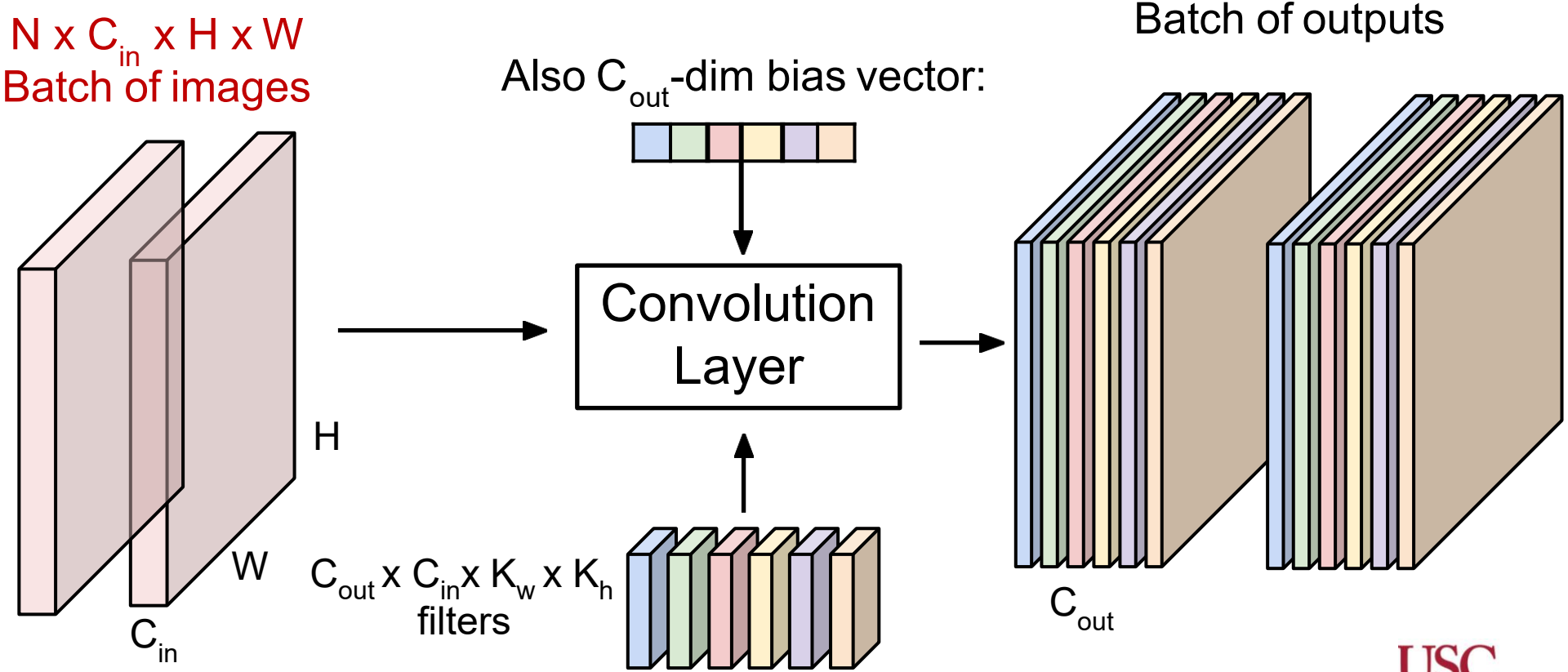
Convolutional layer



Slide inspiration: Justin Johnson

Slide credit: CS 231N

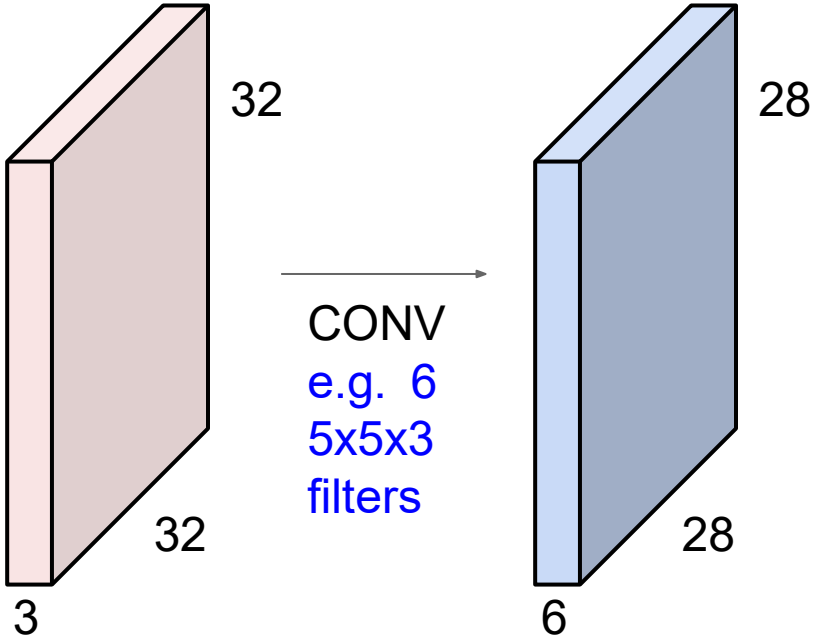
Convolutional layer



Slide inspiration: Justin Johnson

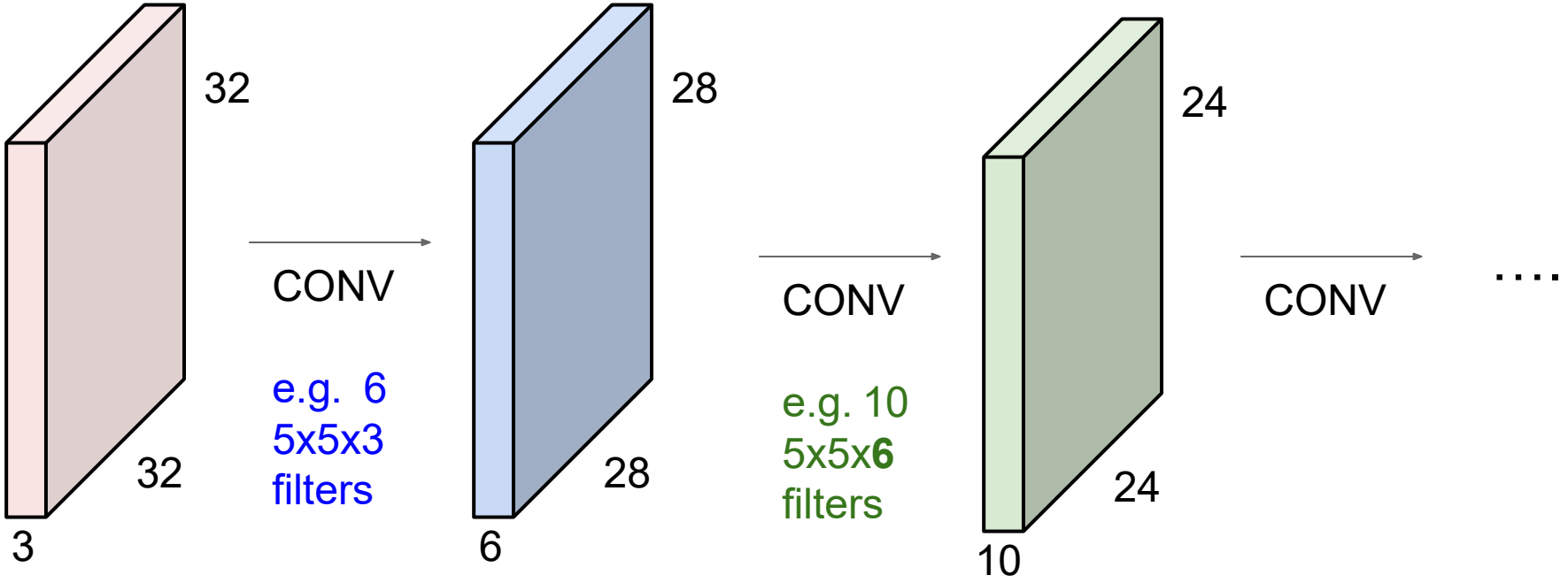
Slide credit: CS 231N

Preview: ConvNet is a sequence of Convolution Layers



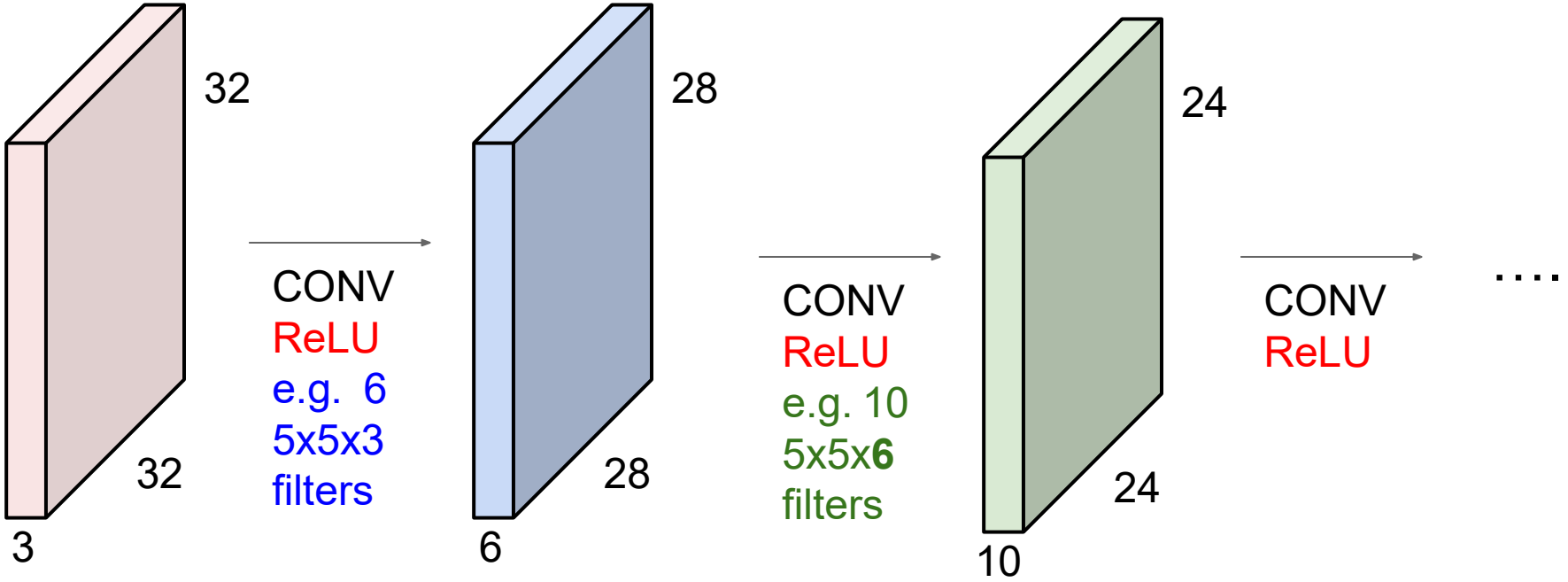
Slide credit: CS 231N

Preview: ConvNet is a sequence of Convolution Layers



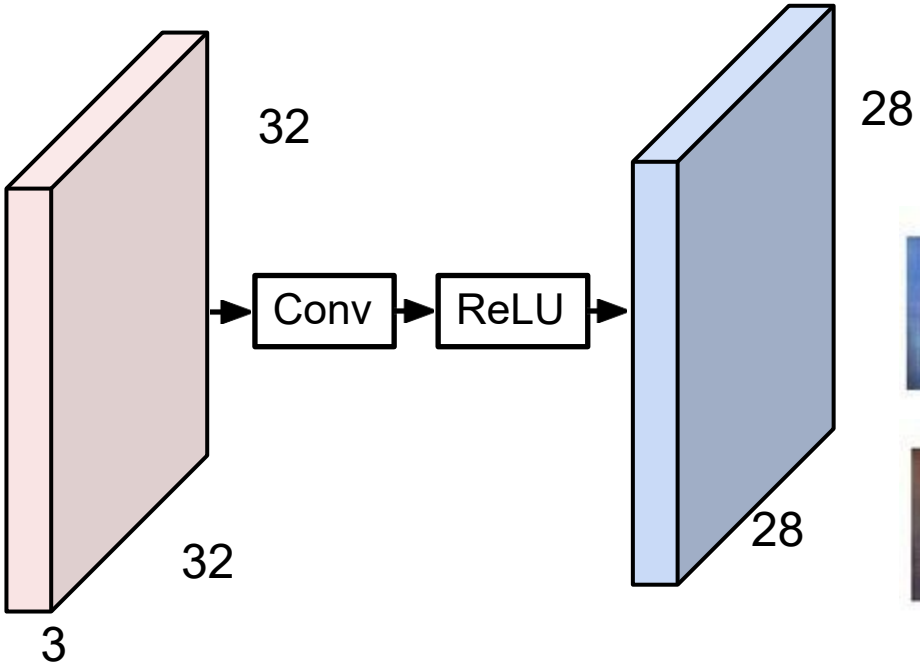
Slide credit: CS 231N

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Slide credit: CS 231N

Preview: What do convolutional filters learn?

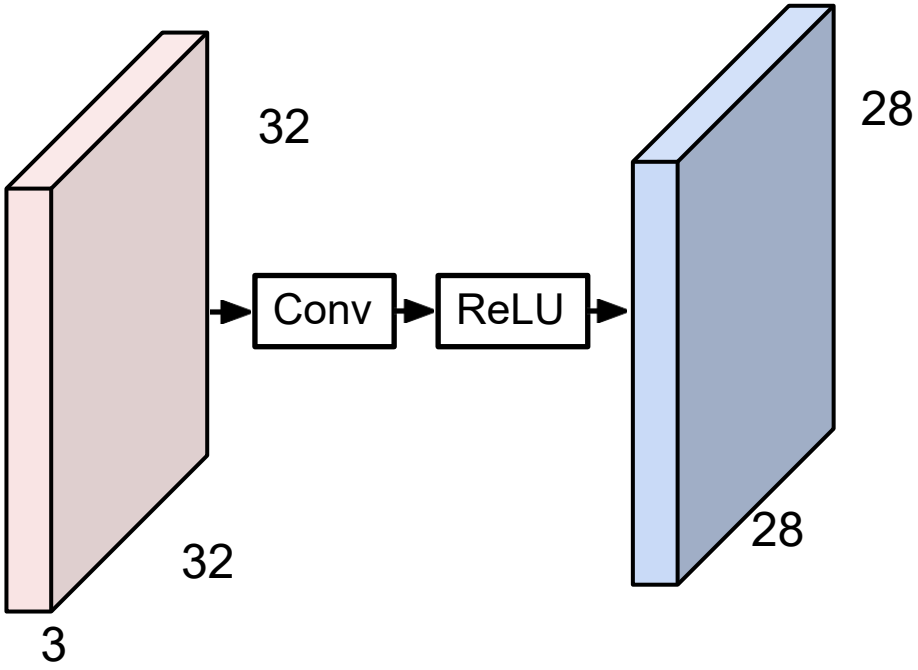


Linear classifier: One template per class



Slide credit: CS 231N

Preview: What do convolutional filters learn?

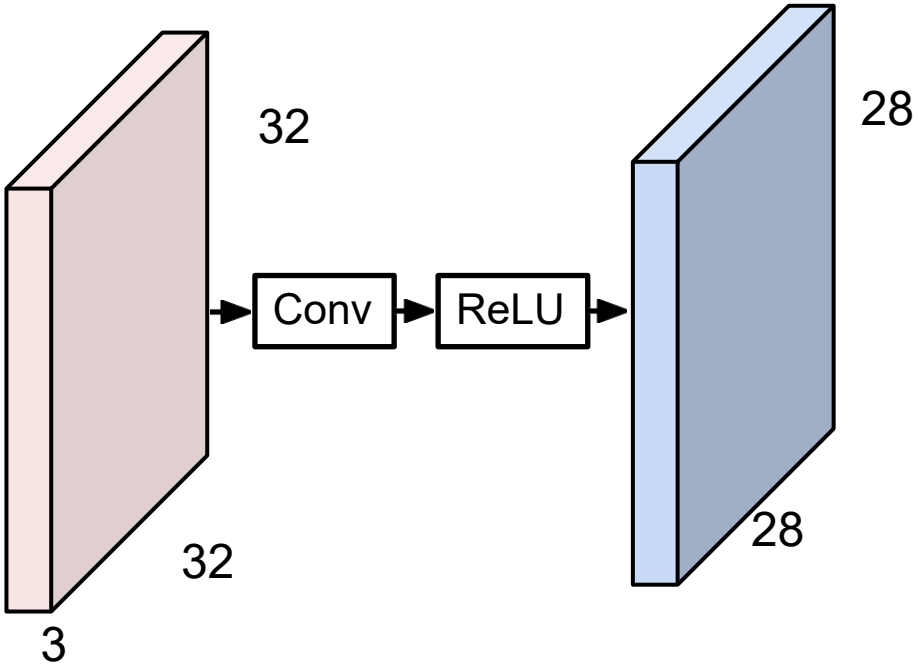


MLP: Bank of whole-image templates

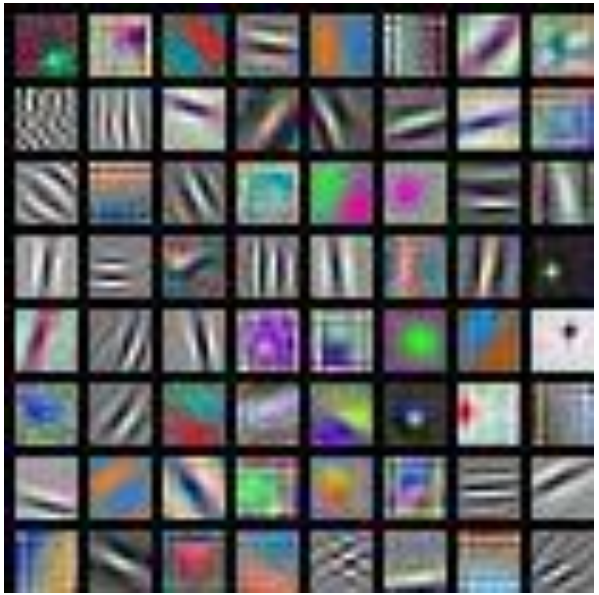


Slide credit: CS 231N

Preview: What do convolutional filters learn?



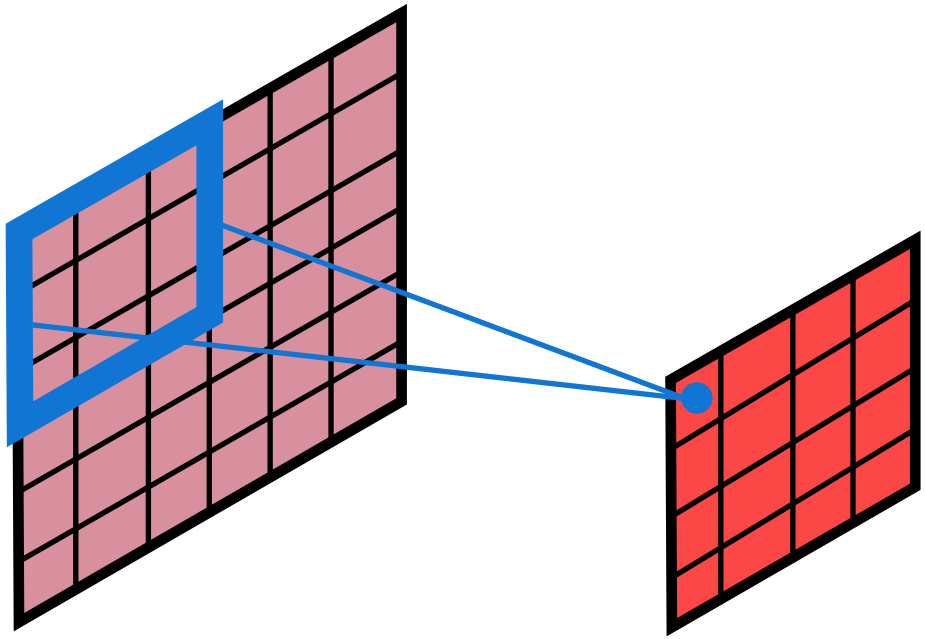
First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)



AlexNet: 64 filters, each 3x11x11

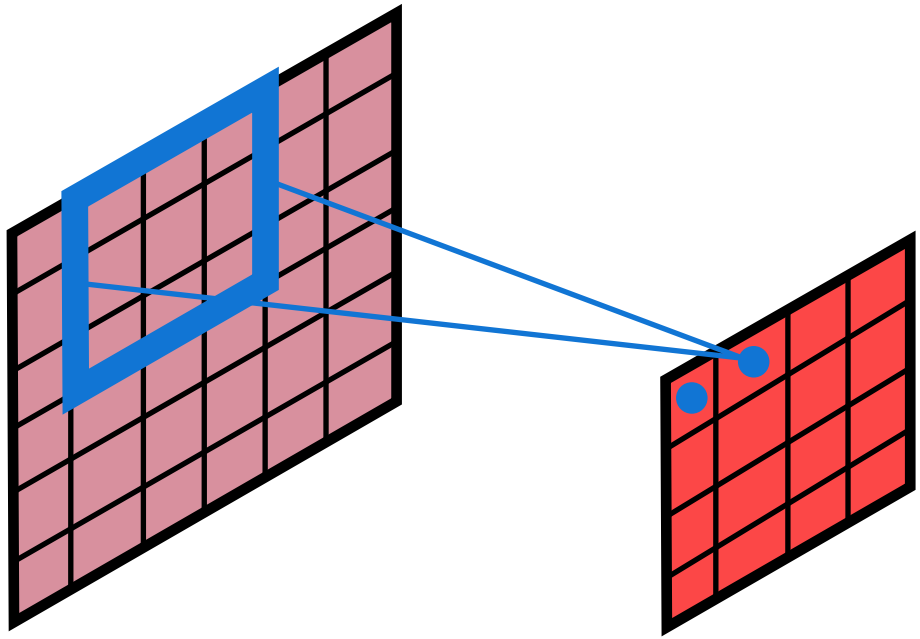
Slide credit: CS 231N

Convolutional layer



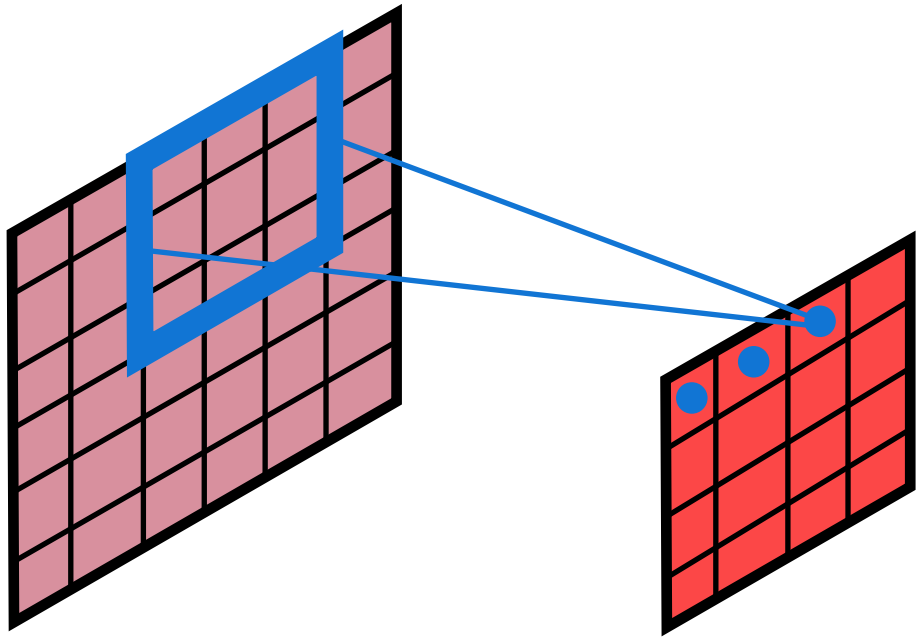
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



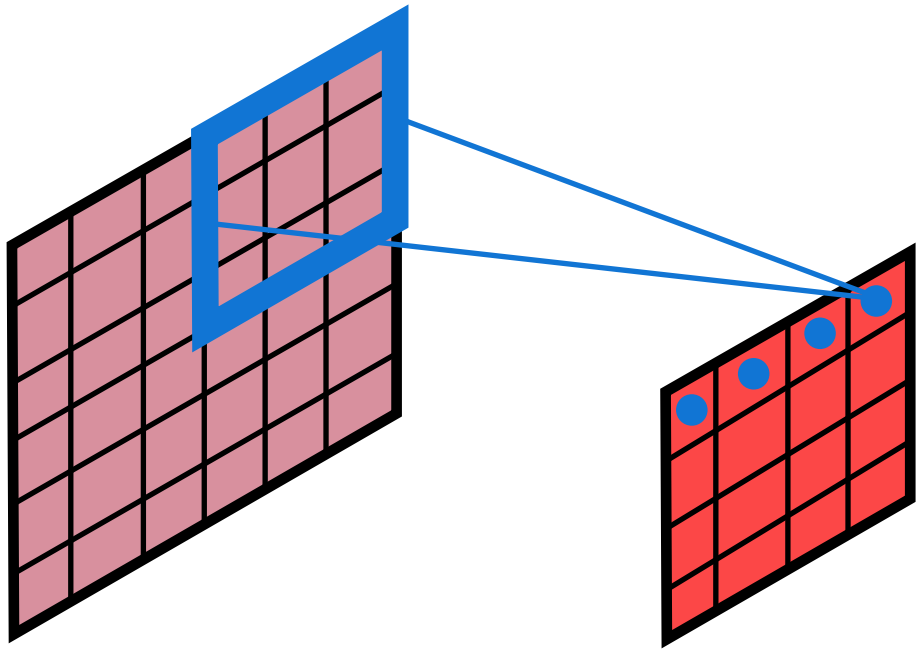
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



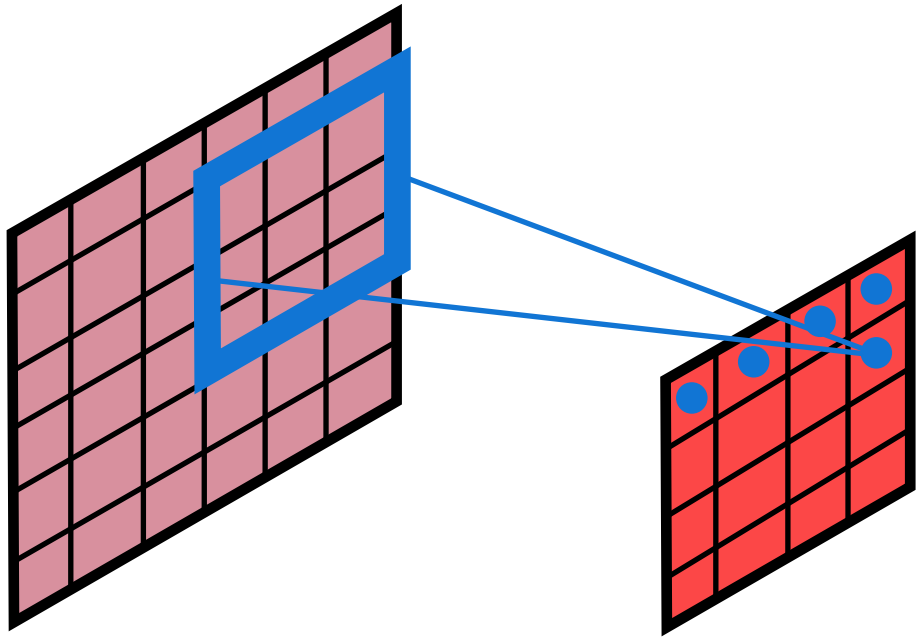
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



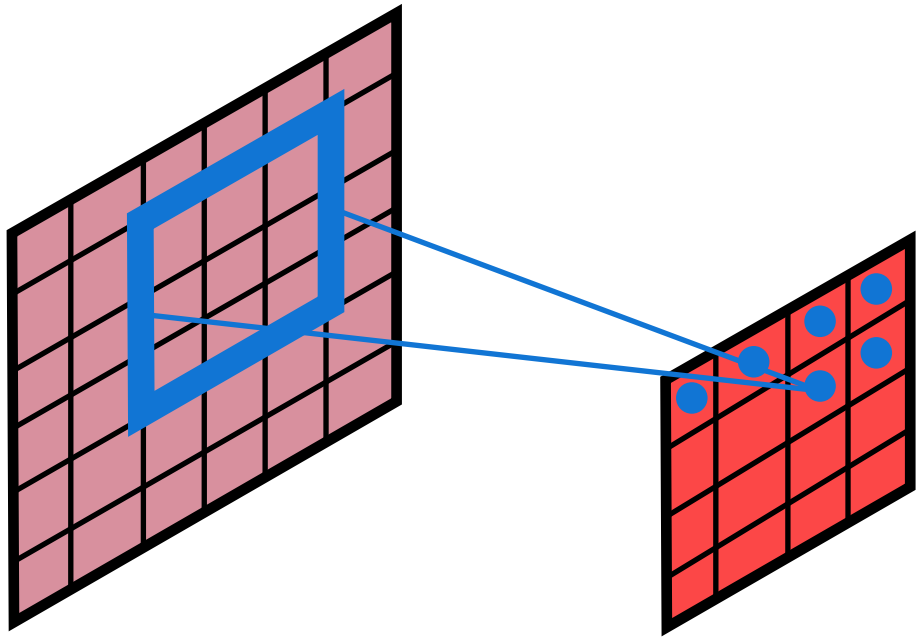
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



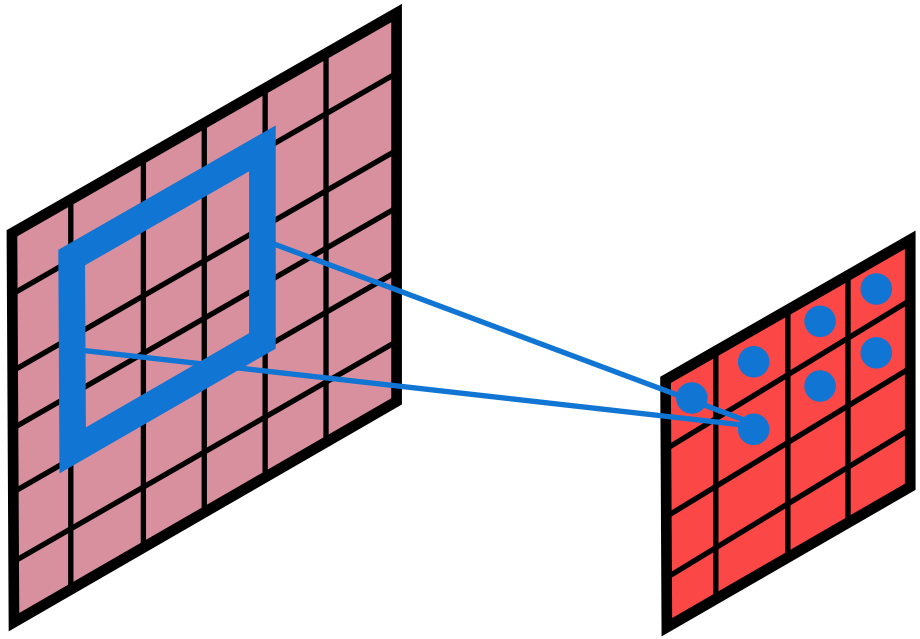
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



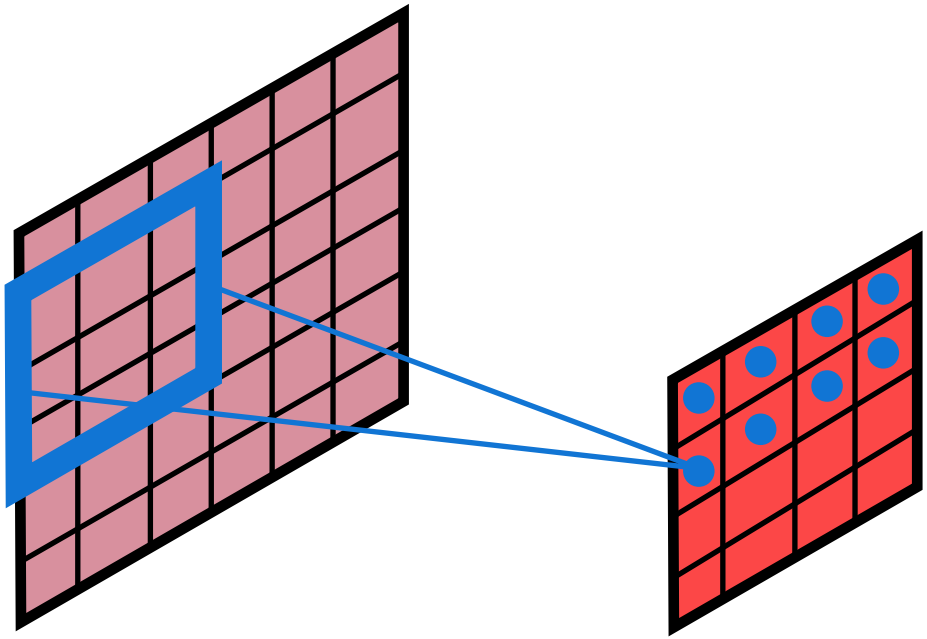
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



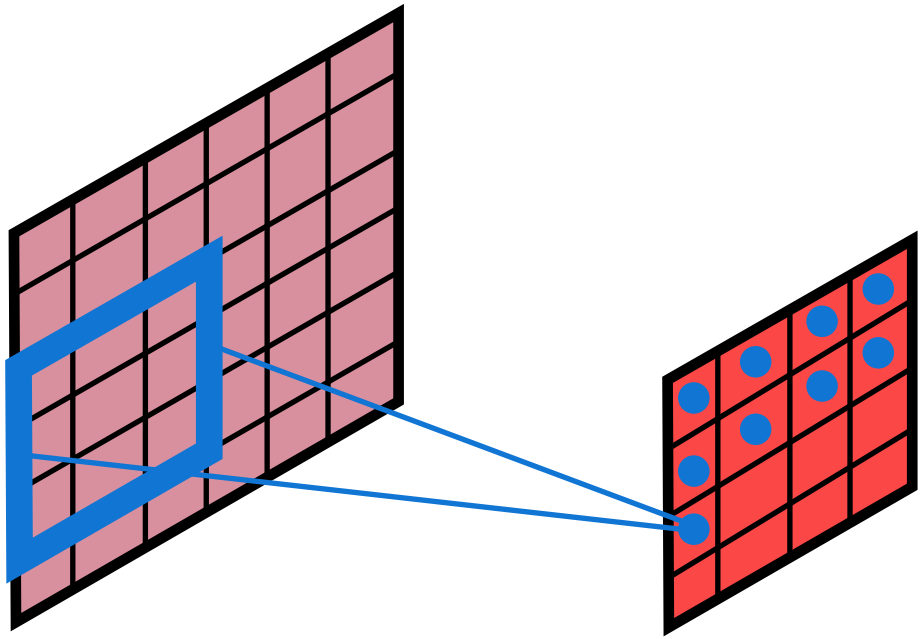
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



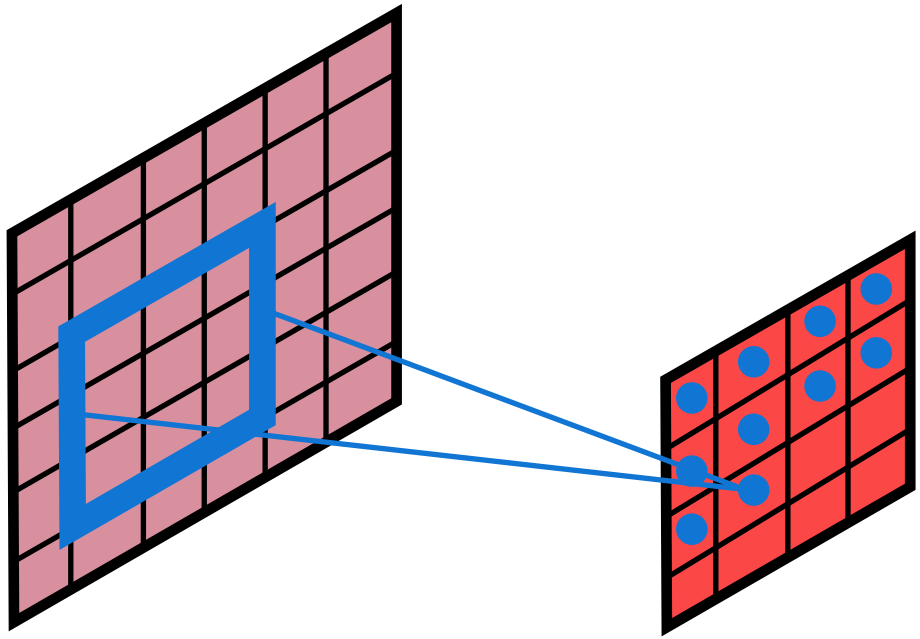
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



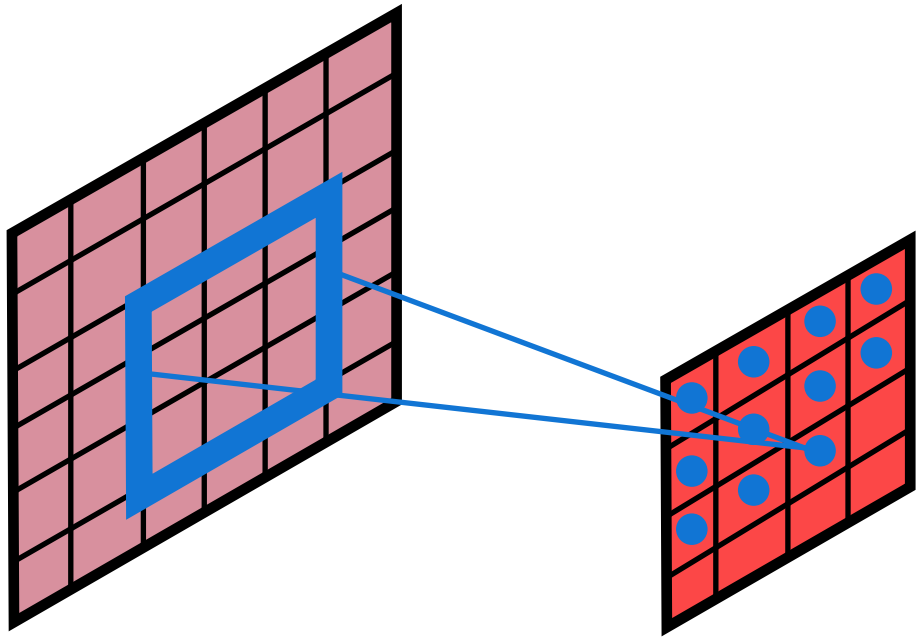
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



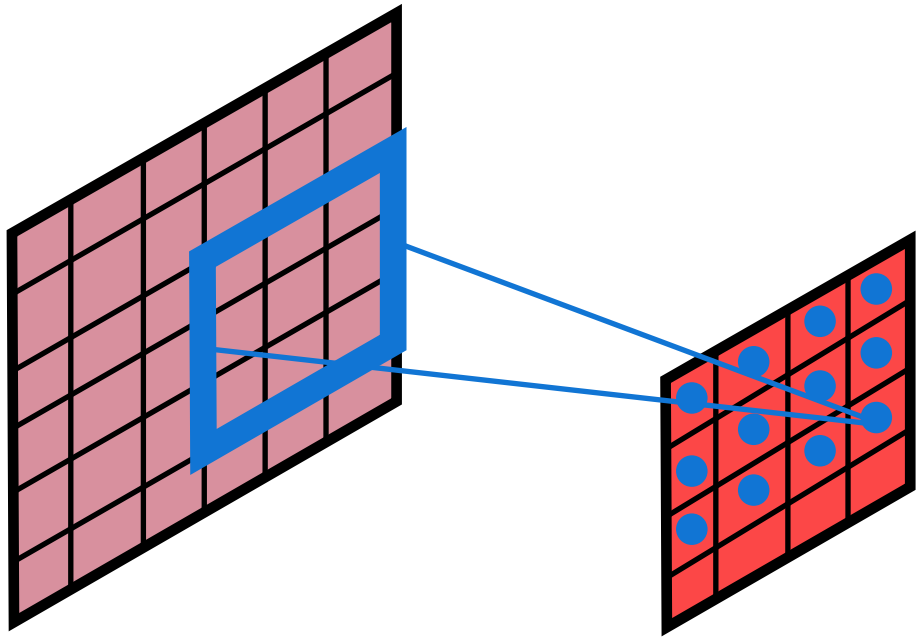
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



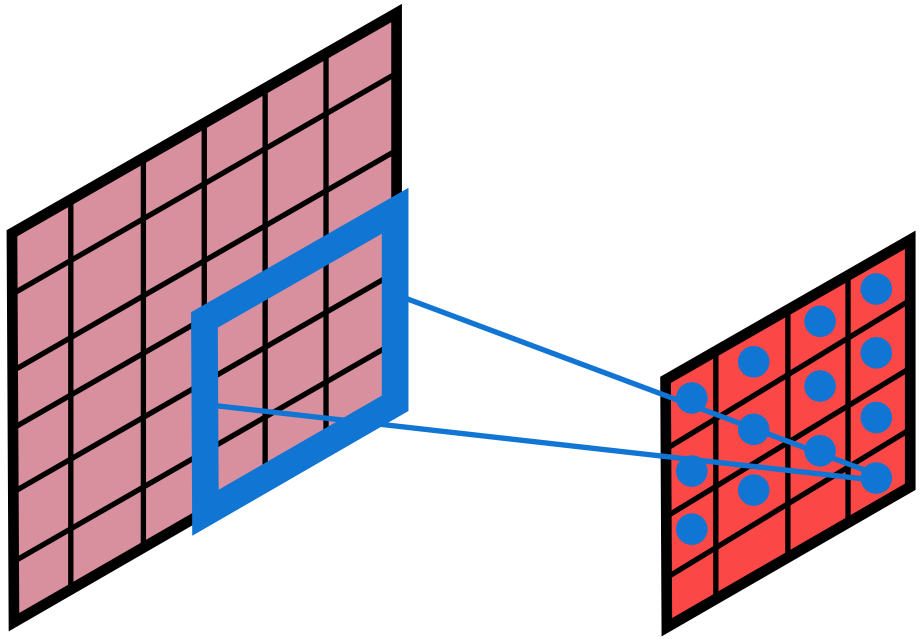
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



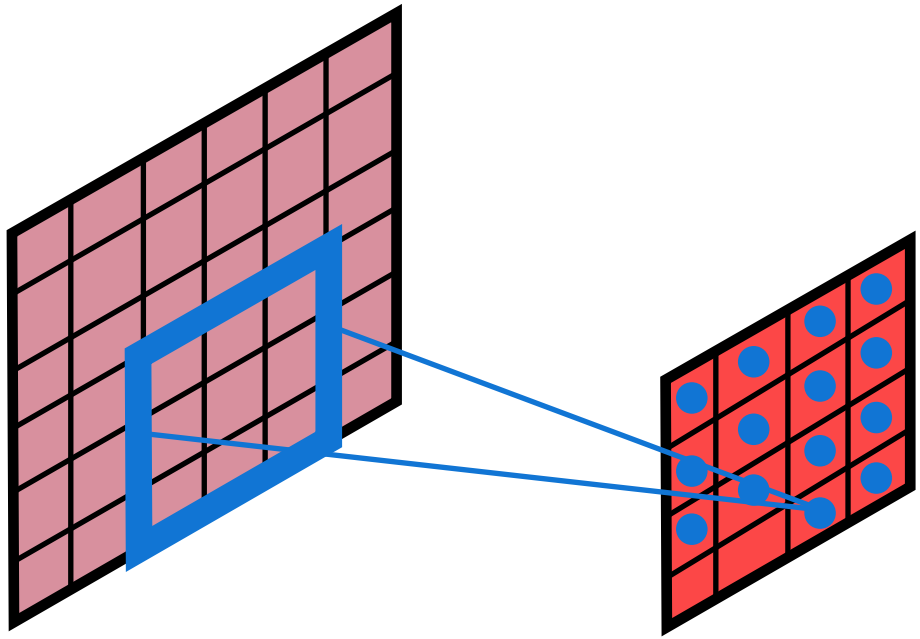
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



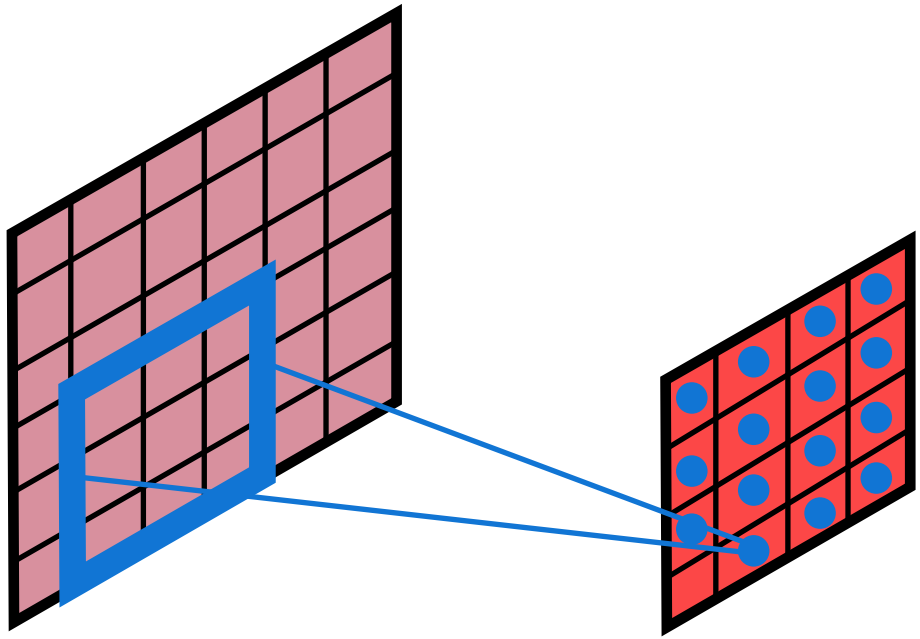
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



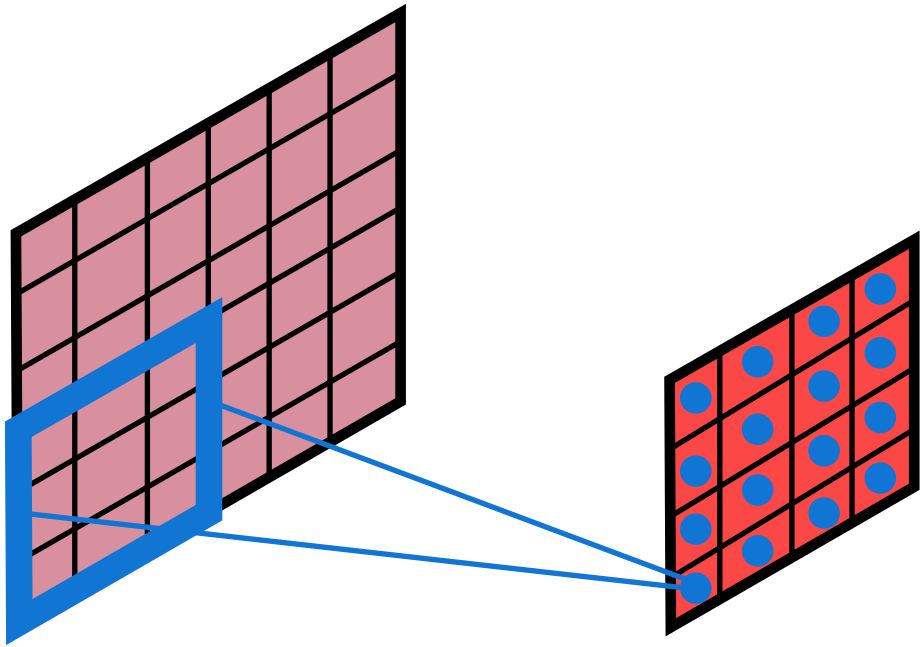
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer



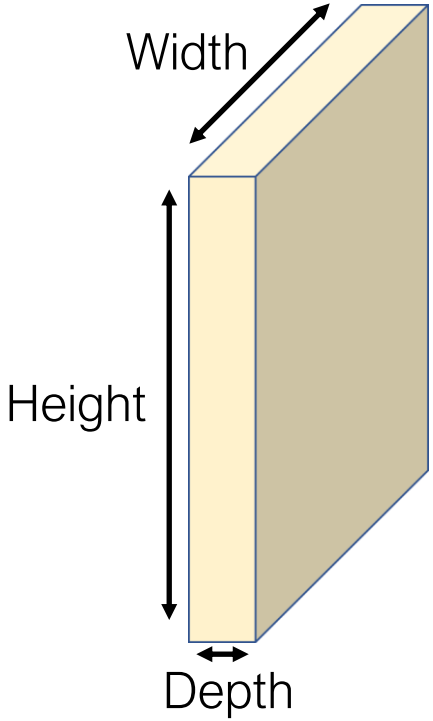
Inspired by Marc'Aurelio Ranzato's slides

Convolutional layer

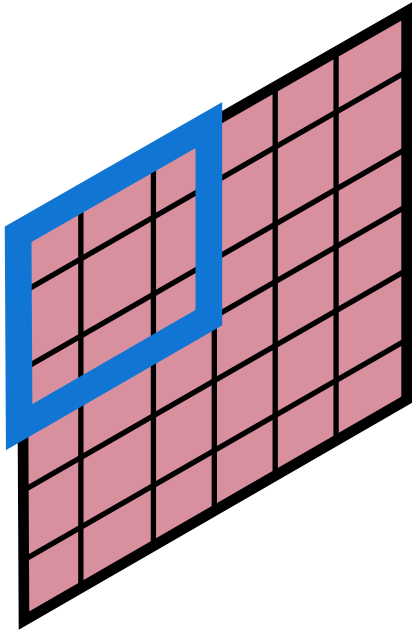


Inspired by Marc'Aurelio Ranzato's slides

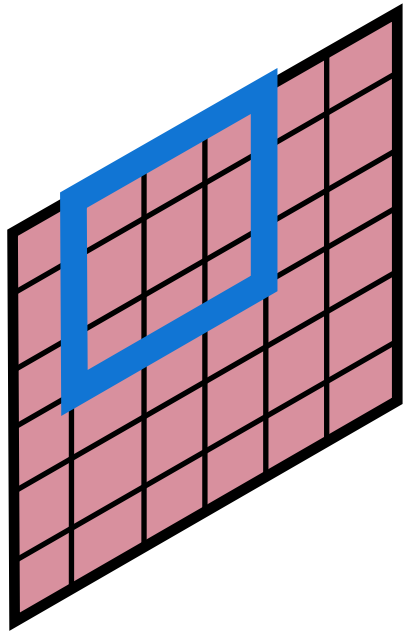
Terminologies



Terminologies



Stride 1

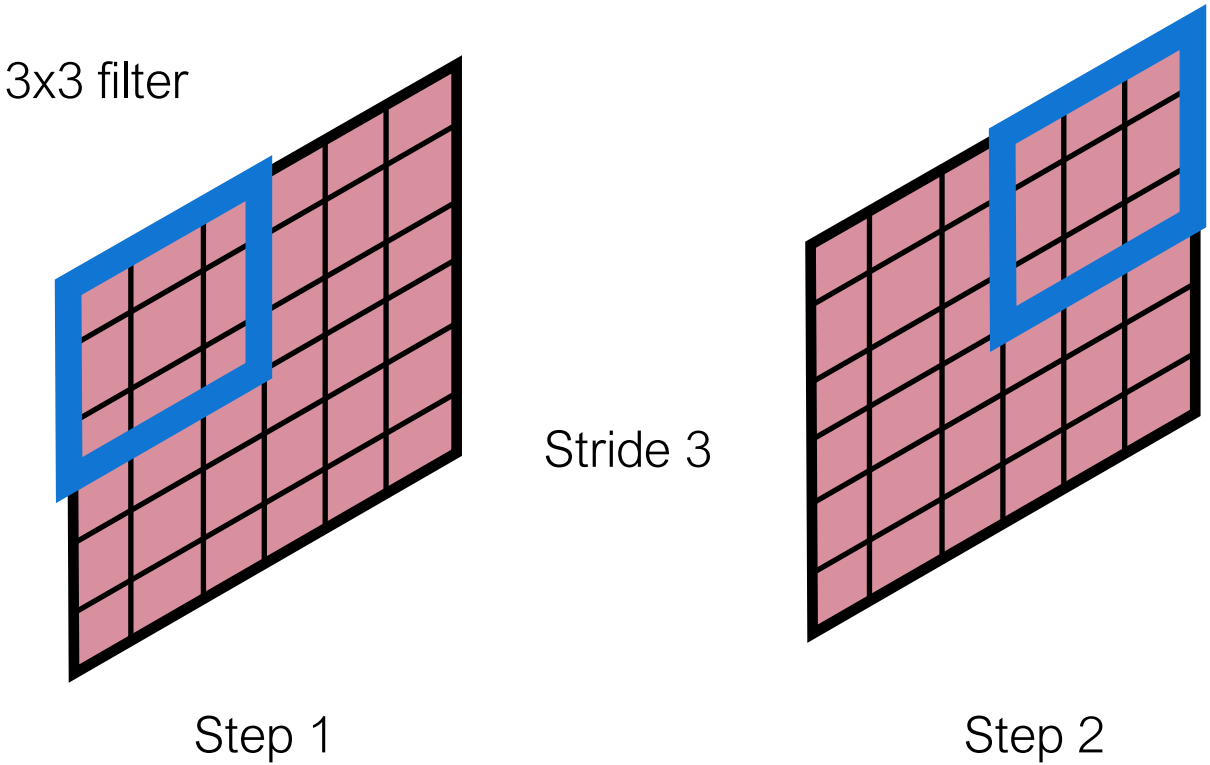


Step 1

Step 2

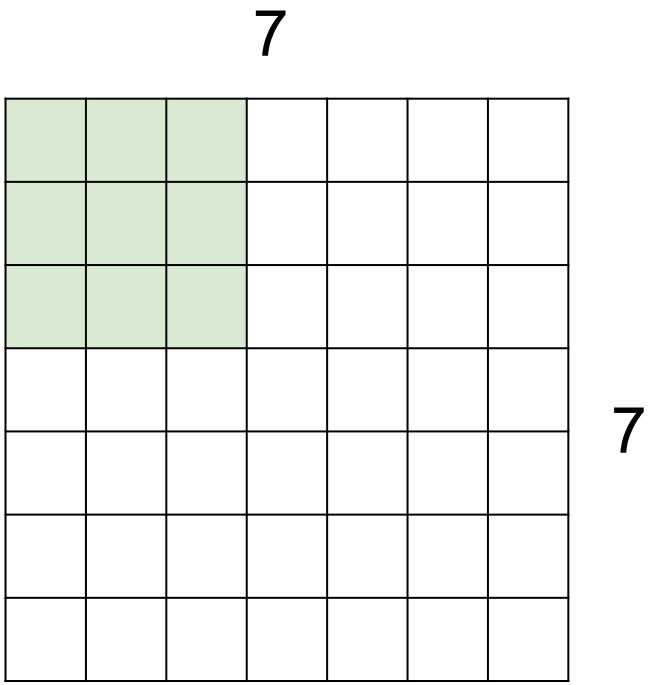
Inspired by Marc'Aurelio Ranzato's slides

Terminologies



Inspired by Marc'Aurelio Ranzato's slides

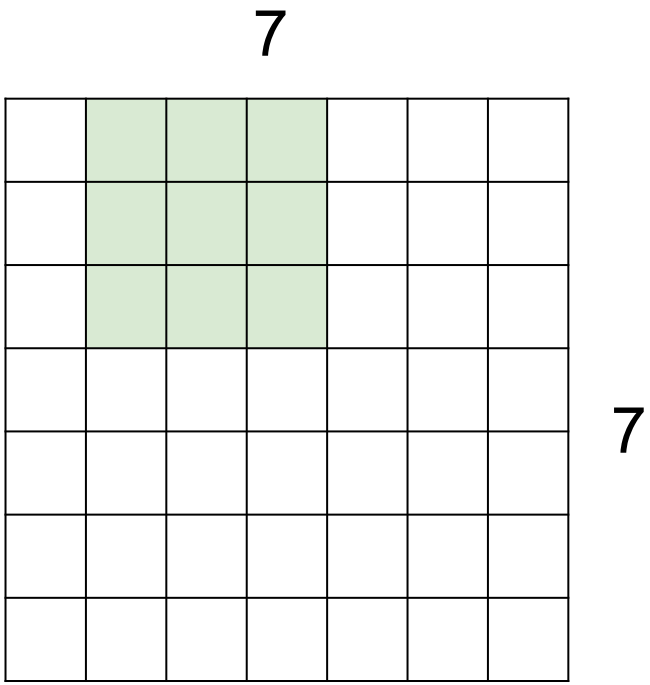
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

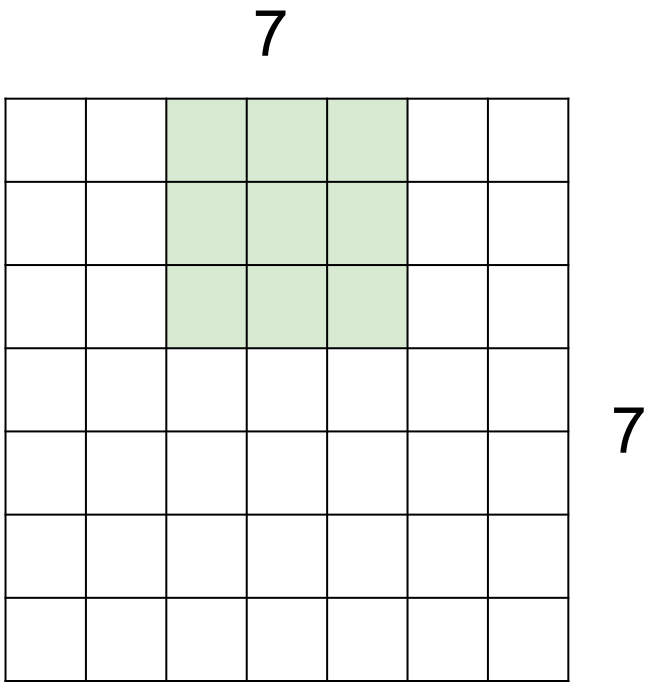
Slide credit: CS 231N

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

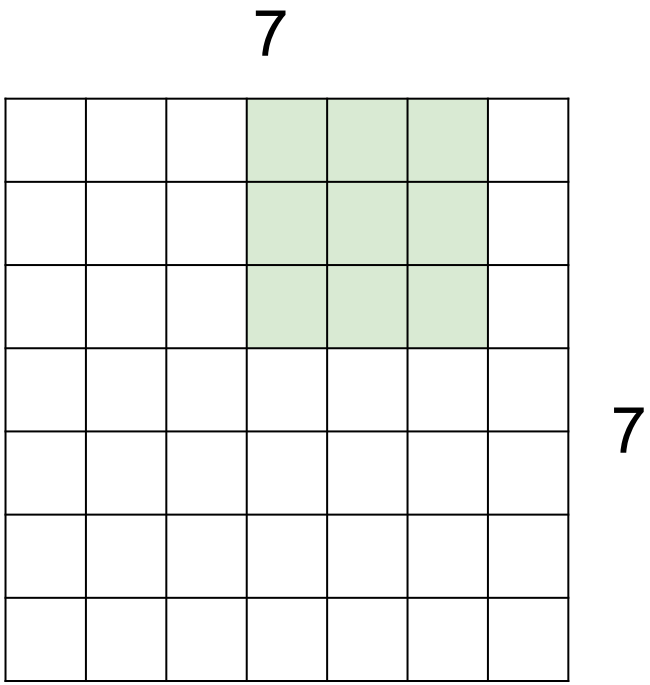
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

Slide credit: CS 231N

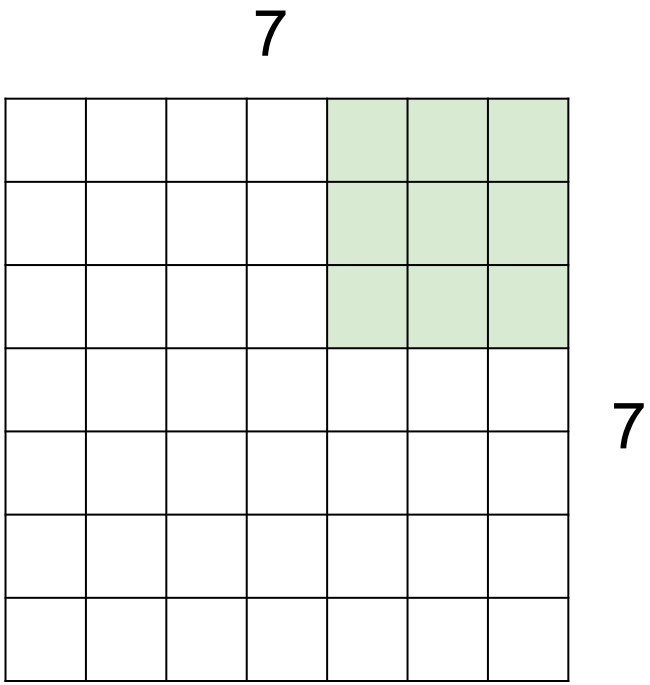
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

Slide credit: CS 231N

A closer look at spatial dimensions:

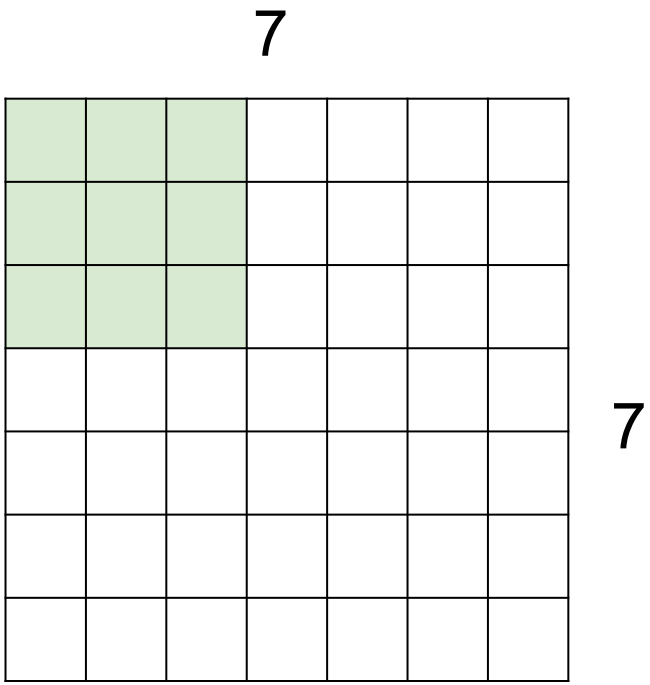


7x7 input (spatially)
assume 3x3 filter

=> 5x5 output

USC Viterbi School of Engineering

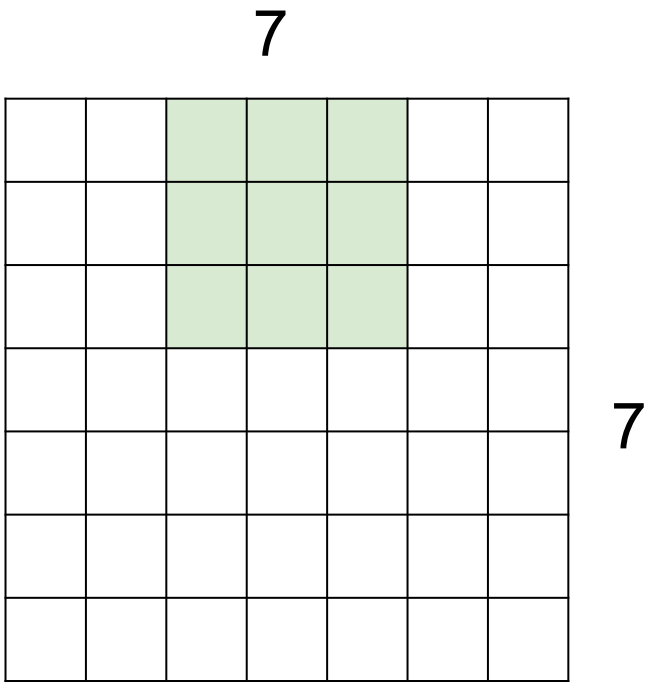
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Slide credit: CS 231N

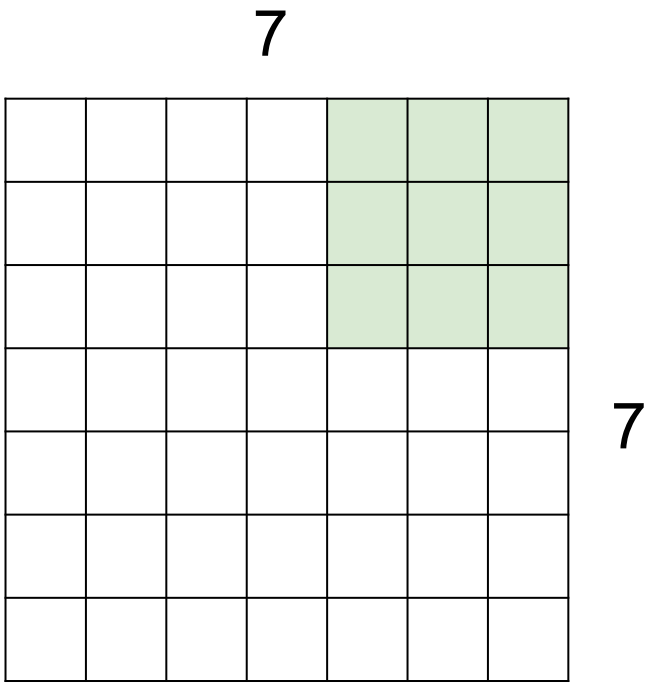
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

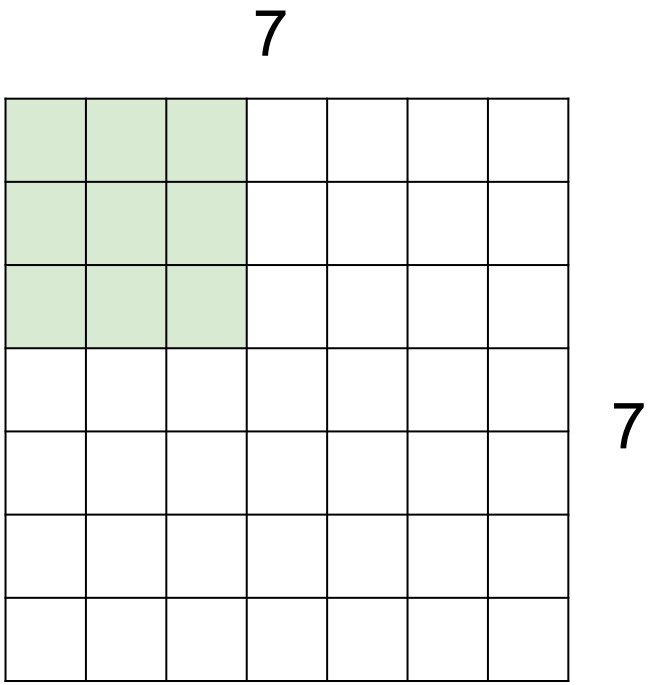
Slide credit: CS 231N

A closer look at spatial dimensions:



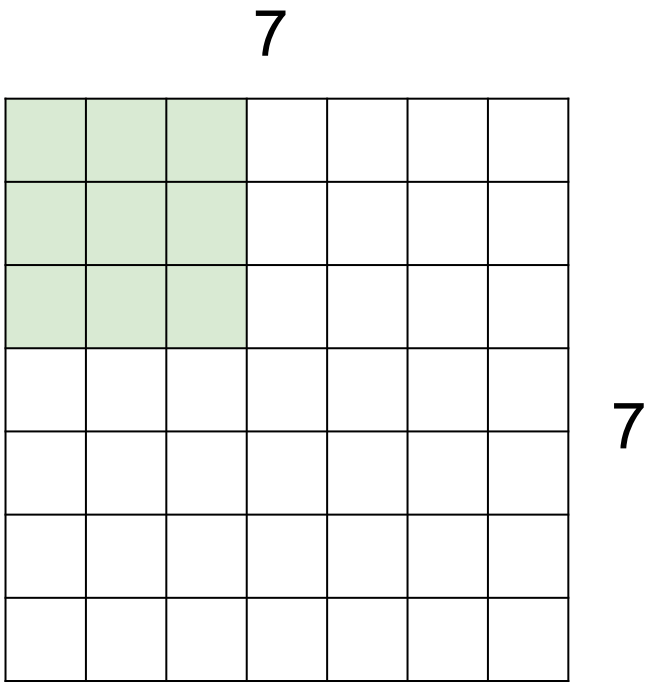
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

Terminologies

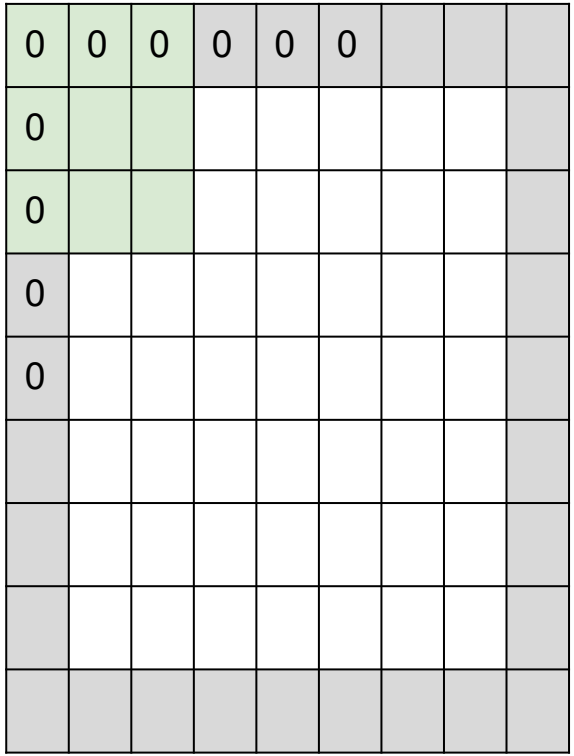
a	b	c	d	e
f	g	h	i	j
k	l	m	n	o
p	q	r	s	t
u	v	w	x	y

Terminologies

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	a	b	c	d	e	0	0
0	0	f	g	h	i	j	0	0
0	0	k	l	m	n	o	0	0
0	0	p	q	r	s	t	0	0
0	0	u	v	w	x	y	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Zero-padding: 2

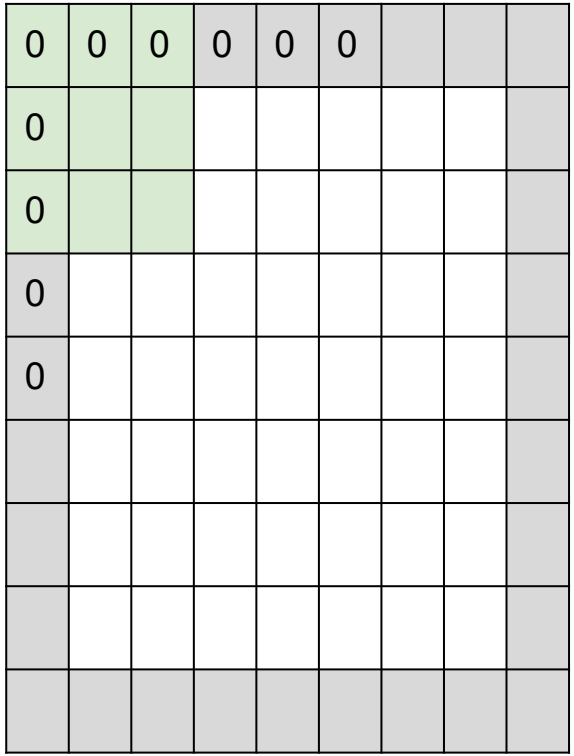
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

In practice: Common to zero pad the border

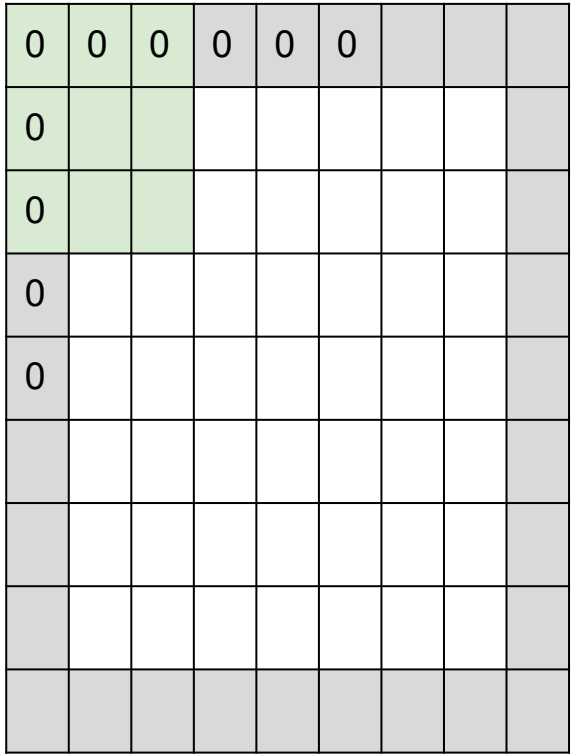


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

(recall:)
(N + 2P - F) / stride + 1

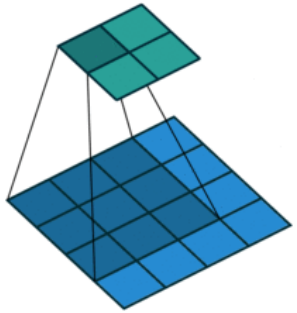
In practice: Common to zero pad the border



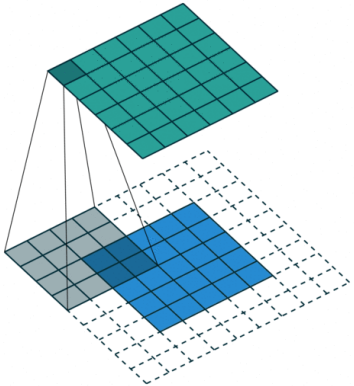
e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

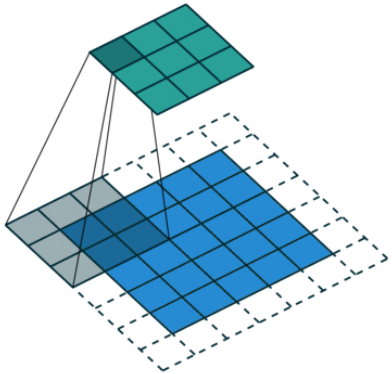
Convolutional layer



padding 0, stride 1



padding 2, stride 1



padding 1, stride 2

Note: it can have **depth** (3rd dimension).

Image credit: vdumoulin (github): https://github.com/vdumoulin/conv_arithmetic

Convolution layer: summary

Let's assume input is W_1 x H_1 x C Conv layer needs 4 hyperparameters:

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

This will produce an output of W_2 x H_2 x K where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

Number of parameters: F^2CK and K biases

Convolution layer: summary

Let's assume input is W_1 x H_1 x C Conv layer needs 4 hyperparameters:

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

Common settings:
K = (powers of 2, e.g. 32, 64, 128, 512)
F = 3, S = 1, P = 1
- F = 5, S = 1, P = 2
- F = 5, S = 2, P = ? (whatever fits)
- F = 1, S = 1, P = 0

This will produce an output of W_2 x H_2 x K where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

Number of parameters: F^2CK and K biases

Fully connected vs Convolutional

Fully connected layer

Locally connected layer

Convolutional layer

Which layer is the best? Why?

Fully connected vs Convolutional

Fully connected layer

Every neuron in a fully connected layer is connected to every neuron in the previous layer. This type of layer is often used toward **the end of neural network** architectures to make decisions based on the features extracted by previous layers.

It is **parameter-heavy** and does **NOT** account for the **spatial hierarchy** in the input data, making it less suitable for tasks like image recognition, where spatial relationships are key.

Fully connected vs Convolutional

Locally connected layer

Each neuron is connected only to **a subset of the input data**, maintaining spatial relationships similar to convolutional layers.

However, unlike convolutional layers, **the weights are not shared across different spatial locations**. This allows the model to learn different features at different locations, but it can lead to **a large number of parameters**, making the network prone to overfitting and computationally expensive.

Fully connected vs Convolutional

Convolutional layer

These layers apply a convolution operation to the input, effectively sliding a number of filters across the input data to produce feature maps.

This method **respects the spatial hierarchy of the data**, is **parameter-efficient** due to weight sharing, and is translation equivariant. Convolutional layers are typically used in tasks involving images or time series data where such properties are advantageous.

Fully connected vs Convolutional

Which layer is “best” cannot be universally answered:

For image-related tasks, convolutional layers encode locality and weight sharing cheaply; vision transformers (Dosovitskiy et al., ICLR 2021) match or beat them at large pretraining scale.

For problems where the precise location of features within the input data is important, and the input data is consistently structured, locally connected layers might be preferable.

For tasks that require holistic understanding of the input data, where spatial relationships are not as important, fully connected layers can be effective, especially as a part of the network's output layer.

What are these layers?

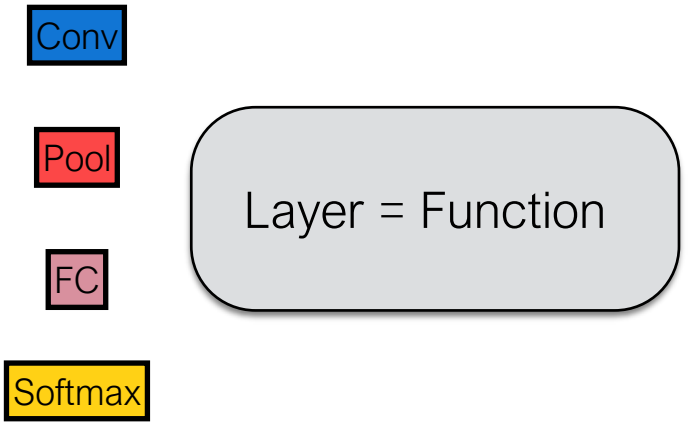
Convolutional layer

Pool layer

Fully connected layer

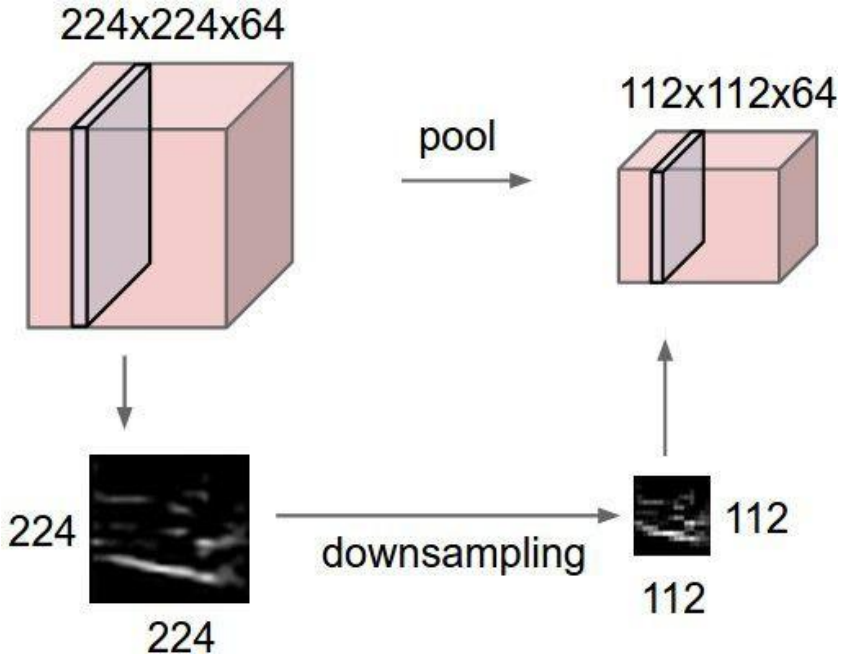
Softmax layer





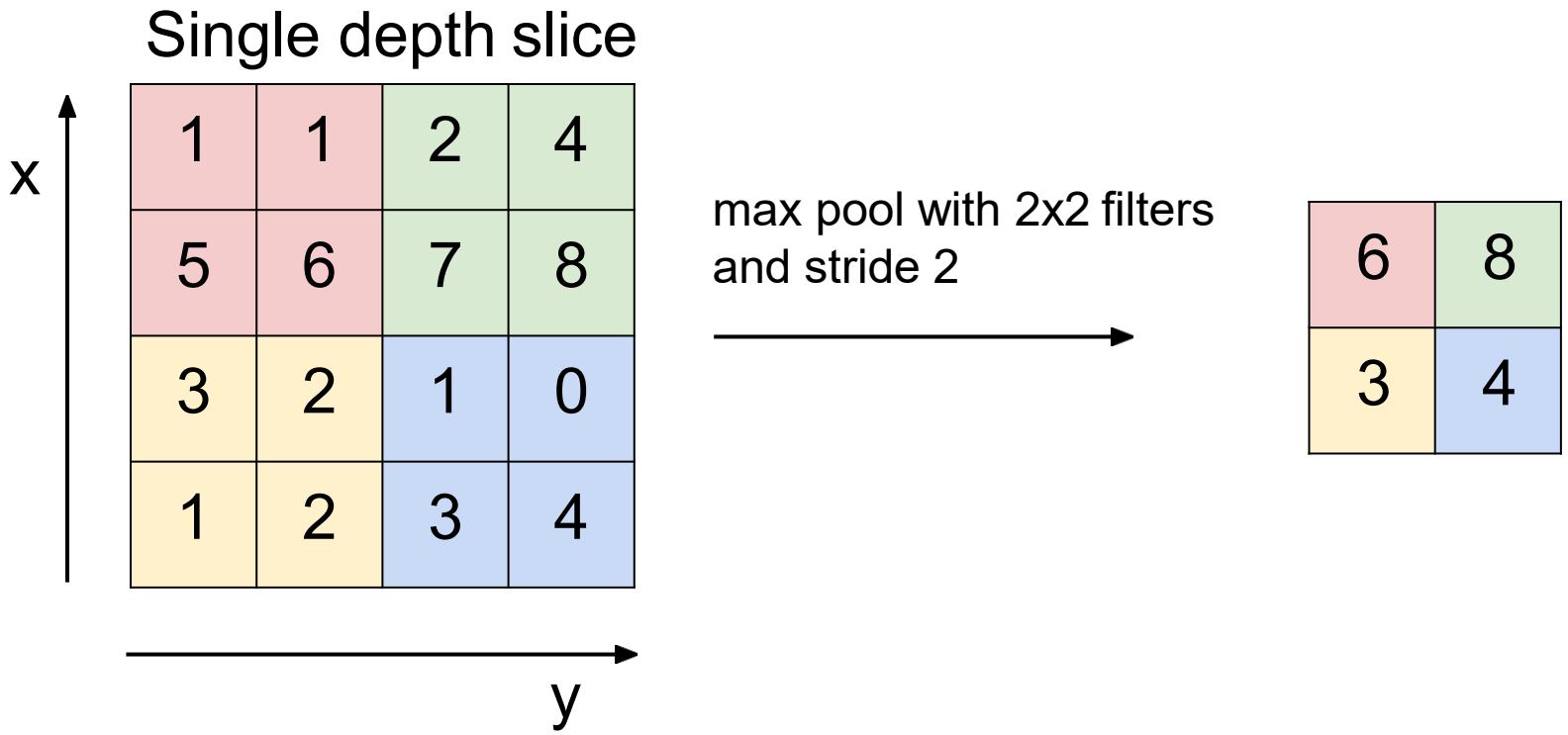
Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently



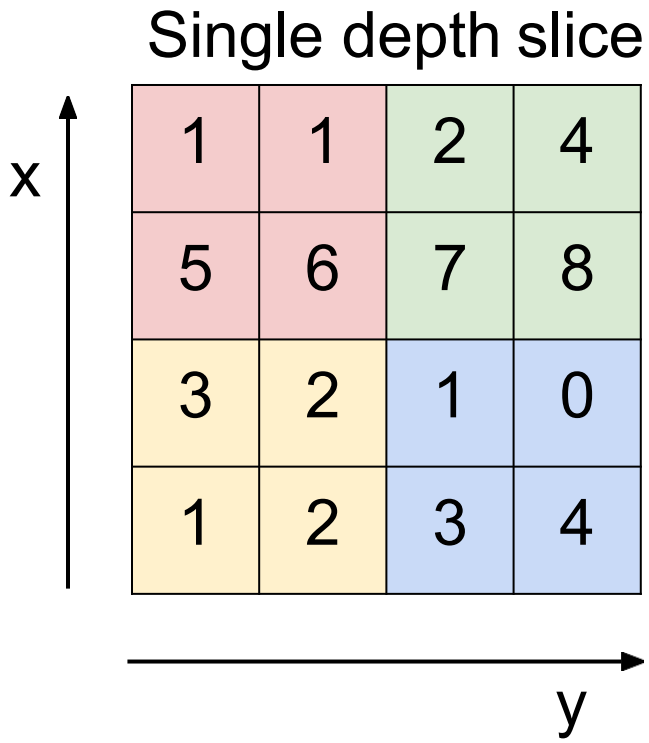
Slide credit: CS 231N

Max Pooling



Slide credit: CS 231N

Max Pooling



max pool with 2x2 filters
and stride 2

6	8
3	4

- No learnable parameters
- Some invariance to **small** shifts

Slide credit: CS 231N

Vote: will max pooling just shift?

Setup. 2x2 max pooling, stride 2, no padding. Shift the input one pixel right, zero-filling the left edge.

Original X.

```
0 1 0 0
0 9 0 0
0 0 8 0
0 0 7 0
```

Vote. Will the pooled output be the old output shifted one position? A: yes. B: no. C: only the maximum matters.

Vote first. Do not compute until the room commits.

Reveal: one pixel breaks the pattern

Outputs. Left is P(X); right is P of the shifted input.

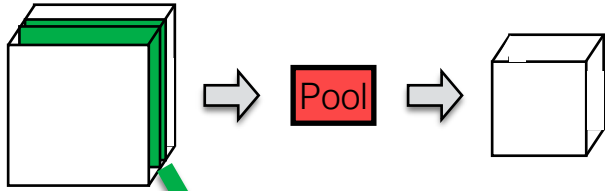
```
9 0      0 9
0 8      0 8
```

Changed. The top row moved but the bottom row did not. That is not one uniform shift.

Correct claim. Pooling lowers resolution and gives limited local shift tolerance. Convolution is shift-equivariant; stride, padding, and boundaries can break even that.

Equivariance tracks shifts; invariance erases them. Pooling guarantees neither.

Max Pooling

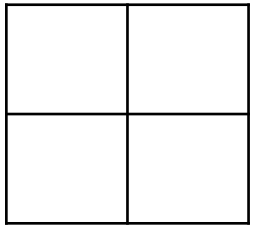


M x N x P

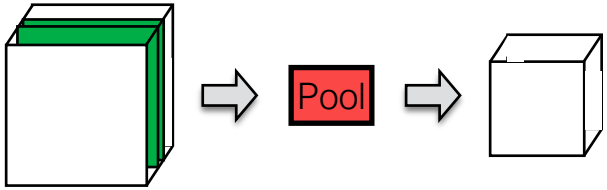
m x n x P

1	3	2	7	2
5	1	3	0	5
4	8	5	0	6
3	2	6	0	8
0	7	4	6	9

max pool with
3x3 filters & stride 2

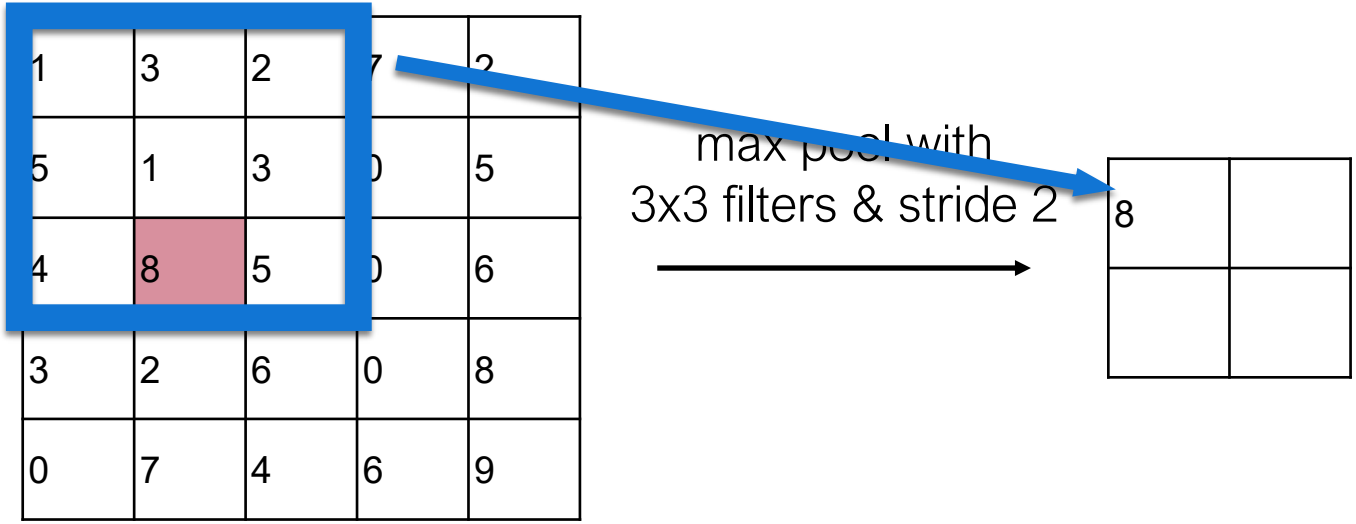


Max pooling layer

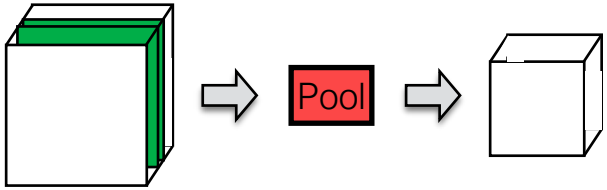


M x N x P

m x n x P



Max pooling layer



M x N x P

m x n x P

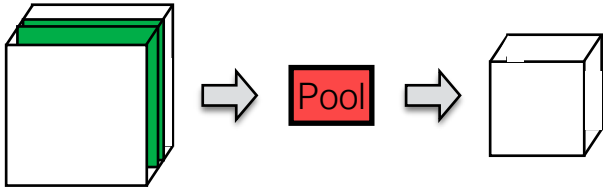
1	3	2	7	2
5	1	3	0	5
4	8	5	0	6
3	2	6	0	8
0	7	4	6	9

max pool with
3x3 filters & stride 2

8	7

USC Viterbi School of Engineering

Max pooling layer



M x N x P

m x n x P

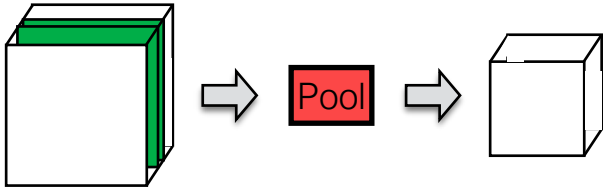
1	3	2	7	2
5	1	3	0	5
4	8	5	0	6
3	2	6	0	8
0	7	4	6	9

max pool with
3x3 filters & stride 2

8	7
8	

USC Viterbi School of Engineering

Max pooling layer



M x N x P

m x n x P

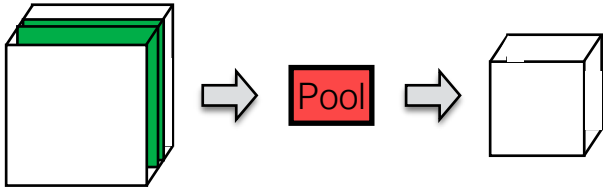
1	3	2	7	2
5	1	3	0	5
4	8	5	0	6
3	2	6	0	8
0	7	4	6	9

max pool with
3x3 filters & stride 2

8	7
8	9

USC Viterbi School of Engineering

Max pooling layer



M x N x P

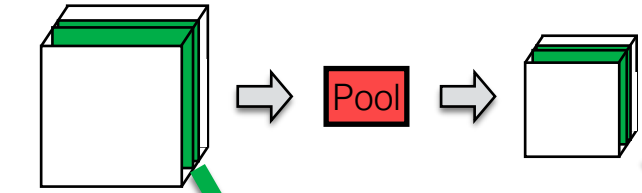
m x n x P

1	3	2	7	2
5	1	3	0	5
4	8	5	0	6
3	2	6	0	8
0	7	4	6	9

max pool with
3x3 filters & stride 2

8	7
8	9

Max pooling layer



M x N x P

m x n x P

1	3	2	7	2
5	1	3	0	5
4	8	5	0	6
3	2	6	0	8
0	7	4	6	9

max pool with
3x3 filters & stride 2

8	7
8	9

Pooling layers

Max pooling (most common)

Average pooling

L2-norm pooling

…

Why pooling layers?

Reduces # of parameters in the following layers

Why pooling layers?

Reduces # of parameters in the following layers

- Hence, less overfitting!—reducing model complexity

Pooling layer: summary

Let's assume input is W_1 x H_1 x C Pooling layer needs 2 hyperparameters:

- The spatial extent **F**
- The stride **S**

This will produce an output of W_2 x H_2 x C where:

- $W_2 = (W_1 - F)/S + 1$
- $H_2 = (H_1 - F)/S + 1$

Number of parameters: 0 -> nothing to be learned

Slide credit: CS 231N

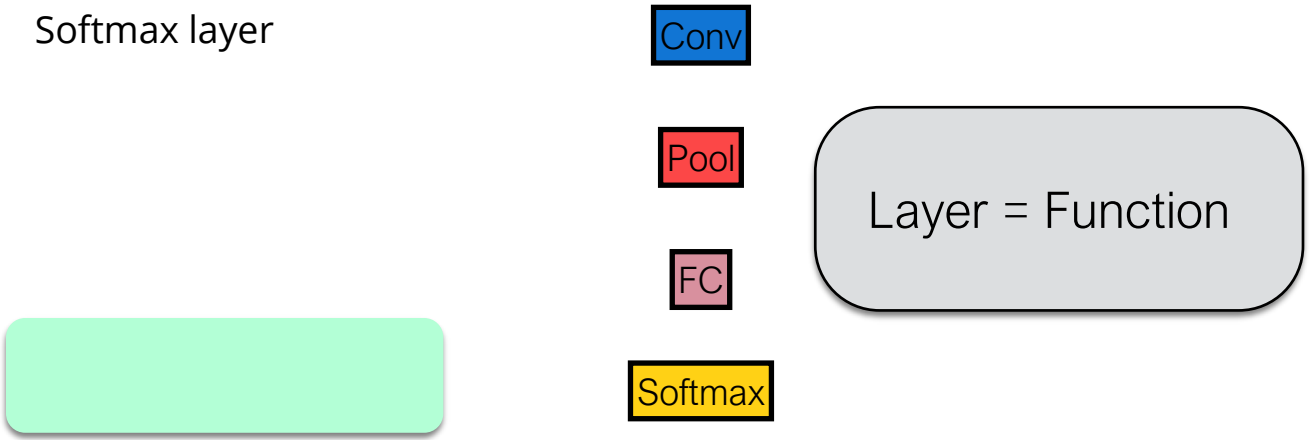
What are these layers?

Convolutional layer

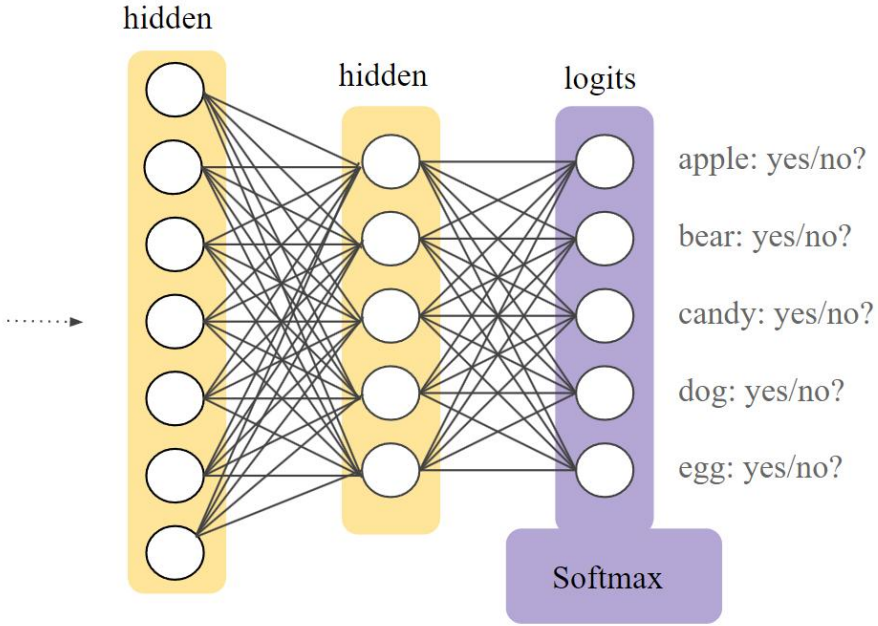
Pool layer

Fully connected layer

Softmax layer

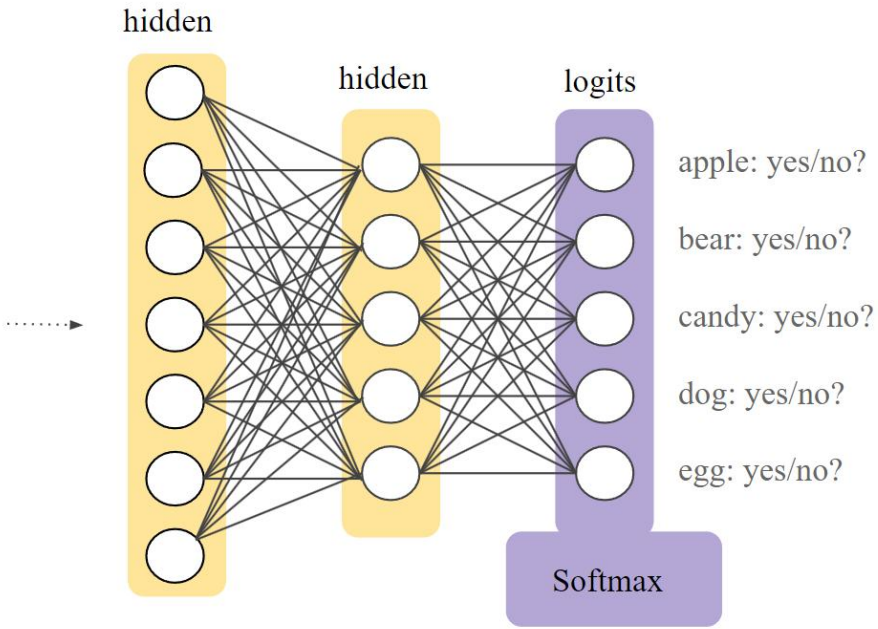


Softmax Layer



Softmax is implemented through a neural network layer just ***before*** the output layer. The Softmax layer must have the same number of nodes as the output layer.

Softmax Layer



The goal is to map the non-normalized output of a network to a **probability distribution** over predicted output classes.

a given class *I* in a classification problem with K classes is:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- *z* is the input vector to the softmax function, containing the raw class scores from the network
- $\sigma(z)_i$ is the output probability of the *i*-th class.

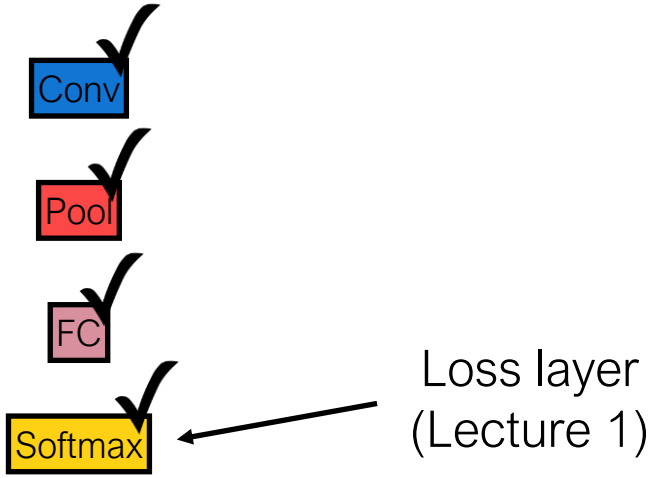
Only two of these are linear!!!

Convolutional layer: linear

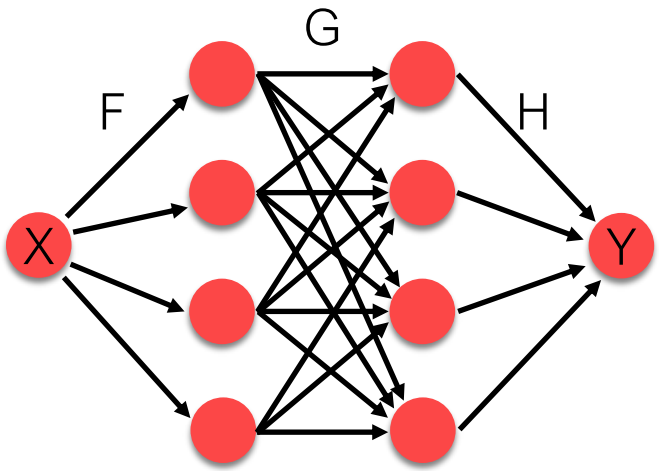
Pool layer: max pool is nonlinear

Fully connected layer: linear

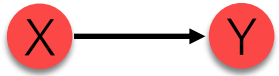
Softmax layer: nonlinear



Activation functions





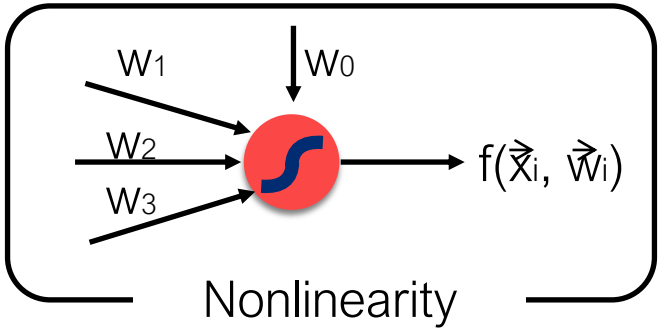


Y=H(G(F(X)))

Y=(HGF)(X)

Without nonlinear activation,
multiple layers are reduced to one matrix multiplication.

Activation functions

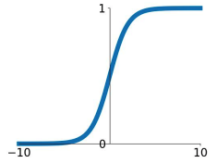


Nonlinearity makes a neural network deeper and more expressive

Recap on Activation functions

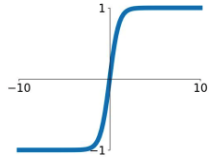
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



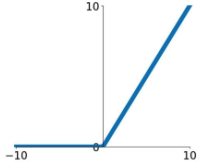
tanh

$$\tanh(x)$$



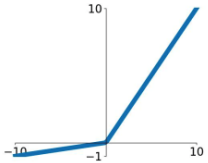
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

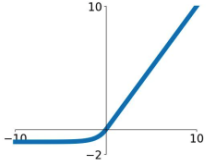
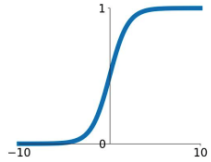


Figure from Stanford cs231n lecture slides

Recap on Activation functions

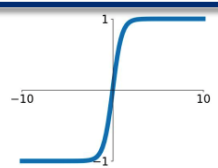
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



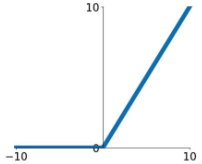
tanh

$$\tanh(x)$$



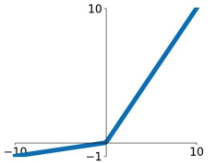
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

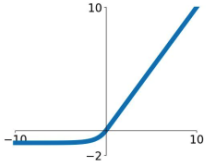


Figure from Stanford cs231n lecture slides

Recap on Activation functions

Similar to step functions, but differentiab

(-) easily saturated (no gradients)

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

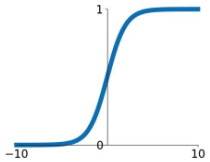
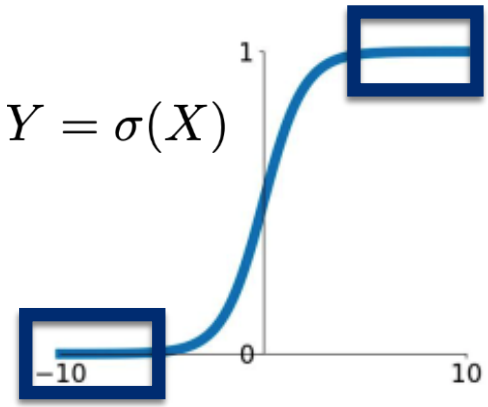


Figure from Stanford cs231n lecture slides

Recap on Gradient vanishing problem



Chain rule

$$\frac{\partial Y}{\partial X} = \frac{\partial Y}{\partial \sigma}\frac{\partial \sigma}{\partial X}$$

is almost 0 for most of X

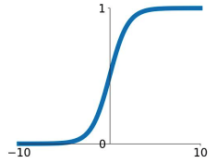
Thus, $\frac{\partial Y}{\partial X}$ also goes to 0.

Figure from Stanford cs231n lecture slides

Recap on Activation functions

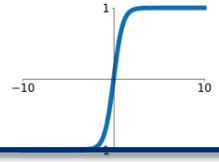
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



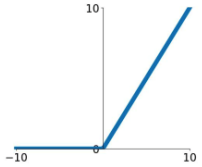
tanh

$$\tanh(x)$$



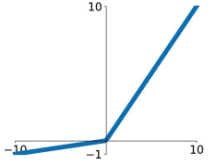
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

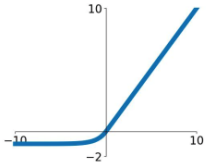


Figure from Stanford cs231n lecture slides

Recap on Activation functions

Prevent gradient vanishing when x > 0

Computationally efficient

Biologically plausible

(-) Still loose gradient when x < 0

ReLU

$$\max(0, x)$$

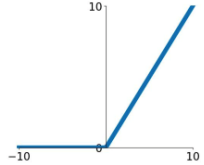
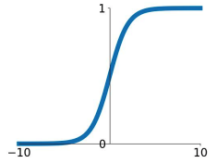


Figure from Stanford cs231n lecture slides

Recap on Activation functions

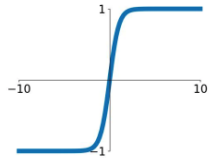
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



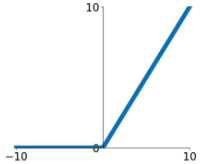
tanh

$$\tanh(x)$$



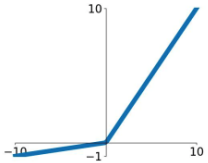
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

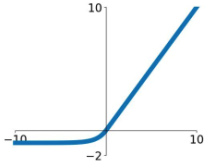
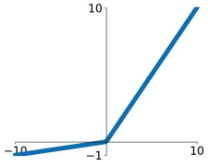


Figure from Stanford cs231n lecture slides

Leaky ReLU/ELU

Leaky ReLU

$$\max(\alpha x, x)$$



g. Leaky R

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

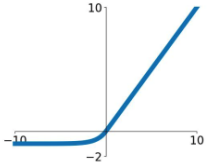
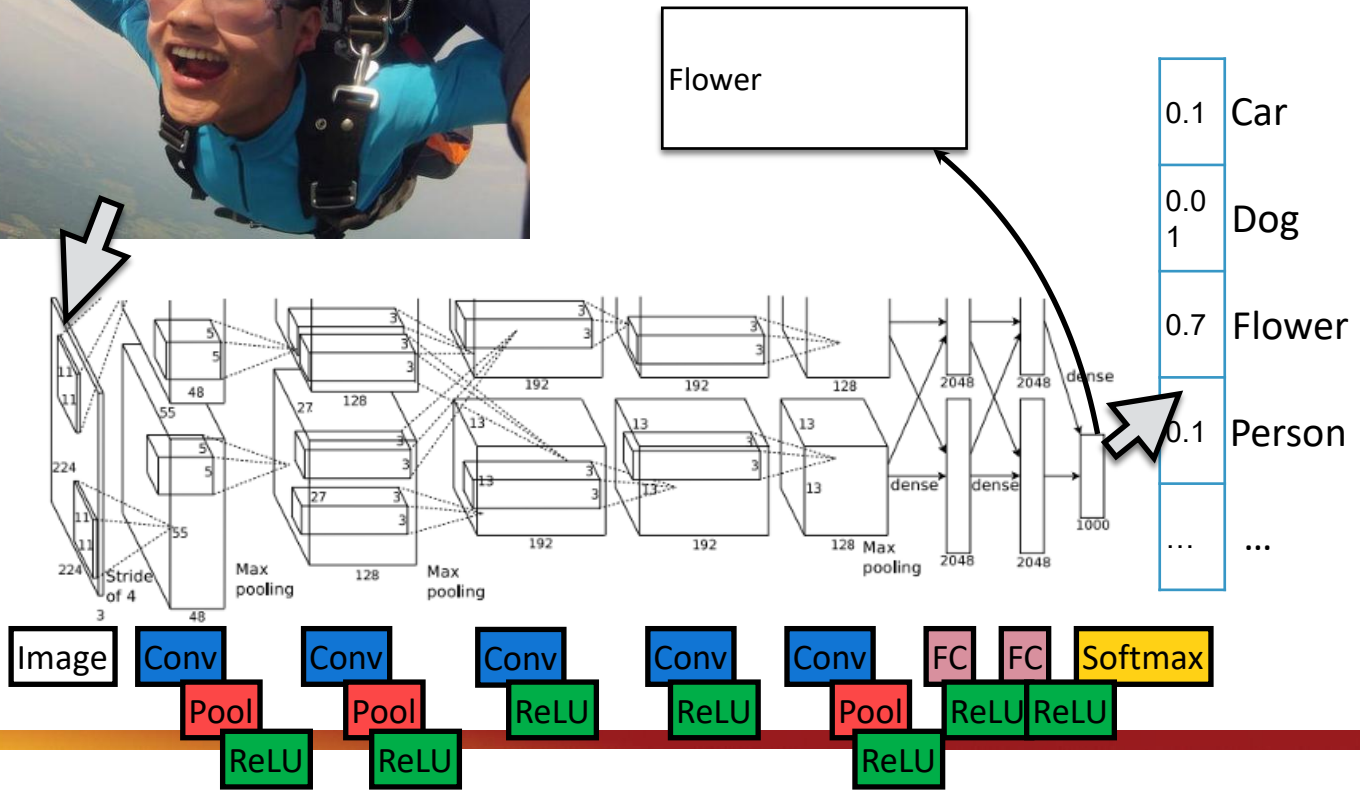


Figure from Stanford cs231n lecture slides

AlexNet in details



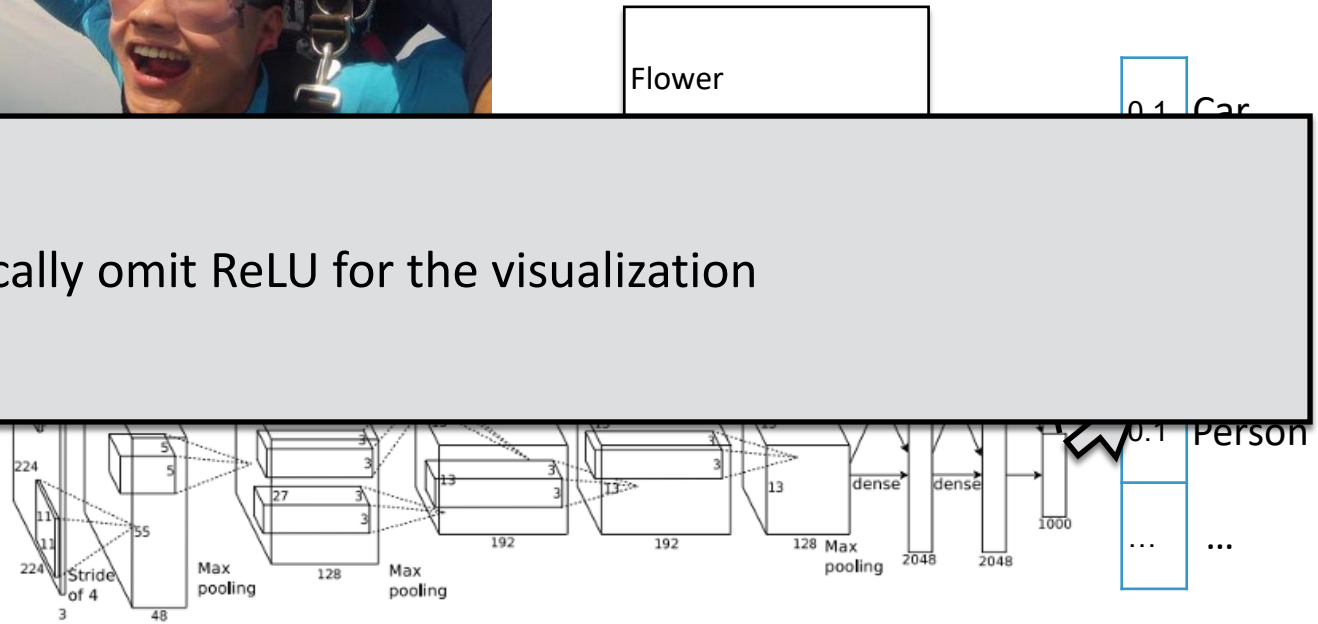


USC Viterbi School of Engineering

AlexNet in details



Typically omit ReLU for the visualization

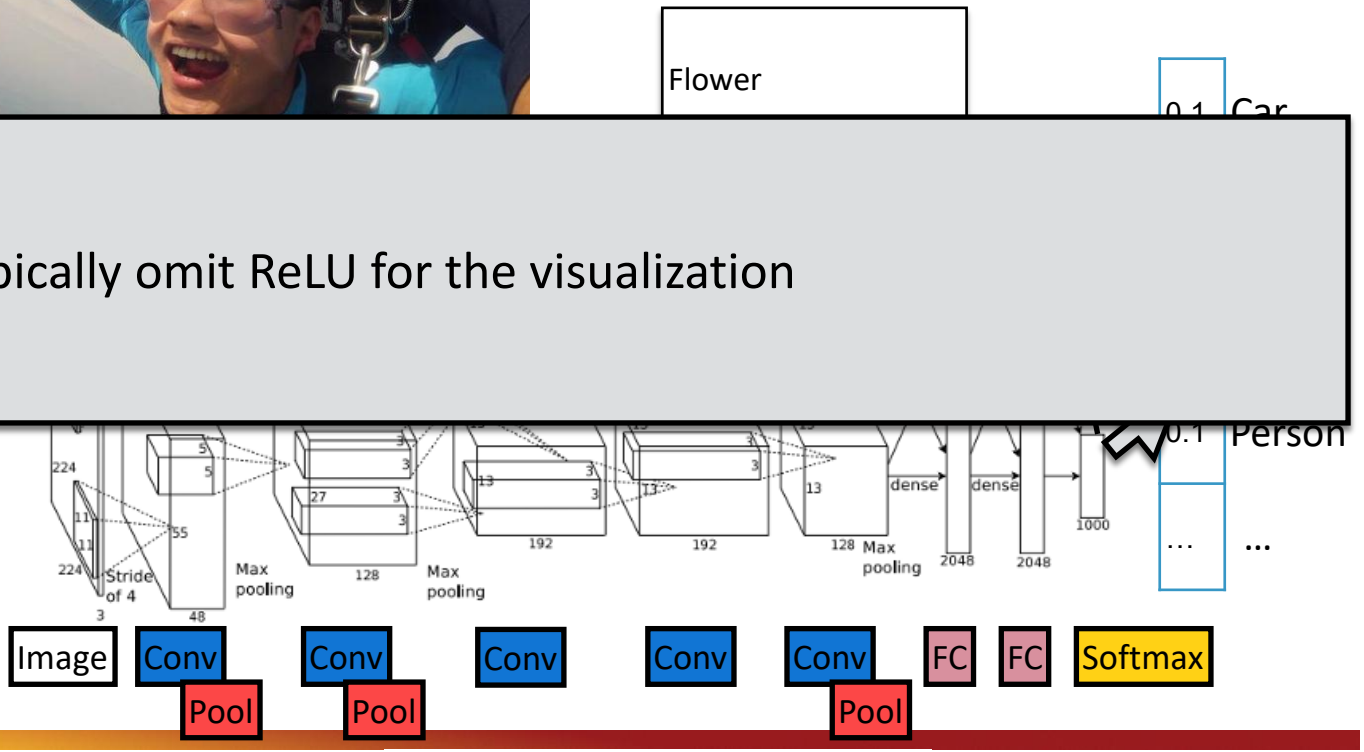


USC Viterbi School of Engineering

AlexNet in details

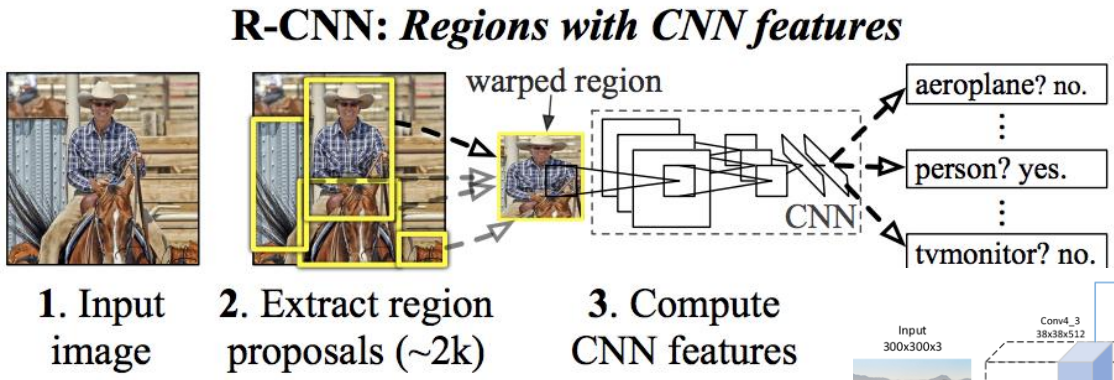


Typically omit ReLU for the visualization



USC Viterbi School of Engineering

More CNNs



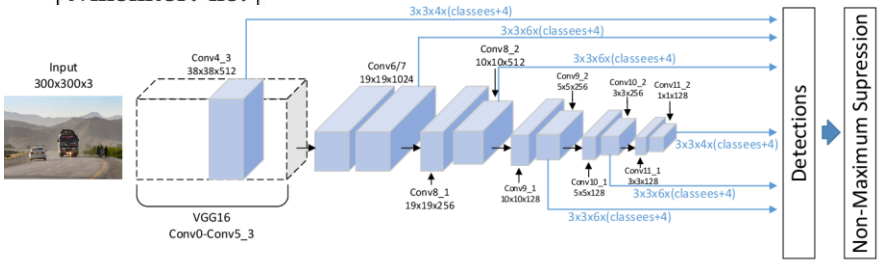
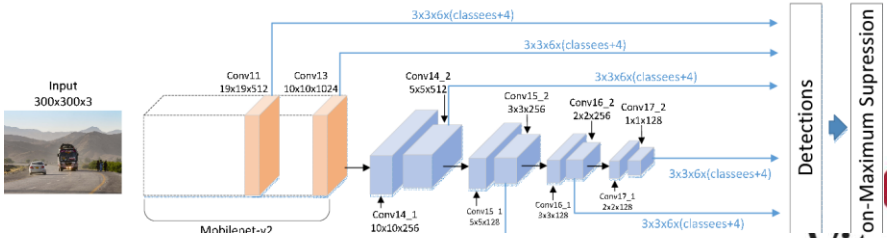


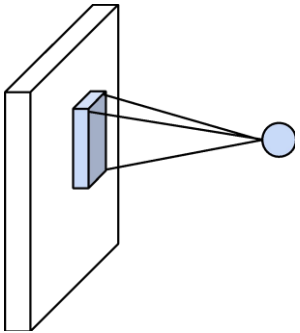
Fig. 1. Network architecture of VGG16-SSD.

RCNN, MobileNet, and many more

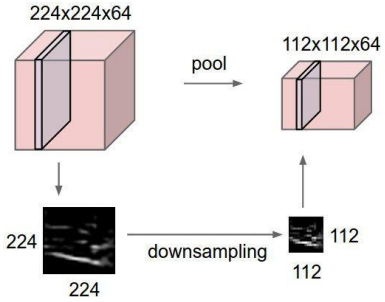


Components of CNNs

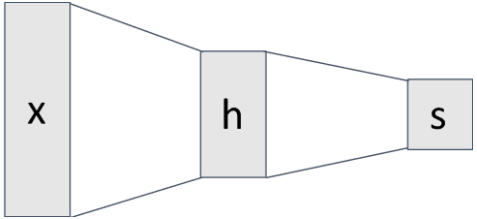
Convolution Layers



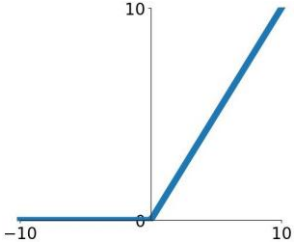
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Figure from Stanford cs231n lecture slides

Batch Normalization

Consider a single layer $y = Wx$

The following could lead to tough optimization:

- Inputs x are not *centered around zero* (need large bias)
- Inputs x have different scaling per-element (entries in W will need to vary a lot)

Idea: force inputs to be “nicely scaled” at each layer!

Batch Normalization

[Ioffe and Szegedy, 2015]

"you want zero-mean unit-variance activations? just make them so."

consider a batch of activations at some layer. To make each dimension zero-mean unit-variance, apply:

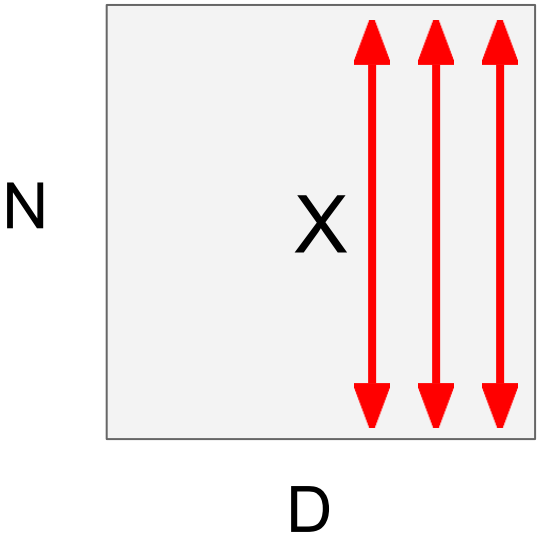
$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

this is a vanilla differentiable function...

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$

Per-channel var, shape is D

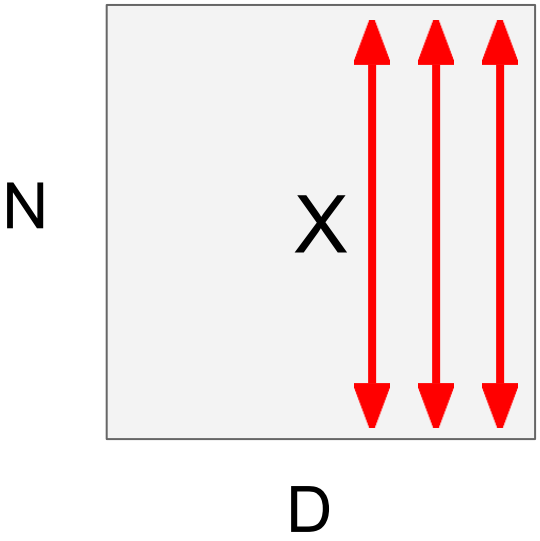
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$



$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$

Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

Problem: What if zero-mean, unit variance is too hard of a constraint?

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^{N} x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^{N} (x_{i,j} - \mu_j)^2$$

Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

Batch Normalization: Test-Time

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

During testing batchnorm becomes a linear operator!
Can be fused with the previous fully-connected or conv layer

$$\mu_j = \text{(Running) average of values seen during training}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \text{(Running) average of values seen during training}$$

Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x, Shape is N x D

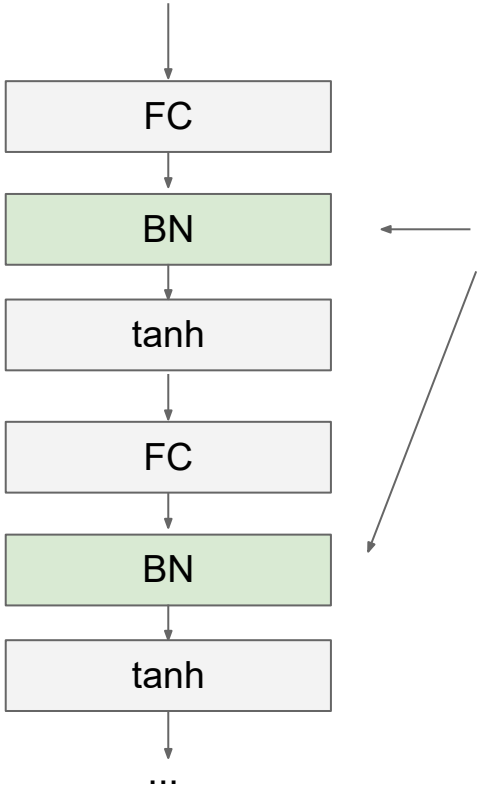
$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output, Shape is N x D

Figure from Stanford cs231n lecture slides

Batch Normalization

[Ioffe and Szegedy, 2015]

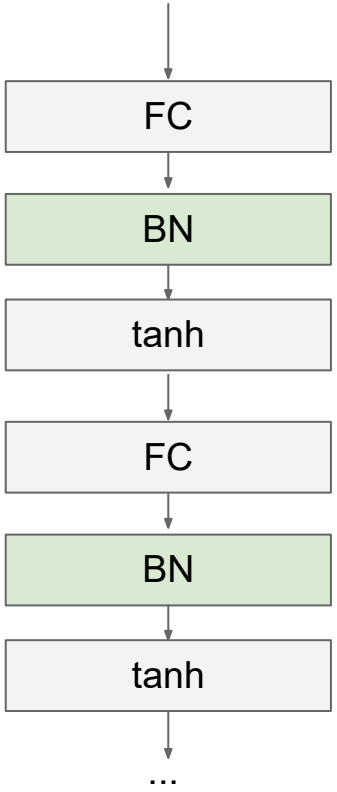


Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

Batch Normalization

[Ioffe and Szegedy, 2015]



- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!
- Behaves differently during training and testing: this is a very common source of bugs!

BatchNorm: One Layer, Two Functions

Train. bn.train() uses the current batch statistics and updates the running estimates.

Eval. bn.eval() uses the stored running estimates, so the same x can produce a different output.

```
bn = nn.BatchNorm1d(1); x = torch.tensor([[1.], [3.]])
bn.train(); print(bn(x).flatten())   # [-1.0, 1.0]
bn.eval();  print(bn(x).flatten())   # [0.7628, 2.6697]
```

Mode is part of the function. Call model.eval() before inference.

Small Batches Change BatchNorm

Batch of 2. x = [1, 3] gives mean 2 and biased variance 1, so the normalized values are [-1, +1] before scale and shift.

Batch of 1. x = [3] gives mean 3 and variance 0. PyTorch BatchNorm1d raises an error on a [1, C] input in training mode.

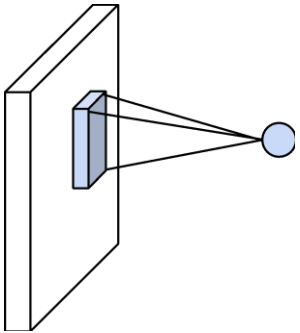
Scope. Ioffe and Szegedy (ICML 2015) report a regularizing effect in some settings, not as a universal rule.

Alternatives. Group Normalization (Wu and He, ECCV 2018) targets exactly this small-batch dependence.

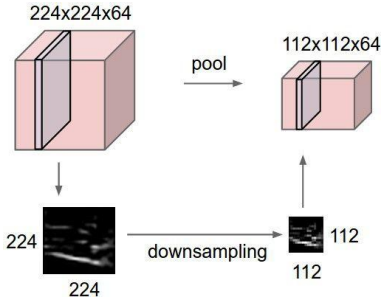
With small batches, who else is in the batch becomes part of the model.

Components of CNNs

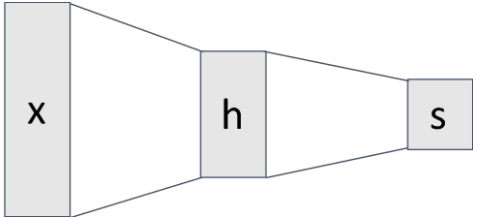
Convolution Layers



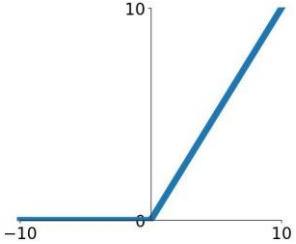
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

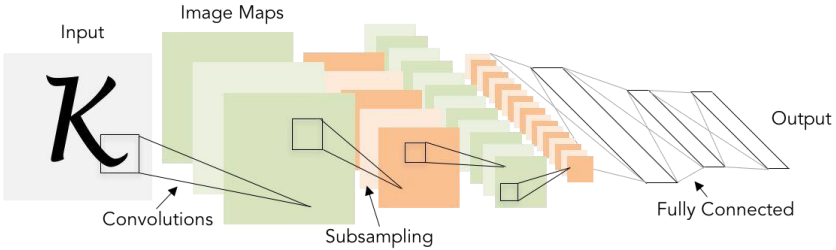
Question: How should we put them together?

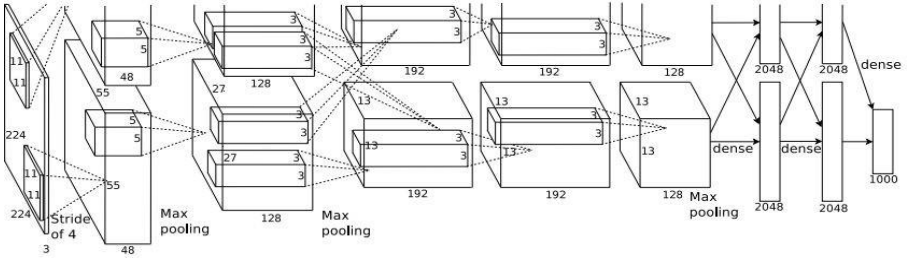
Figure from Stanford cs231n lecture slides

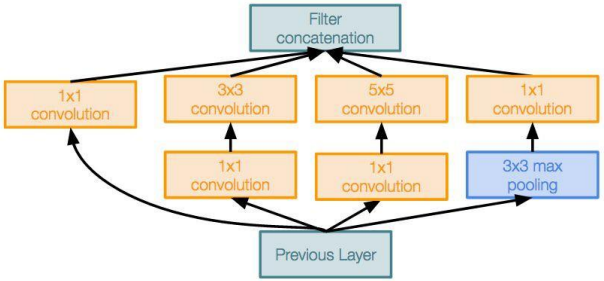
Convolutional Neural Networks (CNN)

CNN Architectures & Applications

Today: CNN Architectures







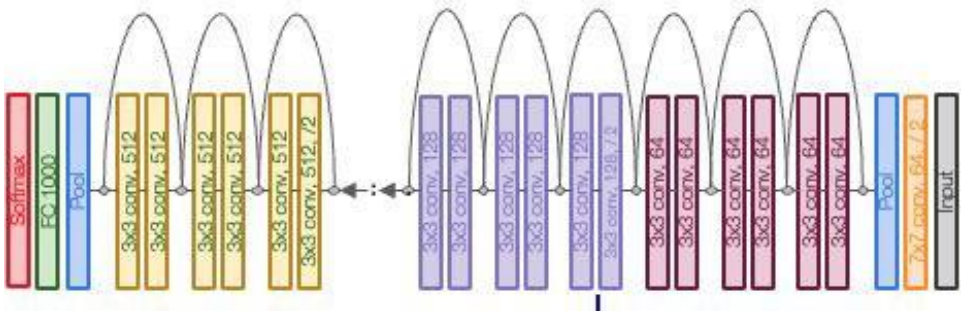


Figure from Stanford cs231n lecture slides

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

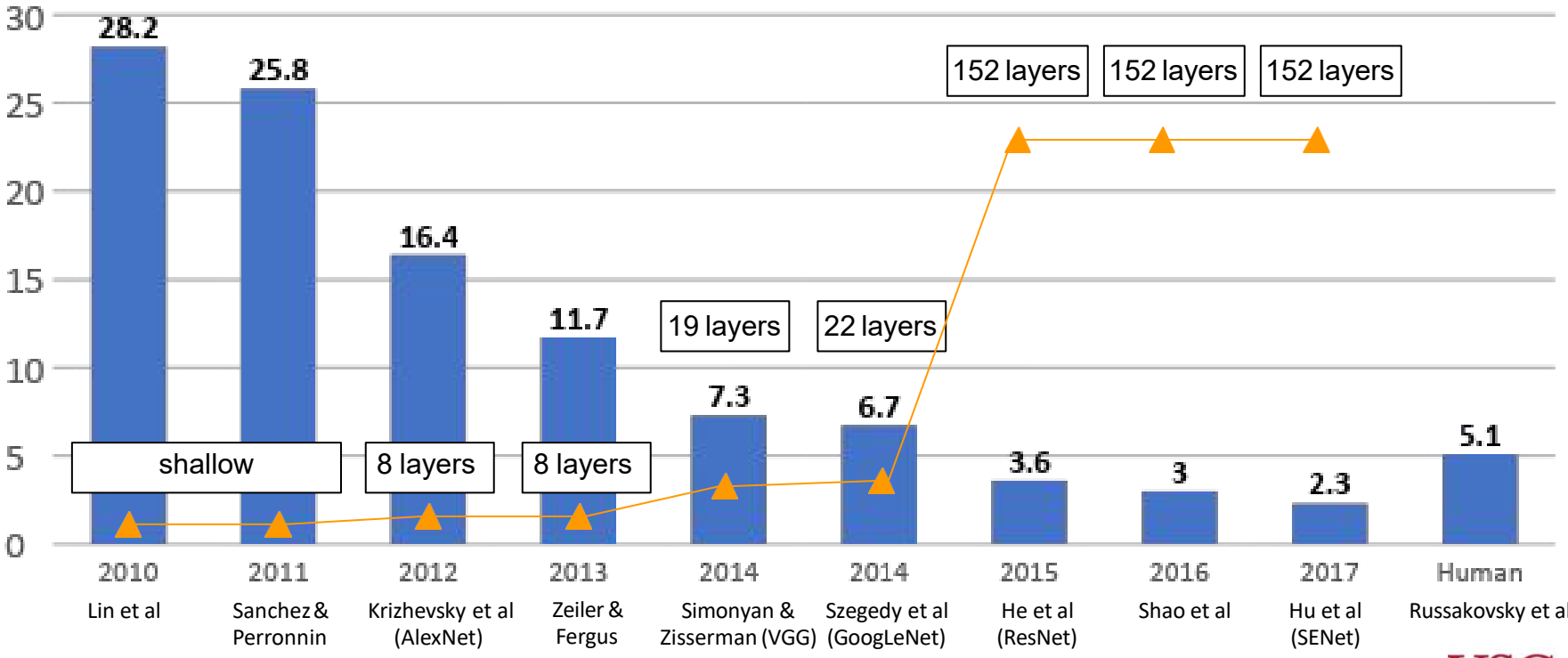


Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

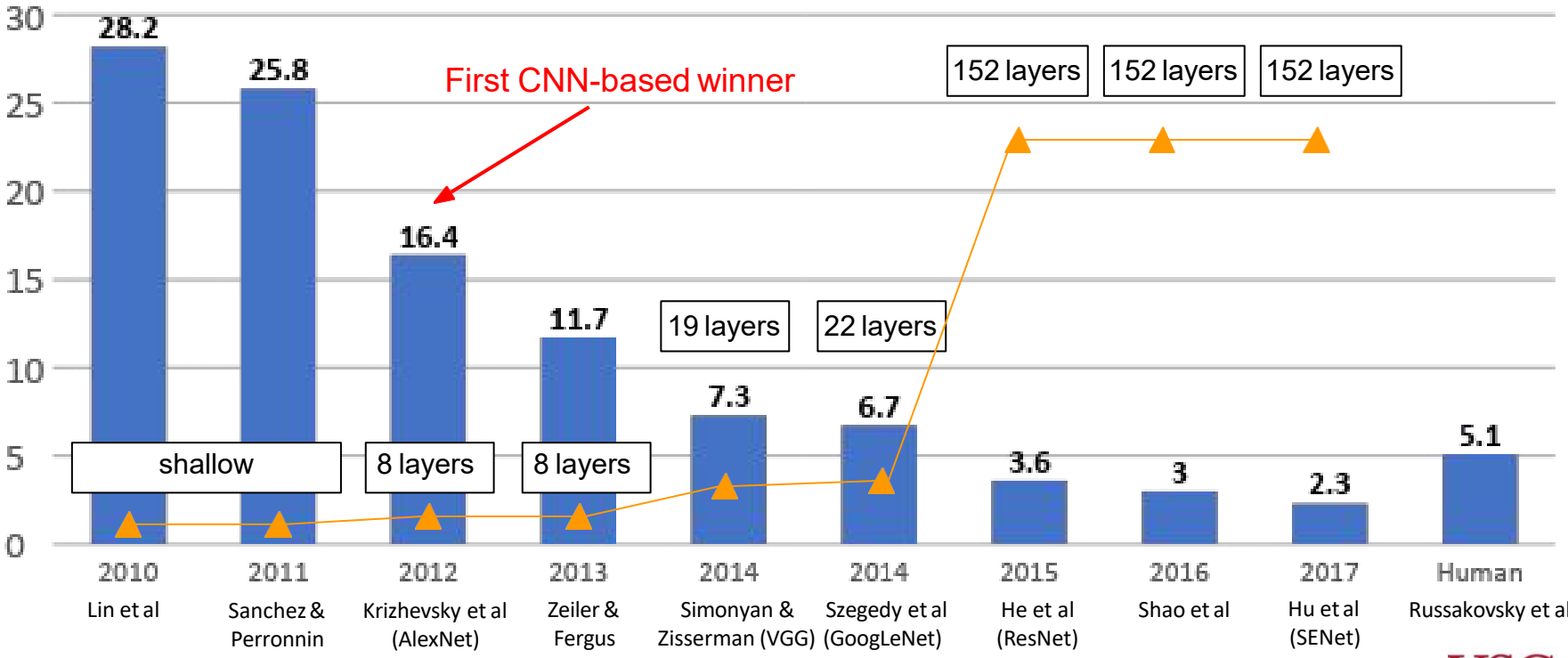


Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

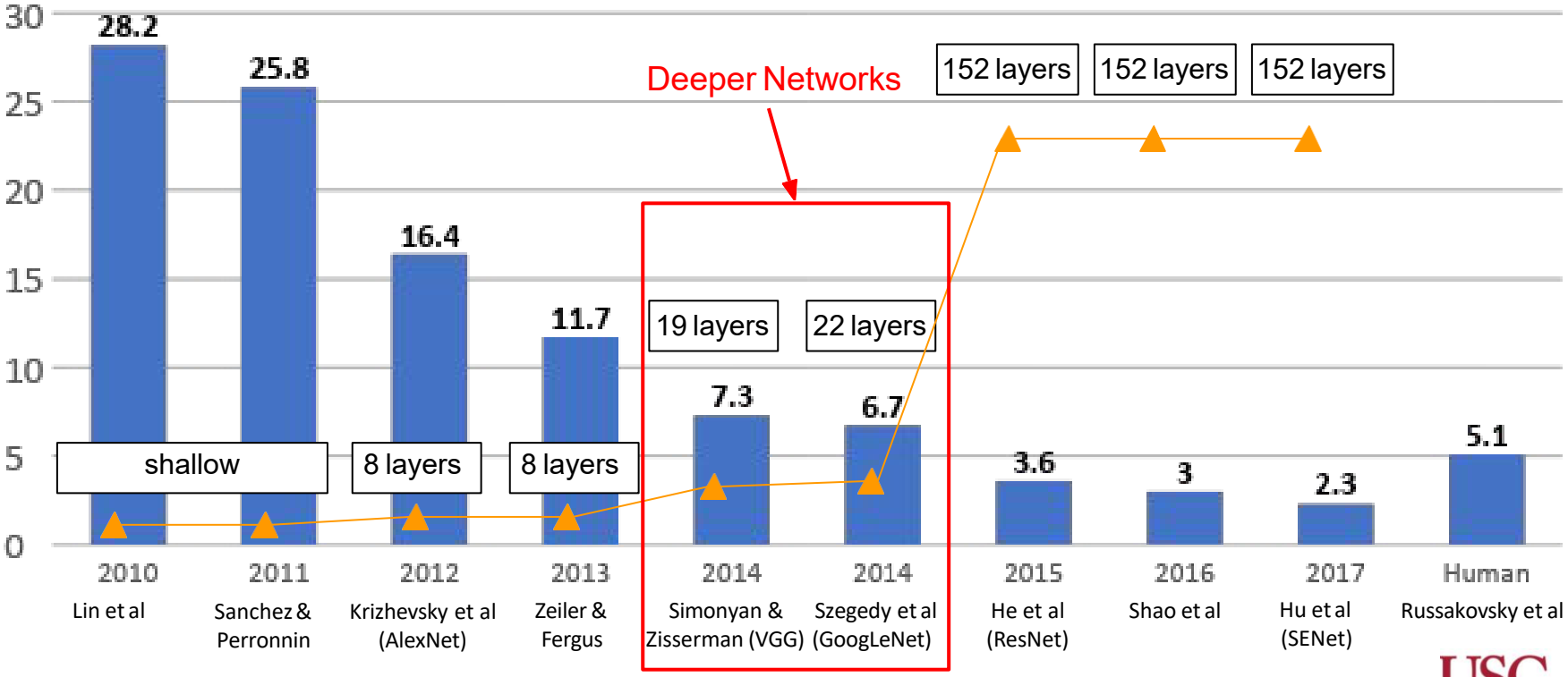


Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Small filters, Deeper networks

8 layers (AlexNet)
-> 16 - 19 layers (VGG16Net)

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

11.7% top 5 error in ILSVRC'13 (ZFNet)
-> 7.3% top 5 error in ILSVRC'14

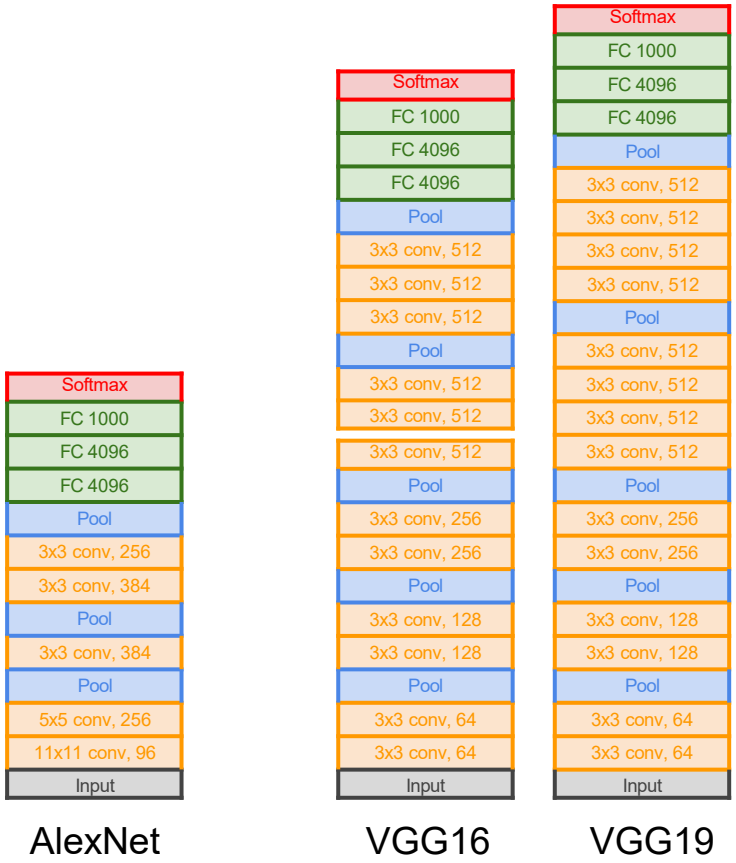


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

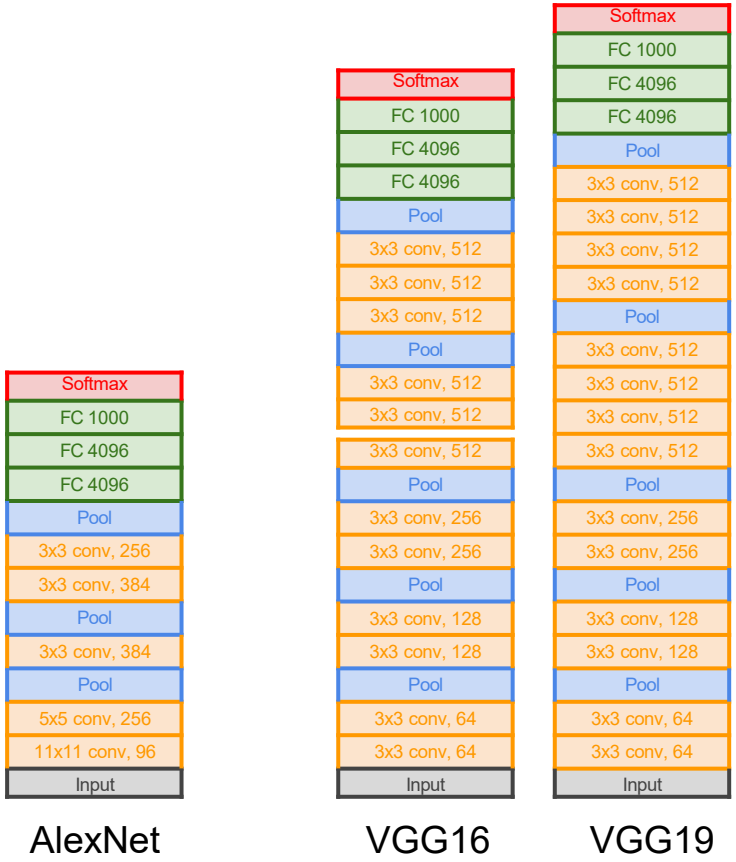


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?

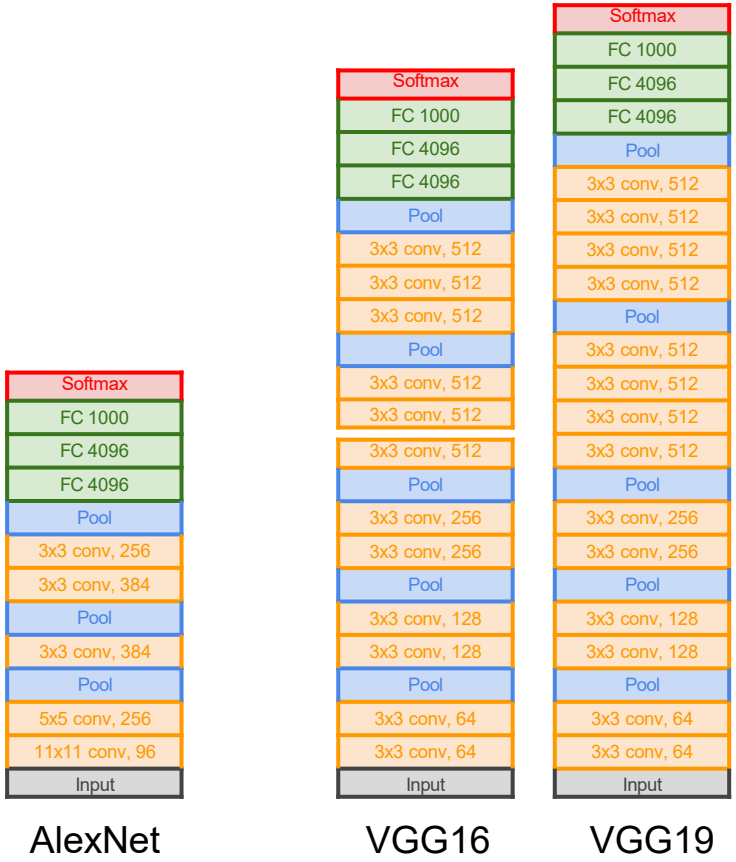
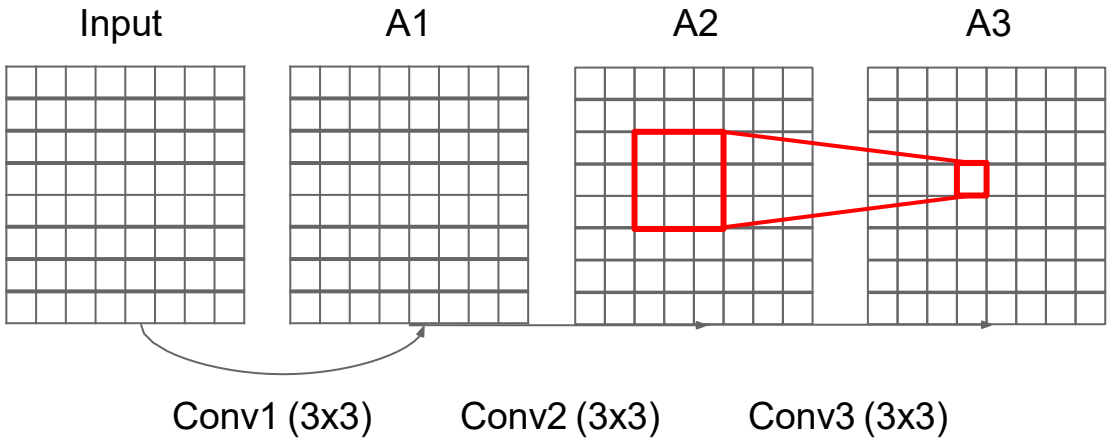


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



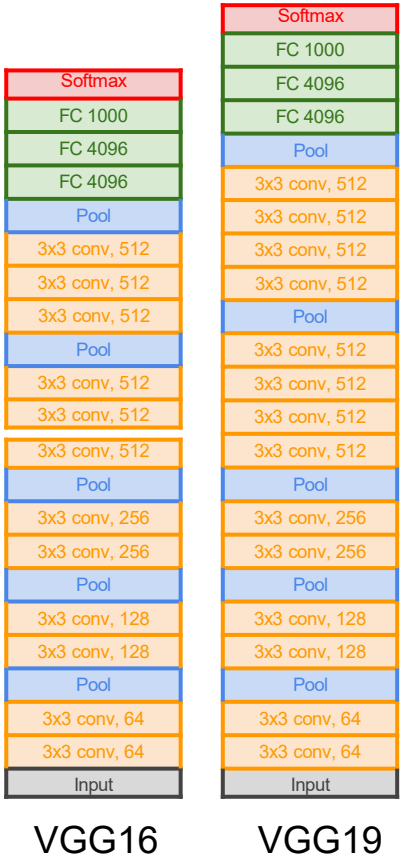
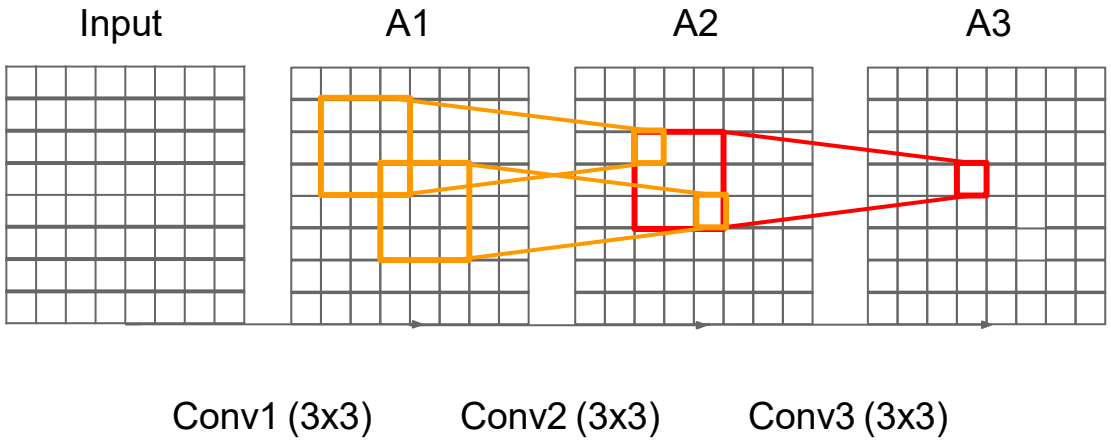


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



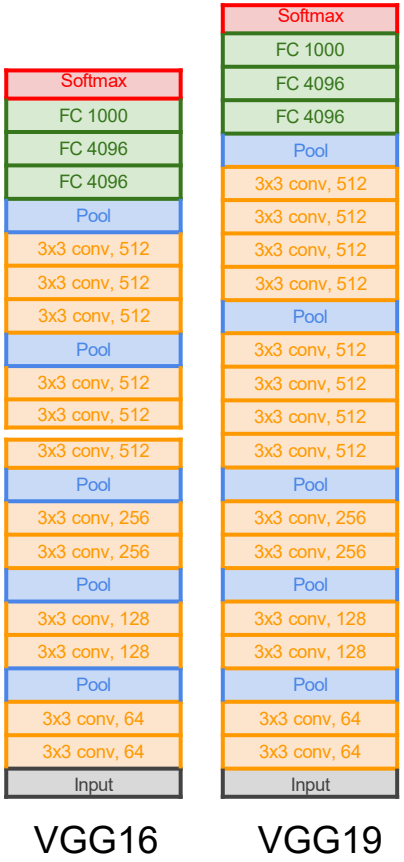


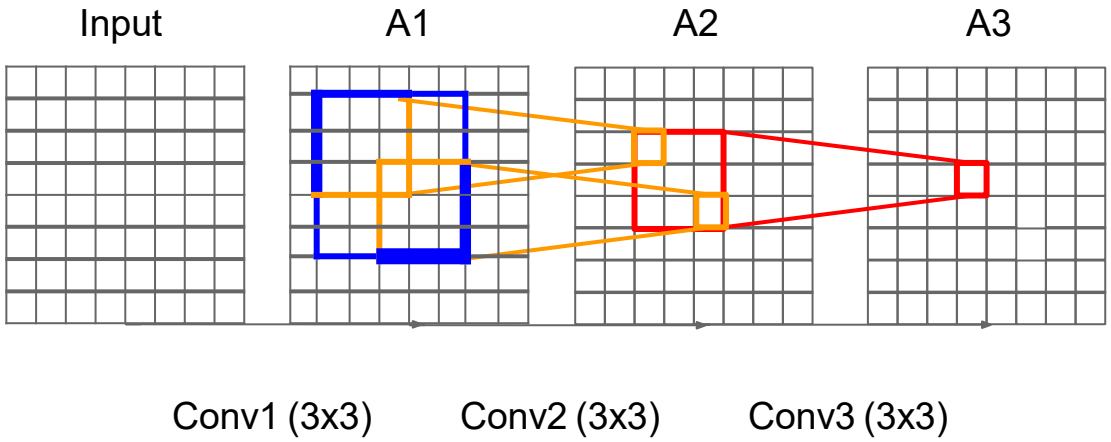
Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



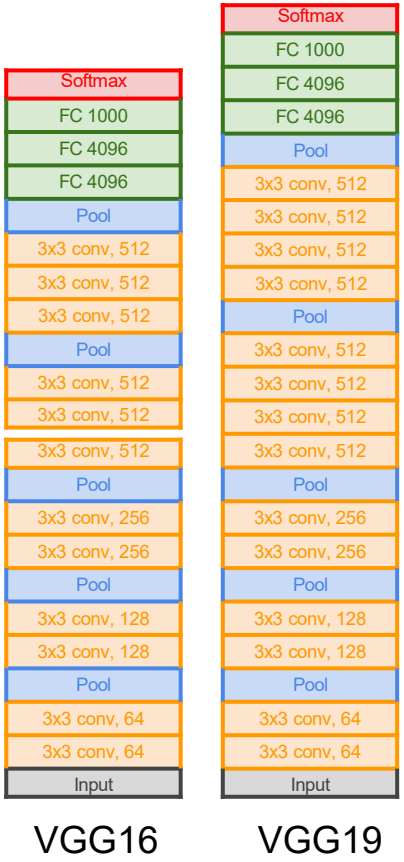
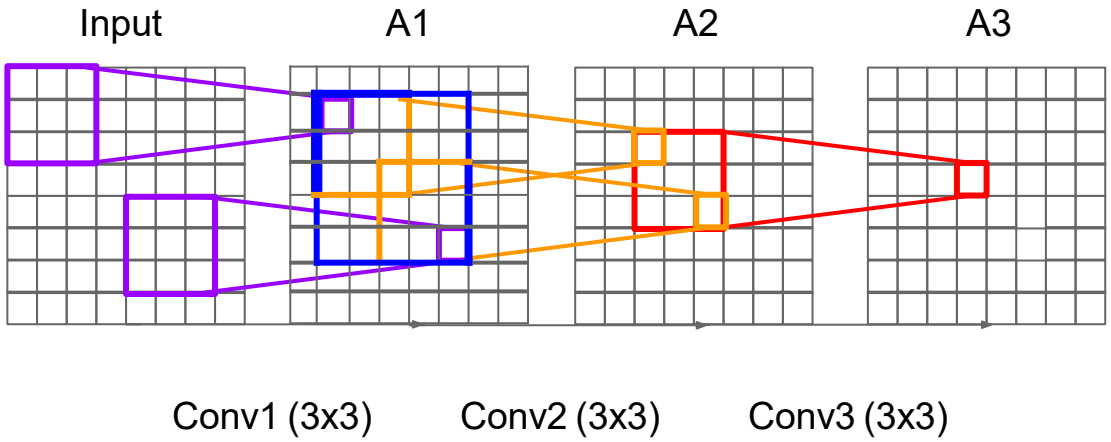


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



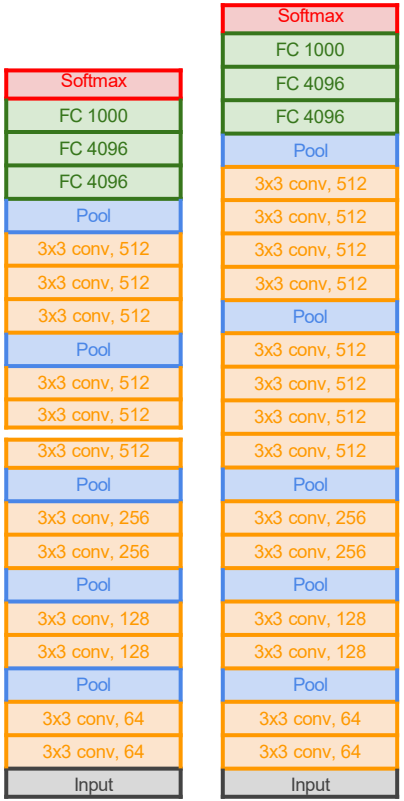
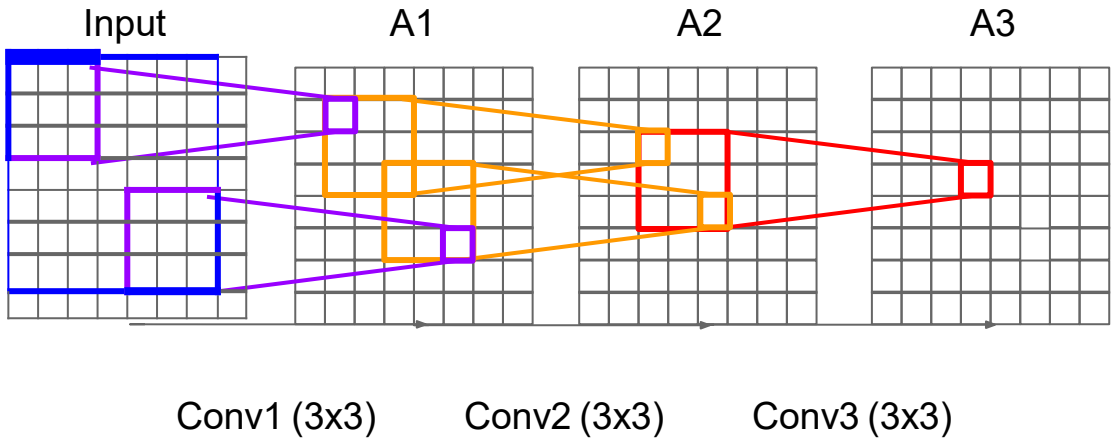


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



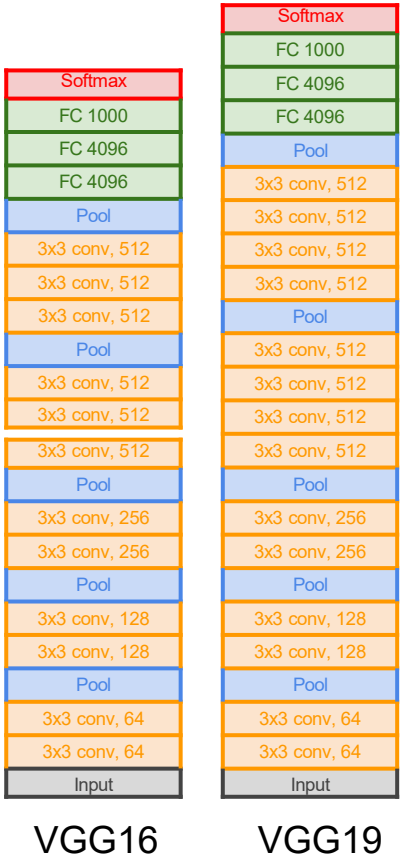


Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

[7x7]

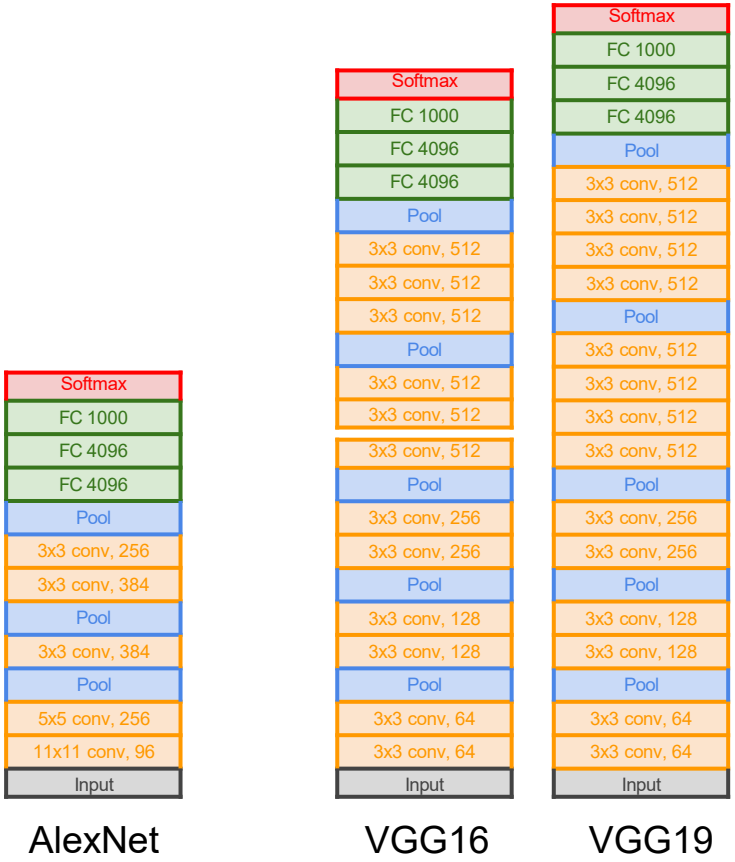


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

But deeper, more non-linearities

And **fewer parameters**: 3 * (3^2C^2) vs. 7^2C^2 for C channels per layer & **better convergence**.

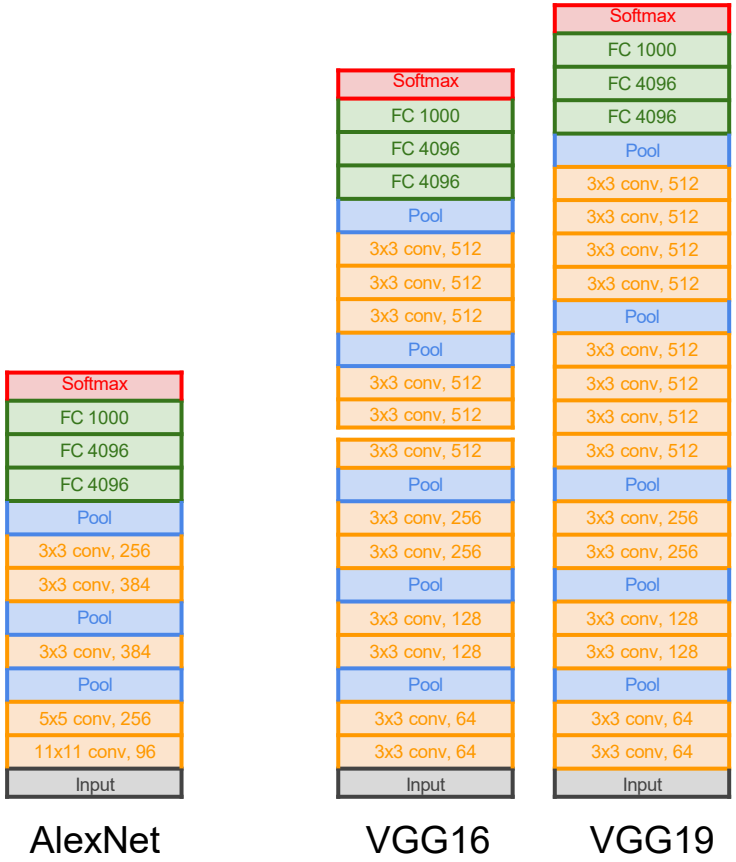


Figure from Stanford cs231n lecture slides

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

Deeper Hierarchical Feature Learning: Smaller filters allow the network to learn a hierarchy of features, from simple edges in the first layer to more complex patterns in the deeper layers. This hierarchical structure can be more effective for learning[2] [2] representations of complex objects.

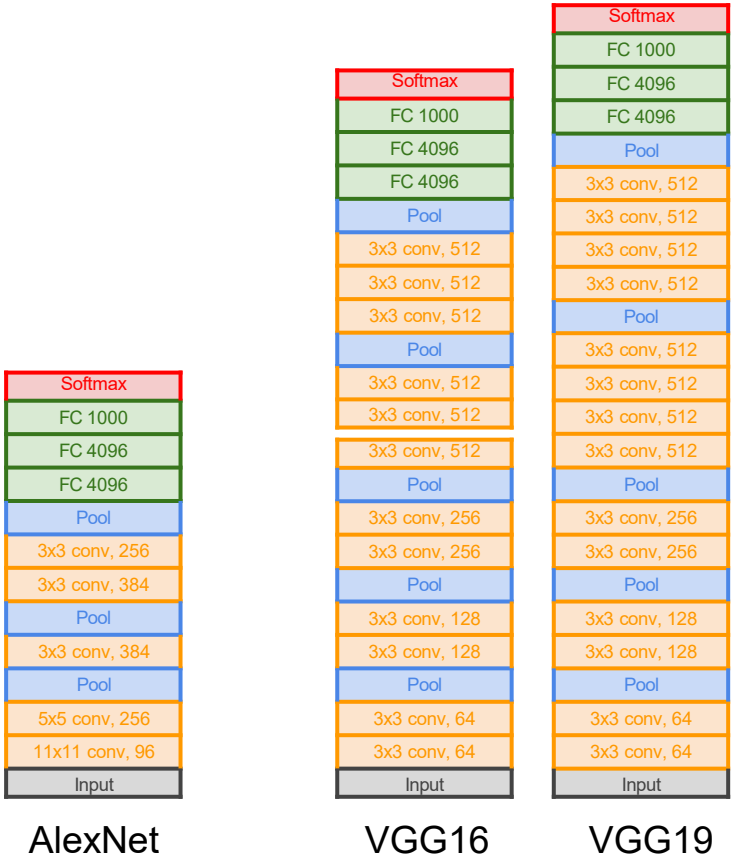
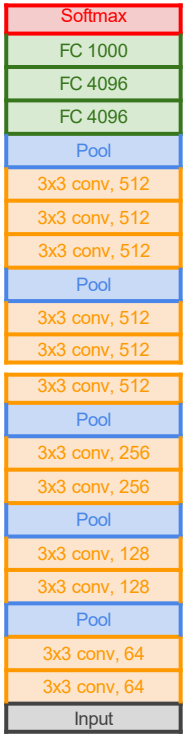


Figure from Stanford cs231n lecture slides

INPUT: [224x224x3] memory: 224*224*3=150K params: 0 (not counting biases)
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

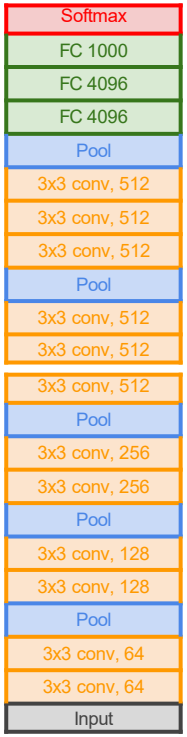


VGG16

Figure from Stanford cs231n lecture slides

INPUT: [224x224x3] memory: 224*224*3=150K params: 0 (not counting biases)
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

TOTAL memory: 24M * 4 bytes ~= 96MB / image (for a forward pass)
TOTAL params: 138M parameters



VGG16

Figure from Stanford cs231n lecture slides

INPUT: [224x224x3] memory: 224*224*3=150K params: 0 (not counting biases)
CONV3-64: [224x224x64] memory: **224*224*64=3.2M** params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: **224*224*64=3.2M** params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = **102,760,448**
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

Note:

Most memory is in early CONV

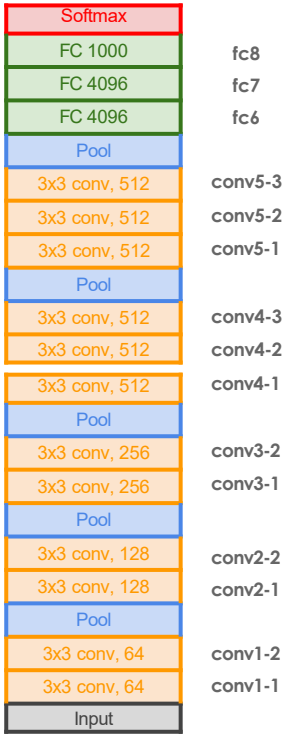
Most params are in late FC

TOTAL memory: 24M * 4 bytes ~= 96MB / image (only forward! ~*2 for bwd)
TOTAL params: 138M parameters

Figure from Stanford cs231n lecture slides

INPUT: [224x224x3] memory: 224*224*3=150K params: 0 (not counting biases)
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
POOL2: [112x112x64] memory: 112*112*64=800K params: 0
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
POOL2: [56x56x128] memory: 56*56*128=400K params: 0
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
POOL2: [28x28x256] memory: 28*28*256=200K params: 0
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
POOL2: [14x14x512] memory: 14*14*512=100K params: 0
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
POOL2: [7x7x512] memory: 7*7*512=25K params: 0
FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

TOTAL memory: 24M * 4 bytes ~= 96MB / image (only forward! ~*2 for bwd)
TOTAL params: 138M parameters



VGG16

Common names

Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Details:

- ILSVRC'14 2nd in classification, 1st in localization
- Similar training procedure as Krizhevsky 2012
- No Local Response Normalisation (LRN)
- Use VGG16 or VGG19 (VGG19 only slightly better, more memory)
- Use ensembles for best results
- FC7 features generalize well to other tasks

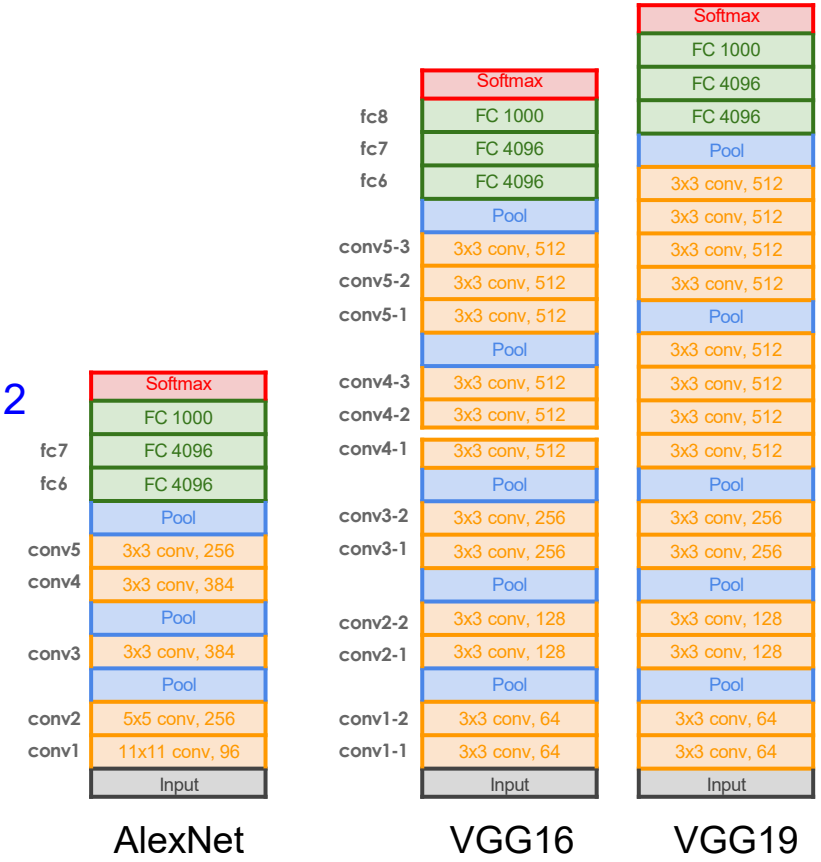


Figure from Stanford cs231n lecture slides

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

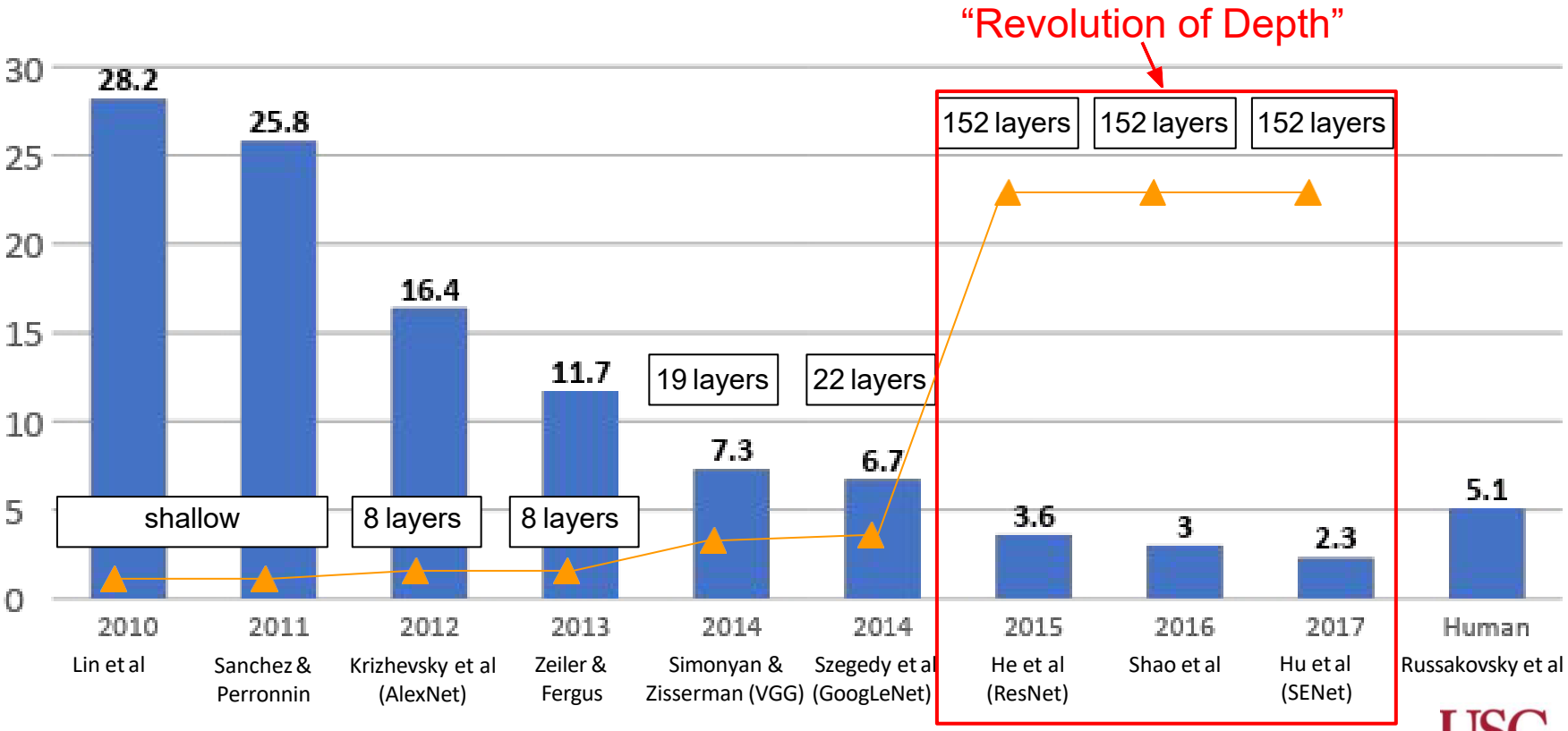


Figure from Stanford cs231n lecture slides

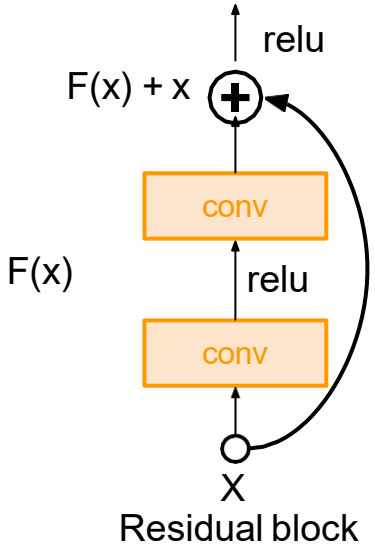
USC Viterbi School of Engineering

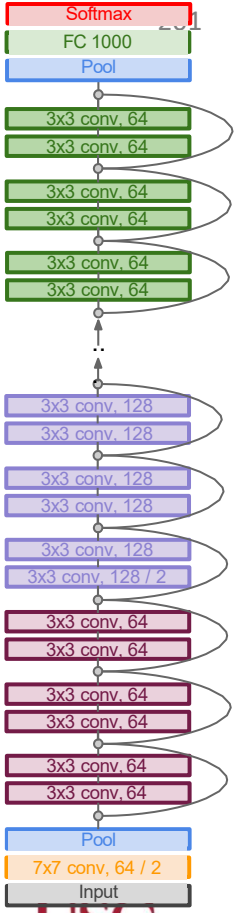
Case Study: ResNet

[He et al., 2015]

Very deep networks using residual connections

- 152-layer model for ImageNet
- ILSVRC'15 classification winner (3.57% top 5 error)
- Swept all classification and detection competitions in ILSVRC'15 and COCO'15!





Case Study: ResNet

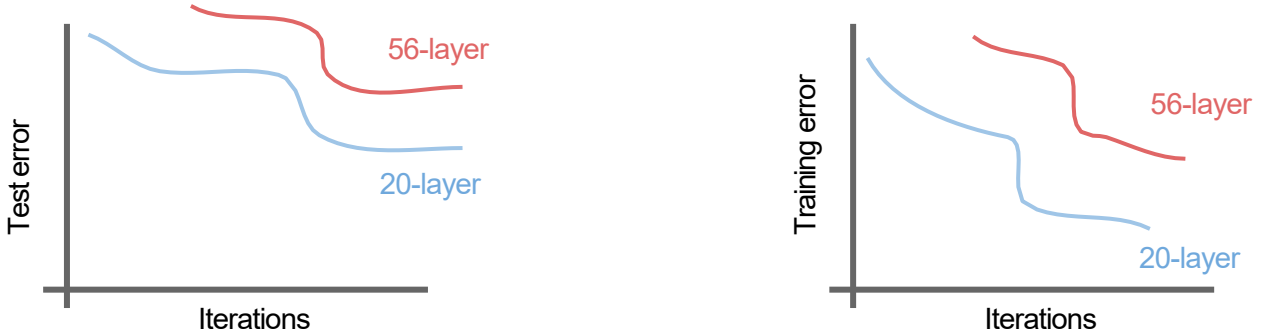
[He et al., 2015]

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

Case Study: ResNet

[He et al., 2015]

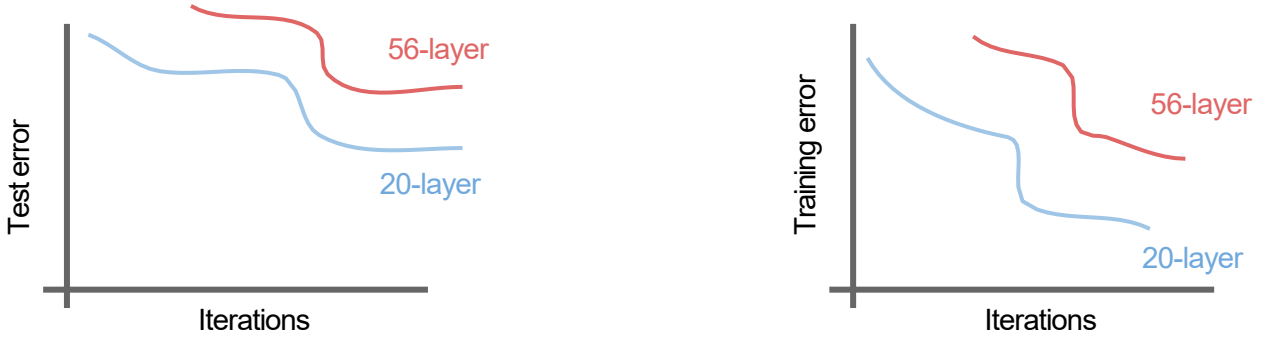
What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



Case Study: ResNet

[He et al., 2015]

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



56-layer model performs worse on both test and training error
-> The deeper model performs worse, but it’s not caused by overfitting!

Case Study: ResNet

[He et al., 2015]

Fact: Deep models have more representation power (more parameters) than shallower models.

Hypothesis: the problem is an *optimization* problem, **deeper models are harder to optimize**

Case Study: ResNet

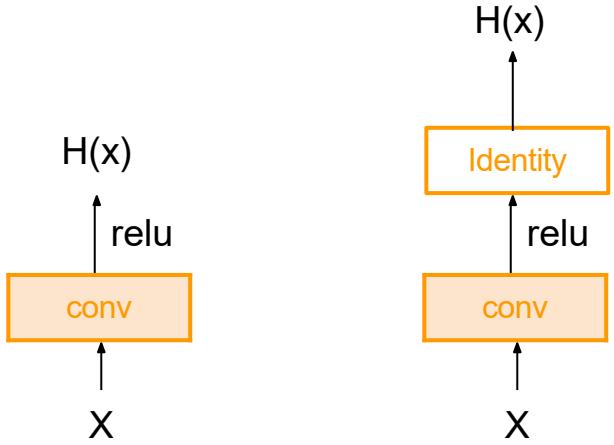
[He et al., 2015]

Fact: Deep models have more representation power (more parameters) than shallower models.

Hypothesis: the problem is an *optimization* problem, deeper models are harder to optimize

What should the deeper model learn to be at least as good as the shallower model?

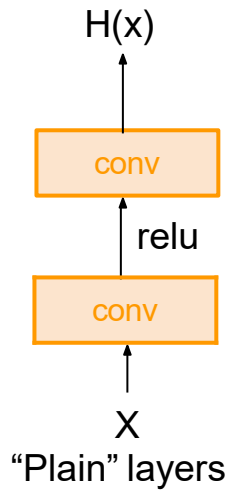
A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.



Case Study: ResNet

[He et al., 2015]

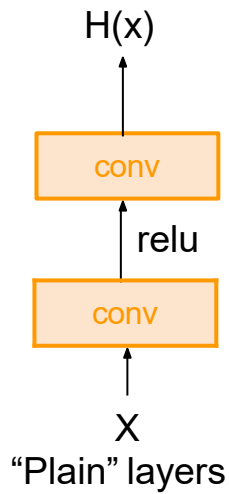
Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

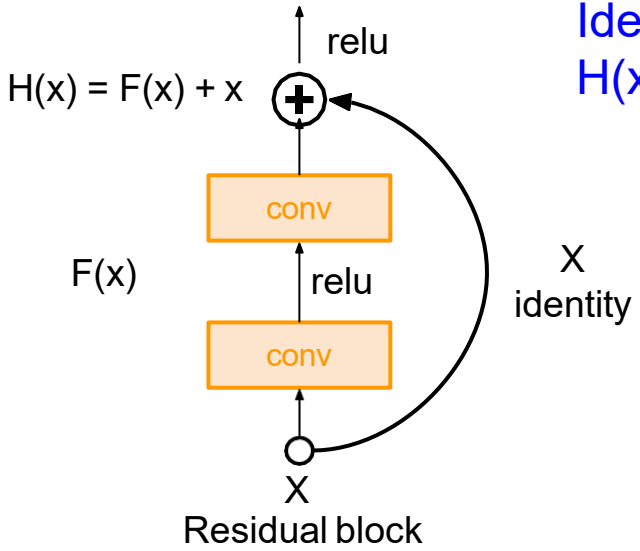


Case Study: ResNet

[He et al., 2015]

Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



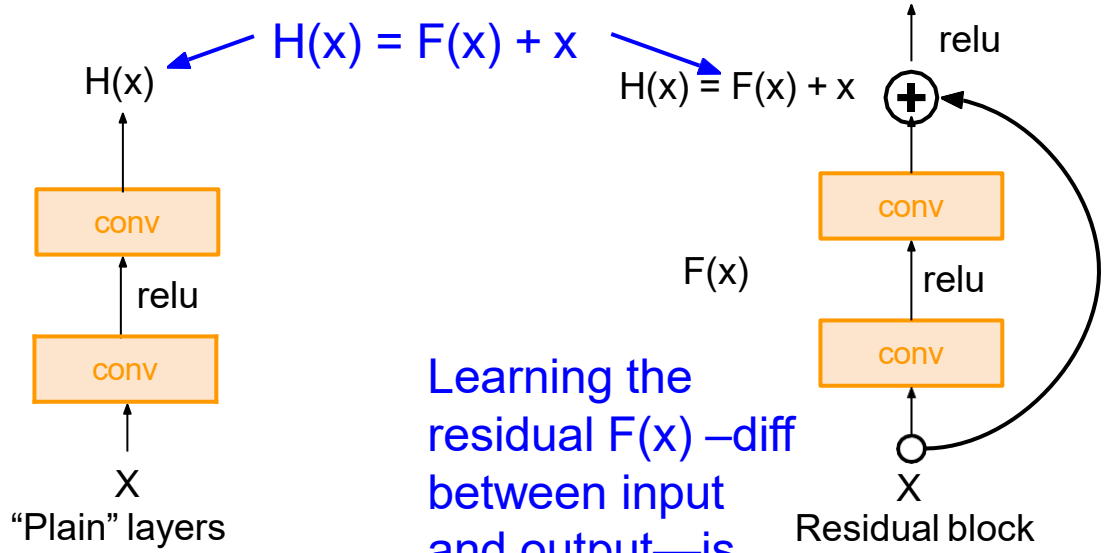


Identity mapping:
H(x) = x if F(x) = 0

Case Study: ResNet

[He et al., 2015]

Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



Identity mapping:
H(x) = x if F(x) = 0

Use layers to fit **residual** F(x) = H(x) - x instead of H(x) directly

Learning the residual F(x) –diff between input and output—is easier than H(x)

Residual Jacobians: A Tiny Example

Block. For y = x + f(x), the Jacobian is J_y = I + J_f. In one dimension, y' = 1 + f'.

Numbers. Take three scalar blocks with f' = 0.10, 0.20, 0.10.

Residual. (1.10)(1.20)(1.10) = 1.452.

Plain. Without the identity path, (0.10)(0.20)(0.10) = 0.002.

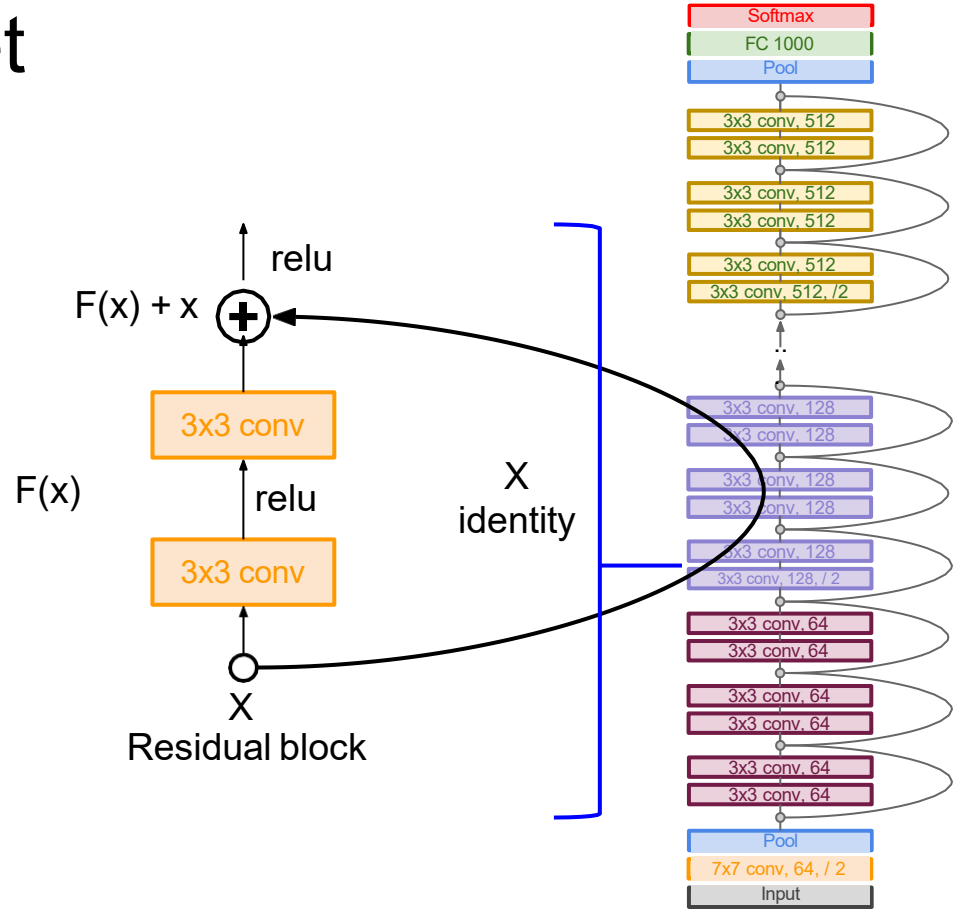
The expanded product contains an all-identity term, so one path escapes repeated shrinking. This is not a non-vanishing guarantee.

Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers

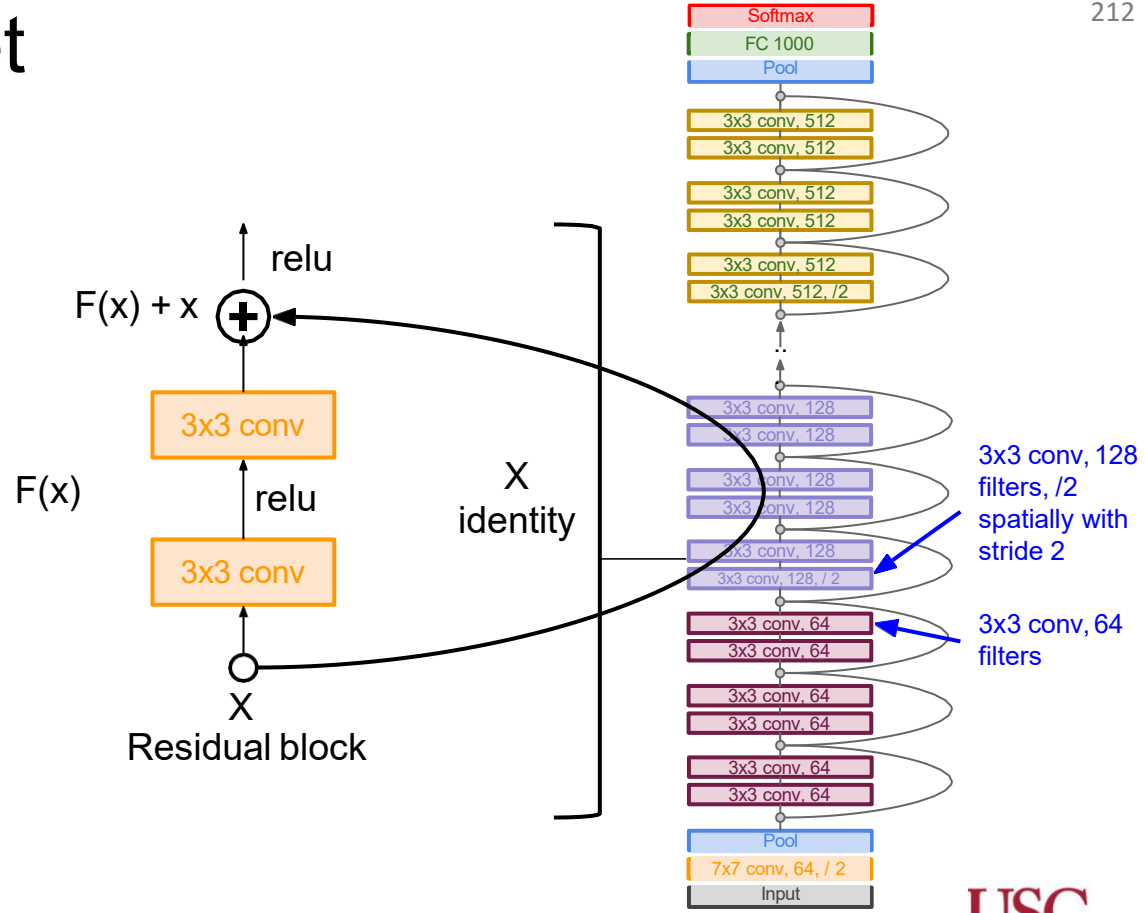


Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension) Reduce the activation volume by half.

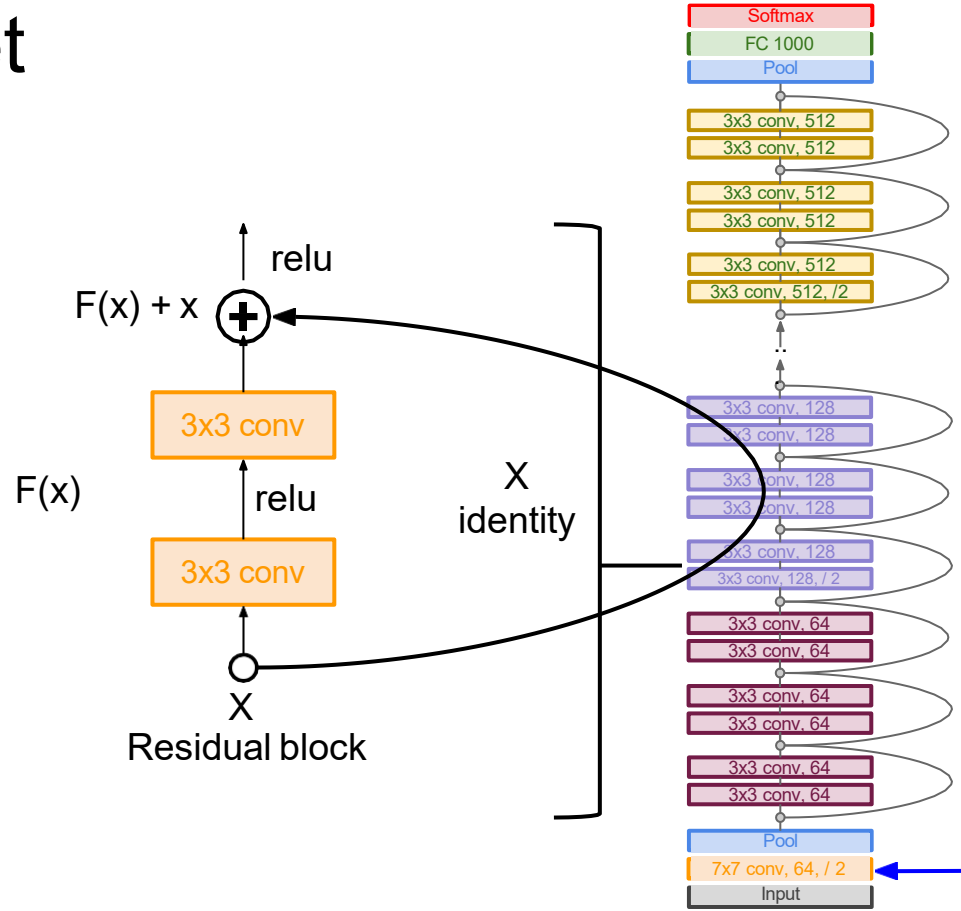


Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning (stem)

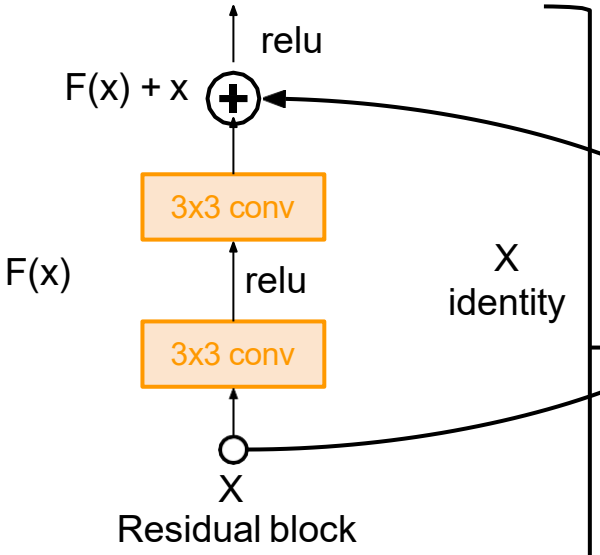


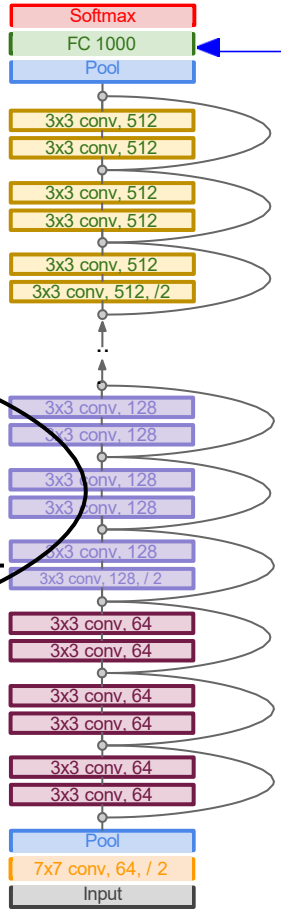
Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning (stem)
- No FC layers at the end (only FC 1000 to output classes)
- (In theory, you can train a ResNet with input image of variable sizes)





No FC layers besides FC 1000 to output classes

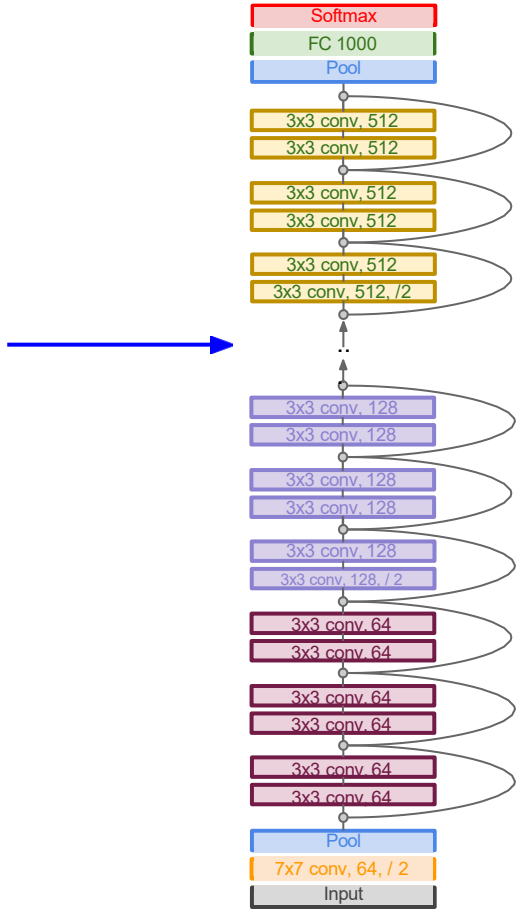
Global average pooling layer after last conv layer

USC Viterbi School of Engineering

Case Study: ResNet

[He et al., 2015]

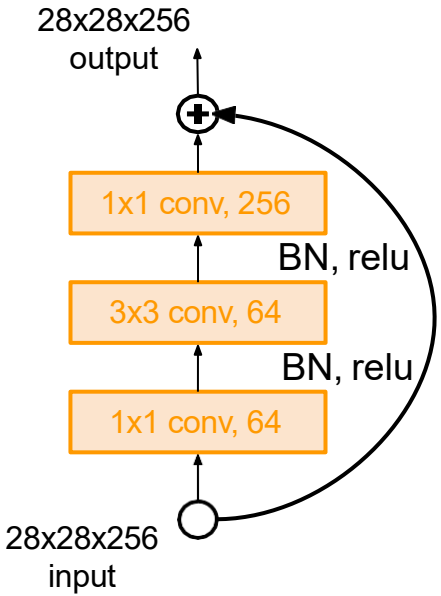
Total depths of 18, 34, 50, 101, or 152 layers for ImageNet



Case Study: ResNet

[He et al., 2015]

For deeper networks (ResNet-50+), use “bottleneck” layer to improve efficiency



Case Study: ResNet

[He et al., 2015]

For deeper networks (ResNet-50+), use "bottleneck" layer to improve efficiency

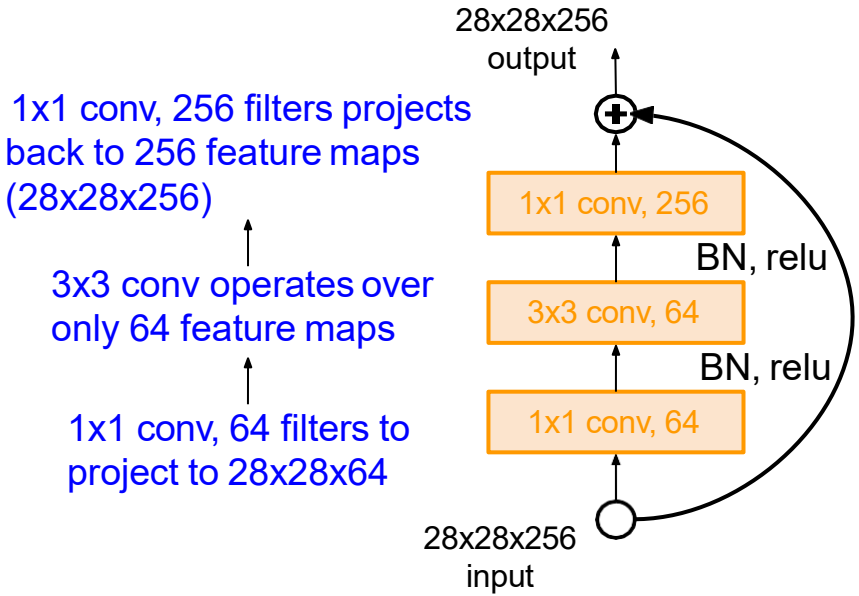


Figure from Stanford cs231n lecture slides

Case Study: ResNet

[He et al., 2015]

Training ResNet in practice (we will cover how to train later!):

- Batch Normalization (**briefly mentioned**) after every CONV layer
- He (Kaiming) initialization
- SGD + Momentum (0.9)
- Learning rate: 0.1, divided by 10 when validation error plateaus
- Mini-batch size 256
- Weight decay of 1e-4
- No dropout used

Case Study: ResNet

[He et al., 2015]

Experimental Results

- Able to train very deep networks without degrading (152 layers on ImageNet, 1202 on Cifar)
- Deeper networks now achieve lower training error as expected
- Swept 1st place in all ILSVRC and COCO 2015 competitions



Figure from Stanford cs231n lecture slides

Case Study: ResNet

[He et al., 2015]

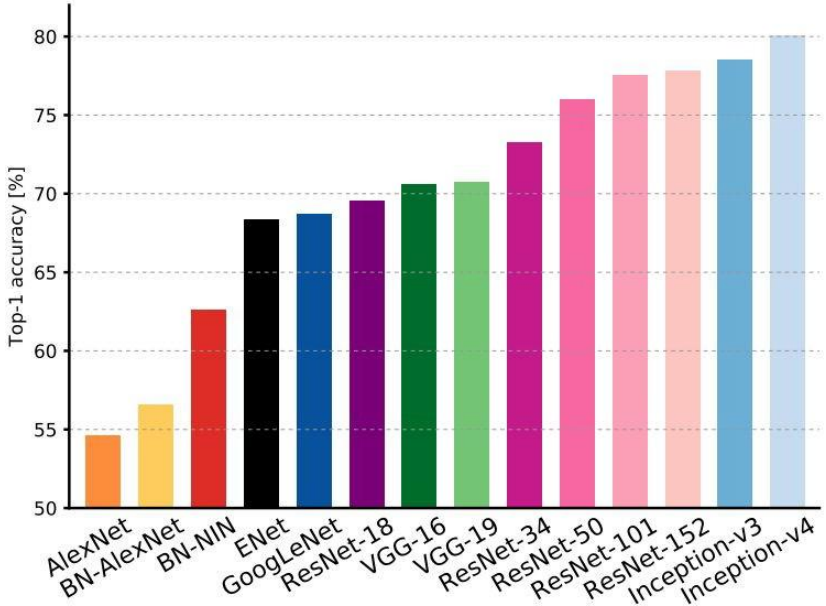
Experimental Results

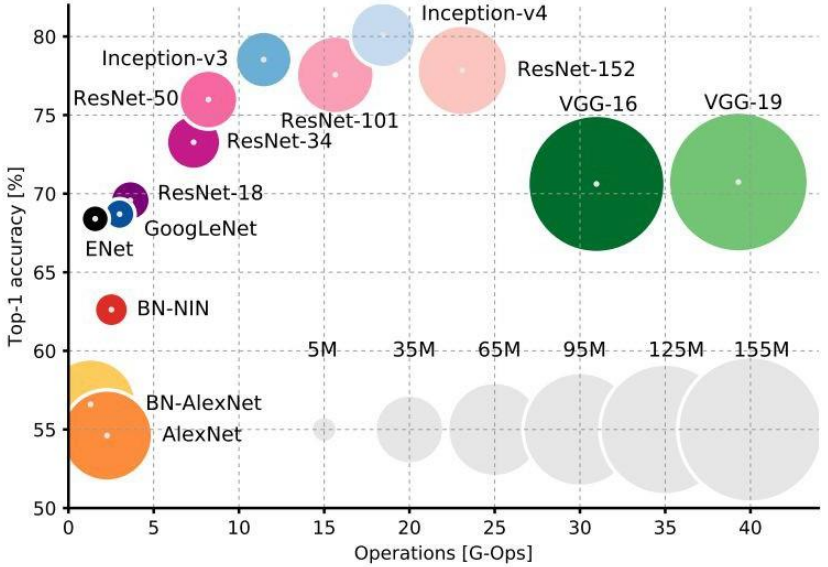
- Able to train very deep networks without degrading (152 layers on ImageNet, 1202 on Cifar)
- Deeper networks now achieve lower training error as expected
- Swept 1st place in all ILSVRC and COCO 2015 competitions



ILSVRC 2015 classification winner (3.6% top 5 error) -- better than the 5.1% top-5 error of one trained human annotator on 1500 images (Russakovsky 2014)

Comparing complexity...





An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Figures copyright Alfredo Canziani, Adam Paszke, Eugenio Culurciello, 2017. Reproduced with permission.

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

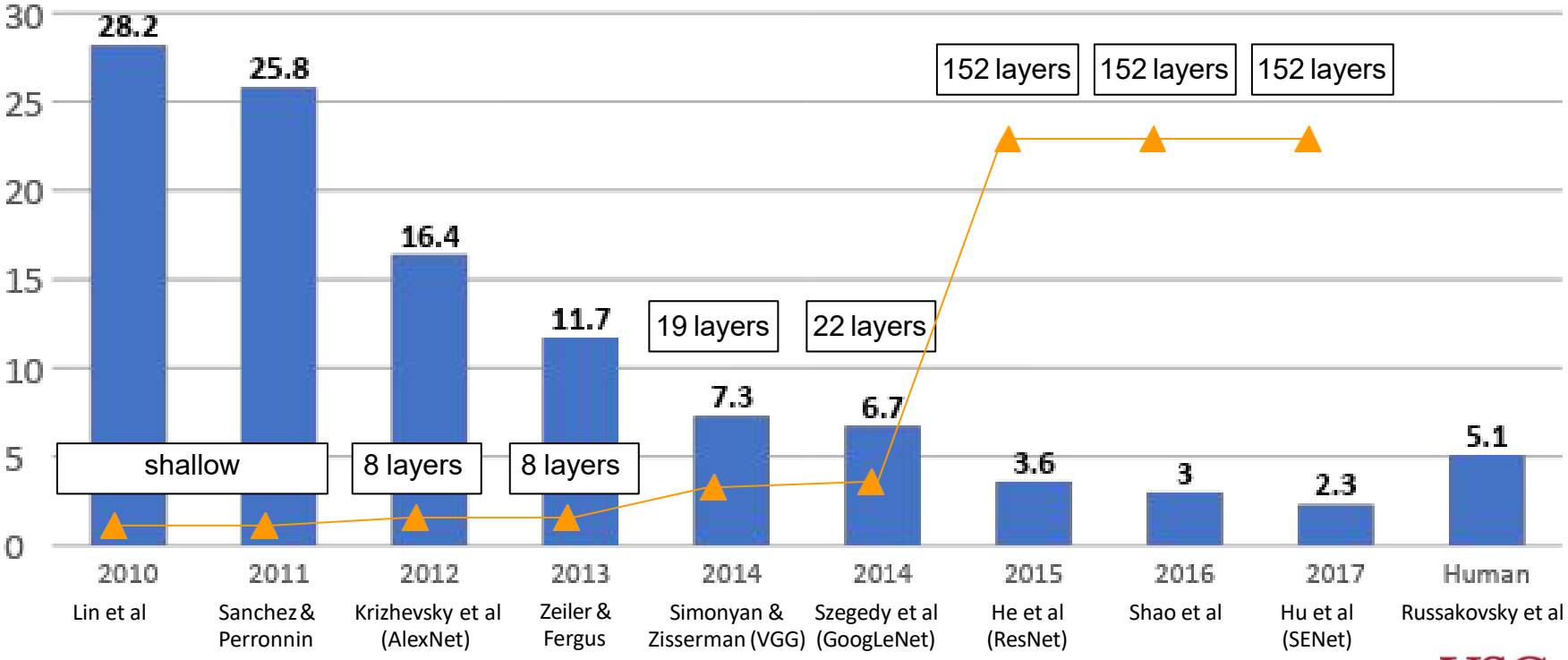


Figure from Stanford cs231n lecture slides

USC Viterbi School of Engineering

Identity Mappings in Deep Residual Networks

[He et al. 2016]

- Improved ResNet block design from creators of ResNet
- Creates a more direct path for propagating information throughout network
- Gives better performance

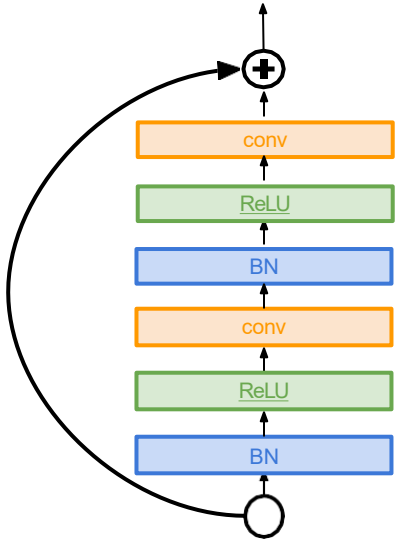
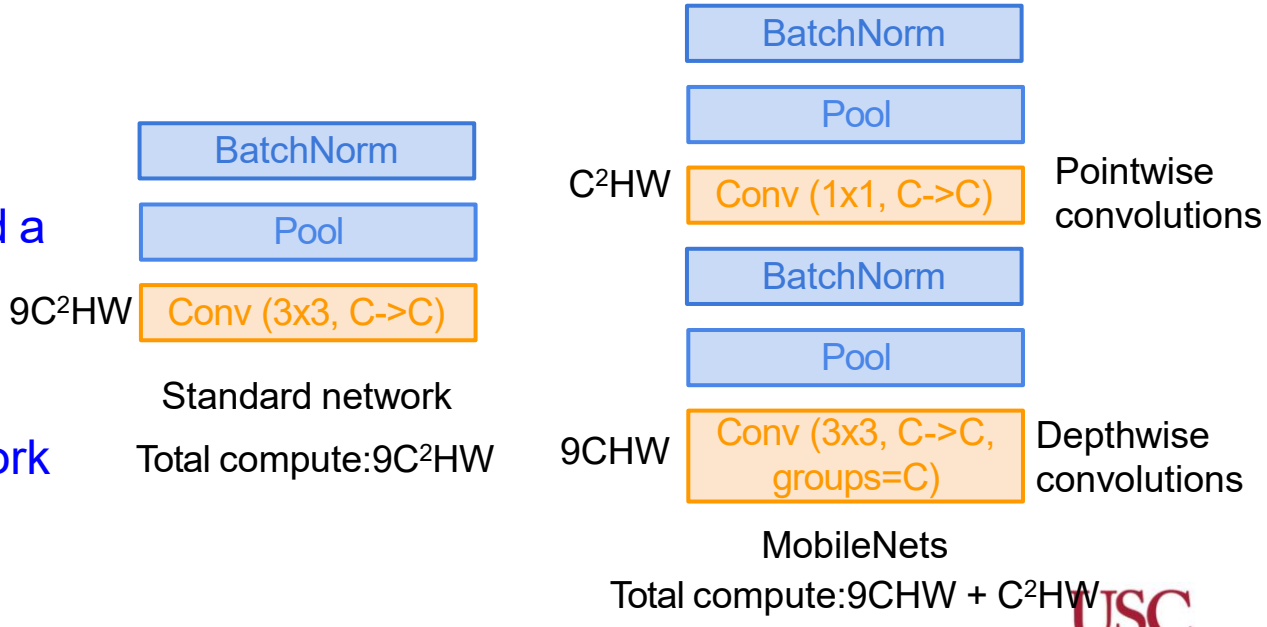


Figure from Stanford cs231n lecture slides

Efficient networks...

MobileNets: Efficient Convolutional Neural Networks for Mobile Applications *[Howard et al. 2017]*

- Depthwise separable convolutions replace standard convolutions by factorizing them into a depthwise convolution and a 1x1 convolution
- Much more efficient, with little loss in accuracy
- Follow-up MobileNetV2 work in 2018 (Sandler et al.)
- ShuffleNet: Zhang et al, CVPR 2018



USC Viterbi School of Engineering

Same shape, different bill

Scope. C_in=64, C_out=128, K=3, H=W=56. Parameters count one bias per output channel; MACs exclude bias additions.

Standard. Params = 3x3x64x128 + 128 = 73,856. MACs = 56x56x128x(3x3x64) = 231,211,008.

Depthwise. Params = 3x3x64 + 64 = 640. MACs = 56x56x64x9 = 1,806,336.

Pointwise. Params = 1x1x64x128 + 128 = 8,320. MACs = 56x56x128x64 = 25,690,112.

Separable total 8,960 params and 27,496,448 MACs: 8.24x fewer parameters, 8.41x fewer MACs.

What the 8x does not buy

Same geometry. Output stays 56x56x128 and the receptive field stays 3x3.

Lost freedom. Per-channel filtering then 1x1 mixing cannot represent every full-convolution coupling.

Accuracy. At matched parameters, factorization often costs some accuracy; width or depth can recover it.

Latency. Fewer MACs may not run faster: kernels, memory bandwidth, and hardware decide. FasterNet (CVPR 2023) measures this gap.

Counts are budgets, not accuracy or latency guarantees.

Summary: CNN Architectures

Case Studies

- AlexNet
- VGG
- GoogLeNet
- ResNet

Also....

- SENet
- Wide ResNet
- ResNeXT
- DenseNet
- MobileNets
- NASNet

ILSVRC Ended; Evaluation Moved

What ended. The official 2017 workshop called it the last ImageNet Challenge competition.

What moved. The same site said its three challenge tracks would be hosted on Kaggle.

What followed. Kaggle's localization challenge ran from 2018 to 2020; ImageNet retains historical data and an evaluation server.

Protocol. Reported ImageNet numbers now differ in pretraining data, resolution, and augmentation.

Cite the year, the split, and the training data alongside the dataset name.

EfficientNet: Search, Then Scale

Order matters. EfficientNet first used NAS to obtain the B0 baseline, then applied compound scaling.

Step 1. A grid search on B0 estimates depth, width, and resolution coefficients under a resource constraint.

Step 2. Fix the coefficients; one scale factor produces B1 through B7.

Coefficients. Tan and Le report 1.2 for depth, 1.1 for width, 1.15 for resolution, with 1.2 x 1.1^2 x 1.15^2 near 2. Check it: 1.92.

Search once for the baseline, then scale it with one factor.

Start From a Pretrained Encoder

Shift. A common workflow starts from a pretrained visual encoder, not random initialization.

Evidence. DINOv2 tests frozen features on global and dense tasks; the 2025 DINOv3 report releases ViT and ConvNeXt encoders.

Adapt. Freeze and train a head with scarce labels or compute; fine-tune for justified task shift.

Boundary. Week 11 covers self-supervised objectives; here the encoder is a component.

The checkpoint, data, license, and adaptation plan are part of the choice.

Use Model Cards, Not a Zoo Shot

Entry. Start from maintained weight APIs and model cards, not a static screenshot.

Record. Architecture, checkpoint, preprocessing, input size, license, data disclosure, and evaluation protocol belong together.

Tools. TorchVision bundles weights, transforms, and metadata; timm and Hugging Face expose model cards and encoders.

Serving. TorchServe is archived and is not a 2026 model-selection map.

A model name without its checkpoint contract is incomplete.

A Constraint-First Decision Map

Scratch. With enough labels and compute, compare a modern ConvNet with ViT or Swin under matched protocols and budgets.

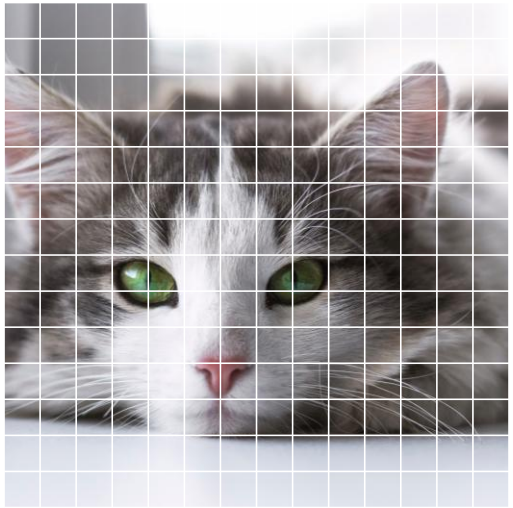
Transfer. If pretrained encoders fit the task and license, compare frozen and fine-tuned paths.

Edge. Measure latency and memory on the actual device; keep parameters and FLOPs as diagnostics.

Report. State data, checkpoint, resolution, precision, hardware, runtime, and metric.

There is no universal default; the constraint set selects the experiment.

A Photograph Becomes a Grid of Patches



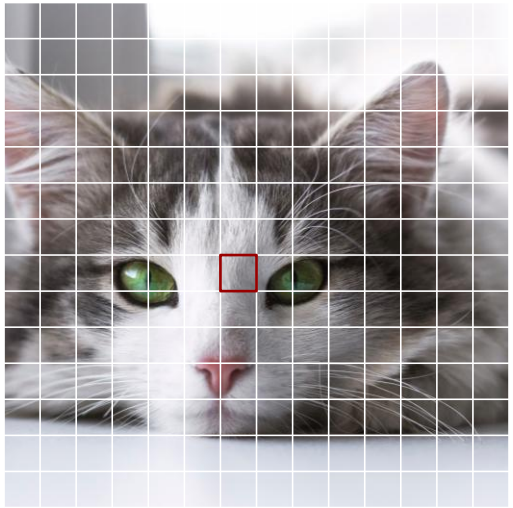
224 × 224

14 × 14
196 patches

Each patch: 16 × 16 RGB pixels

ViT: Dosovitskiy et al. (ICLR 2021). Photo: M. Vasilyev, CC0; center crop. Tokens exclude class token.

Every Patch Uses the Same Projection



224 × 224

768 numbers

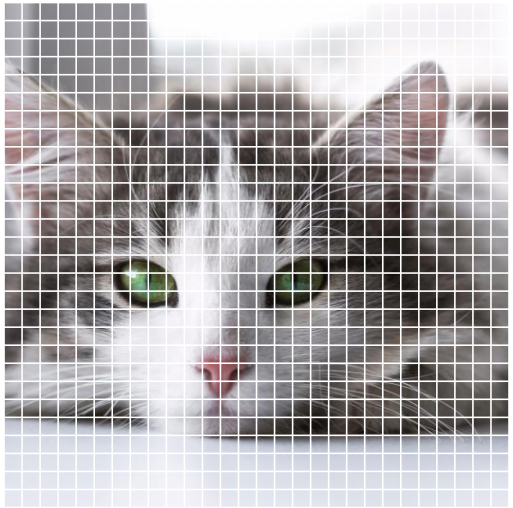
Shared linear map

D features

Same weights at every spatial location

ViT: Dosovitskiy et al. (ICLR 2021). Photo: M. Vasilyev, CC0; center crop. Tokens exclude class token.

More Pixels Can Mean Many More Tokens



448 × 448

28 × 28
784 patches

2× each side → 4× patch tokens

ViT: Dosovitskiy et al. (ICLR 2021). Photo: M. Vasilyev, CC0; center crop. Tokens exclude class token.



OpenAI

Learning Transferable Visual Models From Natural Language Supervision

ICML 2021

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, Ilya Sutskever
OpenAI



OpenAI

Learning Transferable Visual Models From Natural Language Supervision

ICML 2021

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, Ilya Sutskever
OpenAI

- This paper explores how vision models can be trained using **natural language descriptions** instead of fixed class labels.
- The goal is to develop a **general-purpose vision model** that understands concepts directly from text descriptions, enabling zero-shot learning.
- This approach moves away from traditional supervised learning by leveraging **large-scale image-text pairs** from the internet.

Contrastive learning

CLIP is based on **contrastive learning**, a technique where models learn by distinguishing correct and incorrect pairs.

In CLIP, the model sees a batch of (image, text) **pairs** and learns to associate the correct text with the corresponding image while pushing apart incorrect matches.

We will cover contrastive learning later

Contrastive learning

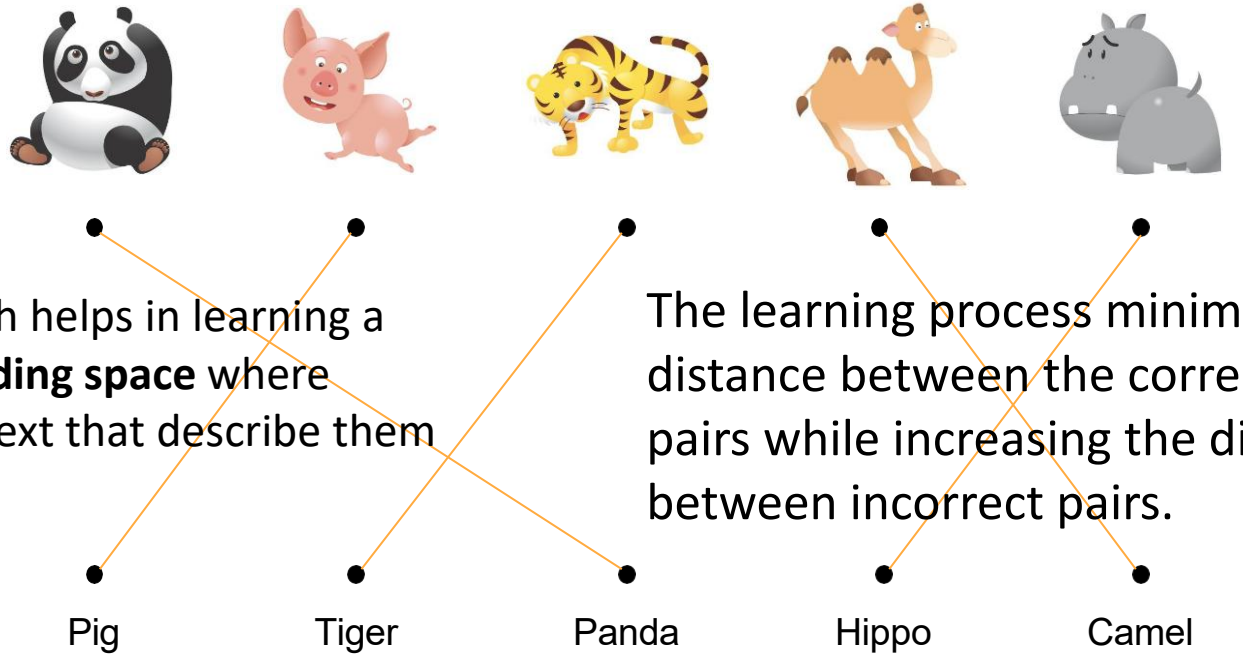






Pig Tiger Panda Hippo Camel

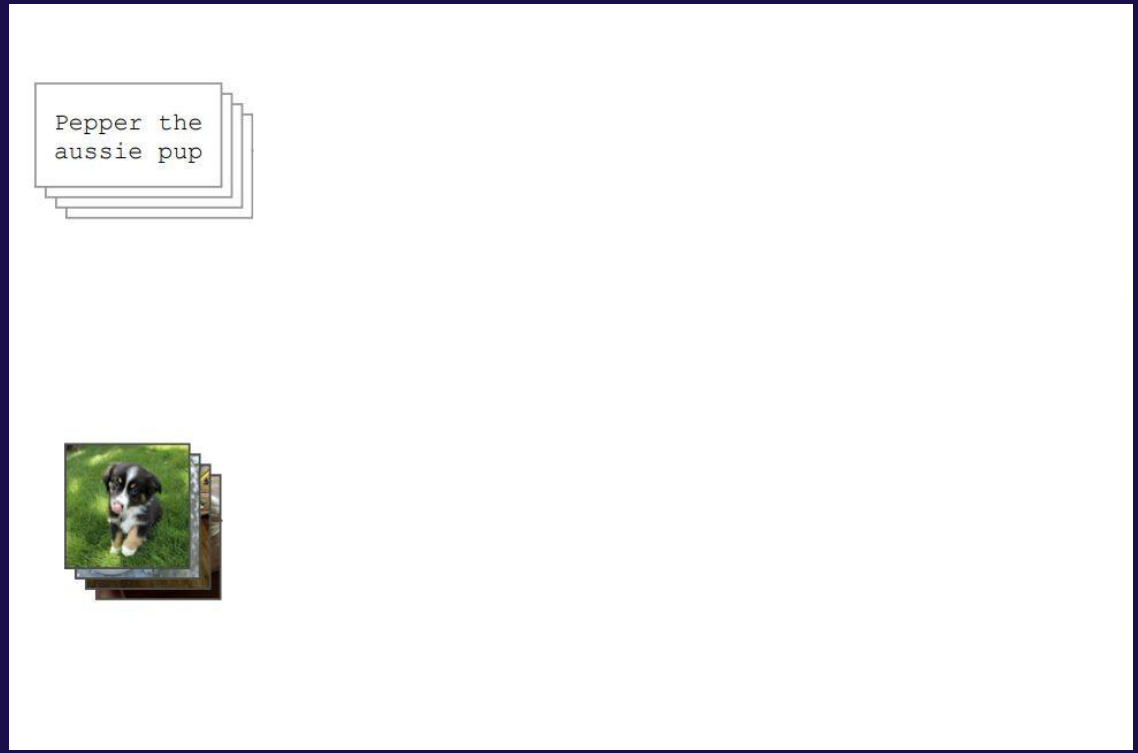
Contrastive learning



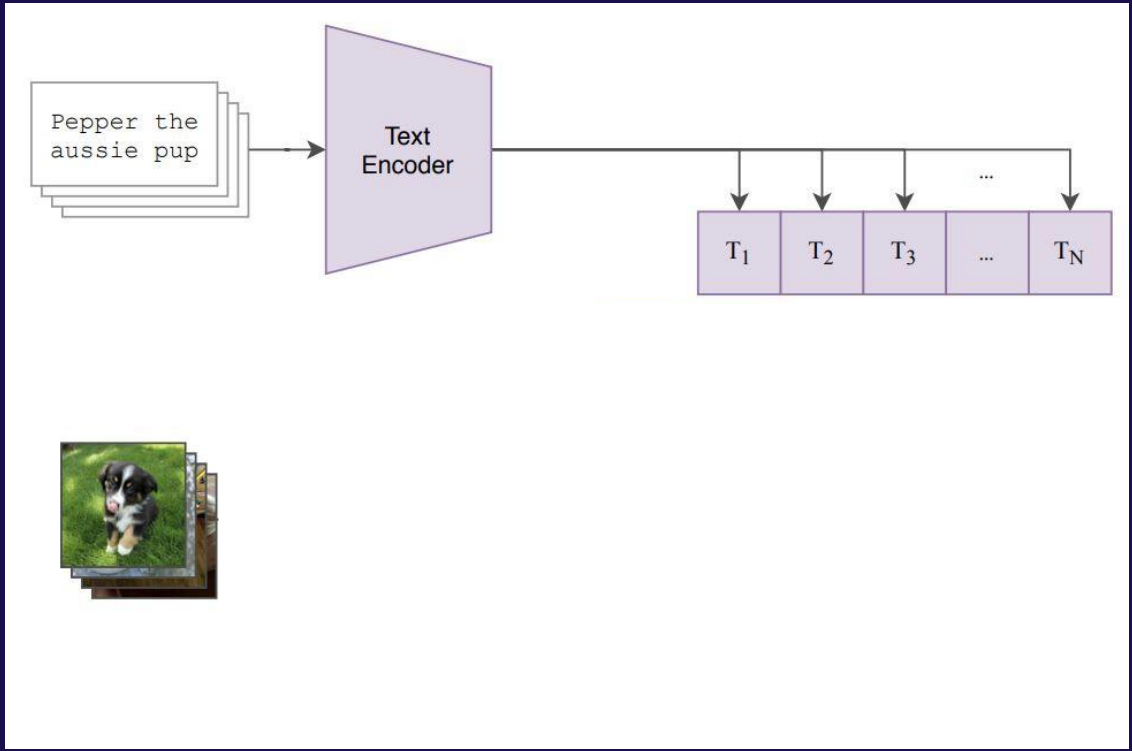
This approach helps in learning a **joint embedding space** where images and text that describe them are closer.

The learning process minimizes the distance between the correct image-text pairs while increasing the distance between incorrect pairs.

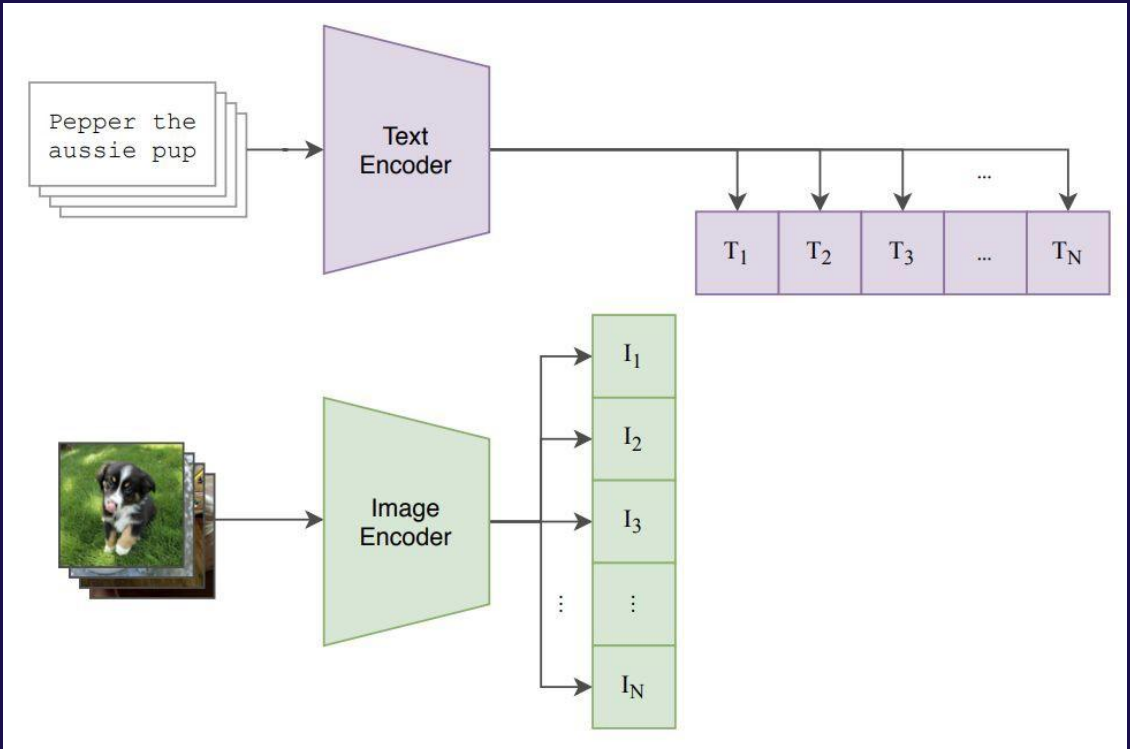
CLIP: Contrastive Language-Image Pre-training



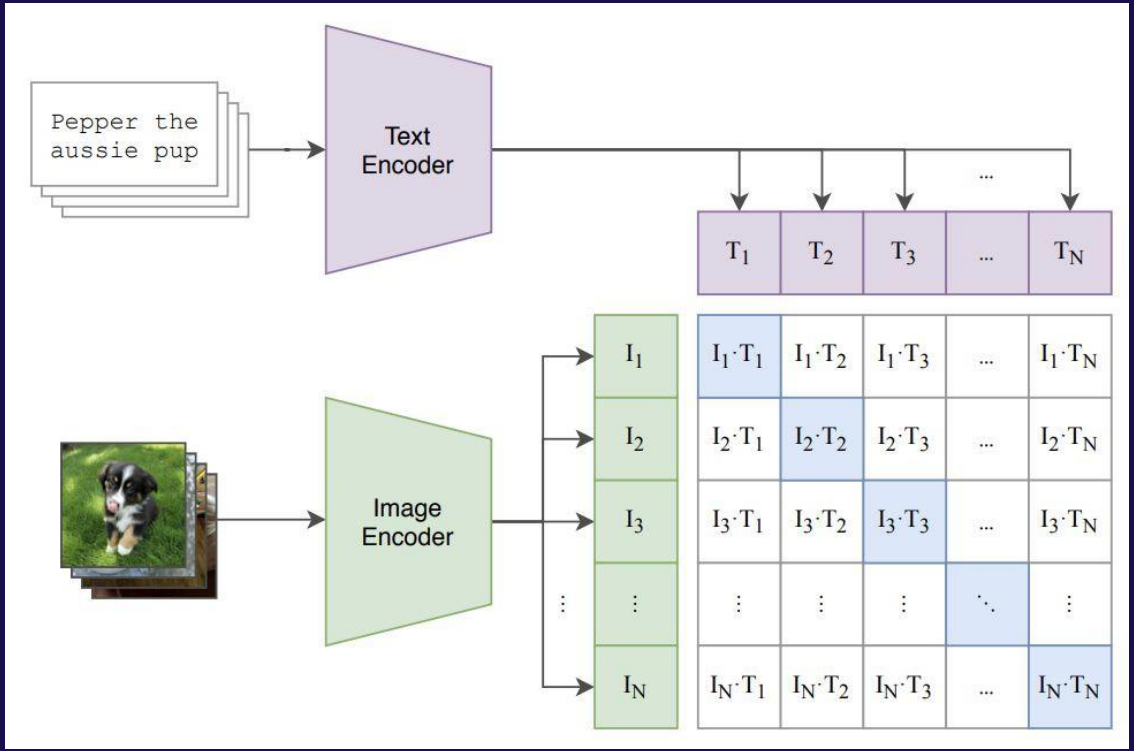
CLIP: Contrastive Language-Image Pre-training



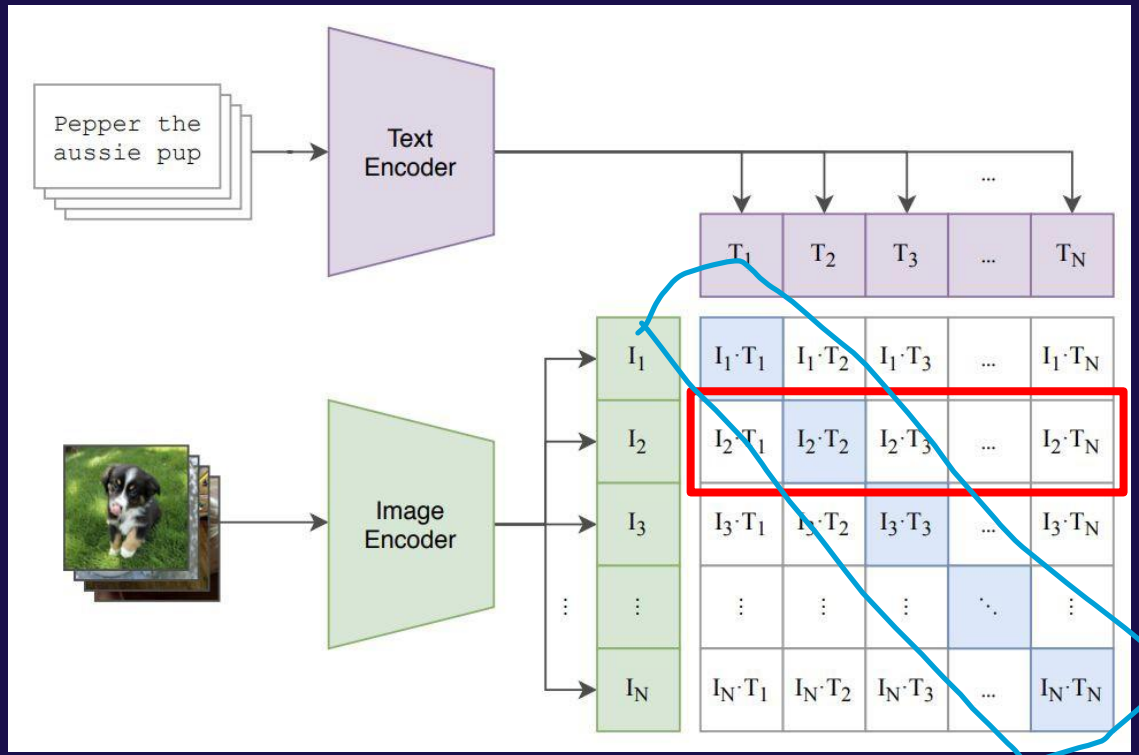
CLIP: Contrastive Language-Image Pre-training



CLIP: Contrastive Language-Image Pre-training



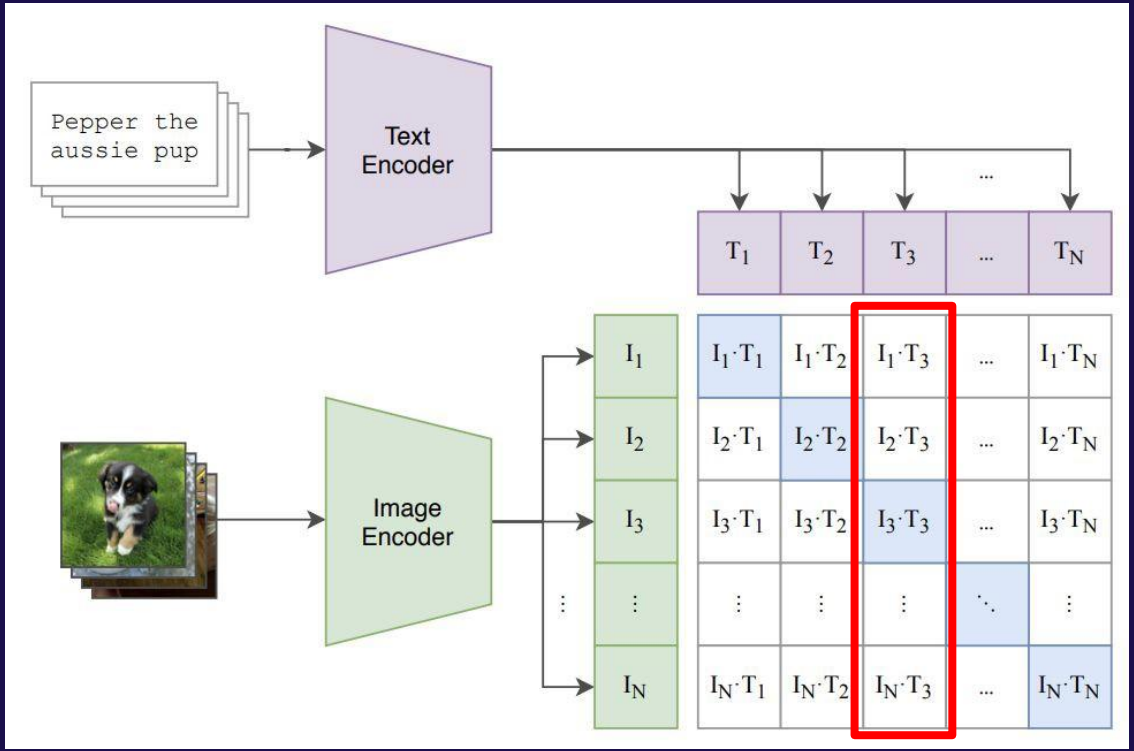
CLIP: Contrastive Language-Image Pre-training



The model is trained using large batches where each image is paired with its correct caption, and all other captions in the batch are treated as negatives.

the model can classify images based on natural language descriptions without explicit training on those categories

CLIP: Contrastive Language-Image Pre-training



Two Photos: Which Captions Match?

	“a cat”	“a dog”
	?	?
	?	?

Which photo belongs with each caption?

Predict the two positive cells.

Radford et al. (ICML 2021), Fig. 3. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

Start With Four Similarity Scores

	“a cat”	“a dog”
	4.0	1.0
	0.5	4.5

Logits = cosine × 5

Example:
0.8 × 5 = 4.0

Toy scores: 5 stands in for CLIP’s learned scale exp(t).

Radford et al. (ICML 2021), Fig. 3. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

Each Image Chooses Among Captions

	"a cat"	"a dog"
	.953	.047
	.018	.982

Each row sums to 1

Image → caption
Mean CE = .0334

Each row has its own softmax.

Radford et al. (ICML 2021), Fig. 3. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

Each Caption Chooses Among Images





	“a cat”	“a dog”
	.971	.029
	.029	.971

Each column sums to 1

Caption → image
Mean CE = .0298

Each row or column has its own softmax.

Radford et al. (ICML 2021), Fig. 3. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

CLIP Learns Both Retrieval Directions

	“a cat”	“a dog”
	4.0	1.0
	0.5	4.5

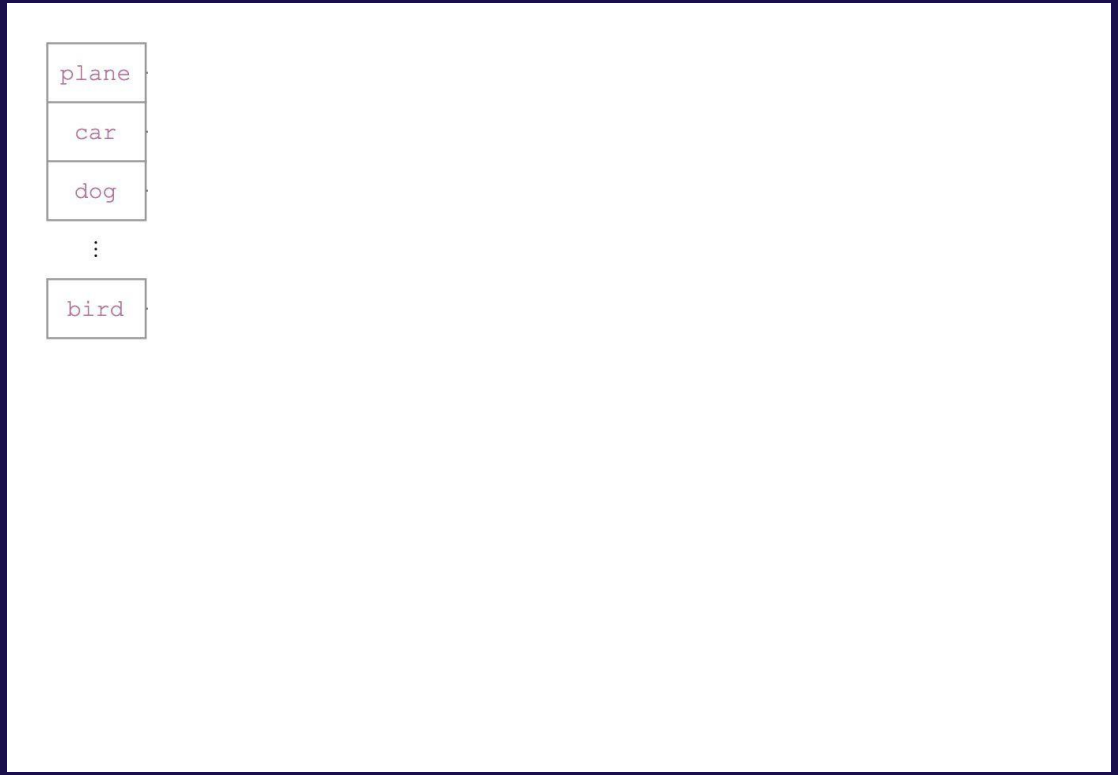
Logits shown

L = (Lrow + Lcol)/2
= 0.0316

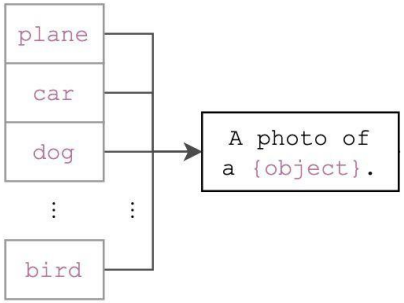
Each row or column has its own softmax.

Radford et al. (ICML 2021), Fig. 3. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

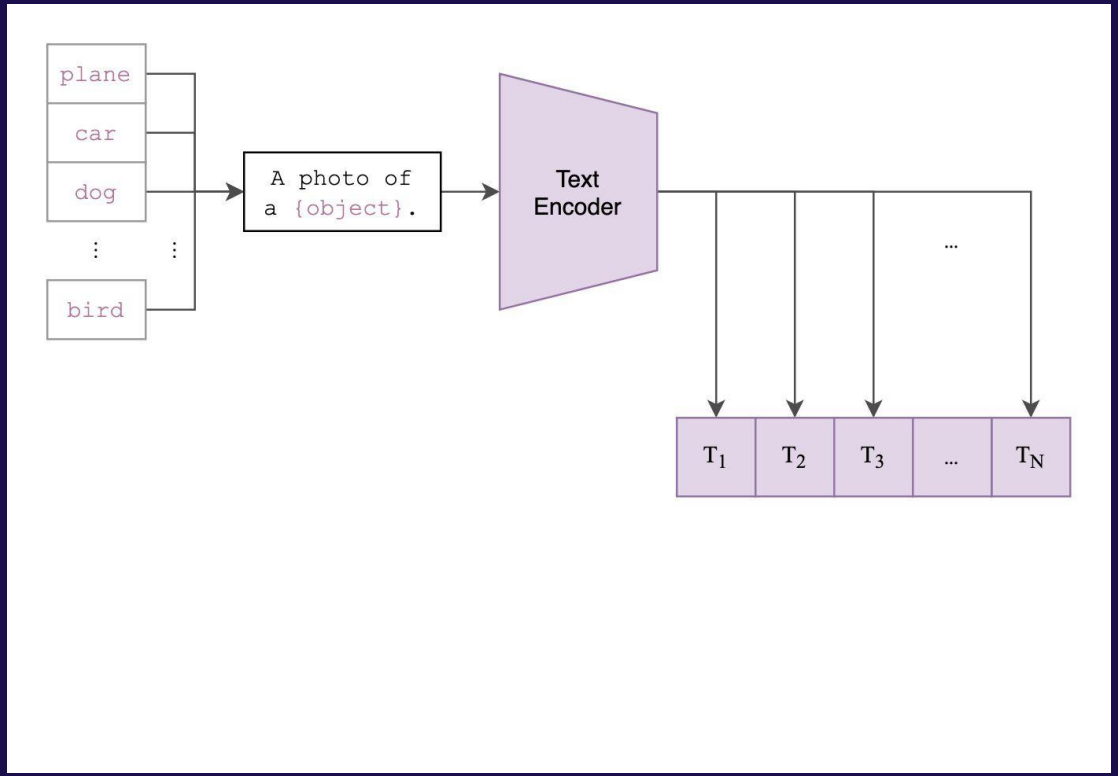
Zero-shot image classification



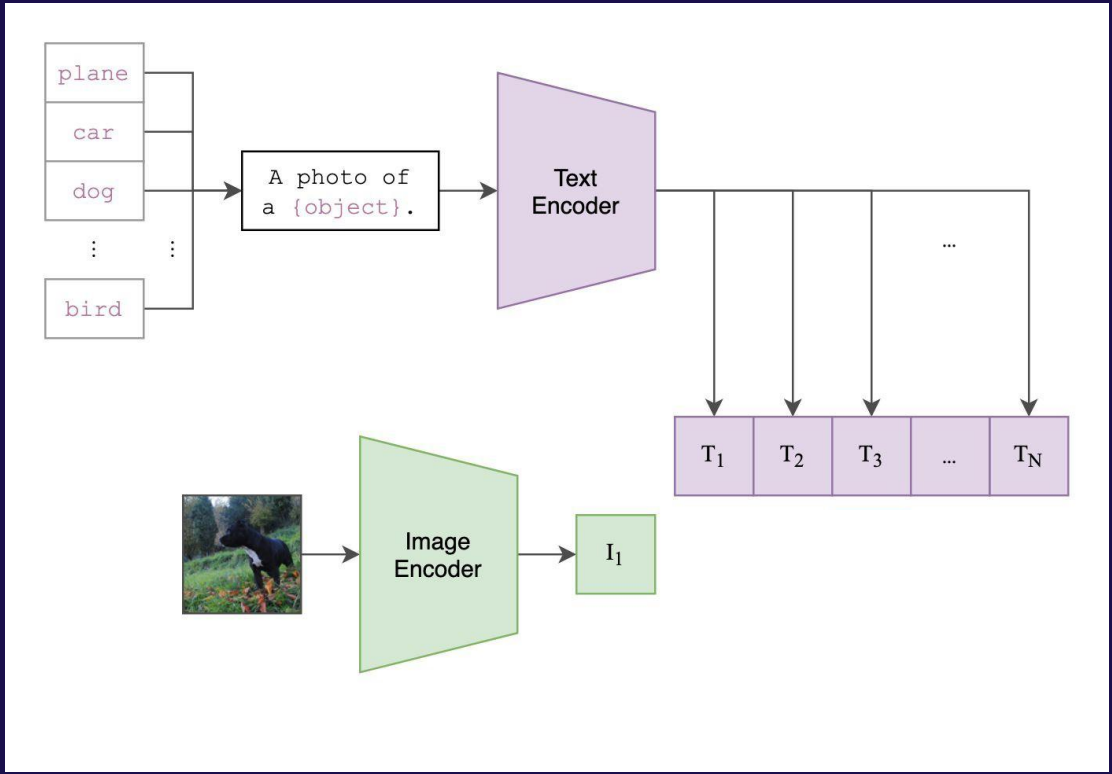
Zero-shot image classification



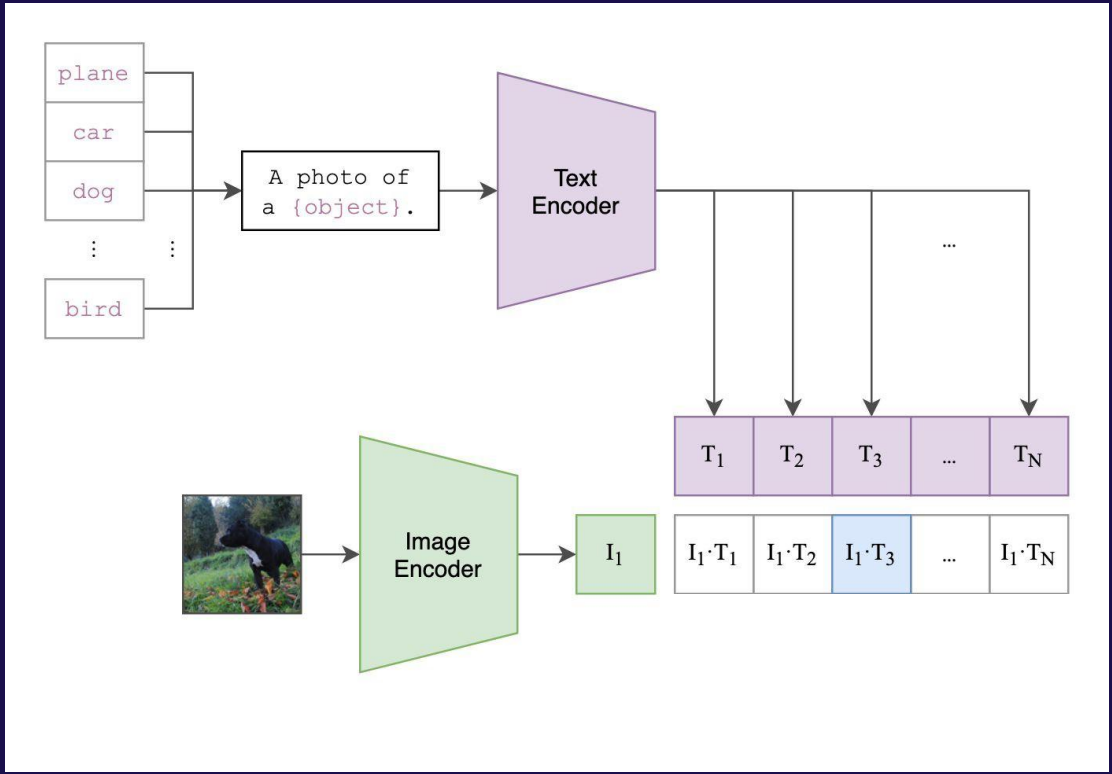
Zero-shot image classification



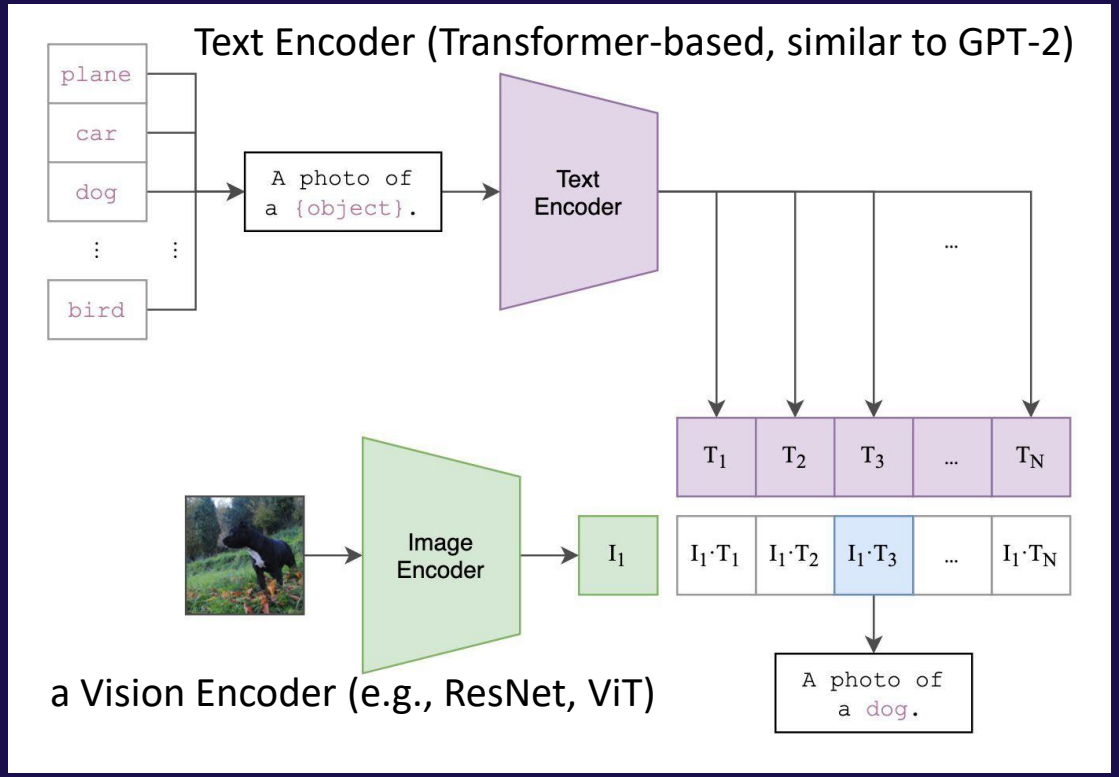
Zero-shot image classification



Zero-shot image classification

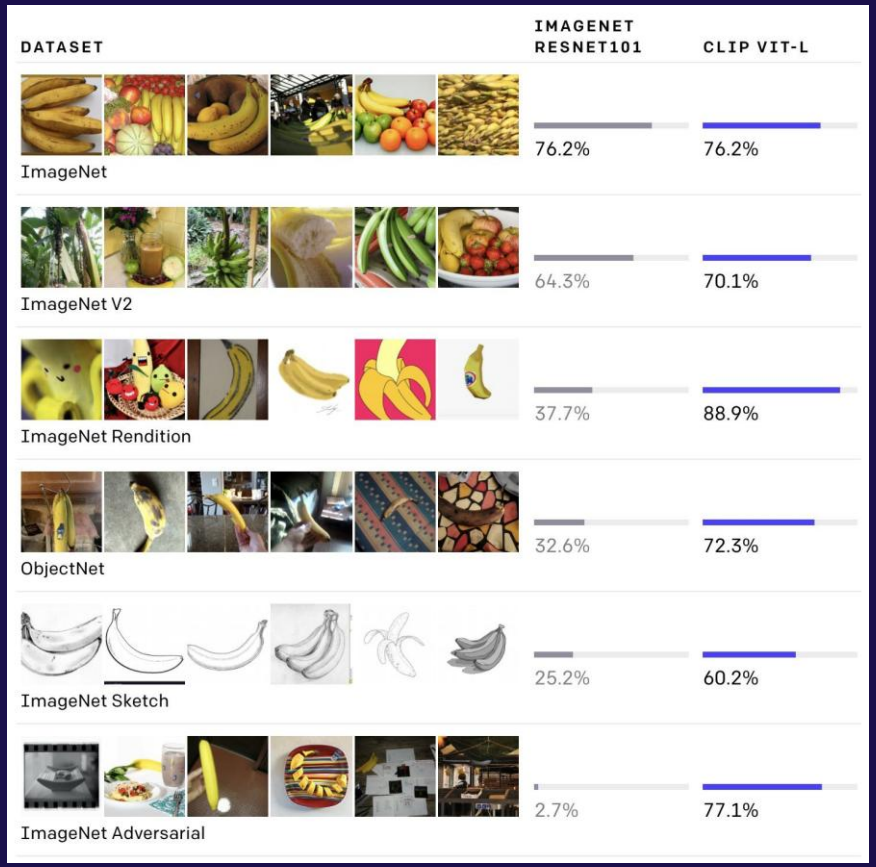


Zero-shot image classification

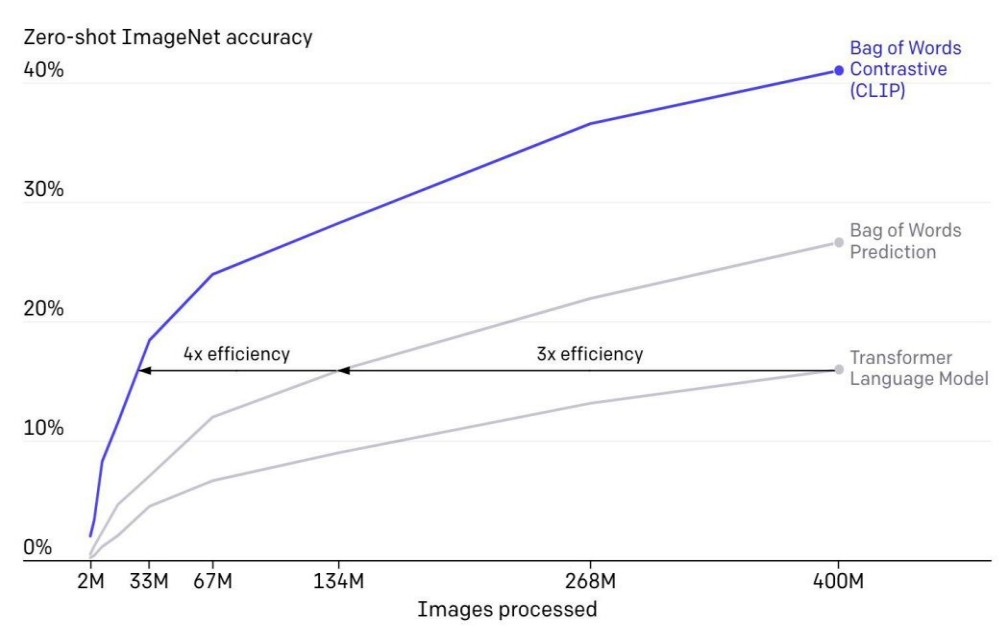


Given a set of text labels (e.g., “a photo of a dog,” “a photo of a cat”), CLIP determines which label best matches the image.

CLIP on Selected ImageNet Distribution Shifts



Why contrastive?



- In this CLIP study, contrastive training improves zero-shot learning efficiency over the caption-prediction baselines.
- The comparison is specific to these objectives, models and data.
- Image-caption pairs still provide supervision.

Some CLIP details

Training

- Trained on 400M image-text pairs from the internet
- Batch size of 32,768
- 32 epochs over the dataset
- Cosine learning rate decay

Architecture

- ResNet-based or ViT-based image encoder
- Transformer-based text encoder

Representation Learning

Linear probe (a Way for Evaluating Representation)

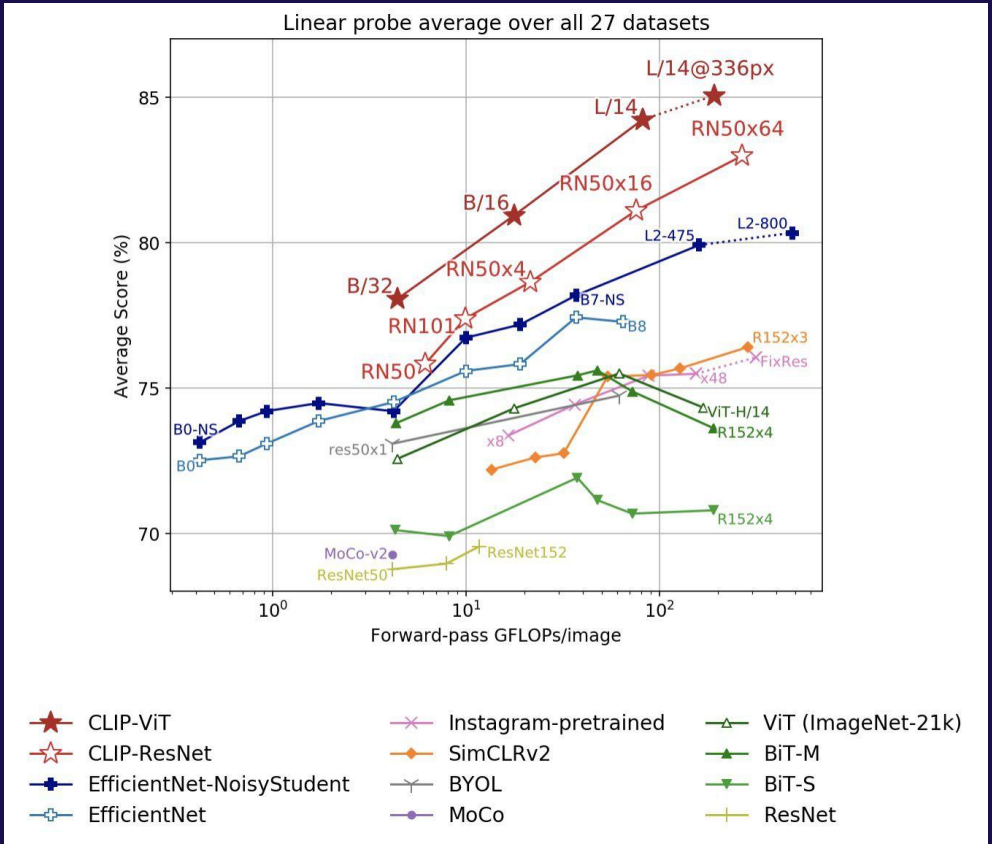
Logistic regression classifier on image features

- L-BFGS
- Only one hyperparameter
- Allows “fair” comparisons with other vision models
- A frozen-feature baseline; fine-tuning can perform worse.

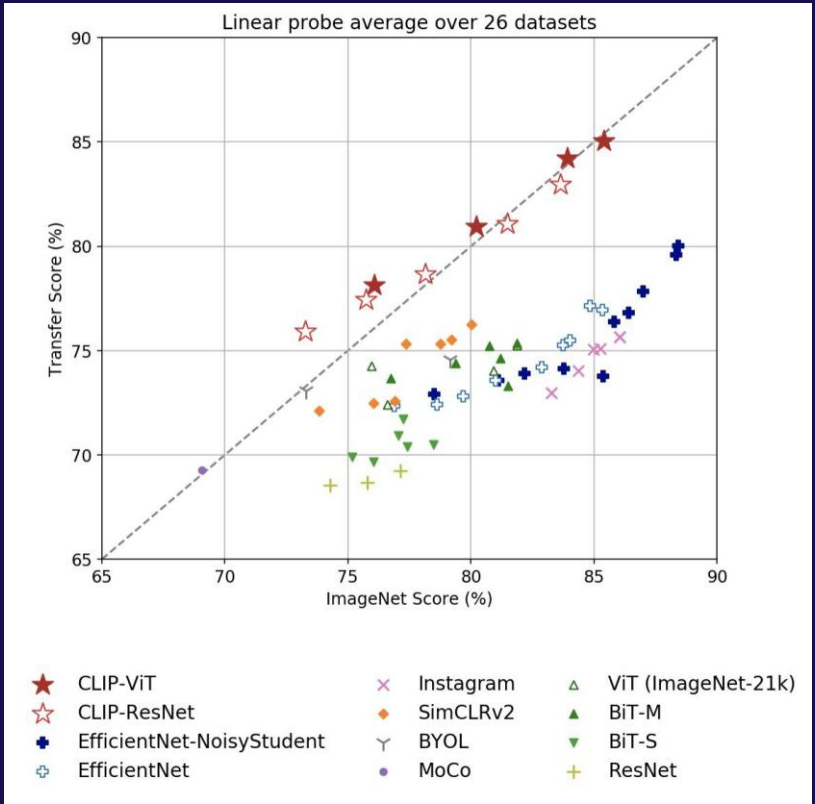
Evaluated on 27 image datasets × 65 vision models



Linear probe performance vs SOTA vision models



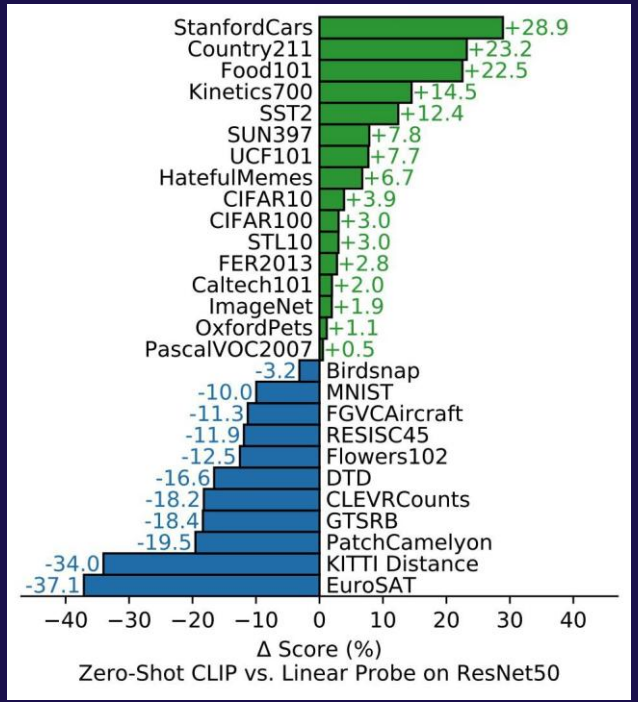
vs ImageNet score



Zero-Shot Transfer

Zero-shot vs Linear-probe ResNet-50

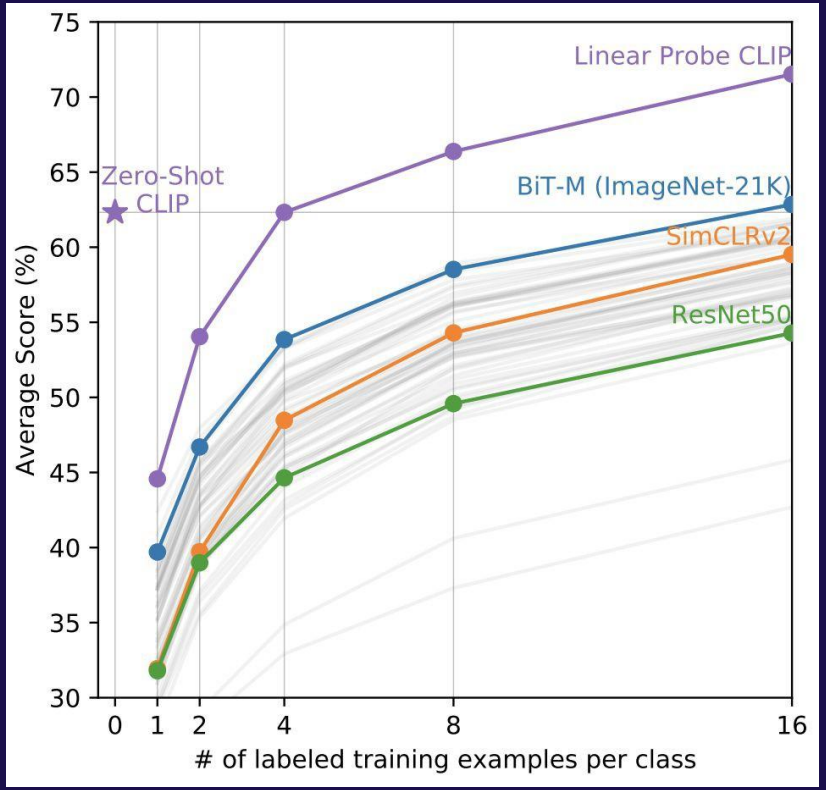
Zero-shot CLIP matches fully supervised ResNet-50 across eval suite



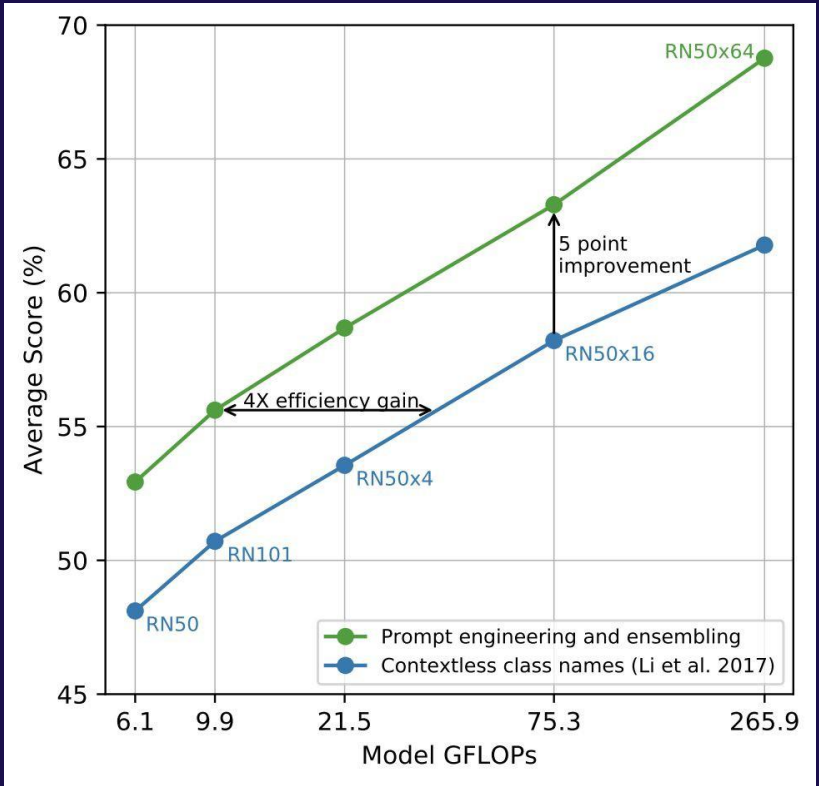
Zero-shot CLIP vs Few-shot linear probes

Zero-shot CLIP is as good as

- 4-shot linear-probe CLIP
- 16-shot BiT-M



Prompt engineering



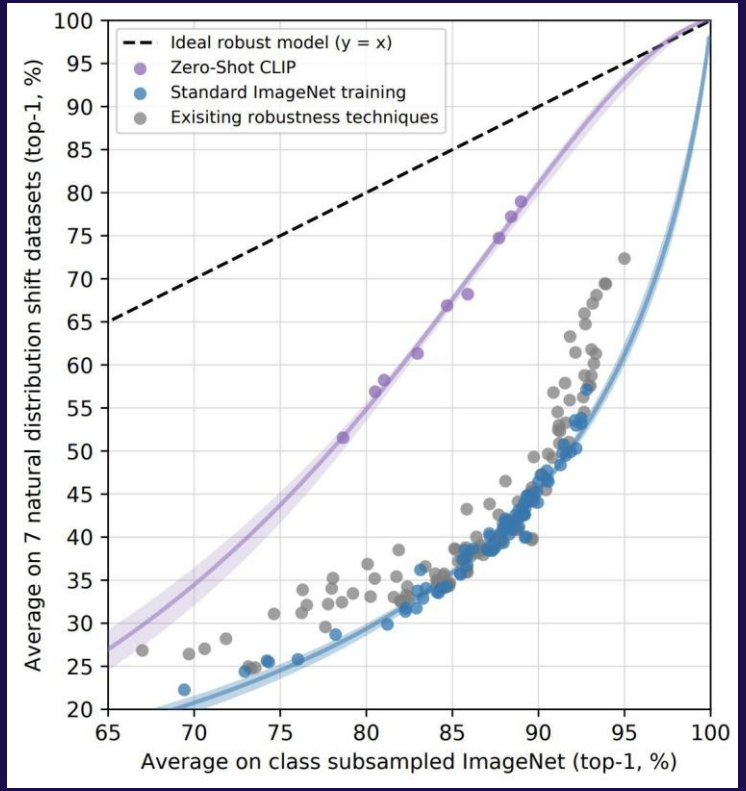
Robustness to Natural Distribution Shift

Robustness to natural distribution shift

Higher average robustness in this evaluation suite.

7 ImageNet-like Datasets (Taori et al.)

- ImageNetV2
- ImageNet-A
- ImageNet-R
- ImageNet Sketch
- ObjectNet
- ImageNet Vid
- Youtube-BB



Limitations and Broader Impacts

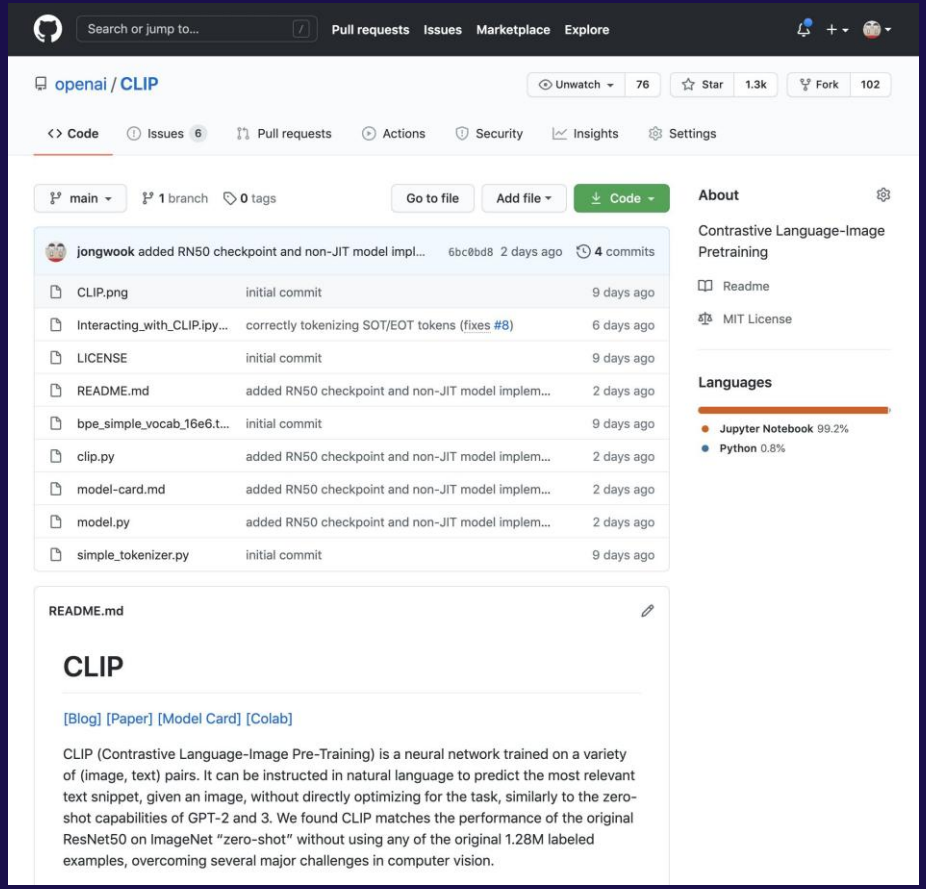
Limitations of CLIP

- Zero-shot performance is well below the SOTA in some tasks
- Especially weak on abstract tasks such as counting
- Poor on out-of-distribution data such as MNIST
- Susceptible to adversarial attacks
- Dataset selection in the eval suite, use of large validation sets for prompt engineering
- Social biases

Try CLIP today!

https://github.com/openai/CLIP

- PyTorch implementation
- Colab notebook
- Zero-Shot prediction reference
- Linear probe reference
- YFCC100M dataset
- Released models



Matching Selects From Supplied Text



a cat

a dog

a bicycle

Rank the given candidates

Liu et al. (NeurIPS 2023), Visual Instruction Tuning. Photo: M. Vasilyev, CC0. Illustrative outputs.

Generation Needs a Language Interface



Vision encoder
+ learned projector

LLM → “Its eyes are green.”

Question: What color are its eyes?

Liu et al. (NeurIPS 2023), Visual Instruction Tuning. Photo: M. Vasilyev, CC0. Illustrative outputs.

Cropping to a Square Throws Content Away

Center square crop



Keep the full aspect ratio



What information disappeared when the image became square?

Tschannen et al. (2025), SigLIP 2. Photo: D. Vorderstraße, CC BY 2.0; display crops.

Same Token Budget, Different Grid Shape

Center square crop



Keep the full aspect ratio



16-px patches: $256^2 \rightarrow 16 \times 16$; $128 \times 512 \rightarrow 8 \times 32$.
Both use 256 tokens. These shapes are separate from the photos.

Tschannen et al. (2025), SigLIP 2. Photo: D. Vorderstraße, CC BY 2.0; display crops.

Sigmoid Pairing: One Target per Cell

	“a cat”	“a dog”
	y = 1	y = 0
	y = 0	y = 1

One binary decision per cell

SigLIP 2 (2025), sigmoid image-text pairing. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

Pairwise Probabilities Need Not Sum to One

	“a cat”	“a dog”
	σ(2) = .881	σ(2) = .881
	negative	positive

Top row: .881 + .881 ≠ 1

Cat + “a dog”: negative pair; loss pushes its logit down.

SigLIP 2 (2025), sigmoid image-text pairing. Photos: M. Vasilyev (CC0); D. Vorderstraße (CC BY 2.0).

A Visual Agent Needs Evidence It Can Read



OBSERVE

What is the printed serial number?

Locate the label first.

Which region should the agent inspect next?

Google, Agentic Vision (Jan 2026). Photo: GautamSudhanshu, CC BY-SA 4.0; annotated/cropped views, same license.

Choose the Region Before Reading Its Digits



What is the printed serial number?

Action: crop the serial-number region from the original-resolution image.

ACT

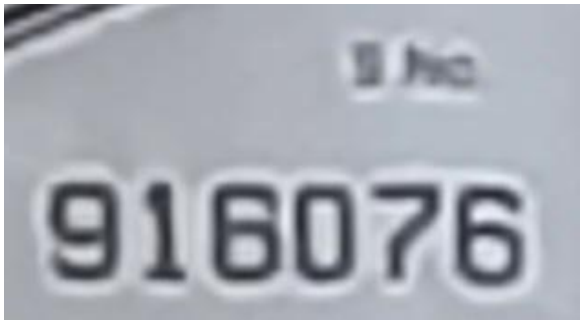
Google, Agentic Vision (Jan 2026). Photo: GautamSudhanshu, CC BY-SA 4.0; annotated/cropped views, same license.

Observe the Crop, Then Answer the Question



OBSERVE AGAIN

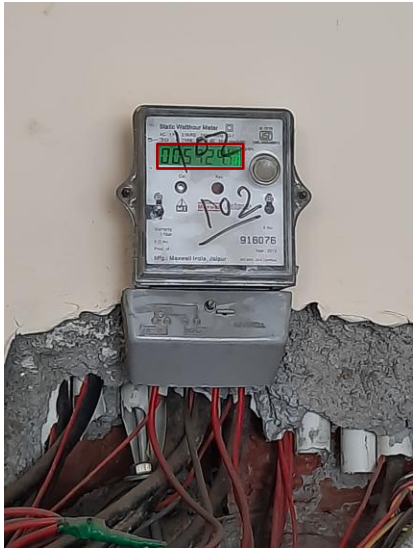
What is the printed serial number?



Answer: 916076

Google, Agentic Vision (Jan 2026). Photo: GautamSudhanshu, CC BY-SA 4.0; annotated/cropped views, same license.

Another Crop Can Still Be the Wrong Evidence



What is the printed serial number?



LCD display, partly obscured.

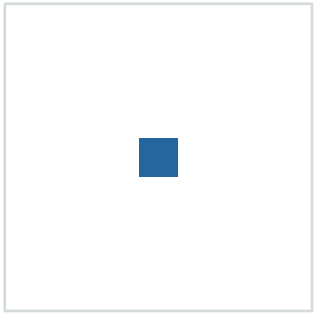
Wrong region: the LCD reading is not the printed serial number.

Google, Agentic Vision (Jan 2026). Photo: GautamSudhanshu, CC BY-SA 4.0; annotated/cropped views, same license.

Allocate Pixels to the Detail You Need

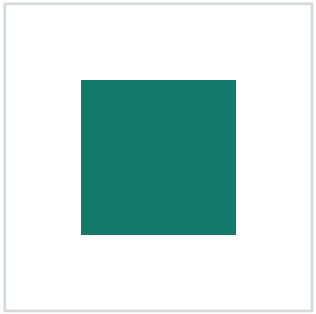
Source: 1792 × 1792 pixels; a printed digit is 16 px tall.
Magnified detail below, shown at the same display scale.

Whole image → 224²



Resize ÷8
16 → 2 px

448² crop → 224²



Resize ÷2
16 → 8 px

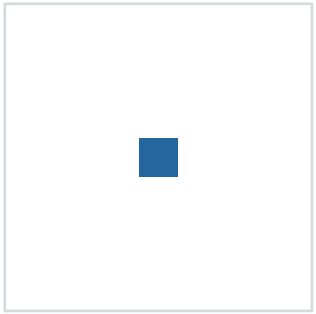
The crop must come from the original-resolution image.

Teaching geometry; blocks show magnified detail, not an OCR result or a specific model pipeline.

Cropping Trades Context for Local Detail

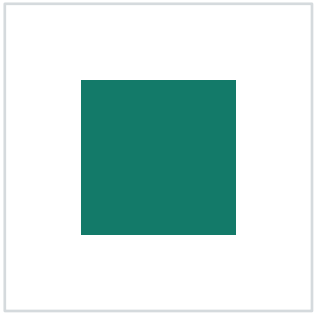
Source: 1792 × 1792 pixels; a printed digit is 16 px tall.
Magnified detail below, shown at the same display scale.

Whole image → 224²



Resize ÷8
16 → 2 px

448² crop → 224²



Resize ÷2
16 → 8 px

More pixels cannot fix occlusion or a wrong region choice.

Teaching geometry; blocks show magnified detail, not an OCR result or a specific model pipeline.

CNN applications

Image classification

Object detection

Object segmentation

Image resolution

Human pose detection

Action classification

Image captioning

Image Classification

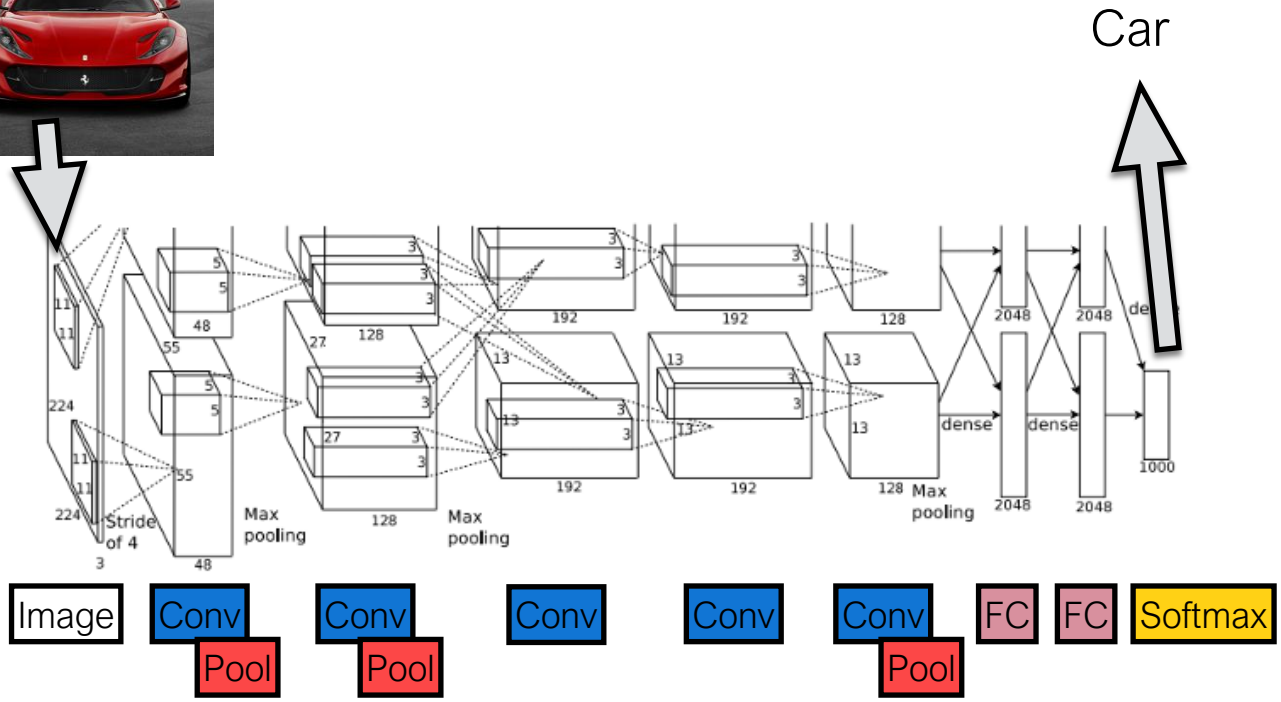




Car

Image Classification

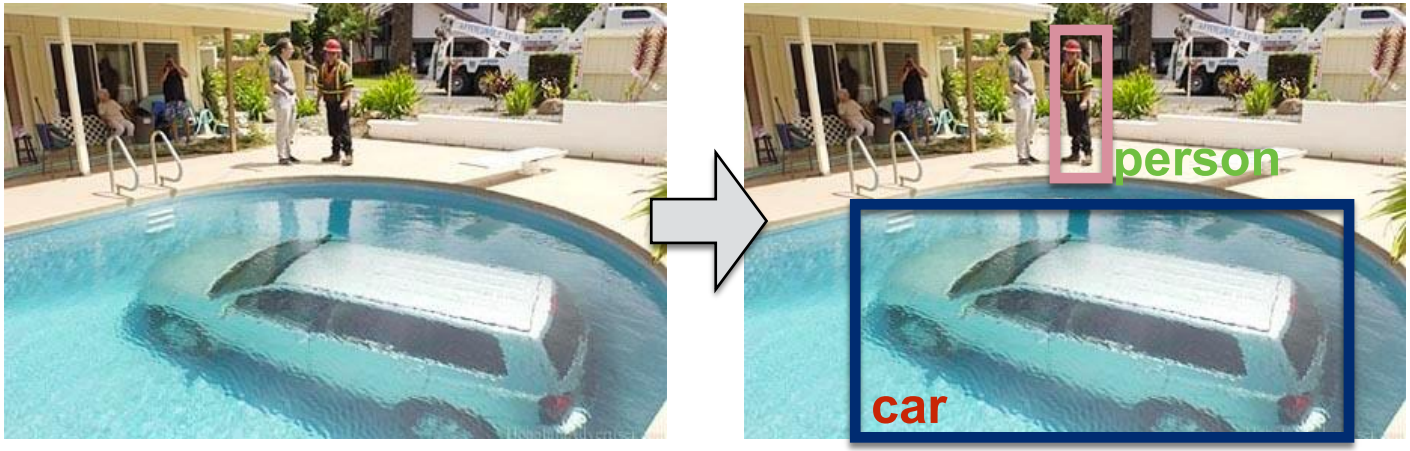




Object Detection



Object Detection



Object Detection



Object Detection



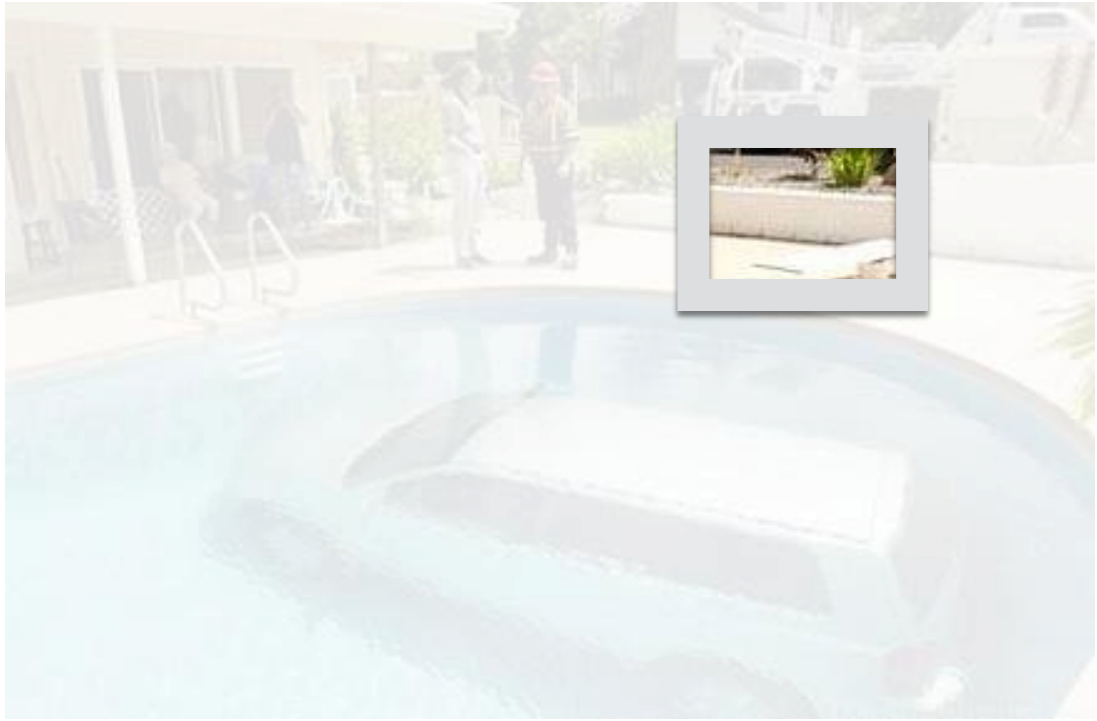
1. Object proposal

Object Detection



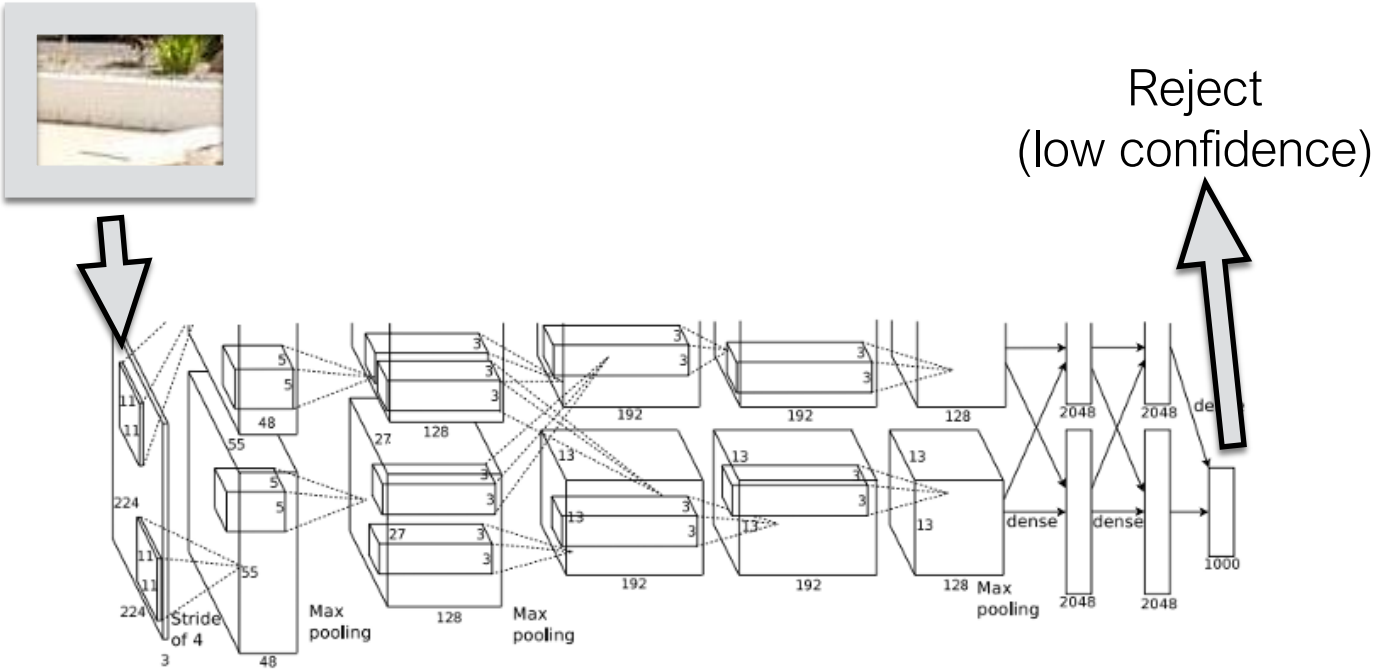
2. Evaluate each proposal

Object Detection



2. Evaluate each proposal

Object Detection



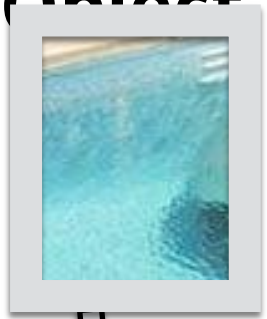
2. Evaluate each proposal

Object Detection

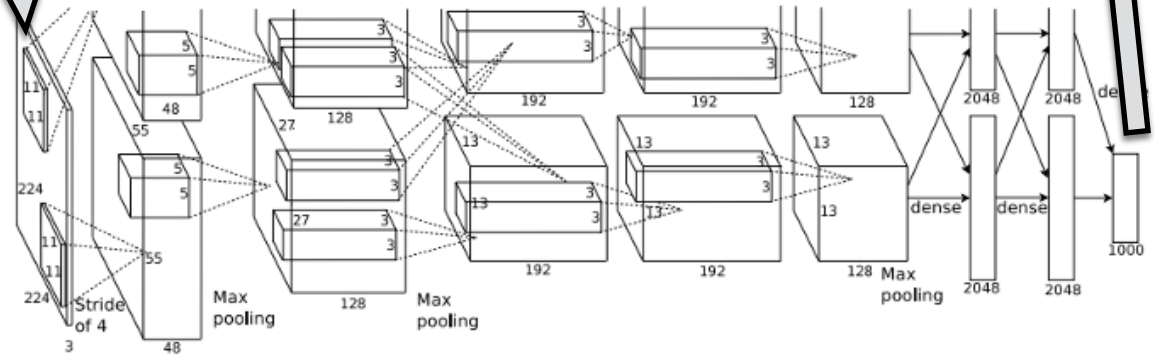


2. Evaluate each proposal

Object Detection



Reject
(low confidence)



2. Evaluate each proposal

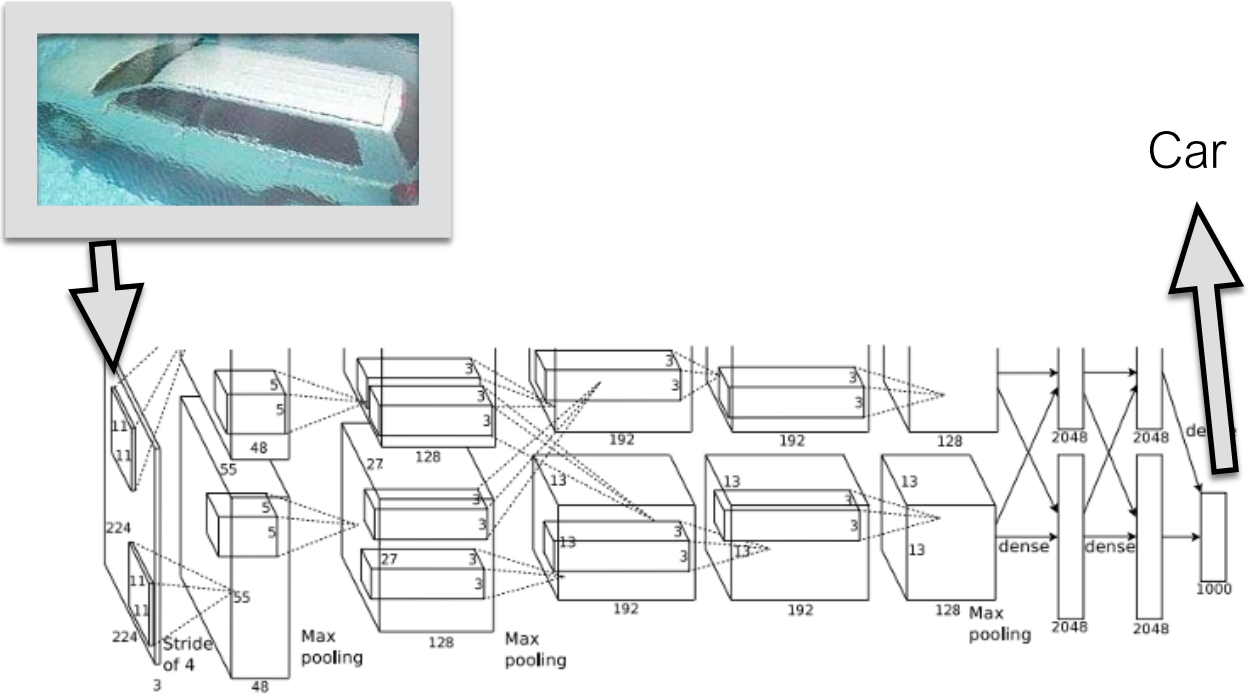
USC Viterbi School of Engineering

Object Detection



2. Evaluate each proposal

Object Detection



2. Evaluate each proposal

Object Detection



For more:
R-CNN (https://github.com/rbgirshick/rcnn) and Fast R-CNN (https://github.com/rbgirshick/fast-rcnn)

Demo

3D Face Reconstruction from a Single Image

http://www.cs.nott.ac.uk/~psxasj/3dme/

Demo

3D Face Reconstruction from a Single Image

http://www.cs.nott.ac.uk/~psxasj/3dme/



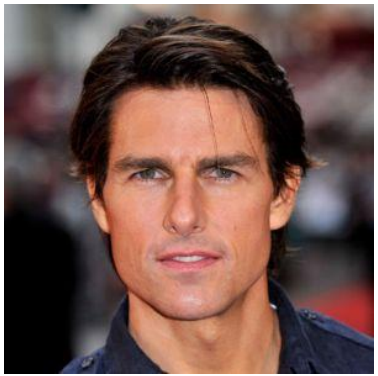




Demo

3D Face Reconstruction from a Single Image

http://www.cs.nott.ac.uk/~psxasj/3dme/



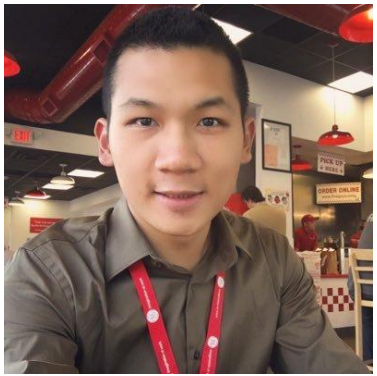




Demo

3D Face Reconstruction from a Single Image

http://www.cs.nott.ac.uk/~psxasj/3dme/







Demo

3D Face Reconstruction from a Single Image

http://www.cs.nott.ac.uk/~psxasj/3dme/





Virtual Clothing Try On

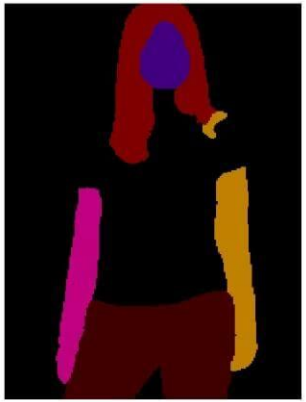


Virtual Clothing Try On

Orig. person segmentation



Transformed segmentation



Target cloth img.



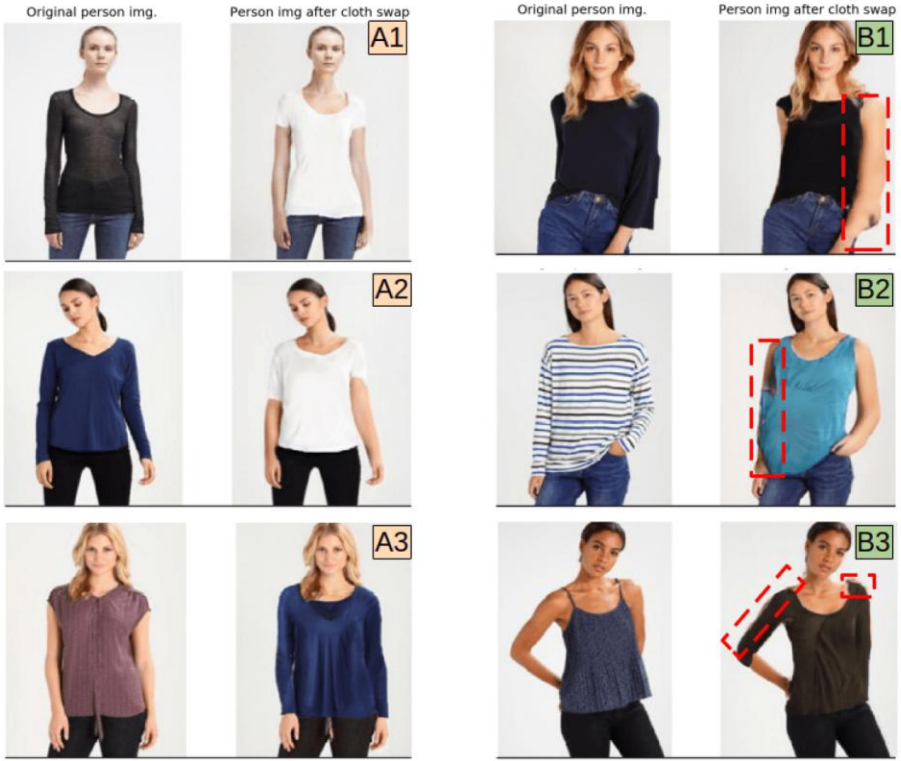
Original person img.



Person img after cloth swap

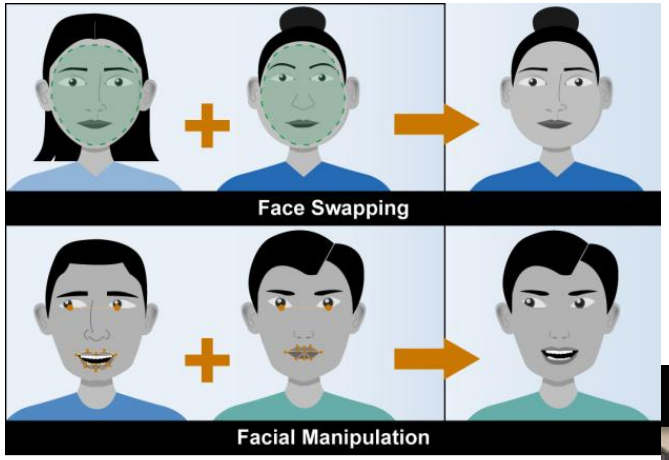


Virtual Clothing Try On



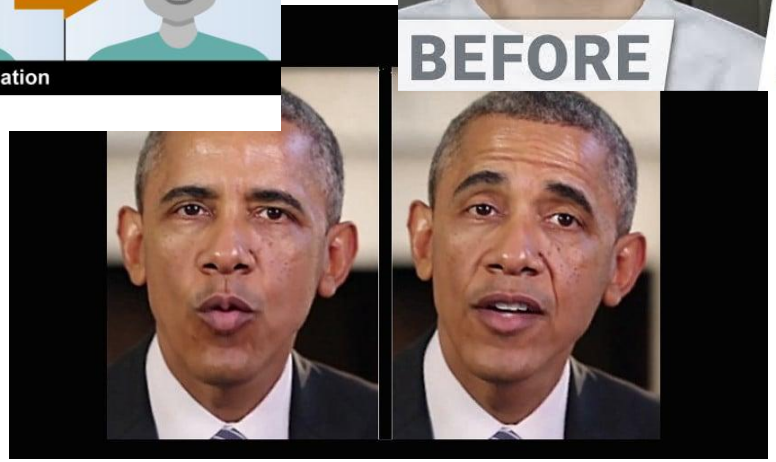
mobidev

DeepFake

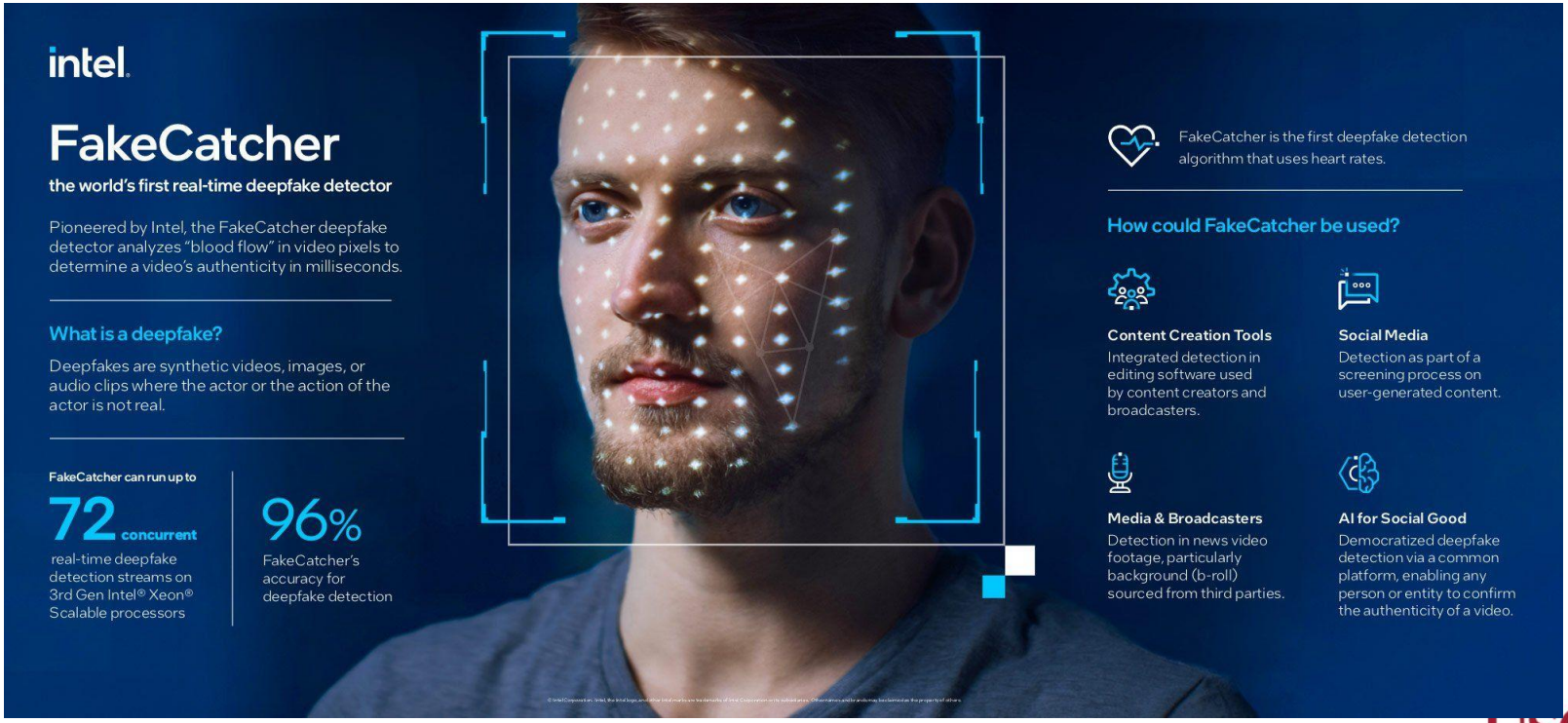


Source: GAO. | GAO-20-379SP





Detecting DeepFake

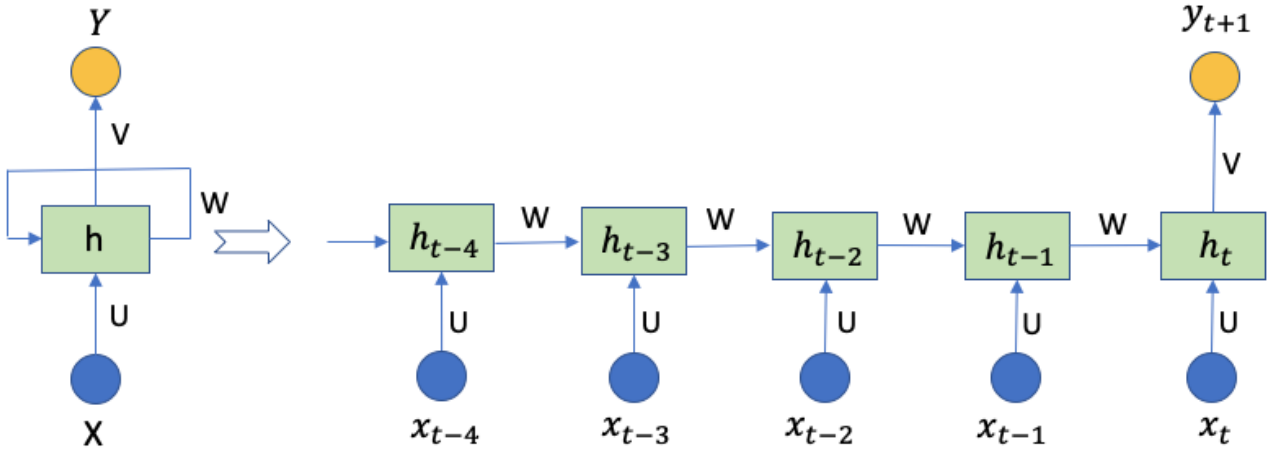


USC Viterbi School of Engineering

Next: From Image Grids to Sequences and Graphs

What changes when the input is no longer an image grid?

- Images: local neighborhoods lie on a fixed grid.
- Sequences: past information must persist across ordered observations.
- Graphs: neighborhoods follow edges, not image coordinates.

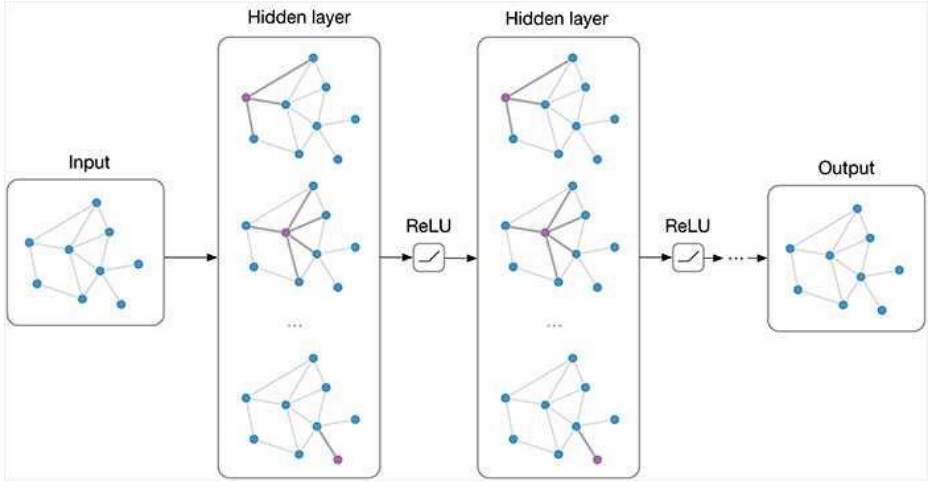


Recurrent neural networks (RNN) for sequences

Next: From Image Grids to Sequences and Graphs

What changes when the input is no longer an image grid?

- Images: local neighborhoods lie on a fixed grid.
- Sequences: past information must persist across ordered observations.
- Graphs: neighborhoods follow edges, not image coordinates.



Graph neural networks (GNN) for graphs